



Using SQL

Version 2026.1
2026-04-20

Using SQL

PDF generated on 2026-04-20

InterSystems IRIS Version 2026.1

Copyright © 2026 InterSystems Corporation

All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)

Tel: +1-617-621-0700

Tel: +44 (0) 844 854 2917

Email: support@InterSystems.com

Table of Contents

1 Using InterSystems SQL	1
1.1 Get Started	1
1.2 Topics	1
1.2.1 InterSystems SQL Syntax	1
1.2.2 SQL Execution Interfaces	1
1.2.3 Schema Definitions	2
1.2.4 Data Management and Queries	2
1.2.5 SQL Security	2
1.2.6 SQL Settings	2
1.2.7 SQL Import/Export	2
2 InterSystems SQL Features	3
2.1 Architecture	4
2.2 Features	4
2.2.1 SQL-92 Compliance	4
2.2.2 Extensions	5
2.3 Interoperability	5
2.3.1 JDBC	5
2.3.2 ODBC	6
2.3.3 Embedded SQL	6
2.3.4 Dynamic SQL	6
2.4 Limitations	7
3 InterSystems SQL Basics	9
3.1 Tables	9
3.1.1 Schemas	9
3.2 Queries	10
3.3 Privileges	11
3.4 SelectMode	11
3.5 Data Collation	12
3.6 Executing SQL	12
4 Language Elements	15
4.1 Commands and Keywords	15
4.2 Functions: Intrinsic and Extrinsic	15
4.3 Literals	16
4.3.1 String Delimiters	17
4.3.2 Concatenation	17
4.4 NULL and the Empty String	17
4.4.1 NULL Processing	17
4.4.2 NULL in Expressions	18
4.4.3 The Length of NULL	18
4.4.4 ObjectScript and SQL	19
4.5 Arithmetic Operators and Functions	20
4.5.1 Resulting Data Type	20
4.5.2 Operator Precedence	21
4.5.3 Precision and Scale	22
4.5.4 Arithmetic and Trigonometric Functions	22
4.6 Relational Operators	23

4.6.1 Contains and Follows Operators	24
4.7 Logical Operators	24
4.7.1 NOT Unary Operator	24
4.7.2 AND and OR Operators	25
4.8 Comments	26
4.8.1 Single Line Comments	26
4.8.2 Multiple Line Comments	27
4.8.3 SQL Code Retained as Comments	27
5 Implicit Joins (Arrow Syntax)	29
5.1 Property Reference	29
5.2 Child Table Reference	31
5.3 Arrow Syntax Privileges	31
6 Identifiers	33
6.1 Simple Identifiers	33
6.1.1 Naming Conventions	33
6.1.2 Case of Letters	34
6.1.3 Testing Valid Identifiers	34
6.1.4 Namespace Names	35
6.1.5 Identifiers and Class Entity Names	35
6.1.6 Identifier Length Considerations	36
6.2 Delimited Identifiers	36
6.2.1 Delimited Identifier Valid Names	37
6.2.2 Disabling Delimited Identifier Support	37
6.3 SQL Reserved Words	38
7 Using Embedded SQL	39
7.1 Compiling Embedded SQL	39
7.1.1 Embedded SQL and the Macro Preprocessor	40
7.2 Embedded SQL Syntax	41
7.2.1 The &sql Directive	42
7.2.2 &sql Alternative Syntax	42
7.2.3 &sql Marker Syntax	43
7.2.4 Embedded SQL and Line Offsets	43
7.3 Embedded SQL Code	43
7.3.1 Simple SQL Statements	44
7.3.2 Schema Name Resolution	44
7.3.3 Literal Values	45
7.3.4 Data Format	46
7.3.5 Privilege Checking	47
7.4 Host Variables	47
7.4.1 Host Variable Examples	49
7.4.2 Host Variable Subscripted by Column Number	50
7.4.3 NULL and Undefined Host Variables	51
7.4.4 Validity of Host Variables	52
7.4.5 Host Variables and Procedure Blocks	53
7.5 SQL Cursors	54
7.5.1 The DECLARE Cursor Statement	55
7.5.2 The OPEN Cursor Statement	55
7.5.3 The FETCH Cursor Statement	56
7.5.4 The CLOSE Cursor Statement	56

7.6 Embedded SQL Variables	56
7.6.1 %msg	57
7.6.2 %ROWCOUNT	57
7.6.3 %ROWID	59
7.6.4 SQLCODE	60
7.6.5 \$TLEVEL	60
7.6.6 \$USERNAME	61
7.7 Embedded SQL in Methods of a Persistent Class	61
7.8 Validating Embedded SQL Code	61
7.8.1 Compile with /compileembedded Qualifier	62
7.8.2 Test with Show Plan	62
7.9 Auditing Embedded SQL	63
8 Using Dynamic SQL	65
8.1 Introduction to Dynamic SQL	65
8.1.1 Dynamic SQL versus Embedded SQL	65
8.2 The %SQL.Statement Class	66
8.3 Creating an Object Instance	66
8.3.1 %SelectMode Property	67
8.3.2 %SchemaPath Property	68
8.3.3 %Dialect Property	69
8.3.4 %ObjectSelectMode Property	70
8.4 Preparing an SQL Statement	71
8.4.1 %Prepare()	71
8.4.2 %PrepareClassQuery()	73
8.4.3 Results of a Successful Prepare	75
8.4.4 The prepare() Method	76
8.5 Executing an SQL Statement	76
8.5.1 %Execute()	76
8.5.2 %ExecDirect()	79
8.5.3 %ExecDirectNoPriv()	80
8.6 Returning the Full Result Set	80
8.6.1 %Display() Method	81
8.6.2 %DisplayFormatted() Method	81
8.6.3 Paginating a Result Set	82
8.7 Returning Specific Values from the Result Set	83
8.7.1 %Print() Method	84
8.7.2 %GetRow() and %GetRows() Methods	85
8.7.3 rset.name Property	86
8.7.4 %Get("fieldname") Method	89
8.7.5 %GetData(n) Method	90
8.8 Returning Multiple Result Sets	91
8.9 SQL Metadata	91
8.9.1 Statement Type Metadata	92
8.9.2 Select-item Metadata	92
8.9.3 Query Arguments Metadata	97
8.9.4 Query Result Set Metadata	98
8.10 Auditing Dynamic SQL	99
9 Using the SQL Shell Interface	101
9.1 Invoking the SQL Shell	101
9.1.1 GO Command	103

9.1.2 Input Parameters	103
9.1.3 Executing ObjectScript Commands	103
9.1.4 Browsing the Namespace	104
9.1.5 CALL Command	105
9.1.6 Executing an SQL Script File	105
9.2 Storing and Recalling SQL Statements	105
9.2.1 Recall by Number	105
9.2.2 Recall by Name	106
9.3 Purging Cached Queries	107
9.4 Configuring the SQL Shell	107
9.4.1 Configuring SQL Shell System-wide Defaults	108
9.4.2 Configuring Parameters for an SQL Shell	108
9.4.3 Setting COLALIGN	109
9.4.4 Setting DISPLAYMODE and DISPLAYTRANSLATETABLE	109
9.4.5 Setting EXECUTEMODE	110
9.4.6 Setting ECHO	112
9.4.7 Setting MESSAGES	112
9.4.8 Setting LOG	113
9.4.9 Setting PATH	113
9.4.10 Setting SELECTMODE	114
9.5 SQL Metadata, Query Plan, and Performance Metrics	114
9.5.1 Displaying Metadata	114
9.5.2 SHOW STATEMENT	114
9.5.3 EXPLAIN and Show Plan	115
9.5.4 SQL Shell Performance	115
9.6 Transact-SQL Support	116
9.6.1 Setting DIALECT	116
9.6.2 Setting COMMANDPREFIX	116
9.6.3 RUN Command	117
9.6.4 TSQL Examples	118
9.7 Other Ways of Executing SQL	119
10 Using the Management Portal SQL Interface	121
10.1 Management Portal SQL Facilities	121
10.1.1 Selecting a Namespace	122
10.1.2 User Customization	122
10.2 Executing SQL Query	122
10.2.1 Writing SQL Statements	123
10.2.2 Execute Query Options	123
10.2.3 Show Plan Button	124
10.2.4 SQL Statement Results	124
10.2.5 Show History	126
10.2.6 Other SQL Interfaces	127
10.3 Filtering Schema Contents	127
10.3.1 Browse Tab	128
10.4 Catalog Details	128
10.4.1 Catalog Details for a Table	128
10.4.2 Catalog Details for a View	130
10.4.3 Catalog Details for a Stored Procedure	131
10.4.4 Catalog Details for a Cached Query	131
10.5 Wizards	131

10.6 Actions	131
10.7 Open Table	132
10.8 Tools	132
10.9 SQL Process View	133
10.10 Diagnostic Logs	134
11 Defining Tables	135
11.1 Table Names and Schema Names	135
11.2 Schema Name	135
11.2.1 Schema Naming Considerations	136
11.2.2 Reserved Schema Names	136
11.2.3 Default Schema Name	137
11.2.4 Schema Search Path	138
11.2.5 Platform-Specific Schema Name Inclusion	139
11.2.6 Listing Schemas	139
11.3 Table Name	139
11.4 RowID Field	141
11.4.1 RowID Based on Fields	142
11.4.2 RowID Hidden?	142
11.5 Primary Key	143
11.6 RowVersion, AutoIncrement, and Serial Counter Fields	143
11.6.1 RowVersion Field	144
11.6.2 Serial Counter Field	145
11.6.3 AutoIncrement Field	145
11.7 Defining a Table by Using DDL	146
11.7.1 Using DDL in Embedded SQL	146
11.7.2 Using a Class Method to Execute DDL	148
11.7.3 Defining Tables by Importing and Executing a DDL Script	148
11.8 Defining a Table by Creating a Persistent Class	149
11.8.1 Defining Property Parameters	150
11.8.2 Embedded Object (%SerialObject)	154
11.8.3 Class Methods	155
11.8.4 Defining a Sharded Table by Creating a Persistent Class	155
11.9 Defining a Sharded Table	156
11.9.1 Convert a Non-Sharded Table to Sharded	157
11.10 Defining a Table by Querying an Existing Table	157
11.11 Listing Tables	158
11.12 Listing Column Names and Numbers	158
11.12.1 The Get Columns Methods	159
11.13 Listing Constraints	159
12 Defining and Using Views	161
12.1 Creating a View	161
12.1.1 Management Portal Create View Interface	162
12.1.2 Views and Corresponding Classes	162
12.2 Altering a View	163
12.3 Updateable Views	163
12.3.1 The WITH CHECK Option	163
12.4 Read-only Views	164
12.5 View ID: %VID	164
12.6 Listing View Properties	165
12.7 Listing View Dependencies	166

13 Relationships Between Tables	169
13.1 Defining a Foreign Key	169
13.2 Foreign Key Referential Integrity Checking	169
13.2.1 Defining Referential Integrity Checking with Persistent Classes	170
13.3 Parent and Child Tables	170
13.3.1 Defining Parent and Child Tables	171
13.3.2 Inserting Data into Parent and Child Tables	171
13.3.3 Identifying Parent and Child Tables	172
14 Using Triggers	173
14.1 Defining Triggers	173
14.2 Types of Triggers	174
14.2.1 AFTER Triggers	175
14.2.2 Recursive Triggers	175
14.3 ObjectScript Trigger Code	175
14.3.1 %ok, %msg, and %oper System Variables	176
14.3.2 {fieldname} Syntax	176
14.3.3 Macros within Trigger Code	177
14.3.4 {name*O}, {name*N}, and {name*C} Trigger Code Syntax	177
14.3.5 Additional ObjectScript Trigger Code Syntax	178
14.4 Python Trigger Code	178
14.5 Pulling Triggers	179
14.6 Triggers and Object Access	180
14.6.1 Not Pulling Triggers During Object Access	180
14.7 Triggers and Transactions	180
14.8 Listing Triggers	181
15 Collation	183
15.1 Collation Types	184
15.2 Namespace-wide Default Collation	184
15.3 Table Field/Property Definition Collation	185
15.4 Index Definition Collation	186
15.5 Query Collation	187
15.5.1 select-item Collation	187
15.5.2 DISTINCT and GROUP BY Collation	188
15.6 Legacy Collation Types	189
15.7 SQL and NLS Collations	189
16 Federated Tables	191
16.1 Requirements for Creating a Federated Table	191
16.2 Creating a Federated Table	192
16.2.1 Defining a Federated Table	192
16.2.2 Connecting Source Tables to a Federated Table	193
16.3 Querying a Federated Table	193
16.4 Dropping or Disconnecting a Federated Table	194
17 Foreign Tables	195
17.1 Introduction to Foreign Tables	195
17.2 Creating a Foreign Table	195
17.2.1 Step 1: Define a Foreign Server	195
17.2.2 Step 2: Define a Foreign Table	196
17.3 Querying a Foreign Table	196
17.3.1 Issuing a Passthrough Query	197

17.4 Deleting a Foreign Table	198
18 Partitioned Tables	199
18.1 Create a Partitioned Table	199
18.1.1 Partition Key Types	200
18.1.2 Multi-Field Partition Keys	201
18.1.3 Improved Query Performance Through Partition Pruning	201
18.2 View Partition Metadata	202
18.3 Move Partitions Between Databases	202
18.4 Partitioned Table Conversions	205
18.4.1 Surrogate IDs	206
18.4.2 Limitations of Partitioned Table Conversions	206
18.5 Drop or Truncate a Partition	207
19 Modifying the Database	209
19.1 Inserting Data	209
19.1.1 Insert Data Using SQL	209
19.1.2 Insert Data Using Object Properties	210
19.2 UPDATE Statements	210
19.3 Computed Field Values on INSERT or UPDATE	210
19.4 Validating Data	212
19.5 DELETE Statements	213
19.6 Transaction Processing	213
19.6.1 Transactions and Savepoints	214
19.6.2 Non-transaction Operations	214
19.6.3 Transaction Locks	215
19.6.4 Transaction Size Limitations	215
19.6.5 Reading Uncommitted Data	215
19.6.6 ObjectScript Transaction Commands	216
20 Querying the Database	217
20.1 Types of Queries	217
20.2 Using a SELECT Statement	217
20.2.1 SELECT Clause Order of Execution	218
20.2.2 Selecting Fields	218
20.2.3 The JOIN Operation	219
20.2.4 Queries Selecting Large Numbers of Fields	220
20.3 Defining and Executing Named Queries	220
20.3.1 CREATE QUERY and CALL	220
20.3.2 Class Queries	221
20.4 Queries Invoking User-defined Functions	221
20.5 Querying Serial Object Properties	222
20.6 Querying Collections	223
20.6.1 Usage Notes and Restrictions	224
20.7 Queries Invoking Free-text Search	224
20.8 Pseudo-Field Variables	224
20.9 Query Metadata	225
20.10 Queries and Enterprise Cache Protocol (ECP)	226
20.11 Cached Queries	226
20.11.1 Listing Cached Queries	227
20.11.2 Purge Cached Queries	228
21 Defining and Using Stored Procedures	229

21.1 Overview	229
21.2 Defining a Stored Procedure Using DDL	229
21.3 SQL to Class Name Transformations	230
21.4 Defining a Method Stored Procedure using Classes	231
21.5 Defining a Query Stored Procedure using Classes	232
21.6 Customized Class Queries	234
21.7 Using Stored Procedures	235
21.7.1 Stored Functions	235
21.7.2 Privileges	236
21.8 Listing Procedures	236
22 Storing and Using Stream Data (BLOBs and CLOBs)	239
22.1 Stream Fields and SQL	239
22.1.1 BLOBs and CLOBs	239
22.1.2 Defining Stream Data Fields	239
22.1.3 Inserting Data into Stream Data Fields	240
22.1.4 Querying Stream Field Data	241
22.1.5 DISTINCT, GROUP BY, and ORDER BY	243
22.1.6 Predicate Conditions and Streams	243
22.1.7 Aggregate Functions and Streams	243
22.1.8 Scalar Functions and Streams	244
22.2 Stream Field Concurrency Locking	244
22.3 Using Stream Fields within InterSystems IRIS Methods	245
22.4 Using Stream Fields from ODBC	245
22.5 Using Stream Fields from JDBC	245
23 SQL Users, Roles, and Privileges	247
23.1 SQL Privileges and System Privileges	248
23.2 %Admin_Secure Permission	248
23.3 %Admin_RoleEdit Permission	249
23.4 %Admin_UserEdit Permission	249
23.5 Users	249
23.5.1 User Name as Schema Name	249
23.5.2 Use ObjectScript to List Users	250
23.6 Roles	250
23.6.1 Use ObjectScript to List Roles	251
23.7 SQL Privileges	251
23.7.1 Granting SQL Privileges	252
23.7.2 Listing SQL Privileges	252
23.7.3 Auditing Privilege Errors	252
24 Row-Level Security	253
24.1 Introduction	253
24.2 Setting Up with the %READERLIST Field	253
24.3 Setting Up with a Custom Field	255
24.4 After Adding Row-Level Security to a Table with Existing Data	255
24.5 Security Tips	256
24.6 See Also	256
25 Using Vector Search	257
25.1 Vectors and Embeddings	257
25.2 Inserting VECTOR-typed Data	258
25.2.1 Translate Text to Embeddings	258

25.2.2 Perform an INSERT	258
25.3 Inserting EMBEDDING-typed Data	260
25.3.1 Create an Embedding Configuration	260
25.3.2 Define a Table with EMBEDDING-type Columns	262
25.4 Define a Vector Index	262
25.4.1 Hierarchical Navigable Small World Index	262
25.5 Perform Vector Search	264
25.5.1 Example	264
25.6 See More	265
26 SQL Settings Reference	267
26.1 How to Change Each SQL Setting	267
26.2 Settings that Require Permissions	269
27 Importing SQL Code	271
27.1 Importing SQL with LOAD SQL	271
27.2 Importing InterSystems SQL with ObjectScript	272
27.2.1 Import File Format	273
27.2.2 Supported SQL Commands	273
27.3 Importing non-InterSystems SQL with ObjectScript	273
Appendix A: Importing and Exporting SQL Data	275
A.1 Importing Data with LOAD DATA	275
A.2 Exporting Data to a Text File	275

List of Tables

Table 11–1: Available DDL Commands in InterSystems SQL 146

Table 25–1: 261

Table 25–2: 261

Table 26–1: 267

Table 26–2: 270

1

Using InterSystems SQL

InterSystems SQL provides a full set of standard relational features, including the ability to define table schema, execute queries, and define and execute stored procedures.

1.1 Get Started

- [InterSystems SQL Features](#) provides an overview of InterSystems SQL as it relates to software standards and interoperability.
- [InterSystems SQL Basics](#) describes the fundamental features of InterSystems SQL, such as tables and queries, and how to execute InterSystems SQL.

1.2 Topics

1.2.1 InterSystems SQL Syntax

- [Language Elements](#) describes InterSystems SQL handling of literals, NULL, operators, and comments.
- [Implicit Joins \(Arrow Syntax\)](#) provides an InterSystems SQL shorthand extension to perform a left outer join on a column specified in a SELECT clause.
- [Identifiers](#) describes the conventions used for naming entities within InterSystems SQL.

1.2.2 SQL Execution Interfaces

- [Using Embedded SQL](#) describes how to embed InterSystems SQL code within ObjectScript code.
- [Using Dynamic SQL](#) describes how to prepare and execute SQL code at runtime from ObjectScript code.
- [Using the SQL Shell Interface](#) describes how to write and execute SQL statements from the InterSystems Terminal using a shell interface.
- [Using the Management Portal SQL Interface](#) describes how to execute SQL statements and view and manage SQL features from the InterSystems Management Portal interface.

1.2.3 Schema Definitions

- [Defining Tables](#) describes fundamental elements of tables (schema and table names, RowID and Primary Key), and how to define a table in InterSystems SQL, either by defining a persistent class or by using DDL commands.
- [Defining and Using Views](#) describes how to define views and the view ID (%VID) in InterSystems SQL.
- [Relationships Between Tables](#) describes how to define and maintain foreign keys and parent/child relationships in InterSystems SQL.
- [Using Triggers](#) describes triggers that are automatically executed when a record is added, modified, or deleted in InterSystems SQL.
- [Collation](#) describes collation types that specify how data values are ordered and compared in InterSystems SQL.

1.2.4 Data Management and Queries

- [Modifying the Database](#) describes how to insert, update, and delete data, and how to use transactions to group multiple data modifications.
- [Querying the Database](#) describes how to create and execute queries in InterSystems SQL.
- [Defining and Using Stored Procedures](#) describes how to define and use stored procedures in InterSystems SQL.
- [Storing and Using Stream Data \(BLOBs and CLOBs\)](#) describes binary stream data and character stream data in InterSystems SQL.

1.2.5 SQL Security

- [SQL Users, Roles, and Privileges](#) describes security features of InterSystems SQL, including defining users, associating users with roles, and assigning privileges to a user or a role.

1.2.6 SQL Settings

- [SQL Settings Reference](#) describes the mechanisms you can use to change various SQL settings.

1.2.7 SQL Import/Export

- [Importing SQL Code](#) describes how to import and execute SQL code from a text file, either InterSystems SQL or SQL code from other vendors.
- [Importing and Exporting SQL Data](#) describes InterSystems SQL tools to import data from a text file to a table and to export data from a table to a text file.

2

InterSystems SQL Features

InterSystems SQL provides uncompromising, standard relational access to data stored within an InterSystems IRIS® data platform database.

InterSystems SQL offers the following benefits:

- *High performance and scalability* — InterSystems SQL offers performance and scalability superior to other relational database products. In addition, InterSystems SQL runs on a wide variety of hardware and operating systems; from laptop computers to high-end, multi-CPU systems.
- *Integration with InterSystems IRIS objects technology* — InterSystems SQL is tightly integrated with InterSystems IRIS object technology. You can mix relational and object access to data *without* sacrificing the performance of either approach.
- *Low maintenance* — Unlike other relational databases, InterSystems IRIS applications do not require table compression in deployed applications.
- *Support for standard SQL queries* — InterSystems SQL supports SQL-92 standard syntax and commands. In most cases, you can migrate existing relational applications to InterSystems IRIS with little difficulty and automatically take advantage of the higher performance and object capabilities of InterSystems IRIS.

You can use InterSystems SQL for many purposes including:

- *Object- and web-based applications* — You can use SQL queries within InterSystems IRIS applications to perform powerful database operations such as lookups and searches.
- *Online transaction processing* — InterSystems SQL offers outstanding performance for insert and update operations as well as the types of queries typically found within transaction processing applications.
- *Business intelligence and data warehousing* — The combination of the InterSystems IRIS multidimensional database engine and bitmap indexing technology make it an excellent choice for data warehouse-style applications.
- *Ad hoc queries and reports* — You can use the full-featured ODBC and JDBC drivers included with InterSystems SQL to connect to popular reporting and query tools.
- *Enterprise application integration* — The InterSystems SQL Gateway gives you seamless SQL access to data stored in external relational databases that are ODBC- or JDBC-compliant. This makes it easy to integrate data from a variety of sources within InterSystems IRIS applications.

2.1 Architecture

The core of InterSystems SQL consists of the following components:

- *The Unified Data Dictionary* — a repository of all meta-information stored as a series of class definitions. InterSystems IRIS automatically creates relational access (tables) for every persistent class stored within the Unified Dictionary.
- *The SQL Processor and Optimizer* — a set of programs that parse and analyze SQL queries, determine the best search strategy for a given query (using a sophisticated cost-based optimizer), and generate code that executes the query.
- *The InterSystems SQL Server* — a set of InterSystems IRIS server processes that are responsible for all communications with the InterSystems ODBC and JDBC drivers. It also manages a cache of frequently used queries; when the same query is executed multiple times, its execution plan can be retrieved from the query cache instead of having to be processed by the Optimizer again.

2.2 Features

InterSystems SQL includes a full set of standard, relational features. These include:

- The ability to define tables and views (DDL or Data Definition Language).
- The ability to execute queries against tables and views (DML or Data Manipulation Language).
- The ability to execute transactions, including INSERT, UPDATE, and DELETE operations. When performing concurrent operations, InterSystems SQL uses row-level locks.
- The ability to define and use indexes for more efficient queries.
- The ability to use a wide variety of data types, including user-defined types.
- The ability to define users and roles and assign privileges to them.
- The ability to define foreign keys and other integrity constraints.
- The ability to define INSERT, UPDATE, and DELETE triggers.
- The ability to define and execute stored procedures.
- The ability to return data in different formats: ODBC mode for client access; Display mode for use within server-based applications.

Note: We continue to add support for additional features within InterSystems SQL. If you require a feature that is not supported within this release, please feel free to check with the [InterSystems Worldwide Response Center \(WRC\)](#) to see if it will be included in a newer release.

2.2.1 SQL-92 Compliance

The SQL-92 standard is imprecise with regard to arithmetical operator precedence; assumptions on this matter differ amongst SQL implementations. InterSystems SQL supports configuring your system for either of the following system-wide alternatives for [SQL arithmetic operator precedence](#):

- InterSystems SQL can be configured to parse arithmetic expressions in strict left-to-right order, with no operator precedence. This is the same convention used in ObjectScript. Thus, $3+3*5=30$. You can use parentheses to enforce the desired precedence. Thus, $3+(3*5)=18$.

- InterSystems SQL can be configured to parse arithmetic expressions using ANSI precedence, which gives higher precedence to multiplication and division operators than addition, subtraction, and concatenation operators. Thus, $3+3*5=18$. You can use parentheses to override this precedence, where desired. Thus, $(3+3)*5=30$.

The default for SQL operator precedence depends on your version of InterSystems IRIS. Refer to [SQL arithmetic operator precedence](#) for details.

InterSystems SQL supports the complete entry-level SQL-92 standard with the following exceptions:

- There is no support for adding additional CHECK constraints to a table definition.
- The SERIALIZABLE isolation level is not supported.
- Delimited identifiers are not case-sensitive; the standard says that they should be case-sensitive.
- Within a subquery contained in a HAVING clause, one is supposed to be able to refer to aggregates which are “available” in that HAVING clause. This is not supported.

2.2.2 Extensions

InterSystems SQL supports a number of useful extensions. Many of these are related to the fact that InterSystems IRIS provides simultaneous object and relational access to data.

Some of these extensions include:

- Support for user-definable data type and functions.
- Special syntax for following object references.
- Support for subclassing and inheritance.
- Support for queries against external tables stored within other databases.
- A number of mechanisms for controlling the storage structures used for tables to achieve maximum performance.

2.3 Interoperability

InterSystems SQL supports a number of ways to interoperate *relationally* with other applications and software tools.

2.3.1 JDBC

InterSystems IRIS includes a standards-compliant, level 4 (all pure Java code) JDBC client.

The InterSystems JDBC driver offers the following features:

- High-performance
- A pure Java implementation
- Unicode support
- Thread-safety

You can use InterSystems JDBC with any tool, application, or development environment that supports JDBC. If you encounter problems or have questions about compatibility, contact the InterSystems [InterSystems Worldwide Response Center \(WRC\)](#). You can contact the WRC from the Management Portal by using the **Contact** button found in the upper right corner.

2.3.2 ODBC

The C-language call level interface for InterSystems SQL is ODBC. Unlike other database products, the InterSystems ODBC driver is a *native* driver — it is not built on top of any other proprietary interface.

The InterSystems ODBC driver offers the following features:

- High-performance
- Portability
- Native Unicode support
- Thread-safety

You can use InterSystems ODBC with any tool, application, or development environment that supports ODBC. If you encounter problems or have questions about compatibility, contact the InterSystems [InterSystems Worldwide Response Center \(WRC\)](#). You can contact the WRC from the Management Portal by using the **Contact** button found in the upper right corner.

2.3.3 Embedded SQL

Within ObjectScript, InterSystems SQL supports Embedded SQL: the ability to place an SQL statement within the body of a method (or other code). Using Embedded SQL, you can query a single record, or define a cursor and use that to query multiple records. Embedded SQL is compiled. By default, it is [compiled the first time it is executed](#) (runtime), not when the routine that contains it is compiled. For this reason, it is important to check for SQLCODE errors at runtime. It is also possible to [compile Embedded SQL at the same time as the ObjectScript routine](#) that contains it.

Embedded SQL is quite powerful when used in conjunction with the object access capability of InterSystems IRIS. For example, the following method finds the RowID of the record with a given Name value:

Class Member

```
ClassMethod FindByName(fullname As %String)
{
    &sql(SELECT %ID INTO :id FROM Sample.MyTable WHERE Name = :fullname)
    IF SQLCODE<0 {SET baderr="SQLCODE ERROR: "_SQLCODE_" "_%msg
                  RETURN baderr }
    ELSEIF SQLCODE=100 {SET nodata="Query returns no data"
                      RETURN nodata }
    RETURN "RowID="_id
}
```

For more details, see [Using Embedded SQL](#).

2.3.4 Dynamic SQL

As part of its standard library, InterSystems IRIS provides an %SQL.Statement class that you can use to execute dynamic (that is, defined at runtime) SQL statements. You can use Dynamic SQL within ObjectScript methods. For example, the following method queries for a specified number of people born in the 21st century. The query selects all people born after December 31, 1999, orders the selected records by date of birth, then selects the top x records:

Class Member

```
ClassMethod Born21stC(x) [ language=objectscript ]
{
    SET myquery=2
    SET myquery(1) = "SELECT TOP ? Name,%EXTERNAL(DOB) FROM Sample.Person "
    SET myquery(2) = "WHERE DOB > 58073 ORDER BY DOB"
    SET tStatement = ##class(%SQL.Statement).%New()
    SET qStatus = tStatement.%Prepare(.myquery)
    IF qStatus'=1 {WRITE "%Prepare failed:" DO $System.Status.DisplayError(qStatus) QUIT}
    SET rset = tStatement.%Execute(x)
    DO rset.%Display()
    WRITE !,"End of data"
}
```

When you prepare a query, an optimized version of that query is stored as a cached query. This cached query is executed for subsequent invocations of the query, avoiding the overhead of re-optimizing a query each time it is executed.

For more details, see [Using Dynamic SQL](#).

2.4 Limitations

Note the following limitations of InterSystems SQL:

- NLS can be used to specify the behavior of **\$ORDER** for a particular national locale behavior for individual globals, as well as for local variables in the currently running process. InterSystems SQL can be used and works well within any National Language locale. However, a current limitation of InterSystems SQL is that for any particular process, all the relevant globals it references have to be using the same national locale as the current process locale. See [SQL Collation and NLS Collations](#).

3

InterSystems SQL Basics

This topic provides an overview of the features of InterSystems SQL, especially those that are not covered by the SQL standard or are related to the InterSystems IRIS® data platform unified data architecture. It assumes prior knowledge of SQL and is not designed to serve as an introduction to SQL concepts or syntax.

3.1 Tables

Within InterSystems SQL, data is presented within *tables*. Each table is defined to contain a number of *columns*. A table may contain zero or more rows of data values. The following terms are roughly equivalent:

Data Terms	Relational Database Terms	InterSystems IRIS Terms
database	schema	package
	table	persistent class
field	column	property
record	row	

For further details, see [Introduction to the Default SQL Projection](#).

There are two basic types of tables: *base tables* (which contain data and are usually referred to simply as tables) and *views* (which present a logical view based on one or more tables).

To find out more on how to define tables, see [Defining Tables](#).

To find out more on how to define views, see [Defining Views](#).

In order to make queries against tables more efficient, you can define indexes on tables. See [Defining and Building Indexes](#).

In order to enforce referential integrity you can define foreign keys and triggers on tables. See [Defining Foreign Keys](#) and [Defining Triggers](#).

3.1.1 Schemas

SQL schemas provides a means of grouping sets of related tables, views, stored procedures, and cached queries. The use of schemas helps prevent naming collisions at the table level, because a table, view, or stored procedure name must only be unique within its schema. An application can specify tables in multiple schemas.

SQL schemas correspond to persistent class packages. Commonly a schema has the same name as its corresponding package, but these names may differ because of different [schema naming conventions](#) or because different names have been deliberately specified. Schema-to-package mapping is further described in [SQL to Class Name Transformations](#).

Schemas are defined within a specific namespace. A schema name must be unique within its namespace. A schema (and its corresponding package) is automatically created when the first item is assigned to it and automatically deleted when the last item is deleted from it.

You can specify an SQL name as qualified or unqualified. A qualified name specifies the schema: `schema . name`. An unqualified name does not specify the schema: `name`. If you do not specify the schema, InterSystems IRIS supplies the schema as follows:

- For DDL operations, InterSystems IRIS uses the [system-wide default schema name](#). This default is configurable. It applies to all namespaces.
- For DML operations, InterSystems IRIS can use either a user-supplied [schema search path](#) or the [system-wide default schema name](#). Different techniques are used to supply a schema search path in [Dynamic SQL](#), [Embedded SQL](#), and the [SQL Shell](#).

To view all the existing schemas within a namespace:

1. From the Management Portal select **System Explorer**, then **SQL**. Select a namespace by clicking the name of the current namespace displayed at the top of the page; this displays the list of available namespaces. Select a namespace.
2. Select the **Schema** drop-down list on the left side of the screen. This displays a list of the schemas in the current namespace. Select a schema from this list; the selected name appears in the **Schema** box.
3. The **applies to** drop-down list allows you to select Tables, Views, Procedures, or Cached Queries, or All of these that belong to the schema. After setting this option, click the triangles to view a list of the items. If there are no items, clicking a triangle has no effect.

3.2 Queries

Within InterSystems SQL, you view and modify data within tables by means of *queries*. Roughly speaking, queries come in two flavors: those that retrieve data ([SELECT](#) statements), and those that modify data ([INSERT](#), [UPDATE](#), and [DELETE](#) statements).

You can use SQL queries in a number of ways:

- Using [Embedded SQL](#) within ObjectScript.
- Using [Dynamic SQL](#) within ObjectScript.
- Calling a [stored procedure](#) created using [CREATE PROCEDURE](#) or [CREATE QUERY](#).
- Using a [class query](#). For further details, refer to [Defining and Using Class Queries](#).
- Using the ODBC or JDBC interfaces from a variety of other environments.

SELECT queries are described in [Querying the Database](#).

Queries are part of [InterSystems IRIS objects](#) or [ObjectScript routines](#).

3.3 Privileges

InterSystems SQL provides a way to limit access to tables, views, and so on via *privileges*. You can define a set of *users* and *roles* and grant various privileges (read, write, and so on) to them. See [SQL Users, Roles, and Privileges](#).

3.4 SelectMode

InterSystems SQL uses a [SelectMode option](#) to specify how data is to be displayed or stored. The available options are Logical, Display, and ODBC. Data is stored internally in Logical mode, and can be displayed in any of these modes. Every data type class can define transformations between internal Logical format and Display format or ODBC format by using the **LogicalToDisplay()**, **LogicalToOdbc()**, **DisplayToLogical()**, and **OdbcToLogical()** methods. When SQL SelectMode is Display, the LogicalToDisplay transformation is applied, and returned values are formatted for display. The default SQL SelectMode is Logical; thus by default returned values are displayed in their storage format.

SelectMode affects the format that in which query result set data is displayed. It also affects the format in which data values should be supplied, for example in the WHERE clause. InterSystems IRIS applies the appropriate transformation method based on the storage mode and the specified SelectMode. A mismatch between a supplied data value and the SelectMode can result in an error or in erroneous results. For example, if DOB is a date stored in \$HOROLOG Logical format, and a WHERE clause specifies WHERE DOB > 2000-01-01 (ODBC format), SelectMode = ODBC returns the intended results. SelectMode = Display generates SQLCODE -146 Unable to convert date input to a valid logical date value. SelectMode = Logical attempts to parse 2000-01-01 as a Logical date value, and returns zero rows.

SelectMode is applied to complete expressions returned from SQL queries, not to individual fields specified in the query. This behavior primarily affects:

- **Queries that apply functions to returned fields:** When a function is applied to a field, the SelectMode does not format the field before the function is applied. Instead, the SelectMode formats the result of applying the function to the field. As a result, the function is always performed on the Logical representation of the field.
- **Queries that concatenate a field to a string:** When a field is concatenated to a string (or to another field), the SelectMode does not format the field (or the string) before performing the concatenation. Instead, the SelectMode formats the result of the concatenation. As a result, the concatenation is always performed on the Logical representation of the field.

For most data types, the three SelectMode modes return the same results. The following data types are affected by the SelectMode option:

- [Date, Time, and Timestamp data types](#). InterSystems SQL supports numerous Date, Time, and Timestamp data types (%Library.Date, %Library.Time, %Library.PosixTime, %Library.TimeStamp, and %MV.Date). With the exception of %Library.TimeStamp, these data types use different representations for Logical, ODBC, and Display modes.

In Logical mode, InterSystems IRIS displays the dates in the format they were stored in. For most of these data types, dates are stored (and therefore displayed) in the [\\$HOROLOG](#) format. This format consists of an integer count of the number of days from an arbitrary starting date (December 31st, 1840), a comma separator, and an integer count of the number of seconds since midnight of the current day. However, %Library.PosixTime timestamps are not stored in the \$HOROLOG format, and are instead stored and displayed as an encoded 64-bit signed integer.

In ODBC mode, dates and times are always represented as YYYY-MM-DD hh:mm:ss.fff. The %Library.TimeStamp data type also uses this ODBC format for Logical and Display modes.

In Display mode, dates and times commonly appear in the format specified by the data type's FORMAT parameter or the date and time format defaults for the current locale in %SYS.NLS.Format. The default for the American locale is MM/DD/YYYY hh:mm:ss.

- [%List data type](#). InterSystems IRIS Logical mode stores lists using two non-printing characters that appear before the first item in the list, and appear as a separator between list items. In ODBC SelectMode, list items are displayed with a comma separator between list items. In Display SelectMode, list items are displayed with a blank space separator between list items.
- Data types that specify [VALUELIST](#) and [DISPLAYLIST](#). For required fields, if you are in display mode and you insert a value into a table where the field has a DISPLAYLIST, the display value you enter must exactly match one of the items in the DISPLAYLIST. For non-required fields, non-matching values are converted to NULL values.
- Empty strings, and empty BLOBs (stream fields). In Logical mode empty strings and BLOBs are represented by the non-display character \$CHAR(0). In Display mode they are represented by an empty string ("").

The SQL SelectMode may be specified as follows:

- For the current process, using the **SetOption("SelectMode")** method.
- For a [InterSystems SQL Shell session](#), using the SET SELECTMODE command.
- For a query result set from the Management Portal [Execute Query](#) user interface (**System Explorer, SQL**), using the "Display Mode" drop-down list.
- For a [Dynamic SQL %SQL.Statement instance](#), using the [%SelectMode](#) property.
- For Embedded SQL, using the ObjectScript [#sqlcompile select](#) preprocessor directive setting. This directive allows for a fourth value, Runtime, which sets the select mode to whatever the *RuntimeMode* property setting is: Logical, Display, or ODBC. The RuntimeMode default is Logical.
- For the SQL commands [CREATE QUERY](#), [CREATE METHOD](#), [CREATE PROCEDURE](#), and [CREATE FUNCTION](#) using the SELECTMODE keyword.
- For an individual column within an SQL query by using the [%EXTERNAL](#), [%INTERNAL](#), and [%ODBCOUT](#) functions.

3.5 Data Collation

Collation specifies how values are ordered and compared, and is part of both InterSystems SQL and InterSystems IRIS objects.

You can specify a collation type as part of field/property definition. Unless otherwise specified, a string field/property defaults to the namespace default collation. By default, the namespace default collation for strings is SQLUPPER. SQLUPPER collation transforms strings into uppercase for the purposes of sorting and comparing. Thus, unless otherwise specified, string ordering and comparison is not case-sensitive.

You can specify a collation type as part of index definition, or use the collation type of the indexed field.

An SQL query can override the defined field/property collation type by applying a collation function to a field name. The [ORDER BY](#) clause specifies the result set sequence for a query; if a specified string field is defined as SQLUPPER, query results order is not case-sensitive.

For further details refer to [Collation](#).

3.6 Executing SQL

InterSystems IRIS supports numerous ways to write and execute SQL code. These include:

- [Embedded SQL](#): SQL code embedded within ObjectScript code.
- [Dynamic SQL](#): SQL code executed from within ObjectScript, using the %SQL.Statement class.
- [Execute\(\) method](#): execute SQL code using the **Execute()** method of the %SYSTEM.SQL class.
- [Stored Procedure](#) containing SQL code, created using [CREATE PROCEDURE](#) or [CREATE QUERY](#).
- The [SQL Shell](#): SQL statements executed from the Terminal interface.
- [Execute Query Interface](#): SQL statements executed from the Management Portal.

You can use InterSystems IRIS objects (classes and methods) to:

- [Define a persistent class \(an SQL table\)](#).
- [Define an index](#).
- [Define and Use a Class Query](#).

4

Language Elements

This page introduces the language elements in InterSystems SQL.

4.1 Commands and Keywords

An InterSystems SQL command (also known as an SQL statement) begins with a keyword followed by one or more arguments. Some of these arguments may be clauses or functions, identified by their own keywords.

- InterSystems SQL commands do not have a command terminator, except in specific cases such as SQL [procedure code](#) or [trigger code](#), in which case SQL commands are terminated by a single semicolon (;). Otherwise, InterSystems SQL commands do not require or accept a semicolon command terminator. Specifying a semicolon command terminator in InterSystems SQL results in an SQLCODE -25 error. InterSystems IRIS® data platform implementation of [TSQL \(Transact-SQL\)](#) accepts, but does not require, a semicolon command terminator. When importing SQL code to InterSystems SQL, semicolon command terminators are stripped out.
- InterSystems SQL commands have no whitespace restrictions. If command items are separated by a space, at least one space is required. If command items are separated by a comma, no space is required. No space is required before or after arithmetic operators. You may insert line breaks or multiple spaces between space-separated items, between items in a comma-separated list of arguments, or before or after arithmetic operators.

InterSystems SQL keywords include command names, function names, predicate condition names, data type names, field constraints, optimization options, and special variables. They also include the AND, OR, and NOT logical operators, the NULL column value indicator, and ODBC function constructs such as {d dateval} and {fn CONCAT(str1,str2)}.

- Keywords are not case-sensitive. By convention, keywords are represented by capital letters in this documentation, but InterSystems SQL has no letter case restriction.
- Many, but not all, keywords are [SQL Reserved Words](#). InterSystems SQL only reserves those keywords that cannot be unambiguously parsed. SQL reserved words can be used as [delimited identifiers](#).

4.2 Functions: Intrinsic and Extrinsic

A function performs an operation and returns a value. Commonly in InterSystems SQL a function is specified in a **SELECT** statement as a *select-item* or in a WHERE clause, performing an operation either on a table field value or on a literal value.

- Intrinsic: InterSystems SQL supports a large number of intrinsic (system-supplied) functions. These include [numeric functions](#), [string functions](#), and [date and time functions](#). These functions are described in the [InterSystems SQL Reference](#). The [arithmetic and trigonometric functions](#) are also listed on that page.

Aggregate functions are SQL intrinsic functions that evaluate all of the values of a column and return a single aggregate value. [Aggregate functions](#) are described separately.

- Extrinsic: InterSystems SQL can also support [user-supplied ObjectScript function calls \(extrinsic functions\)](#), as shown in the following example:

ObjectScript

```
MySQL
&sql(SELECT Name,$$MyFunc() INTO :n,:f FROM Sample.Person)
  IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg QUIT}
  ELSEIF SQLCODE=100 {WRITE "Query returns no results" QUIT}
  WRITE "name is: ",n,!
  WRITE "function value is: ",f,!
  QUIT
MyFunc(){
  SET x="my text"
  QUIT x
}
```

An SQL statement can only invoke user-supplied (extrinsic) functions if their use is configured as a system-wide option. The default is “No”; by default, attempting to invoke user-supplied functions issues an SQLCODE -372 error. You can configure SQL use of extrinsic functions system-wide using the `$$SYSTEM.SQL.Util.SetOption()` method, as follows: `SET status=$$SYSTEM.SQL.Util.SetOption("AllowExtrinsicFunctions",1,.oldval)`. To determine the current setting, call `$$SYSTEM.SQL.CurrentSettings()` which displays the `Allow extrinsic functions in SQL statements` option.

You cannot use a user-supplied function to call a % routine (a routine with a name that begins with the % character). Attempting to do so issues an SQLCODE -373 error.

4.3 Literals

InterSystems SQL literals have the following syntax:

```
literal ::=          number | string-literal
number ::=          {digit}[.]digit{digit}[E[+|-]digit{digit}]
digit ::=          0..9
string-literal ::=  std-string-literal | ObjectScript-empty-string
std-string-literal ::= {std-character-representation}
std-character-representation ::=
  nonquote-character | quote-symbol quote-symbol ::= ' ' ObjectScript-empty-string ::= ''
```

A literal is a series of characters that represents an actual (literal) value. It can be either a number or a character string.

- A number does not require any delimiter character. It can consist of the digits 0 through 9, the decimal point character, the exponent symbol and the plus and minus signs. Only one decimal point character can be used in a number. This decimal point can only be used in the base portion of a number, not in the exponent portion. The decimal point does not need to be followed by a digit. Leading and trailing zeros are permitted. The exponent (scientific notation) symbol is the letter E; both uppercase and lowercase E are accepted, but uppercase E is the preferred usage. A plus or minus sign can prefix a base number or an exponent. Multiple plus and minus signs can prefix a base number; SQL treats these signs as operators. Only a single plus or minus sign can prefix an exponent; SQL treats this sign as part of the literal. No commas or blanks are permitted in a number.
- A character string literal consists of a pair of delimiter characters enclosing a string of characters of any type. The preferred delimiter character is the single-quote character (see below). To specify a delimiter character as a literal within a character string, double the character; for example: 'Mary's office'.

The empty string is a literal string; it is represented by two single-quote characters ('). NULL is *not* a literal value; it represents the absence of any value. For further details, see [NULL and the Empty String](#).

Note: In Embedded SQL, a few character sequences that begin with ## are not permitted within a string literal, as described in [Literal Values](#). This restriction does not apply to other invocations of SQL, such as Dynamic SQL.

4.3.1 String Delimiters

Use single quote (') characters as string delimiters. The use of the double-quote character (") is supported for SQL compatibility, but this use is strongly discouraged because of conflict with the [delimited identifier](#) standard. A pair of double quote characters "" is parsed as an invalid delimited identifier and generates an SQLCODE -1 error.

To specify a single quote character as a literal character within a string, specify a pair of these characters as the literal escape sequence. For example, 'a ''normal'' string'.

4.3.2 Concatenation

The double vertical bar (||) is the preferred SQL concatenation operator. It can be used to concatenate two numbers, two character strings, or a number and a character string.

The underscore (_) is provided as an SQL concatenation operator for ObjectScript compatibility. This concatenation operator can only be used to concatenate two character strings.

If the two operands are both character strings, and both strings have the same [collation type](#), the resulting concatenated string has that collation type. In all other cases, the result of concatenation is of collation type EXACT.

4.4 NULL and the Empty String

Use the NULL keyword to indicate that a value is not specified. NULL is always the preferred way in SQL to indicate that a data value is unspecified or nonexistent for any reason.

The SQL zero-length string (empty string) is specified by two single quote characters. The empty string (') is *not* the same thing as NULL.

Note: The SQL zero-length string is *not recommended* for use as a field input value or a field default value; in ObjectScript, this corresponds to a string of length one that contains the \$CHAR(0) character. Use NULL to represent the absence of a data value, which corresponds to the ObjectScript empty string ("). See “[ObjectScript and SQL](#)” for more information.

The SQL zero-length string should be avoided in SQL coding. However, because many SQL operations delete trailing blank spaces, a data value that contains only whitespace characters (spaces and tabs) may result in an SQL zero-length string.

Note that different SQL length functions return different values: [LENGTH](#), [CHAR_LENGTH](#), and [DATALENGTH](#) return SQL lengths. [\\$LENGTH](#) returns ObjectScript representation length. See [The Length of NULL](#) below. **LENGTH** does not count trailing blank spaces; all other length functions count trailing blank spaces.

4.4.1 NULL Processing

The NOT NULL data constraint requires that a field must receive a data value; specifying NULL rather than a value is not permitted. This constraint does not prevent the use of an empty string value. For further details, refer to the [CREATE TABLE](#) command.

The **IS NULL** predicate in the **WHERE** or **HAVING** clause of a **SELECT** statement selects NULL values; it does not select empty string values.

The **IFNULL** function evaluates a field value and returns the value specified in its second argument if the field evaluates to NULL. It does not treat an empty string value as a non-NULL value.

The **COALESCE** function selects the first non-NULL value from supplied data. It treats empty string values as non-NULL.

When the **CONCAT** function or the concatenate operator (||) concatenate a string and a NULL, the result is NULL. This is shown in the following example:

SQL

```
SELECT {fn CONCAT('fred',NULL)} AS FuncCat,    -- returns <null>
       'fred' || NULL AS OpCat                -- returns <null>
```

The **AVG**, **COUNT**, **MAX**, **MIN**, and **SUM** aggregate functions ignore NULL values when performing their operations. (COUNT * counts all rows, because there cannot be a record with NULL values for all fields.) The **DISTINCT** keyword of the **SELECT** statement includes NULL in its operation; if there are NULL values for the specified field, **DISTINCT** returns one NULL row.

The **AVG**, **COUNT**, and **MIN**, aggregate functions are affected by empty string values. The **MIN** function considers an empty string to be the minimum value, even when there are rows that have a value of zero. The **MAX** and **SUM** aggregate functions are not affected by empty string values.

4.4.2 NULL in Expressions

Supplying NULL as an operand to most SQL functions returns NULL.

Any SQL arithmetic operation that has NULL as an operand returns a value of NULL. Thus, 7+NULL=NULL. This includes the binary operations addition (+), subtraction (-), multiplication (*), division (/), integer division (\), and modulo (#), and the unary sign operators plus (+) and minus (-).

An empty string specified in an arithmetic operation is treated as a value of 0 (zero). Division (/), integer division (\), or modulo (#) by empty string (6/"') results in a <DIVIDE> error.

4.4.3 The Length of NULL

Within SQL, the length of a NULL is undefined (it returns <null>). The length of an empty string, however, is defined as length zero. This is shown in the following example:

SQL

```
SELECT LENGTH(NULL) AS NullLen,    -- returns <null>
       LENGTH('') AS EmpStrLen    -- returns 0
```

As shown in this example, the SQL **LENGTH** function returns the SQL lengths.

You can convert an SQL zero-length string to a NULL by using the **ASCII** function, as shown in the following example:

SQL

```
SELECT LENGTH(NULL) AS NullLen,    -- returns <null>
       LENGTH({fn ASCII('')}) AS AsciiEmpStrLen, -- returns <null>
       LENGTH('') AS EmpStrLen    -- returns 0
```

However, certain InterSystems IRIS extensions to standard SQL treat the length of NULL and the empty string differently. The **\$LENGTH** function returns the InterSystems IRIS internal representation of these values: NULL is represented as a defined value with length 0, the SQL empty string is represented as a string of length 0. This functionality is compatible with ObjectScript.

SQL

```
SELECT $LENGTH(NULL) AS NullLen,      -- returns 0
$LENGTH('') AS EmpStrLen,           -- returns 0
$LENGTH('a') AS OneCharStrLen,       -- returns 1
$LENGTH(CHAR(0)) AS CharZero         -- returns 0
```

Another place where the internal representation of these values is significant is in the **%STRING**, **%SQLSTRING** and **%SQLUPPER** functions, which append a blank space to a value. Since a NULL truly has no value, appending a blank to it creates a string of length 1. But an empty string does have a character value, so appending a blank to it creates a string of length 2. This is shown in the following example:

SQL

```
SELECT CHAR_LENGTH(%STRING(NULL)) AS NullLen,  -- returns 1
CHAR_LENGTH(%STRING('')) AS EmpStrLen         -- returns 2
```

Note that this example uses **CHAR_LENGTH**, not **LENGTH**. Because the **LENGTH** function removes trailing blanks, **LENGTH(%STRING(NULL))** returns a length of 0; **LENGTH(%STRING(' '))** returns a length of 2, because **%STRING** appends a leading blank, not a trailing blank.

4.4.4 ObjectScript and SQL

When an SQL NULL is output to ObjectScript, it is represented by an ObjectScript empty string (""), a string of length zero.

When an SQL zero-length string data is output to ObjectScript, it is represented by a string containing \$CHAR(0), a string of length 1.

ObjectScript

```
&sql(SELECT NULL, ''
      INTO :a,:b)

WRITE !,"NULL length: ", $LENGTH(a)      // returns 0
WRITE !,"empty string length: ", $LENGTH(b) // returns 1
```

In ObjectScript, the absence of a value is usually indicated by an empty string (""). When this value is passed into embedded SQL, it is treated as a NULL value, as shown in the following example:

ObjectScript

```
set x=""
set myquery="SELECT NULL As NoVal,:x As EmpStr"
set tStatement=##class(%SQL.Statement).%New()

set qStatus = tStatement.%Prepare(myquery)
if $$$ISERR(qStatus) {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}

set rset = tStatement.%Execute()
if (rset.%SQLCODE != 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

while rset.%Next()
{
    write "NoVal:", rset.%Get("NoVal"), " length ", $LENGTH(rset.%Get("NoVal")), ! // length 0
    write "EmpStr:", rset.%Get("EmpStr"), " length ", $LENGTH(rset.%Get("EmpStr")), ! // length 0
}
if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

write "End of data"
```

If you specify an input host variable that is not defined, embedded SQL treats its value as NULL.

In the following example, the SQL empty string with an appended blank is passed out as string of length 2:

ObjectScript

```
&sql(SELECT %SQLUPPER('')
      INTO :y )
WRITE !,"SQL empty string length: ", $LENGTH(y)
```

4.5 Arithmetic Operators and Functions

InterSystems SQL supports the following arithmetic operators:

Operator	Description
+	Addition operator. For example, 17+7 equals 24.
–	Subtraction operator. For example, 17 - 7 equals 10. Note that a pair of these characters is the InterSystems SQL comment indicator. Therefore, to specify two or more subtraction operators or negative signs you must use either spaces or parentheses. For example, 17 - -7 or 17 - (-7) equals 24.
*	Multiplication operator. For example, 17*7 equals 119.
/	Division operator. For example, 17/7 equals 2.428571428571428571.
\	Integer division operator. For example, 17\7 equals 2.
#	Modulo operator. For example, 17 # 7 equals 3. Note that because the # character is also a valid identifier character, to use it as a modulo operator you should specify it separated from its operands by spaces before and after.
E	Exponentiation (scientific notation) operator. Can be uppercase or lowercase. For example, 7E3 equals 7000. A too-large exponent results in an SQLCODE -7 “Exponent out of range” error. For example, 1E309 or 7E308.
()	Grouping operators. Used to nest arithmetic operations. Operations are executed according to ANSI operator precedence . For example, 17+7*2 equals 31, but (17+7)*2 equals 48.
	Concatenate operator. For example, 17 7 equals 177.

Arithmetic operations are performed on numbers in their [canonical form](#).

4.5.1 Resulting Data Type

When performing an arithmetic operation on two numeric values having different [data types](#), the resulting data type is determined as follows:

For addition (+), subtraction (-), integer division (\), and modulo (#):

Data Type	NUMERIC	INTEGER	TINYINT	SMALLINT	BIGINT	DOUBLE
NUMERIC	NUMERIC	NUMERIC	NUMERIC	NUMERIC	NUMERIC	DOUBLE
INTEGER	NUMERIC	BIGINT	BIGINT	BIGINT	BIGINT	DOUBLE
TINYINT	NUMERIC	BIGINT	SMALLINT	INTEGER	BIGINT	DOUBLE
SMALLINT	NUMERIC	BIGINT	INTEGER	INTEGER	BIGINT	DOUBLE
BIGINT	NUMERIC	BIGINT	BIGINT	BIGINT	BIGINT	DOUBLE
DOUBLE	DOUBLE	DOUBLE	DOUBLE	DOUBLE	DOUBLE	DOUBLE

For multiplication (*) or division (/):

Data Type	NUMERIC	INTEGER	TINYINT	SMALLINT	BIGINT	DOUBLE
NUMERIC	NUMERIC	NUMERIC	NUMERIC	NUMERIC	NUMERIC	DOUBLE
INTEGER	NUMERIC	NUMERIC	NUMERIC	NUMERIC	NUMERIC	DOUBLE
TINYINT	NUMERIC	NUMERIC	NUMERIC	NUMERIC	NUMERIC	DOUBLE
SMALLINT	NUMERIC	NUMERIC	NUMERIC	NUMERIC	NUMERIC	DOUBLE
BIGINT	NUMERIC	NUMERIC	NUMERIC	NUMERIC	NUMERIC	DOUBLE
DOUBLE	DOUBLE	DOUBLE	DOUBLE	DOUBLE	DOUBLE	DOUBLE

Concatenating two numbers of any data type results in a VARCHAR string.

In Dynamic SQL you can use [SQL column metadata](#) to determine the data type of a result set field. For further details on numeric data types refer to SQL [Data Types](#).

4.5.2 Operator Precedence

The SQL-92 standard is imprecise with regard to operator precedence; assumptions on this matter differ amongst SQL implementations. InterSystems SQL can be configured to support either type of precedence:

- At InterSystems IRIS 2019.1 and subsequent, InterSystems SQL supports ANSI precedence of arithmetic operators by default. This is a system-wide configuration setting. When ANSI precedence is configured, the "*", "\", "/", and "#" operators have a higher precedence than the "+", "-", and "||" operators. Operators with a higher precedence are executed before operators with a lower precedence. Thus, $3+3*5$ equals 18. You can use parentheses to override precedence when desired. Thus, $(3+3)*5$ equals 30.

Default ANSI precedence is supported for a clean install of InterSystems IRIS 2019.1; when you upgrade InterSystems IRIS 2018.1 to InterSystems IRIS 2019.1, the operator precedence remains configured to the InterSystems IRIS 2018.1 default: strict left-to-right order.

- At InterSystems IRIS 2018.1, InterSystems SQL does not provide precedence of arithmetic operators by default. By default, InterSystems SQL executes arithmetic expressions in strict left-to-right order, with no operator precedence. This is the [same convention used in ObjectScript](#). Thus, $3+3*5$ equals 30. You can use parentheses to enforce the desired precedence. Thus, $3+(3*5)$ equals 18. Careful developers should use parentheses to explicitly state their intentions.

You can configure either type of SQL operator precedence system-wide using the `$$SYSTEM.SQL.Util.SetOption()` method, as follows: `SET status=$$SYSTEM.SQL.Util.SetOption("ANSIPrecedence",1,.oldval)` sets ANSI precedence; `SET status=$$SYSTEM.SQL.Util.SetOption("ANSIPrecedence",0,.oldval)` sets strict left-to-

right evaluation. To determine the current setting, call **\$\$SYSTEM.SQL.CurrentSettings()** which displays the Apply ANSI operator precedence option. Changing this SQL option takes effect immediately system-wide. Changing this option causes all cached queries to be purged system-wide.

Changing SQL precedence has no effect on ObjectScript. ObjectScript always follows strict left-to-right execution of arithmetic operators.

4.5.3 Precision and Scale

The *precision* (maximum number of digits present in the number) for a NUMERIC result for:

- addition or subtraction is determined using the following algorithm: $\text{resultprecision} = \max(\text{scale1}, \text{scale2}) + \max(\text{precision1} - \text{scale1}, \text{precision2} - \text{scale2}) + 1$. If the calculated resultprecision is greater than 36, the precision value is set to 36.
- multiplication is determined using the following algorithm: $\text{resultprecision} = \min(36, \text{precision1} + \text{precision2} + 1)$.
- division ($\text{value1} / \text{value2}$) is determined using the following algorithm: $\text{resultprecision} = \min(36, \text{precision1} - \text{scale1} + \text{scale2} + \max(6, \text{scale1} + \text{precision2} + 1))$.

The *scale* (maximum number of fractional digits) for a NUMERIC result for:

- addition or subtraction is determined using the following algorithm: $\text{resultscale} = \max(\text{scale1}, \text{scale2})$.
- multiplication is determined using the following algorithm: $\text{resultscale} = \min(17, \text{scale1} + \text{scale2})$.
- division ($\text{value1} / \text{value2}$) is determined using the following algorithm: $\text{resultscale} = \min(17, \max(6, \text{scale1} + \text{precision2} + 1))$.

For further details on data types, precision, and scale, refer to SQL [Data Types](#).

4.5.4 Arithmetic and Trigonometric Functions

InterSystems SQL supports the following arithmetic functions:

Function	Description
ABS	Returns the absolute value of a numeric expression.
CEILING	Returns the smallest integer greater than or equal to a numeric expression.
EXP	Returns the log exponential (base e) value of a numeric expression.
FLOOR	Returns the largest integer less than or equal to a numeric expression.
GREATEST	Returns the largest number from a comma-separated list of numbers.
ISNUMERIC	Returns a boolean code specifying whether an expression is a valid number.
LEAST	Returns the smallest number from a comma-separated list of numbers.
LOG	Returns the natural log (base e) value of a numeric expression.
LOG10	Returns the base-10 log value of a numeric expression.
MOD	Returns the modulus value (remainder) of a division operation. Same as the # operator.
PI	Returns the numeric constant pi.
POWER	Returns the value of a numeric expression raised to a specified power.
ROUND	Returns a numeric expression rounded (or truncated) to a specified number of digits.

Function	Description
SIGN	Returns a numeric code specifying whether a numeric expression evaluates to positive, zero, or negative.
SQRT	Returns the square root of a numeric expression.
SQUARE	Returns the square of a numeric expression.
TRUNCATE	Returns a numeric expression truncated to a specified number of digits.

InterSystems SQL supports the following trigonometric functions:

Function	Description
ACOS	Returns the arc-cosine of a numeric expression.
ASIN	Returns the arc-sine of a numeric expression.
ATAN	Returns the arc-tangent of a numeric expression.
COS	Returns the cosine of a numeric expression.
COT	Returns the cotangent of a numeric expression.
SIN	Returns the sine of a numeric expression.
TAN	Returns the tangent of a numeric expression.

InterSystems SQL also supports the following angle conversion functions:

Function	Description
DEGREES	Converts radians to degrees.
RADIANS	Converts degrees to radians.

4.6 Relational Operators

A conditional expression evaluates to a boolean value. A conditional expression can use the following relational operators:

Operator	Description
=	Equals operator.
!=	Does not equal operator. The two syntactical forms are functionally identical.
<>	
<	Less than operator.
>	Greater than operator.
<=	Less than or equal to operator.
>=	Greater than or equal to operator.

When comparing a table field value, these equality operators use the field's default collation. The InterSystems IRIS default is not case-sensitive. When comparing two literals, the comparison is case-sensitive.

Equality operators (equals, does not equal) should be avoided when comparing floating point numbers. Floating point numbers ([data types](#) classes %Library.Decimal and %Library.Double) are stored as binary values, not as fixed-precision numbers. During conversion, rounding operations may result in two floating point numbers that are intended to represent the same number not being precisely equal. Use less-than / greater-than tests to determine if two floating point numbers are “the same” to the desired degree of precision.

4.6.1 Contains and Follows Operators

InterSystems SQL also supports the Contains and Follows comparison operators:

Operator	Description
[	Contains operator. Returns all values that contain the operand, including values equal to the operand. This operator uses EXACT (case-sensitive) collation. The inverse is NOT[.

- The Contains operator determines if a value contains a specified character or string of characters. It is case-sensitive.
- The [%STARTSWITH](#) predicate condition determines if a value starts with a specified character or string of characters. It is not case-sensitive.
- [InterSystems SQL Search](#) can be used to determine if a value contains a specified word or phrase. SQL Search performs context-aware matching. It is not case-sensitive.

Operator	Description
]	Follows operator. Returns all values that follow the operand in collation sequence. Excludes the operand value itself. This operator uses the field's default collation. The InterSystems IRIS default is not case-sensitive. The inverse is NOT].

For example, `SELECT Age FROM MyTable WHERE Age ] 88` returns 89 and greater, but also returns 9 because 9 is after 88 in the collation sequence. `SELECT Age FROM MyTable WHERE Age > 88` returns 89 and greater; it does not return 9. A string operand such as 'ABC' collates before any string that contains additional characters, such as 'ABCA'; therefore, to exclude the operand string from a] operator or a > operator you must specify the entire string. `Name ] 'Smith, John'` excludes 'Smith,John' but not 'Smith,John P.'

4.7 Logical Operators

SQL logical operators are used in condition expressions that are evaluated as being True or False. These conditional expressions are used in the **SELECT** statement WHERE and HAVING clauses, in the **CASE** statement WHEN clauses, in the **JOIN** statement ON clause, and the **CREATE TRIGGER** statement WHEN clause.

4.7.1 NOT Unary Operator

You can use the NOT unary logical operator to specify the logical inverse of a condition, as shown in the following examples:

SQL

```
SELECT Name, Age FROM Sample.Person
WHERE NOT Age > 21
ORDER BY Age
```

SQL

```
SELECT Name, Age FROM Sample.Person
WHERE NOT Name %STARTSWITH('A')
ORDER BY Name
```

You can place the NOT operator before the condition (as shown above). Or you can place NOT immediately before a single-character operator; for example, NOT<, NOT[, and so forth. Note that there must be no space between NOT and the single-character operator it inverts.

4.7.2 AND and OR Operators

You can use the AND and OR logical operators between two operands in a series of two or more conditions. These logical operators can be specified by keyword or symbol:

Operator	Description
AND	&
OR	!

Spaces are not required (though recommended for readability) between a symbol operator and its operand. Spaces are required before and after a keyword operator.

These logical operators can be used with the NOT unary logical operator, such as the following: WHERE Age<65 & NOT Age=21.

The following two examples use logical operators to schedule an assessment based on age. People between the ages of 20 and 40 are assessed every three years, people from 40 to 64 are assessed every two years, and those 65 and over are assessed every year. The examples give identical results; the first example uses keywords, the second uses symbols:

SQL

```
SELECT Name, Age FROM Sample.Person
WHERE Age>20
      AND Age<40 AND (Age # 3)=0
      OR Age>=40 AND (Age # 2)=0
      OR Age>=65
ORDER BY Age
```

SQL

```
SELECT Name, Age FROM Sample.Person
WHERE Age>20
      & Age<40 & (Age # 3)=0
      ! Age>=40 & (Age # 2)=0
      ! Age>=65
ORDER BY Age
```

Logical operators can be grouped using parentheses. This establishes a grouping level; evaluation proceeds from the lowest grouping level to the highest. In the first of the following examples, the AND condition is applied only to the second OR condition. It returns persons of any age from MA, and persons with age less than 25 from NY:

SQL

```
SELECT Name, Age, Home_State FROM Sample.Person
WHERE Home_State='MA' OR Home_State='NY' AND Age < 25
ORDER BY Age
```

Using parentheses to group conditions gives a different result. The following example returns persons from MA or NY whose age is less than 25:

SQL

```
SELECT Name, Age, Home_State FROM Sample.Person
WHERE (Home_State='MA' OR Home_State='NY') AND Age < 25
ORDER BY Age
```

Note: To maintain consistent results, when you employ multiple AND or OR operators in a query, InterSystems recommends that you use parentheses to clearly encapsulate the contents of each clause. Otherwise, the system employs a grouping level that you may not expect.

- SQL execution uses short-circuit logic. If a condition fails, the remaining AND conditions will not be tested. If a condition succeeds, the remaining OR conditions will not be tested.
- However, because SQL optimizes WHERE clause execution, the order of execution of multiple conditions (at the same grouping level) cannot be predicted and should not be relied upon.

4.8 Comments

InterSystems SQL supports both single-line comments and multi-line comments. Comment text can contain any characters or strings, except, of course, the character(s) that indicate the end of the comment.

Note: Using Embedded SQL marker syntax (&sql<marker>(. . .)<reversemarker>) imposes a restriction on the contents of SQL comments. If you are using marker syntax, the comments within the SQL code may not contain the character sequence “)<reversemarker>”. For further details, refer to [The &sql Directive](#).

You can use the **preparse()** method to return an SQL DML statement stripped of comments. The **preparse()** method also replaces each query argument with a ? character and returns a %List structure of these arguments. The **preparse()** method in the following example returns a parsed version of the query, stripped of single-line and multi-line comments and whitespace:

ObjectScript

```
SET myq=4
SET myq(1)="SELECT TOP ? Name /* first name */, Age "
SET myq(2)="  FROM Sample.MyTable -- this is the FROM clause"
SET myq(3)="  WHERE /* various conditions "
SET myq(4)="apply */ Name='Fred' AND Age > 21 -- end of query"
DO ##class(%SQL.Statement).preparse(.myq,.stripped,.args)
WRITE stripped,!
WRITE $LISTTOSTRING(args)
```

4.8.1 Single Line Comments

A single-line comment is specified by a two-hyphen prefix. A comment can be on a separate line, or can appear on the same line as SQL code. When a comment follows SQL code on the same line, at least one blank space must separate the code from the double-hyphen comment operator. A comment can contain any characters, including hyphens, asterisks, and slashes. The comment continues to the end of the line.

The following example contains multiple single-line comments:

SQL

```
-- This is a simple SQL query
-- containing -- (double hyphen) comments
SELECT TOP 10 Name, Age, -- Two columns selected
Home_State -- A third column
FROM Sample.Person -- Table name
-- Other clauses follow
WHERE Age > 20 AND -- Comment within a clause
Age < 40
ORDER BY Age, -- Comment within a clause
Home_State
-- End of query
```

4.8.2 Multiple Line Comments

A multiple-line comment is specified by a `/*` opening delimiter and a `*/` closing delimiter. A comment can appear on one or more separate lines, or can begin or end on the same line as SQL code. A comment delimiter should be separated from SQL code by at least one blank space. A comment can contain any characters, including hyphens, asterisks and slashes, with the obvious exception of the `*/` character pair.

Note: The syntax `/*#OPTIONS */`, with no space between the `/*` and the `#`, specifies a comment option. A [comment option](#) is not a comment; it specifies a code option that the query optimizer uses during the compile of the SQL query. A comment option is specified using JSON syntax, commonly a key:value pair such as the following:

```
/*#OPTIONS {"optionName":value} */.
```

The following example contains several multiple-line comments:

SQL

```
/* This is
   a simple
   SQL query. */
SELECT TOP 10 Name, Age /* Two fields selected */
FROM Sample.Person /* Other clauses
could appear here */ ORDER BY Age
/* End of query */
```

When commenting out Embedded SQL code, always begin the comment before the `&sql` directive or within the parentheses. The following example correctly comments out two the Embedded SQL code blocks:

ObjectScript

```
SET a="default name",b="default age"
WRITE "(not) Invoking Embedded SQL",!
/*&sql(SELECT Name INTO :a FROM Sample.Person) */
WRITE "The name is ",a,!
WRITE "Invoking Embedded SQL (as a no-op)",!
&sql(/* SELECT Age INTO :b FROM Sample.Person */)
WRITE "The age is ",b
```

4.8.3 SQL Code Retained as Comments

Embedded SQL statements can be retained as comments in the .INT code version of routines. This is done system-wide by setting the `$$SYSTEM.SQL.Util.SetOption()` method, as follows: `SET status=$$SYSTEM.SQL.Util.SetOption("RetainSQL",1,.oldval)`. To determine the current setting, call `$$SYSTEM.SQL.CurrentSettings()`, which displays the `Retain SQL as Comments` setting. The default is 1 (“Yes”).

Set this option to “Yes” to retain SQL statements as comments in the .INT code version of a routine. Setting this option to “Yes” also lists all non-% variables used by the SQL statements in the comment text. These listed variables should also be listed in the ObjectScript procedure’s PUBLIC variable list and re-initialized using the [NEW](#) command. For further details, refer to [Host Variables](#).

5

Implicit Joins (Arrow Syntax)

InterSystems SQL provides a special $\rightarrow$ operator as a shorthand for getting values from a related table without the complexity of specifying explicit JOINS in certain common cases. This arrow syntax can be used instead of explicit join syntax, or in combination with explicit join syntax. Arrow syntax performs a [left outer join](#).

Arrow syntax can be used for a reference of a property of a class, or a relationship property of a parent table. Other types of relationships and foreign keys do not support arrow syntax. You cannot use arrow syntax ($\rightarrow$) in an ON clause.

You can use arrow syntax in a query involving sharded tables.

For further information, see [JOIN](#).

5.1 Property Reference

You can use the $\rightarrow$ operator as a shorthand for getting values from a “referenced table.”

For example, suppose you define two classes: Company:

Class Definition

```
Class Sample.Company Extends %Persistent [DdlAllowed]
{
  /// The Company name
  Property Name As %String;
}
```

and Employee:

Class Definition

```
Class Sample.Employee Extends %Persistent [DdlAllowed]
{
  /// The Employee name
  Property Name As %String;

  /// The Company this Employee works for
  Property Company As Company;
}
```

The Employee class contains a property that is a *reference* to a Company object. Within an object-based application, you can follow this reference using dot syntax. For example, to find the name of a company that an employee works for:

ObjectScript

```
Set name = employee.Company.Name
```

You can perform the same task using an SQL statement that uses an OUTER JOIN to join the Employee and Company tables:

SQL

```
SELECT Sample.Employee.Name, Sample.Company.Name AS CompName
FROM Sample.Employee LEFT OUTER JOIN Sample.Company
ON Sample.Employee.Company = Sample.Company.ID
```

Using the → operator, you can perform the same OUTER JOIN operation more succinctly:

SQL

```
SELECT Name, Company->Name AS CompName
FROM Sample.Employee
```

You can use the → operator any time you have a *reference column* within a table; that is, a column whose value is the ID of a referenced table (essentially a special case of foreign key). In this case, the Company field of Sample.Employee contains IDs of records in the Sample.Company table. You can use the → operator anywhere you can use a column expression within a query. For example, in a WHERE clause:

SQL

```
SELECT Name, Company AS CompID, Company->Name AS CompName
FROM Sample.Employee
WHERE Company->Name %STARTSWITH 'G'
```

This is equivalent to:

SQL

```
SELECT E.Name, E.Company AS CompID, C.Name AS CompName
FROM Sample.Employee AS E, Sample.Company AS C
WHERE E.Company = C.ID AND C.Name %STARTSWITH 'G'
```

Note that in this case, this equivalent query uses an INNER JOIN.

The following example uses arrow syntax to access the Spouse field in Sample.Person. As the example shows, the Spouse field in Sample.Employee contains the ID of a record in Sample.Person. This example returns those records where the employee has the same Home_State or Office_State as the Home_State of their spouse:

SQL

```
SELECT Name, Spouse, Home_State, Office_State, Spouse->Home_State AS SpouseState
FROM Sample.Employee
WHERE Home_State=Spouse->Home_State OR Office_State=Spouse->Home_State
```

You can use the → operator in a GROUP BY clause:

SQL

```
SELECT Name, Company->Name AS CompName
FROM Sample.Employee
GROUP BY Company->Name
```

You can use the → operator in an ORDER BY clause:

SQL

```
SELECT Name, Company->Name AS CompName
FROM Sample.Employee
ORDER BY Company->Name
```

or refer to a column alias for a → operator column in an ORDER BY clause:

SQL

```
SELECT Name,Company->Name AS CompName
FROM Sample.Employee
ORDER BY CompName
```

Compound arrow syntax is supported, as shown in the following example. In this example, the Cinema.Review table includes the Film field, which contains Row IDs for the Cinema.Film table. The Cinema.Film table includes the Category field, which contains Row IDs for the Cinema.Category table. Thus Film->Category->CategoryName accesses these three tables to return the CategoryName of each film that has a ReviewScore:

SQL

```
SELECT ReviewScore,Film,Film->Title,Film->Category,Film->Category->CategoryName
FROM Cinema.Review
ORDER BY ReviewScore
```

5.2 Child Table Reference

You can use → operator to reference a child table. For example, if LineItems is a child table of the Orders table, you can specify:

SQL

```
SELECT LineItems->amount
FROM Orders
```

Note that there is no property called LineItems in Orders; LineItems is the name of a child table that contains the amount field. This query produces multiple rows in the result set for each Order row. It is equivalent to:

SQL

```
SELECT L.amount
FROM Orders O LEFT JOIN LineItems L ON O.id=L.custorder
```

Where custorder is the parent reference field of the LineItems table.

5.3 Arrow Syntax Privileges

When using arrow syntax, you must have SELECT privileges on the referenced data in both tables. Either you must have a table-level SELECT privilege or a column-level SELECT privilege on the referenced column. With column-level privileges, you need SELECT privilege on the ID of the referenced table, as well as the referenced column.

The following example demonstrates the required column-level privileges:

SQL

```
SELECT Name,Company->Name AS CompanyName
FROM Sample.Employee
GROUP BY Company->Name
ORDER BY Company->Name
```

In the above example, you must have column-level SELECT privilege for Sample.Employee.Name, Sample.Company.Name, and Sample.Company.ID:

ObjectScript

```
SET tStatement = ##class(%SQL.Statement).%New()
SET privchk1="%CHECKPRIV SELECT (Name,ID) ON Sample.Company"
SET privchk2="%CHECKPRIV SELECT (Name) ON Sample.Employee"
CompanyPrivTest
SET qStatus = tStatement.%Prepare(privchk1)
IF qStatus'=1 {WRITE "%Prepare failed:" DO $System.Status.DisplayError(qStatus) QUIT}
SET rset = tStatement.%Execute()
IF rset.%SQLCODE=0 {WRITE !,"have Company privileges",! }
ELSE { WRITE !,"No privilege: SQLCODE=",rset.%SQLCODE,! }
EmployeePrivTest
SET qStatus = tStatement.%Prepare(privchk2)
IF qStatus'=1 {WRITE "%Prepare failed:" DO $System.Status.DisplayError(qStatus) QUIT}
SET rset = tStatement.%Execute()
IF rset.%SQLCODE=0 {WRITE !,"have Employee privilege",! }
ELSE { WRITE !,"No privilege: SQLCODE=",rset.%SQLCODE }
```

6

Identifiers

An identifier is the name of an SQL entity, such as a table, a view, a column (field), a schema, a table alias, a column alias, an index, a stored procedure, a trigger, or some other SQL entity. An identifier name must be unique within its context; for example, two tables in the same schema, or two fields within the same table cannot have the same name. However, two tables in different schemas, or two fields in different tables can have the same name. In most cases, the same identifier name can be used for SQL entities of different types; for example, a schema, a table in that schema, and a field in that table can all have the same name without conflict. However, a table and a view in the same schema cannot have the same name.

InterSystems IRIS® data platform SQL identifiers follow a set of naming conventions, which may be further restricted according to the use of the identifier. Identifiers are not case-sensitive.

An identifier may be either a [simple identifier](#) or a [delimited identifier](#). The InterSystems SQL default is to support both simple identifiers and delimited identifiers.

6.1 Simple Identifiers

A simple identifier has the following syntax:

```
simple-identifier ::= identifier-start { identifier-part }  
    identifier-start ::= letter | % | _  
    identifier-part ::= letter | number | _ | @ | # | $
```

6.1.1 Naming Conventions

The *identifier-start* is the first character of an SQL identifier. It must be one of the following:

- An uppercase or lowercase letter. A letter is defined as any character that passes validation by the ObjectScript [\\$ZNAME](#) function; by default these are the uppercase letters A through Z (ASCII 65–90), the lowercase letters a through z (ASCII 97–122), and the letters with accent marks (ASCII 192–255, exclusive of ASCII 215 and 247). InterSystems IRIS can use any valid Unicode (16-bit) letter character within an SQL identifier in any locale. Simple identifiers are not case-sensitive (however, see below). By convention they are represented with initial capital letters.

The Japanese locale does not support accented Latin letter characters in InterSystems IRIS names. Japanese names may contain (in addition to Japanese characters) the Latin letter characters A-Z and a-z (65–90 and 97–122) and any Unicode character.

- An underscore (_).

- A percent sign (%). InterSystems IRIS names beginning with a % character (except those beginning with %Z or %z) are reserved as system elements and should not be used as identifiers. For further details, refer to [Rules and Guidelines for Identifiers](#).

The *identifier-part* is any of the subsequent characters of an SQL identifier. These remaining characters may consist of zero or more:

- Letters (including Unicode characters).
- Numbers. A number is defined as the digits 0 through 9.
- Underscores (_).
- At signs (@).
- Pound signs (#).
- Dollar signs (\$).

Some symbol characters are also used as operators. In SQL, the # sign is used as the modulo operator. In SQL, the underscore character can be used to concatenate two strings; this usage is provided for compatibility with ObjectScript, the preferred SQL concatenation operator is ||. The interpretation of a symbol as an identifier character always take precedence over its interpretation as an operator. Any ambiguity concerning the correct parsing of a symbol character as an operator can be resolved by adding spaces before and after the operator.

A simple identifier cannot contain blank spaces or non-alphanumeric characters (other than those symbol characters specified above). The InterSystems SQL import tool removes blank spaces from imported table names.

Note: SQL cursor names do not follow identifier naming conventions. For details on cursor naming conventions, refer to the [DECLARE](#) statement.

InterSystems SQL includes reserved words that cannot be used as simple identifiers. For a list of these reserved words, see [Reserved Words](#); to test if a word is a reserved word use the `$SYSTEM.SQL.IsReservedWord()` method. However, a delimited identifier can be the same as an SQL reserved word.

Any identifier that does not follow these naming conventions must be represented as a [delimited identifier](#) within an SQL statement.

6.1.2 Case of Letters

InterSystems SQL identifiers by default are not case-sensitive. InterSystems SQL implements this by comparing identifiers after converting them to all uppercase letters. This has no effect on the actual case of the names being used. (Note that other implementations of SQL may handle case sensitivity of identifiers differently. For this reason, it is recommended that you avoid case-based identifiers.)

Note that cursor names and passwords in InterSystems SQL are case-sensitive.

6.1.3 Testing Valid Identifiers

InterSystems IRIS provides the `IsValidRegularIdentifier()` method of the `%SYSTEM.SQL` class, which tests whether a string is a valid identifier. It tests both for character usage and for reserved words. It also performs a maximum length test of 200 characters (this is an arbitrary length used to avoid erroneous input; it is not an identifier validation). The following ObjectScript example shows the use of this method:

ObjectScript

```
WRITE !,$SYSTEM.SQL.IsValidRegularIdentifier("Fred")
WRITE !,$SYSTEM.SQL.IsValidRegularIdentifier("%Fred#123")
WRITE !,$SYSTEM.SQL.IsValidRegularIdentifier("%#$@_Fred")
WRITE !,$SYSTEM.SQL.IsValidRegularIdentifier("_1Fred")
WRITE !,$SYSTEM.SQL.IsValidRegularIdentifier("%#$")

WRITE !,$SYSTEM.SQL.IsValidRegularIdentifier("1Fred")
WRITE !,$SYSTEM.SQL.IsValidRegularIdentifier("Fr ed")
WRITE !,$SYSTEM.SQL.IsValidRegularIdentifier("%sqlupper")
```

The first three method calls return 1, indicating a valid identifier. The fourth and fifth method calls also return 1; these are valid identifiers, although they are not valid for use as table or field names. The last three method calls return 0, indicating an invalid identifier. Two of these are invalid because they violate the character rules — in these cases by beginning with a number or containing a blank. The final method call returns 0 because the specified string is a reserved word. Note that these rule tests are a minimum requirement; they do not certify an identifier as valid for all SQL uses.

This method can also be called as a stored procedure from ODBC or JDBC: %SYSTEM.SQL_IsValidRegularIdentifier("nnnn").

6.1.4 Namespace Names

A namespace name (also referred to as a database name) follows identifier naming conventions, with additional restrictions on punctuation characters and maximum length. For further details, refer to the [CREATE DATABASE](#) command.

A namespace name can be [delimited identifier](#) and can be the same as an [SQL reserved word](#). However, the same namespace name punctuation restrictions apply to both simple identifiers and delimited identifiers.

6.1.5 Identifiers and Class Entity Names

SQL table names, view names, field names, index names, trigger names, and procedure names are used to generate corresponding persistent class entities by stripping out non-alphanumeric characters. The generated names of class entities and globals follow these translation rules.

Note: Namespace names and SQL schema names and corresponding package names *do not* follow these translation rules.

- Identifiers that differ only in their inclusion of punctuation characters are valid. Because class object names cannot include punctuation characters, InterSystems IRIS generates corresponding unique object names by stripping out all punctuation characters. If stripping out the punctuation characters of an identifier results in a non-unique class object name, InterSystems IRIS creates a unique name by replacing the last alphanumeric character with an incremented character suffix.

For tables, views, fields, triggers, and procedure classmethod names, this is an integer suffix, beginning with 0. For example, myname and my_name generate myname and mynam0, adding my#name generates mynam1. If the number of generated unique names is larger than 10 (mynam9), additional names are generated by substituting a capital letter suffix, starting with A (mynamA). Because tables and views share the same name space, the same suffix counter is incremented for either a table or a view.

For index names, this suffix is a capital letter, beginning with A. For example, myindex and my_index generate myindex and myindeA.

If you have defined a name that ends in a suffix character (for example my_name0 or my_indexA, InterSystems IRIS handles unique name generation by incrementing to the next unused suffix.

- Identifiers that have a punctuation character as the first character and a number as the second character are not valid for table names, view names, or procedure names. They are valid for field names and index names. If the first character

of an SQL field name or index name is a punctuation character (% or _) and the second character is a number, InterSystems IRIS appends a lowercase “n” as the first character of the corresponding property name.

- Identifiers that consist *entirely* of punctuation characters, or begin with two underscore characters (__)name), or contains two pound signs together (nn##nn) are generally invalid as SQL entity names and should be avoided in all contexts.

You can configure translation of specific characters in SQL identifiers to other characters in corresponding object identifiers by creating a list of from/to character pairs. When converting an SQL identifier to an Objects identifier at DDL runtime, the characters in the “From” string are converted to the corresponding characters in the “To” string. These system-wide character translations facilitate the use of identifiers across environments where the rules for permitted identifier characters differ. Use the `$SYSTEM.SQL.Util.SetDDLIdentifierTranslations()` method to set from/to character pairings. To determine the current setting, call `$SYSTEM.SQL.CurrentSettings()`.

6.1.5.1 Specifying SQL Names in a Class Definition

When you define a persistent class that projects SQL entities, the name of each SQL entity are the same as the name of its corresponding persistent class definition element. To make an SQL table, field, or index name different, use the [SqlTableName](#), [SqlFieldName](#), or [SqlName](#) (for an index) keyword to specify the SQL name within your class definition. For example:

Class Member

```
Property LName As %String [SqlFieldName = "Family#Name"];
```

Class Member

```
Index NameIdx As %String [SqlName = "FullNameIndex"];
```

6.1.6 Identifier Length Considerations

The maximum length for SQL identifiers is 128 characters. When InterSystems IRIS maps an SQL identifier to the corresponding object entity, it creates the corresponding property, method, query, or index name with a maximum of 96 characters. If two SQL identifiers are identical for the first 96 characters, InterSystems IRIS replaces the 96th character of the corresponding object name with an integer (beginning with 0) to create a unique name.

The maximum length for schema and table names is subject to additional considerations and restrictions. Refer to [Table Names and Schema Names](#).

6.2 Delimited Identifiers

A delimited identifier has the following syntax:

```
delimited-identifier ::= " delimited-identifier-part { delimited-identifier-part } "
delimited-identifier-part ::= non-double-quote-character | double-quote-symbol
double-quote-symbol ::= ""
```

A delimited identifier is a unique identifier enclosed by delimiter characters. InterSystems SQL supports double quote characters (") as delimiter characters. Delimited identifiers are generally used to avoid the naming restrictions of simple identifiers.

Note that InterSystems SQL uses single quote characters (') to delimit [literals](#). For this reason, delimited identifiers must be specified using double quote characters ("), and literals must be specified using single quote characters ('). For example, '7' is the numeric literal 7, but "7" is a delimited identifier. When an SQL statement is enclosed in double quotes (for example, in Dynamic SQL), double quote characters within that string must be doubled.

A delimited identifier may span multiple lines or, in other words, may contain a newline character.

An SQL empty string should always be specified as a pair of single quote characters ' '. When delimited identifier support is enabled, a pair of double quote characters "" is parsed as an invalid delimited identifier and generates an SQLCODE -1 error.

6.2.1 Delimited Identifier Valid Names

A delimited identifier must be a unique name. Delimited identifiers are not case-sensitive; by convention, identifiers are represented with initial capital letters.

A delimited identifier can be the same as an [SQL reserved word](#). Delimited identifiers are commonly used to avoid concerns about naming conflicts with SQL reserved words.

A delimited identifier may contain almost any printable character, including blank spaces. Most delimited identifier names cannot contain the following characters: comma (,), period (.), caret (^), and the two-character arrow sequence (->); however delimited identifier role names and user names may contain these characters. A delimited identifier classname may contain periods (.). No delimited identifier may begin with an asterisk (*). The following term cannot be used as a delimited identifier: %vid. Violating these naming conventions results in an SQLCODE -1 error.

A delimited identifier used as a table, schema, column, or index name must be able to be converted to a [valid class entity name](#). Therefore, it must contain at least one alphanumeric character. A delimited identifier that begins with a number (or punctuation followed by a number) generates a corresponding class entity name with the letter “n” prefix.

The following example shows a query that makes use of delimited identifiers for both column and table names:

SQL

```
SELECT "My Field" FROM "My Table" WHERE "My Field" LIKE 'A%'
```

Note that the delimited identifiers are delimited with double quotes, and the string literal A% is delimited with single quotes.

When specifying a delimited identifier for a table name, you must separately delimit the table name and the schema name. Thus, "schema"."tablename" or schema."tablename" are valid identifiers, but "schema.tablename" is not a valid identifier.

6.2.2 Disabling Delimited Identifier Support

By default, support is enabled for delimited identifiers.

When delimited identifier support is disabled, characters within double quotes are treated as string literals.

You can set delimited identifier support system-wide using the [SET OPTION](#) command with the SUPPORT_DELIMITED_IDENTIFIERS keyword.

You can set delimited identifier support system-wide using the `$SYSTEM.SQL.Util.SetOption()` method `DelimitedIdentifiers` option. Delimited identifiers are supported by default.

To determine the current setting, call `$SYSTEM.SQL.CurrentSettings()`.

Note: Delimited identifiers are required by other features of InterSystems IRIS, such as InterSystems IRIS Business Intelligence. As such, take care when disabling them to ensure that other parts of your configuration are not affected.

6.3 SQL Reserved Words

SQL includes a long list of reserved words that cannot be used as identifiers. For a list of these reserved words, see [Reserved Words](#).

7

Using Embedded SQL

You can embed SQL statements within ObjectScript code used by InterSystems IRIS® data platform. These Embedded SQL statements are converted to optimized, executable code at runtime.

There are two kinds of Embedded SQL:

- A simple Embedded SQL query can only return values from a single row. Simple Embedded SQL can also be used for single-row insert, update, and delete, and for other SQL operations.
- A cursor-based Embedded SQL query can iterate through a query result set, returning values from multiple rows. Cursor-based Embedded SQL can also be used for multiple row update and delete SQL operations.

Note: Embedded SQL cannot be input to the Terminal command line, or specified in an [XECUTE](#) statement. To execute SQL from the command line, either use the `$SYSTEM.SQL.Execute()` method or the [SQL Shell interface](#).

7.1 Compiling Embedded SQL

Embedded SQL is not compiled when the routine that contains it is compiled. Instead, compilation of Embedded SQL occurs upon the first execution of the SQL code (runtime). First execution defines an executable cached query. This parallels the compilation of Dynamic SQL, where the SQL code is not compiled until the SQL [Prepare operation](#) is executed.

Embedded SQL code is not validated against SQL tables and other entities until the first execution of the routine. Therefore, you can compile a routine or a [method of a persistent class](#) containing Embedded SQL that references tables or other SQL entities that do not exist at routine compilation time. For this reason, most SQL errors are returned upon runtime execution, not compilation.

At routine compilation time SQL syntax checking is performed on Embedded SQL. The ObjectScript compiler fails and generates compile errors for invalid SQL syntax in Embedded SQL.

You can use the Management Portal SQL interface to test for the existence of SQL entities specified in Embedded SQL without executing the SQL code. This is described in [Validating Embedded SQL Code](#), which both validates the SQL syntax and checks for the existence of SQL entities. You can choose to validate Embedded SQL code prior to runtime execution by compiling a routine containing Embedded SQL code using the `/compileembedded=1` qualifier, as described in [Validating Embedded SQL Code](#).

A successfully executed Embedded SQL statement generates a cached query. Subsequent execution of that Embedded SQL uses the cached query, rather than recompiling the Embedded SQL source. This provides the performance benefits of cached queries to Embedded SQL. These cached queries are listed in the Management Portal for each table in the **Catalog Details** Cached Queries listing.

Runtime execution of a cursor-based Embedded SQL statement occurs when the cursor is first opened using an **OPEN** command. At this point in execution an optimized cached query plan is generated, as shown in the [SQL Statements](#) listing in the Management Portal. The **SQL Statements** listed **Location** is the name of the routine containing the Embedded SQL code. Entries in the **Cached Query** listings with hashed class names, such as %sqlcq.HSODS.xE6YUuGgukeA8rvZJUTKCawPmVyd, indicate Embedded SQL queries; however, entries with non-hashed class names, such as %sqlcq.USER.cls1 are created by Dynamic SQL queries.

Note: The [#sqlcompile mode](#) preprocessor statement used in earlier versions of InterSystems IRIS has been deprecated. It is parsed, but no longer performs any operation for most Embedded SQL commands. Most Embedded SQL commands are compiled at runtime regardless of the [#sqlcompile mode](#) setting. However, setting [#sqlcompile mode=deferred](#) is still meaningful for a small number of Embedded SQL commands because it forces runtime compilation of all types of Embedded SQL commands.

7.1.1 Embedded SQL and the Macro Preprocessor

You can use Embedded SQL within methods and within triggers (provided that they are defined to use ObjectScript) or within ObjectScript MAC routines. A MAC routine is processed by the Macro Preprocessor and converted to INT (intermediate) code which is subsequently compiled to executable OBJ code. These operations are performed at compile time on the routine containing the Embedded SQL, but not on the Embedded SQL code itself, which is not compiled until runtime. For further details, see [Using Macros and Include Files](#).

If an Embedded SQL statement itself contains Macro Preprocessor statements (# commands, ## functions, or \$\$\$macro references) these statements are compiled when the routine is compiled and are made available to the SQL code at runtime. This may affect **CREATE PROCEDURE**, **CREATE FUNCTION**, **CREATE METHOD**, **CREATE QUERY**, or **CREATE TRIGGER** statements that contain an ObjectScript code body.

7.1.1.1 Include Files in Embedded SQL

Embedded SQL statements require any macro Include files that they reference to be loaded on the system at runtime.

Because the compilation of Embedded SQL is deferred until it is first referenced, the context in which an Embedded SQL class is compiled will be the runtime environment rather than the compile-time environment of the containing class or routine. If the runtime current namespace is different than the containing routine's compile-time namespace, the Include files in the compile-time namespace may not be visible in the runtime namespace. In this situation the following occurs:

1. If an Include file is not visible in the runtime namespace, Embedded SQL compilation removes all Include files. Because Include files are rarely needed for SQL compilation, the runtime Embedded SQL compile will often succeed without them.
2. If after removing the Include files the compile fails, the InterSystems IRIS error reports the routine compile-time namespace, the Embedded SQL runtime namespace, and the list of Include files not visible from the runtime namespace.

7.1.1.2 The #SQLCompile Macro Directives

The Macro Preprocessor provides three preprocessor directives for use with Embedded SQL:

- [#sqlcompile select](#) specifies the format for data display when returned from a **SELECT** statement, or the required format for data input when specified to an **INSERT** or **UPDATE** statement, or a **SELECT** input host variable. It supports the following six options: Logical (the default), Display, ODBC, Runtime, Text (synonym for Display), and FDBMS (see below). If [#sqlcompile select=Runtime](#), you can use the **\$\$\$SYSTEM.SQL.Util.SetOption("SelectMode",n)** method to change how the data is displayed. The *n* value can be 0=Logical, 1=ODBC, or 2=Display.

Regardless of the [#sqlcompile select](#) option specified, an **INSERT** or **UPDATE** automatically converts the specified data value to its corresponding Logical format for storage.

Regardless of the `#sqlcompile select` option specified, a **SELECT** automatically converts an [input host variable](#) value to its corresponding Logical format for predicate matching.

Using `#sqlcompile select` for query display is shown in the following examples. These examples display the DOB (date of birth) value, then change the `SelectMode` to ODBC format, then display the DOB again. In the first example, changing the `SelectMode` has no effect on the display; in the second example, because `#sqlcompile select=Runtime`, changing the `SelectMode` changes the display:

ObjectScript

```
#sqlcompile select=Display
&sql(SELECT DOB INTO :a FROM Sample.Person)
  IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg  QUIT}
  ELSEIF SQLCODE=100 {WRITE "Query returns no results"  QUIT}
WRITE "1st date of birth is ",a,!
DO $SYSTEM.SQL.Util.SetOption("SelectMode",1)
WRITE "changed select mode to: ",$SYSTEM.SQL.Util.GetOption("SelectMode"),!
&sql(SELECT DOB INTO :b FROM Sample.Person)
WRITE "2nd date of birth is ",b
```

ObjectScript

```
#sqlcompile select=Runtime
&sql(SELECT DOB INTO :a FROM Sample.Person)
  IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg  QUIT}
  ELSEIF SQLCODE=100 {WRITE "Query returns no results"  QUIT}
WRITE "1st date of birth is ",a,!
DO $SYSTEM.SQL.Util.SetOption("SelectMode",1)
WRITE "changed select mode to: ",$SYSTEM.SQL.Util.GetOption("SelectMode"),!
&sql(SELECT DOB INTO :b FROM Sample.Person)
WRITE "2nd date of birth is ",b
```

For further details on `SelectMode` options, refer to [Data Display Options](#).

- `#sqlcompile select=FDBMS` is provided to enable Embedded SQL to format data in the same way as FDBMS. If a query has a constant value in the WHERE clause, FDBMS mode assumes it to be a Display value and converts it using `DisplayToLogical` conversion. If a query has a variable in the WHERE clause, FDBMS mode converts it using `FDBMSToLogical` conversion. The `FDBMSToLogical` conversion method should be designed to handle the three FDBMS variable formats: Internal, Internal_\$(1)_External, and \$(1)_External. If a query selects into a variable, it invokes the `LogicalToFDBMS` conversion method. This method returns Internal_\$(1)_External.
- `#sqlcompile path` (or `#import`) specifies the [schema search path](#) used to resolves unqualified table, view, and stored procedure names in data management commands such as **SELECT**, **CALL**, **INSERT**, **UPDATE**, **DELETE**, and **TRUNCATE TABLE**. If no schema search path is specified, or if the table is not found in the specified schemas, InterSystems IRIS uses the [default schema](#). `#sqlcompile path` and `#import` are ignored by data definition statements such as **ALTER TABLE**, **DROP VIEW**, **CREATE INDEX**, or **CREATE TRIGGER**. Data definition statements use the [default schema](#) to resolve unqualified names.
- `#sqlcompile audit` is a boolean switch specifying whether or not the execution of Embedded SQL statements should be recorded in the system events audit log. For further details, refer to [Auditing Embedded SQL](#).

For further details on these preprocessor directives, refer to [Preprocessor Directives Reference](#).

7.2 Embedded SQL Syntax

The syntax of the Embedded SQL directive is described below.

7.2.1 The &sql Directive

Embedded SQL statements are set off from the rest of the code by the &sql() directive, as shown in the following example:

ObjectScript

```
NEW SQLCODE,a
WRITE "Invoking Embedded SQL",!
&sql(SELECT Name INTO :a FROM Sample.Person)
  IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg  QUIT}
  ELSEIF SQLCODE=100 {WRITE "Query returns no results"  QUIT}
WRITE "The name is ",a
```

Results are returned using the [INTO clause](#) specifying one or more host variables. In this case, the host variable is named :a. For further details, see [Host Variables](#), which includes information on interactions between SQLCODE and host variables.

The &sql directive is not case-sensitive; you can use &sql, &SQL, &Sql, and so on. The &sql directive must be followed by an open parenthesis, with no intervening spaces, line breaks, or comments. The &sql directive can be used on the same line as a label, as shown in the following example:

ObjectScript

```
Mylabel &sql(
  SELECT Name INTO :a
  FROM Sample.Person
)
```

The body of an &sql directive should contain a [valid Embedded SQL statement](#), enclosed in parentheses. You can format your SQL statements in any way you like: white space and new lines are ignored by SQL.

When the Macro Preprocessor encounters an &sql directive, it hands the enclosed SQL statement to the SQL Query Processor. The Query Processor returns the code needed (in ObjectScript INT format) to execute the query. The Macro Preprocessor then replaces the &sql directive with this code (or a call to a label containing the code).

If an &sql directive contains an invalid Embedded SQL statement, the Macro Preprocessor generates a compilation error. An invalid SQL statement may have syntax errors, or refer to tables or columns that do not exist at compile time. Refer to [Validating Embedded SQL Code](#).

An &sql directive can contain [SQL-style comments](#) anywhere within its parentheses, can contain no SQL code, or contain only comment text. If an &sql directive contains no SQL code or only commented text, the directive is parsed as a no-op and the SQLCODE variable is not defined.

ObjectScript

```
NEW SQLCODE
WRITE !,"Entering Embedded SQL"
&sql()
WRITE !,"Leaving Embedded SQL"
```

ObjectScript

```
NEW SQLCODE
WRITE !,"Entering Embedded SQL"
&sql(/* SELECT Name INTO :a FROM Sample.Person */)
WRITE !,"Leaving Embedded SQL"
```

7.2.2 &sql Alternative Syntax

Because complex Embedded SQL programs may contain multiple &sql directives — including nested &sql directives — the following alternative syntax formats are provided:

- `##sql( ... )`: this directive is functionally equivalent to `&sql`. It provides an alternative syntax for clarity of code. However, it cannot include marker syntax.
- `&sql<marker>( ... )<reversemarker>`: this directive allows you to specify multiple `&sql` directives, identifying each with a user-selected marker character or string. This marker syntax is described in the following section.

7.2.3 &sql Marker Syntax

You can identify a specific `&sql` directive using user-defined marker syntax. This syntax consists of a character or string specified between “`&sql`” and the open parenthesis character. The reverse of this marker must appear immediately after the closing parenthesis at the end of the Embedded SQL. The syntax is as follows:

```
&sql<marker>( SQL statement )<reverse-marker>
```

Note that no white space (space, tab, or line return) is permitted between `&sql`, *marker*, and the open parenthesis, and no white space is permitted between the closing parenthesis and *reverse-marker*.

A *marker* can be a single character or a series of characters. A *marker* cannot contain the following punctuation characters:

```
( + - / \ | * )
```

A *marker* cannot contain a whitespace character (space, tab, or line return). It may contain all other printable characters and combinations of characters, including Unicode characters. The *marker* and *reverse-marker* are case-sensitive.

The corresponding *reverse-marker* must contain the same characters as *marker* in the reverse order. For example: `&sqlABC( ... )CBA`. If *marker* contains a `[` or `{` character, *reverse-marker* must contain the corresponding `]` or `}` character. The following are examples of valid `&sql` *marker* and *reverse-marker* pairs:

```
&sql@@( ... )@@    &sql[( ... )]    &sqltest( ... )tset    &sql[Aa{( ... )}aA]
```

When selecting a marker character or string, note the following important SQL restriction: the SQL code cannot contain the character sequence “`)<reversemarker>`” anywhere in the code, *including in literal strings and comments*. For example, if the marker is “ABC”, the character string “)CBA” cannot appear anywhere in the Embedded SQL code. If this occurs, the combination of a valid marker and valid SQL code will fail compilation. Thus it is important to use care in selecting a *marker* character or string to prevent this collision.

7.2.4 Embedded SQL and Line Offsets

The presence of Embedded SQL affects ObjectScript line offsets, as follows:

- Embedded SQL adds (at least) 2 to the total number of INT code lines at that point in the routine. Therefore, a single line of Embedded SQL counts as 3 lines, two lines of Embedded SQL count as 4 lines, and so forth. Embedded SQL that invokes other code can add many more lines to the INT code.

A dummy Embedded SQL statement, containing only a comment counts as 2 INT code lines, as in the following example: `&sql( /* for future use */)`.

- All lines within Embedded SQL count as line offsets, including comments and blank lines.

You can display INT code lines using the [^ROUTINE](#) global.

7.3 Embedded SQL Code

Considerations for writing SQL code in Embedded SQL include the following:

- [Simple \(non-cursor\) Embedded SQL statements](#)
- [Schema name resolution](#)
- [Literal data values](#)
- [Data formatting for %List and date/time data values](#)
- [Privilege Checking](#)

Host variables, which are used to export data values from Embedded SQL are described [later on this page](#).

7.3.1 Simple SQL Statements

You can use a simple SQL statement (a single Embedded SQL statement) for a variety of operations including:

- [INSERT](#), [UPDATE](#), [INSERT OR UPDATE](#), and [DELETE](#) statements.
- [DDL](#) statements.
- [GRANT](#) and [REVOKE](#) statements.
- [SELECT](#) statements that return only a single row (or if you are only interested in the first returned row).

Simple SQL statements are also referred to as *non-cursor-based SQL statements*. Also see [Cursor-based Embedded SQL](#).

For example, the following statement finds the name of the (one and only) Patient with ID of 43:

ObjectScript

```
&sql(SELECT Name INTO :name
      FROM Patient
      WHERE %ID = 43)
```

If you use a simple statement for a query that can return multiple rows, then only the first row is returned:

ObjectScript

```
&sql(SELECT Name INTO :name
      FROM Patient
      WHERE Age = 43)
```

Depending on the query, there is no guarantee which row will actually be returned first.

The INTO clause [output host variables](#) are set to the null string when Embedded SQL is compiled. For this reason, an simple Embedded SQL statement should test for [SQLCODE=100](#) (query returns no data), or SQLCODE=0 (successful execution) before accessing output host variables.

7.3.2 Schema Name Resolution

A table name, view name, or stored procedure name is either qualified (specifies a schema name) or unqualified (does not specify a schema name). If the name does not specify a schema name, InterSystems IRIS resolves the schema name as follows:

- **Data Definition:** InterSystems IRIS uses the [system-wide default schema](#) to resolve an unqualified name. If the default schema does not exist, InterSystems IRIS creates the schema and the corresponding class package. All data definition statements use the system-wide default schema; data definition statements ignore the **#import** and **#sqlcompile path** macro preprocessor directives.
- **Data Management:** InterSystems IRIS uses the [schema search path](#) specified by the **#sqlcompile path** and/or the **#import** macro preprocessor directive(s) in effect for the class or routine that contains the Embedded SQL statement. The **#import** and **#sqlcompile path** directives are mutually independent lists of possible schema names with different

functionality. Either or both may be used to supply a schema name for an unqualified table, view, or stored procedure name. If no schema search path is specified, InterSystems IRIS uses the [system-wide default schema name](#).

See [Packages](#) for more details on schemas.

7.3.3 Literal Values

Embedded SQL queries may contain literal values (strings, numbers, or dates). Strings should be enclosed within single (') quotes. (In InterSystems SQL, double quotes specify a [delimited identifier](#)):

ObjectScript

```
&sql(SELECT 'Employee (' || Name || ' )' INTO :name
      FROM Sample.Employee)
      IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg QUIT}
      ELSEIF SQLCODE=100 {WRITE "Query returns no results" QUIT}
WRITE name
```

Numeric values can be used directly. Literal numbers and timestamp values are “lightly normalized” before InterSystems IRIS compares these literal values to field values, as shown in the following example where +0050.000 is normalized to 50:

ObjectScript

```
&sql(SELECT Name, Age INTO :name, :age
      FROM Sample.Person
      WHERE Age = +0050.000)
      IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg QUIT}
      ELSEIF SQLCODE=100 {WRITE "Query returns no results" QUIT}
WRITE name, " age=", age
```

Arithmetic, function, and special variable expressions can be specified:

ObjectScript

```
&sql(DECLARE C1 CURSOR FOR
      SELECT Name, Age-65, $HOROLOG INTO :name, :retire, :today
      FROM Sample.Person
      WHERE Age > 60
      ORDER BY Age, Name)
&sql(OPEN C1)
      QUIT:(SQLCODE'=0)
&sql(FETCH C1)
      WHILE (SQLCODE = 0) {
        WRITE $ZDATE(today), " ", name, " has ", retire, " eligibility years", !
        &sql(FETCH C1) }
&sql(CLOSE C1)
```

You can also input a literal value using an input host variable. Input host numeric values are also “lightly normalized.” For further details, see [Host Variables](#).

In Embedded SQL, a few character sequences that begin with ## are not permitted within a string literal and must be specified using [##lit](#). These character sequences are: ##;, ##beginlit, ##expression(), ##function(), ##quote(), ##stripq(), and ##unique(). For example, the following example fails:

ObjectScript

```
WRITE "Embedded SQL test", !
&sql(SELECT 'the sequence ##unique( is restricted' INTO :x)
WRITE x
```

The following workaround succeeds:

ObjectScript

```
WRITE "Embedded SQL test",!  
&sql(SELECT 'the sequence ##lit(##unique()) is restricted' INTO :x)  
WRITE x
```

7.3.4 Data Format

Within Embedded SQL, data values are in “Logical mode”; that is, values are in the native format used by the SQL Query Processor. For string, integers, and other data types that do not define a **LogicalToOdbc** or **LogicalToDisplay** conversion, this has no effect. Data format affects %List data, and the %Date and %Time data types.

The %List data type displays in Logical mode as element values prefaced with non-printing list encoding characters. The **WRITE** command displays these values as concatenated elements. For example, the FavoriteColors field of Sample.Person stores data in %List data type, such as the following: \$LISTBUILD('Red', 'Black'). In Embedded SQL this displays in Logical mode as RedBlack, with a length of 12 characters. In Display mode it displays as Red Black; in ODBC mode it displays as Red,Black. This is shown in the following example:

ObjectScript

```
&sql(DECLARE C1 CURSOR FOR  
      SELECT TOP 10 FavoriteColors INTO :colors  
      FROM Sample.Person WHERE FavoriteColors IS NOT NULL)  
&sql(OPEN C1)  
      QUIT:(SQLCODE'=0)  
&sql(FETCH C1)  
WHILE (SQLCODE = 0) {  
    WRITE $LENGTH(colors),": ",colors,!  
    &sql(FETCH C1) }  
&sql(CLOSE C1)
```

The %Date and %Time [data types](#) provided by InterSystems IRIS use the InterSystems IRIS internal date representation (**\$HOROLOG** format) as their Logical format. A %Date data type returns INTEGER data type values in Logical mode; VARCHAR data type values in Display mode, and DATE data type values in ODBC mode. The %TimeStamp data type uses ODBC date-time format (YYYY-MM-DD HH:MM:SS) for its Logical, Display, and ODBC format.

For example, consider the following class definition:

Class Definition

```
Class MyApp.Patient Extends %Persistent  
{  
  /// Patient name  
  Property Name As %String(MAXLEN = 50);  
  
  /// Date of birth  
  Property DOB As %Date;  
  
  /// Date and time of last visit  
  Property LastVisit As %TimeStamp;  
}
```

A simple Embedded SQL query against this table will return values in logical mode. For example, consider the following query:

ObjectScript

```
&sql(SELECT Name, DOB, LastVisit  
      INTO :name, :dob, :visit  
      FROM Patient  
      WHERE %ID = :id)
```

This query returns logical value for the three properties into the host variables *name*, *dob*, and *visit*:

Host Variable	Value
<i>name</i>	"Weiss,Blanche"
<i>dob</i>	44051
<i>visit</i>	"2001-03-15 11:11:00"

Note that *dob* is in [\\$HOROLOG](#) format. You can convert this to a display format using the [\\$ZDATETIME](#) function:

ObjectScript

```
SET dob = 44051
WRITE $ZDT(dob,3), !
```

The same consideration as true within a WHERE clause. For example, to find a Patient with a given birthday, you must use a logical value in the WHERE clause:

ObjectScript

```
&sql(SELECT Name INTO :name
      FROM Patient
      WHERE DOB = 43023)
```

or, alternatively, using a host variable:

ObjectScript

```
SET dob = $ZDH("01/02/1999",1)
&sql(SELECT Name INTO :name
      FROM Patient
      WHERE DOB = :dob)
```

In this case, we use the [\\$ZDATEH](#) function to convert a display format date into its logical, [\\$HOROLOG](#) equivalent.

7.3.5 Privilege Checking

Embedded SQL does not perform SQL privilege checking. You can access all tables, views, and columns and perform any operation, regardless of the privileges assignments. It is assumed that applications using Embedded SQL will check for privileges before using Embedded SQL statements.

You can use the InterSystems SQL [%CHECKPRIV](#) statement in Embedded SQL to determine the current privileges.

For further details, refer to [SQL Users, Roles, and Privileges](#).

7.4 Host Variables

A host variable is a local variable that passes a literal value into or out of Embedded SQL. Most commonly, host variables are used to either pass the value of a local variable as an input value into Embedded SQL, or to pass an SQL query result value as an output host variable from an Embedded SQL query.

A host variable cannot be used to specify an SQL identifier, such as a schema name, table name, field name, or cursor name. A host variable cannot be used to specify an SQL keyword.

- Output host variables are only used in Embedded SQL. They are specified in an [INTO clause](#), an SQL query clause that is only supported in Embedded SQL. Compiling Embedded SQL initializes all INTO clause variables to the null string ("").

- Input host variables can be used in either Embedded SQL or Dynamic SQL. In [Dynamic SQL](#), you can also input a literal to an SQL statement using the “?” input parameter. This “?” syntax cannot be used in Embedded SQL.

Within Embedded SQL, input host variables can be used in any place that a literal value can be used. Output host variables are specified using an [INTO](#) clause of a **SELECT** or **FETCH** statement.

Note: When an SQL NULL is output to ObjectScript, it is represented by an ObjectScript empty string (""), a string of length zero. See [NULL and Undefined Host Variables](#).

To use a variable or a property reference as a host variable, precede it with a colon (:). A host variable in embedded InterSystems SQL can be one of the following:

- One or more ObjectScript [local variables](#), such as :myvar, specified as a comma-separated list. A local variable can be fully formed and can include subscripts. Like all local variables, it is case-sensitive and can contain Unicode letter characters.
- A single ObjectScript local variable array, such as :myvars(). A local variable array can receive only field values from a single table (not joined tables or a view). For details, refer to “[Host Variable Subscripted by Column Number](#)”, below.
- An object reference, such as :oref.Prop, where Prop is a property name, with or without a leading % character. This can be a simple property or a multidimensional array property, such as :oref.Prop(1). It can be an [instance variable](#), such as :i%Prop or :i%%Data. The property name may be delimited; for example :Person."Home City". Delimited property names can be used even when support for delimited identifiers is deactivated. Multidimensional properties may include :i%Prop() and :m%Prop() host variable references. An object reference host variable can include any number of dot syntax levels; for example, :Person.Address.City.

When an oref.Prop is used as a host variable inside a procedure block method, the system automatically adds the oref variable (not the entire oref.Prop reference) to the [PublicList](#) and **NEWs** it.

Double quotes in a host variable specify a literal string, not a delimited identifier. For example,

```
:request.GetValueAt("PID:SetIDPID") or  
:request.GetValueAt("PID:PatientName(1).FamilyName").
```

Host variables should be listed in the ObjectScript procedure’s [PublicList](#) variables list and reinitialized using the **NEW** command. You can configure InterSystems IRIS to also list all host variables used in Embedded SQL in comment text; this is described in [Comment](#).

Host variable values have the following behavior:

- Input host variables are never modified by the SQL statement code. They retain their original values even after Embedded SQL has run. However, input host variable values are “lightly normalized” before being supplied to the SQL statement code: Valid numeric values are stripped of leading and trailing zeros, a single leading plus sign, and a trailing decimal point. Timestamp values are stripped of trailing spaces, trailing zeros in fractional seconds, and (if there are no fractional seconds) a trailing decimal point.
- Output host variables specified in the INTO clause are defined when the query is compiled. They are set to the null string so that referencing them does not result in an <UNDEFINED> error. Host variable values only represent actual values when SQLCODE=0. In DECLARE ... SELECT ... INTO statements, do not modify the output host variables in the INTO clause between two FETCH calls, since that might cause unpredictable query results.

You must check the [SQLCODE](#) value before processing output host variables. Output host variable values should only be used when SQLCODE=0.

When using a comma-separated list of host variables in the [INTO clause](#), you must specify the same number of host variables as the number of [select-items](#) (fields, aggregate functions, scalar functions, arithmetic expressions, literals). Too many or too few host variables results in an SQLCODE -76 cardinality error upon compilation.

This is often a concern when using **SELECT *** in Embedded SQL. For example, **SELECT * FROM Sample.Person** is only valid with a comma-separated list of 15 host variables (the exact number of non-hidden columns, which, depending on the table definition, may or may not include the system-generated **RowID (ID) column**). Note that this number of columns may not be a simple correspondence to the number of properties listed in the *InterSystems Class Reference*.

Because the number of columns can change, it is usually not a good idea to specify **SELECT *** with an **INTO** clause list of individual host variables. When using **SELECT ***, it is usually preferable to use a host variable subscripted array, such as the following:

ObjectScript

```
NEW SQLCODE
&sql(SELECT %ID,* INTO :tflds() FROM Sample.Person )
    IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg  QUIT}
    ELSEIF SQLCODE=100 {WRITE "Query returns no results"  QUIT}
FOR i=0:1:25 {
    IF $DATA(tflds(i)) {
        WRITE "field ",i," = ",tflds(i),! }
    }
```

This example uses %ID to return the RowID as field number 1, whether or not the RowID is hidden. Note that in this example the field number subscripts may not be continuous sequence; some fields may be hidden and are skipped over. Fields that contain NULL are listed with an empty string value. Using a host variable array is described in “[Host Variable Subscripted by Column Number](#)”, below.

It is good programming practice to check the SQLCODE value immediately after exiting Embedded SQL. Output host variable values should only be used when SQLCODE=0.

7.4.1 Host Variable Examples

In the following ObjectScript example, an Embedded SQL statement uses output host variables to return a name and home state address from an SQL query to ObjectScript:

ObjectScript

```
&sql(SELECT Name,Home_State
      INTO :CName,:CAddr
      FROM Sample.Person)
    IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg  QUIT}
    ELSEIF SQLCODE=100 {WRITE "Query returns no results"  QUIT}
    WRITE !,"Name is: ",CName
    WRITE !,"State is: ",CAddr
```

The Embedded SQL uses an INTO clause that specifies the host variables :CName and :CAddr to return the selected customer's name in the local variable CName, and home state in the local variable CAddr.

The following example performs the same operation, using subscripted local variables:

ObjectScript

```
&sql(SELECT Name,Home_State
      INTO :CInfo(1),:CInfo(2)
      FROM Sample.Person)
    IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg  QUIT}
    ELSEIF SQLCODE=100 {WRITE "Query returns no results"  QUIT}
    WRITE !,"Name is: ",CInfo(1)
    WRITE !,"State is: ",CInfo(2)
```

These host variables are simple local variables with user-supplied subscripts (:CInfo(1)). However, if you omit the subscript (:CInfo()), InterSystems IRIS populates the host variable subscripted array using SqlColumnNumber, as described below.

In the following ObjectScript example, an Embedded SQL statement uses both input host variables (in the WHERE clause) and output host variables (in the INTO clause):

ObjectScript

```
SET minval = 10000
SET maxval = 50000
&sql(SELECT Name,Salary INTO :outname, :outsalary
      FROM MyApp.Employee
      WHERE Salary > :minval AND Salary < :maxval)
      IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg QUIT}
      ELSEIF SQLCODE=100 {WRITE "Query returns no results" QUIT}
WRITE !,"Name is: ",outname
WRITE !,"Salary is: ",outsalary
```

The following example performs “light normalization” on an input host variable. Note that InterSystems IRIS treats the input variable value as a string and does not normalize it, but Embedded SQL normalizes this number to 65 to perform the equality comparison in the WHERE clause:

ObjectScript

```
SET x="+065.000"
&sql(SELECT Name,Age
      INTO :a,:b
      FROM Sample.Person
      WHERE Age=:x)
      IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg QUIT}
      ELSEIF SQLCODE=100 {WRITE "Query returns no results" QUIT}
WRITE !,"Input value is: ",x
WRITE !,"Name value is: ",a
WRITE !,"Age value is: ",b
```

In the following ObjectScript example, an Embedded SQL statement uses object properties as host variables:

ObjectScript

```
&sql(SELECT Name, Title INTO :obj.Name, :obj.Title
      FROM MyApp.Employee
      WHERE %ID = :id )
```

In this case, *obj* must be a valid reference to an object that has mutable (that is, they can be modified) properties Name and Title. Note that if a query includes an INTO statement and no data is returned (that is, that SQLCODE is 100), then executing the query may result in the value of the host variable being modified.

7.4.2 Host Variable Subscripted by Column Number

If the FROM clause contains a single table, you can specify a subscripted host variable for fields selected from that table; for example, the local array :myvar(). The local array is populated by InterSystems IRIS, using each field’s SqlColumnName as the numeric subscript. Note that SqlColumnName is the column number in the table definition, *not* the *select-list* sequence. (You cannot use a subscripted host variable for fields of a view.)

A host variable array must be a local array that has its lowest level subscript omitted. Therefore, :myvar(), :myvar(5,), and :myvar(5,2,) are all valid host variable subscripted arrays.

- A host variable subscripted array may be used for input in an **INSERT**, **UPDATE**, or **INSERT OR UPDATE** statement [VALUES clause](#). When used in an **INSERT** or **UPDATE** statement, a host variable array allows you to define which columns are being updated at runtime, rather than at compile time.
- A host variable subscripted array may be used for output in a **SELECT** or **DECLARE** statement [INTO clause](#). Subscripted array usage in **SELECT** is shown in the examples that follow.

In the following example, the **SELECT** populates the Cdata array with the values of the specified fields. The elements of Cdata() correspond to the table column definition, *not* the **SELECT** elements. Therefore, the Name field is column 6, the Age field is column 2, and the date of birth (DOB) field is column 3 in Sample.Person:

ObjectScript

```
&sql(SELECT Name, Age, DOB
      INTO :Cdata()
      FROM Sample.Person)
      IF SQLCODE<0 {WRITE "SQLCODE error ", SQLCODE, " ", %msg  QUIT}
      ELSEIF SQLCODE=100 {WRITE "Query returns no results"  QUIT}
WRITE !, "Name is: ", Cdata(6)
WRITE !, "Age is: ", Cdata(2)
WRITE !, "DOB is: ", $ZDATE(Cdata(3), 1)
```

The following example uses a subscripted array host variable to return all of the field values of a row:

ObjectScript

```
&sql(SELECT * INTO :Allfields()
      FROM Sample.Person)
      IF SQLCODE<0 {WRITE "SQLCODE error ", SQLCODE, " ", %msg  QUIT}
      ELSEIF SQLCODE=100 {WRITE "Query returns no results"  QUIT}
SET x=1
WHILE x '=' {
  WRITE !, x, " field is ", Allfields(x)
  SET x=$ORDER(Allfields(x))
}
```

Note that this **WHILE** loop is incremented using **\$ORDER** rather than a simple $x=x+1$. This is because in many tables (such as `Sample.Person`) there may be hidden columns. These cause the column number sequence to be discontinuous.

If the `SELECT` list contains items that are not fields from that table, such as expressions or arrow-syntax fields, the `INTO` clause must also contain comma-separated non-array host variables. The following example combines a subscripted array host variable to return values that correspond to defined table columns, and host variables to return values that do not correspond to defined table columns:

ObjectScript

```
&sql(SELECT Name, Home_City, {fn NOW}, Age, ($HOROLOG-DOB)/365.25, Home_State
      INTO :Allfields(), :timestamp('now'), :exactage
      FROM Sample.Person)
      IF SQLCODE<0 {WRITE "SQLCODE error ", SQLCODE, " ", %msg  QUIT}
      ELSEIF SQLCODE=100 {WRITE "Query returns no results"  QUIT}
SET x=$ORDER(Allfields(""))
WHILE x '=' {
  WRITE !, x, " field is ", Allfields(x)
  SET x=$ORDER(Allfields(x)) }
WRITE !, "date & time now is ", timestamp("now")
WRITE !, "exact age is ", exactage
```

Note that the non-array host variables must match the non-column **SELECT** items in number and sequence.

The use of a host variable as a subscripted array is subject to the following restrictions:

- A subscripted list can only be used when selecting fields from a single table in the `FROM` clause. This is because when selecting fields from multiple tables, the `SqlColumnName` values may conflict.
- A subscripted list can only be used when selecting table fields. It cannot be used for expressions or aggregate fields. This is because these *select-list* items do not have an `SqlColumnName` value.

For further details on using a host variable array, see [INTO Clause](#).

7.4.3 NULL and Undefined Host Variables

If you specify an input host variable that is not defined, Embedded SQL treats its value as `NULL`.

ObjectScript

```
NEW x
&sql(SELECT Home_State,:x
      INTO :a,:b
      FROM Sample.Person)
      IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg QUIT}
      ELSEIF SQLCODE=100 {WRITE "Query returns no results" QUIT}
WRITE !,"The length of Home_State is: ",$LENGTH(a)
WRITE !,"The length of x is: ",$LENGTH(b)
```

The SQL NULL is equivalent to the ObjectScript "" string (a zero-length string).

When Embedded SQL is compiled, all INTO clause output host variables are defined as the ObjectScript "" string (a zero-length string). If you output a NULL to a host variable, Embedded SQL treats its value as the ObjectScript "" string (a zero-length string). For example, some records in Sample.Person have a NULL Spouse field. After executing this query:

ObjectScript

```
&sql(SELECT Name,Spouse
      INTO :name,:spouse
      FROM Sample.Person
      WHERE Spouse IS NULL)
      IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg QUIT}
      ELSEIF SQLCODE=100 {WRITE "Query returns no results" QUIT}
WRITE !,"Name: ",name," of length ",$LENGTH(name)," defined: ",$DATA(name)
WRITE !,"Spouse: ",spouse," of length ",$LENGTH(spouse)," defined: ",$DATA(spouse)
```

The host variable, *spouse*, will be set to "" (a zero-length string) to indicate a NULL value. Therefore, the ObjectScript **\$DATA** function cannot be used to determine if an SQL field is NULL. **\$DATA** returns true (variable is defined) when passed an output host variable for an SQL field with a NULL value.

In the rare case that a table field contains an SQL zero-length string (""), such as if an application explicitly set the field to an SQL " string, the host variable will contain the special marker value, \$CHAR(0) (a string of length 1, containing only a single, ASCII 0 character), which is the ObjectScript representation for the SQL zero-length string. Use of SQL zero-length strings is strongly discouraged.

The following example compares host variables output from an SQL NULL and an SQL zero-length string:

ObjectScript

```
&sql(SELECT '',Spouse
      INTO :zls,:spouse
      FROM Sample.Person
      WHERE Spouse IS NULL)
      IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg QUIT}
      ELSEIF SQLCODE=100 {WRITE "Query returns no results" QUIT}
WRITE "In ObjectScript"
WRITE !,"ZLS is of length ",$LENGTH(zls)," defined: ",$DATA(zls)
/* Length=1, Defined=1 */
WRITE !,"NULL is of length ",$LENGTH(spouse)," defined: ",$DATA(spouse)
/* Length=0, Defined=1 */
```

Note that this host variable NULL behavior is only true within server-based queries (Embedded SQL and Dynamic SQL). Within ODBC and JDBC, NULL values are explicitly specified using the ODBC or JDBC interface.

7.4.4 Validity of Host Variables

- Input host variables are never modified by Embedded SQL.
- Output host variables are only reliably valid after Embedded SQL when SQLCODE = 0.

For example, the following use of *OutVal* is *not* reliably valid:

ObjectScript

```
InvalidExample
SET InVal = "1234"
SET OutVal = "None"
&sql(SELECT Name
      INTO :OutVal
      FROM Sample.Person
      WHERE %ID=:InVal)
IF OutVal="None" {           ; Improper Use
WRITE !,"No data returned"
WRITE !,"SQLCODE=",SQLCODE }
ELSE {
WRITE !,"Name is: ",OutVal }
```

The value of *OutVal* set before invoking Embedded SQL should *not* be referenced by the **IF** command after returning from Embedded SQL.

Instead, you should code this example as follows, using the SQLCODE variable:

ObjectScript

```
ValidExample
SET InVal = "1234"
&sql(SELECT Name
      INTO :OutVal
      FROM Sample.Person
      WHERE %ID=:InVal)
IF SQLCODE'=0 { SET OutVal="None"
  IF OutVal="None" {
    WRITE !,"No data returned"
    WRITE !,"SQLCODE=",SQLCODE } }
ELSE {
  WRITE !,"Name is: ",OutVal }
```

The Embedded SQL sets the SQLCODE variable to 0 to indicate the successful retrieval of an output row. An SQLCODE value of 100 indicates that no row was found that matches the **SELECT** criteria. An SQLCODE negative number value indicates a SQL error condition.

7.4.5 Host Variables and Procedure Blocks

If your Embedded SQL is within a procedure block, all input and output host variables must be public. This can be done by declaring them in the PUBLIC section at the beginning of the procedure block, or by naming them with an initial % character (which automatically makes them public). Note, however, that a user-defined % host variable is automatically public, but is not automatically NEWed. It is the user's responsibility to perform a **NEW** on such variables, as desired. Some SQL % variables, such as %ROWCOUNT, %ROWID, and %msg, are both automatically public and automatically NEWed, as described in [Embedded SQL Variables](#). You must declare SQLCODE as public. For further details on the SQLCODE variable, see [Embedded SQL Variables](#).

In the following procedure block example, the host variables *zip*, *city*, and *state*, as well as the SQLCODE variable are declared as PUBLIC. The SQL system variables %ROWCOUNT, %ROWID, and %msg are already public, because their names begin with a % character. The procedure code then performs a **NEW** on SQLCODE, the other SQL system variables, and the *state* local variable:

ObjectScript

```
UpdateTest(zip,city)
[SQLCODE,zip,city,state] PUBLIC {
NEW SQLCODE,%ROWCOUNT,%ROWID,%msg,state
SET state="MA"
&sql(UPDATE Sample.Person
      SET Home_City = :city, Home_State = :state
      WHERE Home_Zip = :zip)
  IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg  QUIT}
  QUIT %ROWCOUNT
}
```

7.5 SQL Cursors

A cursor is a pointer to data that allows an Embedded SQL program to perform an operation on the record pointed to. By using a cursor, Embedded SQL can iterate through a result set. Embedded SQL can use a cursor to execute a query that returns data from multiple records. Embedded SQL can also use a cursor to update or delete multiple records.

You must first **DECLARE** an SQL cursor, giving it a name. In the **DECLARE** statement you supply a **SELECT** statement that identifies which records the cursor will point to. You then supply this cursor name to the **OPEN cursor** statement. You then repeatedly issue the **FETCH cursor** statement to iterate through the **SELECT** result set. You then issue a **CLOSE cursor** statement; it is imperative that you close the cursor before exiting the ObjectScript method that contains Embedded SQL.

- A cursor-based query uses **DECLARE cursorname CURSOR FOR SELECT** to select records and (optionally) return select column values into **output host variables**. The **FETCH** statement iterates through the result set, using these variables to return selected column values.
- A cursor-based **DELETE** or **UPDATE** uses **DECLARE cursorname CURSOR FOR SELECT** to select records for the operation. No output host variables are specified. The **FETCH** statement iterates through the result set. The **DELETE** or **UPDATE** statement contains a **WHERE CURRENT OF** clause to identify the current cursor position in order to perform the operation on the selected record. For further details on cursor-based **DELETE** and **UPDATE**, see **WHERE CURRENT OF**.

Note that a cursor cannot span methods. Therefore, you must declare, open, fetch, and close a cursor within the same class method. It is important to consider this with all code that generates classes and methods, such as classes generated from a .CSP file.

The following example, uses a cursor to execute a query and display the results to the principal device:

ObjectScript

```
&sql(DECLARE C1 CURSOR FOR
  SELECT %ID, Name
  INTO :id, :name
  FROM Sample.Person
  WHERE Name %STARTSWITH 'A'
  ORDER BY Name
)

&sql(OPEN C1)
  QUIT:(SQLCODE'=0)
&sql(FETCH C1)

While (SQLCODE = 0) {
  Write id, ": ", name, !
  &sql(FETCH C1)
}

&sql(CLOSE C1)
```

This example does the following:

1. It declares a cursor, *C1*, that returns a set of Person rows ordered by Name.
2. It opens the cursor.
3. It calls **FETCH** on the cursor until it reaches the end of the data. After each call to **FETCH**, the **SQLCODE** variable will be set to 0 if there is more data to fetch. After each call to **FETCH**, the values returned are copied into the host variables specified by the **INTO** clause of the **DECLARE** statement.
4. It closes the cursor.

7.5.1 The DECLARE Cursor Statement

The **DECLARE** statement specifies both the cursor name and the SQL **SELECT** statement that defines the cursor. The **DECLARE** statement must occur within a routine *before* any statements that use the cursor.

A cursor name is case-sensitive.

A cursor name must be unique within a class or routine. For this reason, a routine that is called recursively cannot contain a cursor declaration. In this situation, it may be preferable to use **Dynamic SQL**.

The following example declares a cursor named *MyCursor*:

ObjectScript

```
&sql(DECLARE MyCursor CURSOR FOR
  SELECT Name, DOB
  FROM Sample.Person
  WHERE Home_State = :state
  ORDER BY Name
)
```

A **DECLARE** statement may include an optional **INTO clause** that specifies the names of the local host variables that will receive data as the cursor is traversed. For example, we can add an **INTO** clause to the previous example:

ObjectScript

```
&sql(DECLARE MyCursor CURSOR FOR
  SELECT Name, DOB
  INTO :name, :dob
  FROM Sample.Person
  WHERE Home_State = :state
  ORDER BY Name
)
```

The **INTO** clause may contain a comma-separated list of host variables, a single host variable array, or a combination of both. If specified as a comma-separated list, the number of **INTO** clause host variables must exactly match the number of columns within the cursor's **SELECT** list or you will receive a “Cardinality Mismatch” error when the statement is compiled.

If the **DECLARE** statement does not include an **INTO** clause, then the **INTO** clause must appear within the **FETCH** statement. A small performance improvement may result from specifying the **INTO** clause in the **DECLARE** statement, rather than in the **FETCH** statement.

Because **DECLARE** is a declaration, not an executed statement, it does not set or kill the **SQLCODE** variable.

If a specified cursor has already been declared, compilation fails with a **SQLCODE -52** error, **Cursor name already declared**.

Executing a **DECLARE** statement does not compile the **SELECT** statement. The **SELECT** statement is compiled the first time the **OPEN** statement is executed. Embedded SQL is not compiled at routine compile time, but at SQL execution time (runtime).

7.5.2 The OPEN Cursor Statement

The **OPEN** statement prepares a cursor for subsequent execution:

ObjectScript

```
&sql(OPEN MyCursor)
```

Executing the **OPEN** statement compiles the Embedded SQL code found in the **DECLARE** statement, creates an optimized query plan, and generates a cached query. Error involving missing resources (such as an undefined table or field) are issued when the **OPEN** is executed (at SQL runtime).

Upon a successful call to OPEN, the SQLCODE variable will be set to 0.

You cannot FETCH data from a cursor without first calling OPEN.

7.5.3 The FETCH Cursor Statement

The **FETCH** statement fetches the data for the next row of the cursor (as defined by the cursor query):

ObjectScript

```
&sql(FETCH MyCursor)
```

You must DECLARE and OPEN a cursor before you can call FETCH on it.

A FETCH statement may contain an **INTO clause** that specifies the names of the local host variables that will receive data as the cursor is traversed. For example, we can add an INTO clause to the previous example:

ObjectScript

```
&sql(FETCH MyCursor INTO :a, :b)
```

The INTO clause may contain a comma-separated list of host variables, a single host variable array, or a combination of both. If specified as a comma-separated list, the number of INTO clause host variables must exactly match the number of columns within the cursor's SELECT list or you will receive an SQLCODE -76 "Cardinality Mismatch" error when the statement is compiled.

Commonly, the INTO clause is specified in the DECLARE statement, not the FETCH statement. If both the SELECT query in the DECLARE statement and the FETCH statement contain an INTO clause, only the host variables specified by the DECLARE statement are set. If only the FETCH statement contain an INTO clause, the host variables specified by the FETCH statement are set.

If FETCH retrieves data, the SQLCODE variable is set to 0; if there is no data (or no more data) to FETCH, SQLCODE is set to 100 (No more data). Host variable values should only be used when SQLCODE=0.

Depending on the query, the first call to FETCH may perform additional tasks (such as sorting values within a temporary data structure).

7.5.4 The CLOSE Cursor Statement

The **CLOSE** statement terminates the execution of a cursor:

ObjectScript

```
&sql(CLOSE MyCursor)
```

The CLOSE statement cleans up any temporary storage used by the execution of a query. Programs that fail to call CLOSE will experience resource leaks (such as unneeded increase of the IRISTEMP temporary database).

Upon a successful call to CLOSE, the SQLCODE variable is set to 0. Therefore, before closing a cursor you should check whether the final FETCH set SQLCODE to 0 or 100.

7.6 Embedded SQL Variables

The following local variables have specialized uses in Embedded SQL. These local variable names are case-sensitive. At process initiation, these variables are undefined. They are set by Embedded SQL operations. They can also be set directly

using the **SET** command, or reset to undefined using the **NEW** command. Like any local variable, a value persists for the duration of the process or until set to another value or undefined using **NEW**. For example, some successful Embedded SQL operations do not set %ROWID; following these operations, %ROWID is undefined or remains set to its prior value.

- [%msg](#)
- [%ROWCOUNT](#)
- [%ROWID](#)
- [SQLCODE](#)

These local variables are not set by Dynamic SQL. (Note that the SQL Shell and the Management Portal SQL interface execute Dynamic SQL.) Instead, Dynamic SQL sets corresponding object properties.

The following ObjectScript special variables are used in Embedded SQL. These special variable names are not case-sensitive. At process initiation, these variables are initialized to a value. They are set by Embedded SQL operations. They cannot be set directly using the **SET** or **NEW** commands.

- [\\$TLEVEL](#)
- [\\$USERNAME](#)

As part of the defined InterSystems IRIS Embedded SQL interface, InterSystems IRIS may set any of these variables during Embedded SQL processing.

If the Embedded SQL is in a class method (with [ProcedureBlock=ON](#)), the system automatically places all of these variables in the [PublicList](#) and **NEW**s the SQLCODE, %ROWID, %ROWCOUNT, %msg, and all non-% variables used by the SQL statement. You can pass these variables by reference to/from the method; variables passed by reference will not be **NEW**ed automatically in the class method procedure block.

If the Embedded SQL is in a routine, it is the responsibility of the programmer to [NEW](#) the %msg, %ROWCOUNT, %ROWID, and SQLCODE variables before invoking Embedded SQL. **NEW**ing these variables prevents interference with prior settings of these variables. To avoid a <FRAMESTACK> error, you should not perform this **NEW** operation within an iteration cycle.

7.6.1 %msg

A variable that contains a system-supplied error message string. InterSystems SQL only sets %msg if it has set SQLCODE to a negative integer, indicating an error. If SQLCODE is set to 0 or 100, the %msg variable is unchanged from its prior value.

This behavior differs from the corresponding [Dynamic SQL %Message](#) property, which is set to the empty string when there is no current error.

In some cases, a specific SQLCODE error code may be associated with more than one %msg string, describing different conditions that generated the SQLCODE. %msg can also take a user-defined message string. This is most commonly used to issue a user-defined message from a trigger when trigger code explicitly sets [%ok=0](#) to abort the trigger.

An error message string is generated in the NLS language in effect for the process when the SQL code is executed. The SQL code may be compiled in a different NLS language environment; the message will be generated according to the runtime NLS environment. See *\$SYS.NLS.Locale.Language*.

7.6.2 %ROWCOUNT

An integer counter that indicates the number of rows affected by a particular statement.

- **INSERT**, **UPDATE**, **INSERT OR UPDATE**, and **DELETE** set %ROWCOUNT to the number of rows affected. An **INSERT** command with explicit values can only affect one row, and thus sets %ROWCOUNT to either 0 or 1.

An **INSERT** query results, an **UPDATE**, or a **DELETE** can affect multiple rows, and can thus set %ROWCOUNT to 0 or a positive integer.

- **TRUNCATE TABLE** always sets %ROWCOUNT to -1, regardless of how many rows were deleted or if any rows were deleted. Therefore, to determine the actual number of rows deleted, either perform a **COUNT(*)** on the table before **TRUNCATE TABLE**, or delete all the rows in the table using **DELETE**, rather than **TRUNCATE TABLE**.
- **SELECT** with no declared cursor can only act upon a single row, and thus execution of a simple **SELECT** always sets %ROWCOUNT to either 1 (single row that matched the selection criteria retrieved) or 0 (no rows matched the selection criteria).
- **DECLARE cursorname CURSOR FOR SELECT** does not initialize %ROWCOUNT; %ROWCOUNT is unchanged following the **SELECT**, and remains unchanged following **OPEN cursorname**. The first successful **FETCH** sets %ROWCOUNT. If no rows matched the query selection criteria, **FETCH** sets %ROWCOUNT=0; if **FETCH** retrieves a row that matched the query selection criteria, it sets %ROWCOUNT=1. Each subsequent **FETCH** that retrieves a row increments %ROWCOUNT. Upon **CLOSE** or when **FETCH** issues an SQLCODE 100 (No Data, or No More Data), %ROWCOUNT contains the total number of rows retrieved.

This **SELECT** behavior differs from the corresponding [Dynamic SQL](#) %ROWCOUNT property, which is set to 0 upon completion of query execution, and is only incremented when the program iterates through the result set returned by the query.

If a **SELECT** query returns only [aggregate functions](#), every **FETCH** sets %ROWCOUNT=1. The first **FETCH** always completes with SQLCODE=0, even when there is no data in the table; any subsequent **FETCH** completes with SQLCODE=100 and sets %ROWCOUNT=1.

The following Embedded SQL example declares a cursor and uses **FETCH** to fetch each row in the table. When the end of data is reached ([SQLCODE=100](#)) %ROWCOUNT contains the number of rows retrieved:

ObjectScript

```
SET name="LastName,FirstName",state="###"
&sql(DECLARE EmpCursor CURSOR FOR
    SELECT Name, Home_State
    INTO :name,:state FROM Sample.Person
    WHERE Home_State %STARTSWITH 'M')
WRITE !,"BEFORE: Name=",name," State=",state
&sql(OPEN EmpCursor)
QUIT:(SQLCODE'=0)
FOR { &sql(FETCH EmpCursor)
    QUIT:SQLCODE
    WRITE !,"Row fetch count: ",%ROWCOUNT
    WRITE " Name=",name," State=",state
}
WRITE !,"Final Fetch SQLCODE: ",SQLCODE
&sql(CLOSE EmpCursor)
WRITE !,"AFTER: Name=",name," State=",state
WRITE !,"Total rows fetched: ",%ROWCOUNT
```

The following Embedded SQL example performs an **UPDATE** and sets the number of rows affected by the change:

ObjectScript

```
&sql(UPDATE MyApp.Employee
    SET Salary = (Salary * 1.1)
    WHERE Salary < 50000)
IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg QUIT}
WRITE "Employees: ", %ROWCOUNT,!
```

Keep in mind that all Embedded SQL statements (within a given process) modify the %ROWCOUNT variable. If you need the value provided by %ROWCOUNT, be sure to get its value before executing additional Embedded SQL statements. Depending on how Embedded SQL is invoked, you may have to **NEW** the %ROWCOUNT variable before entering Embedded SQL.

Also note that *explicitly* rolling back a transaction will not affect the value of %ROWCOUNT. For example, the following will report that changes have been made, even though they have been rolled back:

ObjectScript

```
TSTART // start an explicit transaction
NEW SQLCODE,%ROWCOUNT,%ROWID
&sql(UPDATE MyApp.Employee
      SET Salary = (Salary * 1.1)
      WHERE Salary < 50000)
      IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg QUIT}

TROLLBACK // force a rollback; this will NOT modify %ROWCOUNT
Write "Employees: ", %ROWCOUNT,!
```

Implicit transactions (such as if an **UPDATE** fails a constraint check) are reflected by %ROWCOUNT.

7.6.3 %ROWID

When you initialize a process, %ROWID is undefined. When you issue a **NEW %ROWID** command, %ROWID is reset to undefined. %ROWID is set by the Embedded SQL operations described below. If the operation is not successful, or completes successfully but does not fetch or modify any rows, the %ROWID value remains unchanged from its prior value: either undefined, or set to a value by a previous Embedded SQL operation. For this reason, it is important to **NEW %ROWID** before each Embedded SQL operation.

%ROWID is set to the RowID of the last row affected by the following operations:

- **INSERT, UPDATE, INSERT OR UPDATE, or DELETE:** After a single-row operation, the %ROWID variable contains the system-assigned value of the RowID (Object ID) assigned to the inserted, updated, or deleted record. After a multiple-row operation, the %ROWID variable contains the system-assigned value of the RowID (Object ID) of the last record inserted, updated, or deleted. If no record is inserted, updated, or deleted, the %ROWID variable value is unchanged. **TRUNCATE TABLE** does not set %ROWID.
- **Cursor-based SELECT:** The **DECLARE cursorname CURSOR** and **OPEN cursorname** statements do not initialize %ROWID; the %ROWID value is unchanged from its prior value. The first successful **FETCH** sets %ROWID. Each subsequent **FETCH** that retrieves a row resets %ROWID to the current RowID value. **FETCH** sets %ROWID if it retrieves a row of an updateable cursor. An updateable cursor is one in which the top FROM clause contains exactly one element, either a single table name or an updateable view name. If the cursor is not updateable, %ROWID remains unchanged. If no rows matched the query selection criteria, **FETCH** does not change the prior the %ROWID value (if any). Upon **CLOSE** or when **FETCH** issues an SQLCODE 100 (No Data, or No More Data), %ROWID contains the RowID of the last row retrieved.

Cursor-based **SELECT** with a **DISTINCT** keyword or a **GROUP BY** clause does not set %ROWID. The %ROWID value is unchanged from its previous value (if any).

Cursor-based **SELECT** with an **aggregate function** does not set %ROWID if it returns only aggregate function values. If it returns both field values and aggregate function values, the %ROWID value for every **FETCH** is set to the RowID of the last row returned by the query.

- **SELECT** with no declared cursor does not set %ROWID. The %ROWID value is unchanged upon the completion of a simple **SELECT** statement.

In **Dynamic SQL**, the corresponding %ROWID property returns the RowID value of the last record inserted, updated, or deleted. Dynamic SQL does not return a %ROWID property value when performing a **SELECT** query.

You can retrieve the current %ROWID from ObjectScript using the following method call:

ObjectScript

```
WRITE $SYSTEM.SQL.GetROWID()
```


Following an **INSERT**, **UPDATE**, **DELETE**, **TRUNCATE TABLE**, or Cursor-based **SELECT** operation, the **LAST_IDENTITY** SQL function returns the value of the **IDENTITY field** for the most-recently modified record. If the table does not have an **IDENTITY** field, this function returns the RowID for the most-recently modified record.

7.6.4 SQLCODE

After running an embedded SQL Query, you must check the **SQLCODE** before processing the output host variables. In particular, you should always check **SQLCODE<0**; if this condition holds true, then there was error while processing the query and your application should respond accordingly.

If **SQLCODE=0** the query completed successfully and returned data. The output host variables contain field values.

If **SQLCODE=100** the query completed successfully, but output host variable values may differ. Either:

- The query returned one or more rows of data (**SQLCODE=0**), then reached the end of the data (**SQLCODE=100**), in which case output host variables are set to the field values of the last row returned. **%ROWCOUNT>0**.
- The query returned no data, in which case the output host variables are set to the null string. **%ROWCOUNT=0**.

After any invocation of the **&sql()** directive, you should check **SQLCODE<0**. If this condition holds true, then an error arose.

If a query returns only [aggregate functions](#), the first **FETCH** always completes with **SQLCODE=0** and **%ROWCOUNT=1**, even when there is no data in the table. The second **FETCH** completes with **SQLCODE=100** and **%ROWCOUNT=1**. If there is no data in the table or no data matches the query conditions, the query sets output host variables to 0 or the empty string, as appropriate.

If **SQLCODE** is a negative number the query failed with an error condition. For a list of these error codes and additional information, refer to [SQLCODE Values and Error Messages](#).

Depending on how Embedded SQL is invoked, you may have to [NEW](#) the **SQLCODE** variable before entering Embedded SQL. Within trigger code, setting **SQLCODE** to a nonzero value automatically sets **%ok=0**, aborting and rolling back the trigger operation.

In [Dynamic SQL](#), the corresponding **%SQLCODE** property returns SQL error code values.

7.6.5 \$TLEVEL

The transaction level counter. InterSystems SQL initializes **\$TLEVEL** to 0. If there is no current transaction, **\$TLEVEL** is 0.

- An initial [START TRANSACTION](#) sets **\$TLEVEL** to 1. Additional **START TRANSACTION** statements have no effect on **\$TLEVEL**.
- Each [SAVEPOINT](#) statement increments **\$TLEVEL** by 1.
- A [ROLLBACK TO SAVEPOINT pointname](#) statement decrements **\$TLEVEL**. The amount of decrement depends on the savepoint specified.
- A [COMMIT](#) resets **\$TLEVEL** to 0.
- A [ROLLBACK](#) resets **\$TLEVEL** to 0.

You can also use the [%INTRANSACTION](#) statement to determine if a transaction is in progress.

\$TLEVEL is also set by ObjectScript transaction commands. For further details, see [\\$TLEVEL](#).

7.6.6 \$USERNAME

The SQL username is the same as the InterSystems IRIS username, stored in the ObjectScript `$USERNAME` special variable. The username can be used as the [system-wide default schema](#) or as an element in the [schema search path](#).

7.7 Embedded SQL in Methods of a Persistent Class

The following example shows a persistent class containing a class method and an instance method, both of which contain Embedded SQL:

Class Definition

```
Class Sample.MyClass Extends %Persistent [DdlAllowed]
{
  ClassMethod NameInitial(Myval As %String) As %String [SqlProc]
  {
    &sql(SELECT Name INTO :n FROM Sample.Stuff WHERE Name %STARTSWITH :Myval)
    IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE RETURN %msg}
    ELSEIF SQLCODE=100 {WRITE "Query returns no results" RETURN}
    WRITE "Hello " RETURN n
  }
  Method CountRows() As %Integer
  {
    &sql(SELECT COUNT(*) INTO :count FROM Sample.Stuff)
    IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE RETURN %msg}
    ELSEIF SQLCODE=100 {WRITE "Query returns no results" RETURN}
    WRITE "Number of rows is " RETURN count
  }
}
```

The class method is invoked as follows:

ObjectScript

```
WRITE ##class(Sample.MyClass).NameInitial("G")
```

The instance method is invoked as follows:

ObjectScript

```
SET x=##class(Sample.MyClass).%New()
WRITE x.CountRows()
```

SQL entities such as tables and fields do not have to exist for these methods to successfully compile. Because checking for the existence of SQL entities is performed at runtime, Embedded SQL methods should contain SQLCODE test logic.

You can test for the existence of SQL entities specified in Embedded SQL without executing the code. This is described in [Validating Embedded SQL Code](#).

7.8 Validating Embedded SQL Code

You can validate Embedded SQL code without executing the code in three ways:

- Compiling a routine containing Embedded SQL code using the `/compileembedded=1` qualifier.
- Compiling multiple Embedded SQL routines using the `$$SYSTEM.OBJ.GenerateEmbedded()` method.
- Testing the Embedded SQL code using the Management Portal SQL interface **Show Plan** option.

7.8.1 Compile with /compileembedded Qualifier

You can validate Embedded SQL code by using the compile class methods of the `$SYSTEM.OBJ` class and specifying in the *qspec* argument the `/compileembedded=1` qualifier. The `/compileembedded` default is 0.

- `$SYSTEM.OBJ.Compile()` compiles the specified class and all routines within that class.
- `$SYSTEM.OBJ.CompileList()` compiles a list of specified classes and all routines within those classes.
- `$SYSTEM.OBJ.CompilePackage()` compiles all classes/routines in the specified package (schema).
- `$SYSTEM.OBJ.CompileAll()` compiles all classes/routines in the current namespace.
- `$SYSTEM.OBJ.CompileAllNamespaces()` compiles all classes/routines in all namespaces.

You can specify use of the `/compileembedded=1` qualifier by default using the `SetQualifiers()` method from the Terminal:

Terminal

```
USER>DO $SYSTEM.OBJ.SetQualifiers("/compileembedded=1") /* sets /compileembedded for current namespace */
```

Terminal

```
USER>DO $SYSTEM.OBJ.SetQualifiers("/compileembedded=1",1) /* sets /compileembedded for all namespaces */
```

To display a list of *qspec* qualifiers, including `/compileembedded`, invoke:

Terminal

```
USER>DO $SYSTEM.OBJ.ShowQualifiers()
```

The non-default qualifier settings are shown at the end of the `ShowQualifiers()` display.

7.8.2 Test with Show Plan

You can use the Management Portal SQL interface to validate Embedded SQL code without executing the code. This operation both validates the SQL syntax and checks for the existence of the specified SQL entities.

From the Management Portal **System Explorer** option select the **SQL** option to display the Execute Query code area.

1. Input your Embedded SQL query. For example `SELECT Name INTO :n FROM Sample.MyTest` or `DECLARE MyCursor CURSOR FOR SELECT Name, Age INTO :n, :a FROM Sample.MyTest WHERE Age > 21 FOR READ ONLY`.
2. Press the **Show Plan** button to check the code. If the code is valid, **Show Plan** displays a Query Plan. If the code is invalid, **Show Plan** displays an SQLCODE error value and message.

Note that **Show Plan** validation will not issue an error if the `INTO` clause is missing, because the `INTO` clause may be specified in the `FETCH` statement. **Show Plan** will issue appropriate errors if the `INTO` clause contains an error or is in the wrong location.

You cannot use the **Execute** button to execute Embedded SQL code.

7.9 Auditing Embedded SQL

InterSystems IRIS supports optional [auditing](#) of Embedded SQL statements. Embedded SQL auditing is performed when the following two requirements are met:

1. The %System/%SQL/EmbeddedStatement [system audit event](#) is enabled system-wide. By default, this system audit event is not enabled. To enable, go to Management Portal, **System Administration**, select **Security**, then **Auditing**, then **Configure System Events**.
2. The routine containing the Embedded SQL statement must contain the [#sqlcompile audit](#) macro preprocessor directive. If this directive is set to ON, any Embedded SQL statement following it in the compiled routine is audited when executed.

Auditing records information in the Audit Database. To view the Audit Database, go to the Management Portal, **System Administration**, select **Security**, then **Auditing**, then **View Audit Database**. You can set the **Event Name** filter to Embedded-Statement to limit the **View Audit Database** to Embedded SQL statements. The Audit Database lists Time (a local timestamp), User, PID (process ID), and the Description, which specifies the type of Embedded SQL statement. For example, SQL SELECT Statement.

By selecting the **Details** link for an event you can list additional information, including the **Event Data**. The Event Data includes the SQL statement executed and the values of any input arguments to the statement. For example:

```
SELECT TOP :n Name,ColorPreference INTO :name,:color FROM Sample.Stuff WHERE Name %STARTSWITH :letter
Parameter values:
n=5
letter="F"
```

InterSystems IRIS also supports auditing of Dynamic SQL statements (**Event Name**=DynamicStatement) and ODBC and JDBC statements (**Event Name**=XDBCStatement).

8

Using Dynamic SQL

This topic discusses Dynamic SQL, queries and other SQL statements that are prepared and executed at runtime from InterSystems IRIS® data platform.

This topic describes Dynamic SQL programming using the %SQL.Statement class, which is the preferred implementation of Dynamic SQL. All statements about Dynamic SQL here, and throughout our documentation, refer specifically to the %SQL.Statement implementation.

8.1 Introduction to Dynamic SQL

Dynamic SQL refers to SQL statements that are prepared and executed at runtime. In Dynamic SQL preparing and executing an SQL command are separate operations. Dynamic SQL lets you program within InterSystems IRIS in a manner similar to an ODBC or JDBC application (except that you are executing the SQL statement within the same process context as the database engine). Dynamic SQL is invoked from an ObjectScript program.

Dynamic SQL queries are prepared at program execution time, not compilation time. This means that the compiler cannot check for errors at compilation time and preprocessor macros cannot be used within Dynamic SQL. It also means that executing programs can create specialized Dynamic SQL queries in response to user or other input.

Dynamic SQL can be used to perform an SQL query. It can also be used to issue other SQL statements. The examples here perform a **SELECT** query. For Dynamic SQL program examples, see [CREATE TABLE](#), [INSERT](#), [UPDATE](#), [DELETE](#), and [CALL](#).

Dynamic SQL is used in the execution of the InterSystems IRIS [SQL Shell](#), the InterSystems IRIS Management Portal [Execute Query interface](#), the [SQL Code Import methods](#), and the [Data Import and Export Utilities](#).

No row can exceed the [string length limit](#).

8.1.1 Dynamic SQL versus Embedded SQL

Dynamic SQL differs from Embedded SQL in the following ways:

- Initial execution of a Dynamic SQL query is slightly less efficient than Embedded SQL, because it does not generate in-line code for queries. However, re-execution of both Dynamic SQL and Embedded SQL is substantially faster than the first execution of the query because both support cached queries.
- Dynamic SQL can accept a literal value input to a query in two ways: input parameters specified using the “?” character, and input host variables (for example, :var). Embedded SQL uses input and output host variables (for example, :var).

- Dynamic SQL output values are retrieved using the API of the result set object (that is, the Data property). Embedded SQL uses host variables (for example, :var) with the **INTO** clause of a **SELECT** statement to output values.
- Dynamic SQL sets the %SQLCODE, %Message, %ROWCOUNT, and %ROWID object properties. Embedded SQL sets the corresponding SQLCODE, %msg, %ROWCOUNT, and %ROWID local variables. Dynamic SQL does not set %ROWID for a **SELECT** query; Embedded SQL sets %ROWID for a cursor-based **SELECT** query.
- Dynamic SQL provides an easy way to find query metadata (such as quantity and names of columns).
- Dynamic SQL performs SQL privilege checking by default; you must have the appropriate privileges to access or modify a table, field, etc. Embedded SQL does not perform SQL privilege checking. Refer to the SQL [%CHECKPRIV](#) statement for further details.
- Dynamic SQL cannot access a private class method. To access an existing class method, the method must be made public. This is a general SQL limitation. However, Embedded SQL gets around this limitation because the Embedded SQL operation itself is a method of the same class.

Dynamic SQL and Embedded SQL use the same data representation (logical mode by default, but this can be changed) and NULL handling.

8.2 The %SQL.Statement Class

The preferred interface for Dynamic SQL is the %SQL.Statement class. To prepare and execute Dynamic SQL statements, use an instance of %SQL.Statement. The result of executing a Dynamic SQL statement is an SQL statement result object that is an instance of the %SQL.StatementResult class. An SQL statement result object is either a unitary value, a result set, or a context object. In all cases, the result object supports a standard interface. Each result object initializes the %SQLCODE, %Message and other result object properties. The values these properties are set to depends on the SQL statement issued. For a successfully executed **SELECT** statement, the object is a result set (specifically, an instance of %SQL.StatementResult) and supports the expected result set functionality.

The following ObjectScript code prepares and executes a Dynamic SQL query:

ObjectScript

```
/* Simple %SQL.Statement example */
set myquery = "SELECT TOP 5 Name,DOB FROM Sample.Person"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(myquery)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
set rset = tStatement.%Execute()
do rset.%Display()
write !,"End of data"
```

The examples on this page use methods associated with the %SQL.Statement and %SQL.StatementResult classes.

8.3 Creating an Object Instance

You can create an instance of the %SQL.Statement class using the **%New()** class method:

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New()
```

At this point the result set object is ready to prepare an SQL statement. Once you have created an instance of the %SQL.Statement class, you can use that instance to issue multiple Dynamic SQL queries and/or **INSERT**, **UPDATE**, or **DELETE** operations.

%New() accepts three optional comma-separated parameters in the following order:

1. %SelectMode, which specifies the mode used for data input and data display.
2. %SchemaPath, which specifies the search path used to supply the schema name for an unqualified table name.
3. %Dialect, which specifies the Transact-SQL (TSQL) Sybase or MSSQL dialect. The default is IRIS (InterSystems SQL).

There is also an %ObjectSelectMode property, which cannot be set as a %New() parameter. %ObjectSelectMode specifies the data type binding of fields to their related object properties.

In the following ObjectScript example, the %SelectMode is 2 (Display mode), and the %SchemaPath specifies "Sample" as the default schema:

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New(2, "Sample")
```

In the following ObjectScript example, a %SelectMode is not specified (note the placeholder comma), and the %SchemaPath specifies a schema search path containing three schema names:

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New(, "MyTests, Sample, Cinema")
```

8.3.1 %SelectMode Property

The %SelectMode property specifies one of the following modes: 0=Logical (the default), 1=ODBC, 2=Display. These modes specify how a data value is input and displayed. A mode is most commonly used for date and time values and for displaying %List data (a string containing an encoded list). Data is stored in Logical mode.

A **SELECT** query uses the %SelectMode value to determine the format used to display data.

An **INSERT** or **UPDATE** operation uses the %SelectMode value to determine the permitted format(s) for data input.

%SelectMode is used for data display. SQL statements run internally in Logical mode. For example, an **ORDER BY** clause orders records based on their Logical values, regardless of the %SelectMode setting. SQL functions use Logical values, regardless of the %SelectMode setting. Methods projected as SQLPROC also run in Logical mode. SQL routines called as functions in an SQL statement need to return the function value in Logical format.

- For a **SELECT** query, %SelectMode specifies the format used for displaying the data. Setting %SelectMode to ODBC or Display also affects the data format used for specifying comparison predicate values. Some predicate values must be specified in the %SelectMode format, other predicate values must be specified in Logical format, regardless of the %SelectMode. For details, refer to [Overview of SQL Predicates](#).
 - Time data type data in %SelectMode=1 (ODBC) can display fractional seconds, which is not the same as actual ODBC time. The InterSystems IRIS Time data type supports fractional seconds. The corresponding ODBC TIME data type (TIME_STRUCT standard header definition) does not support fractional seconds. The ODBC TIME data type truncates a supplied time value to whole seconds. ADO DotNet and JDBC do not have this restriction.
 - %List data type data in %SelectMode=0 (Logical) does not display the internal storage value, because %List data is encoded using non-printing characters. Instead, Dynamic SQL displays a %List data value as a \$LISTBUILD statement, such as the following: \$lb("White", "Green"). See [%Print\(\) Method](#) for an example. %List data

type data in %SelectMode=1 (ODBC) displays list elements separated by commas. %List data type data in %SelectMode=2 (Display) displays list elements separated by \$CHAR(10, 13) (Line Feed, Carriage Return).

- For an **INSERT** or **UPDATE** operation, %SelectMode specifies the format for input data that will be converted to Logical storage format. For this data conversion to occur, the SQL code must have been compiled with a select mode of RUNTIME (the default) so that a Display or ODBC %SelectMode is used when the **INSERT** or **UPDATE** is executed. For permitted input values for dates and times, refer to the [date and time data types](#). For further details, see [INSERT](#) or [UPDATE](#).

You can specify %SelectMode either as the first parameter of the %New() class method, or set it directly, as shown in the following two examples:

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New(2)
```

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New()  
set tStatement.%SelectMode=2
```

The following example returns the current value of %SelectMode:

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New()  
write !,"default select mode=",tStatement.%SelectMode  
set tStatement.%SelectMode=2  
write !,"set select mode=",tStatement.%SelectMode
```

You can determine the SelectMode default setting for the current process using the **\$SYSTEM.SQL.Util.GetOption("SelectMode")** method. You can change the SelectMode default setting for the current process using the **\$SYSTEM.SQL.Util.SetOption("SelectMode",n)** method, when *n* can be 0=Logical, 1=ODBC, or 2=Display. Setting %SelectMode overrides this default for the current object instance; it does not change the SelectMode process default.

For further details on SelectMode options, see [Data Display Options](#).

8.3.2 %SchemaPath Property

The %SchemaPath property specifies the search path used to supply the schema name for an unqualified table name, view name, or stored procedure name. A schema search path is used for data management operations such as **SELECT**, **CALL**, **INSERT**, and **TRUNCATE TABLE**; it is ignored by data definition operations such as **DROP TABLE**.

The search path is specified as a quoted string containing a schema name or a comma-separated series of schema names. InterSystems IRIS searches the listed schemas in left-to-right order. InterSystems IRIS searches each specified schema until it locates the first matching table, view, or stored procedure name. Because schemas are searched in the specified order, there is no detection of ambiguous table names. Only schema names in the current namespace are searched.

The schema search path can contain both literal schema names and the CURRENT_PATH, CURRENT_SCHEMA, and DEFAULT_SCHEMA keywords.

- CURRENT_PATH specifies the current schema search path, as defined in a prior %SchemaPath property. This is commonly used to add schemas to the beginning or end of an existing schema search path.
- CURRENT_SCHEMA specifies the current schema container class name if the %SQL.Statement call is made from within a class method. If a [#sqlcompile path](#) macro directive is defined in a class method, the CURRENT_SCHEMA is the schema mapped to the current class package. Otherwise, CURRENT_SCHEMA is the same as DEFAULT_SCHEMA.

- `DEFAULT_SCHEMA` specifies the [system-wide default schema](#). This keyword enables you to search the system-wide default schema as a item within the schema search path, before searching other listed schemas. The system-wide default schema is always searched after searching the schema search path if all the schemas specified in the path have been searched without a match.

The `%SchemaPath` is the first place InterSystems IRIS searches schemas for a matching table name. If `%SchemaPath` is not specified, or does not list a schema that contains a matching table name, InterSystems IRIS uses the [system-wide default schema](#).

You can specify a schema search path either by specifying the `%SchemaPath` property, or by specifying the second parameter of the `%New()` class method, as shown in the following two examples:

ObjectScript

```
set path="MyTests,Sample,Cinema"
set tStatement = ##class(%SQL.Statement).%New(,path)
```

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New()
set tStatement.%SchemaPath="MyTests,Sample,Cinema"
```

You can set `%SchemaPath` at any point prior to the `%Prepare()` method which uses it.

The following example returns the current value of `%SchemaPath`:

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New()
write !,"default path=",tStatement.%SchemaPath
set tStatement.%SchemaPath="MyTests,Sample,Cinema"
write !,"set path=",tStatement.%SchemaPath
```

You can use the `%ClassPath()` method to set `%SchemaPath` to the search path defined for the specified class name:

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New()
set tStatement.%SchemaPath=tStatement.%ClassPath("Sample.Person")
write tStatement.%SchemaPath
```

8.3.3 %Dialect Property

The `%Dialect` property specifies the SQL statement dialect. You can specify Sybase, MSSQL, or IRIS (InterSystems SQL). The Sybase or MSSQL setting causes the SQL statement to be processed using the specified Transact-SQL dialect.

The Sybase and MSSQL dialects support a limited subset of SQL statements in these dialects. They support the **SELECT**, **INSERT**, **UPDATE**, **DELETE**, and **EXECUTE** statements. They support the **CREATE TABLE** statement for permanent tables, but not for temporary tables. **CREATE VIEW** is supported. **CREATE TRIGGER** and **DROP TRIGGER** are supported. However, this implementation does not support transaction rollback should the **CREATE TRIGGER** statement partially succeed but then fail on class compile. **CREATE PROCEDURE** and **CREATE FUNCTION** are supported.

The Sybase and MSSQL dialects support the **IF** flow-of-control statement. This command is not supported in the IRIS (InterSystems SQL) dialect.

The default is InterSystems SQL, represented by an empty string (""), or specified as "IRIS"

You can specify `%Dialect` either as the third parameter of the `%New()` class method, or set it directly as a property, or set it using a method, as shown in the following three examples:

Setting `%Dialect` in `%New()` class method:

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New(,,"Sybase")
write "language mode set to=",tStatement.%Dialect
```

Setting the %Dialect property directly:

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New()
set defaultdialect=tStatement.%Dialect
write "default language mode=",defaultdialect,!
set tStatement.%Dialect="Sybase"
write "language mode set to=",tStatement.%Dialect,!
set tStatement.%Dialect="IRIS"
write "language mode reset to default=",tStatement.%Dialect,!
```

Setting the %Dialect property using the **%DialectSet()** instance method, which returns an error status:

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New()
set tStatus = tStatement.%DialectSet("Sybase")
if tStatus'=1 {write "%DialectSet failed:" do $System.Status.DisplayError(tStatus) quit}
write "language mode set to=",tStatement.%Dialect
```

The **%DialectSet()** method returns a **%Status** value: Success returns a status of 1. Failure returns an object expression that begins with 0, followed by encoded error information. For this reason, you cannot perform a `tStatus=0` test for failure; you can perform a `$$$ISOK(tStatus)=0` [macro](#) test for failure.

8.3.4 %ObjectSelectMode Property

The **%ObjectSelectMode** property is a boolean value. If **%ObjectSelectMode=0** (the default) all columns in the SELECT list are bound to properties with literal types in the result set. If **%ObjectSelectMode=1** then columns in the SELECT list are bound to properties with the type defined in the associated property definition.

%ObjectSelectMode allows you to specify how columns whose type class is a swizzleable class will be defined in the result set class generated from a SELECT statement. If **%ObjectSelectMode=0** the property corresponding to the swizzleable column will be defined in result sets as a simple literal type corresponding to the SQL table's RowID type. If **%ObjectSelectMode=1** the property will be defined with the column's declared type. That means that accessing the result set property will trigger [swizzling](#).

%ObjectSelectMode cannot be set as a parameter of **%New()**.

The following example returns the **%ObjectSelectMode** default value, sets **%ObjectSelectMode**, then returns the new **%ObjectSelectMode** value:

ObjectScript

```
set myquery = "SELECT TOP 5 %ID AS MyID,Name,Age FROM Sample.Person"
set tStatement = ##class(%SQL.Statement).%New()
write !,"default ObjectSelectMode=",tStatement.%ObjectSelectMode
set tStatement.%ObjectSelectMode=1
write !,"set ObjectSelectMode=",tStatement.%ObjectSelectMode
```

%ObjectSelectMode=1 is principally used when returning values from a result set using the field name property. This is further described with examples in [Fieldname Property](#).

%ObjectSelectMode=1 can be used when a field in the SELECT list is linked to a collection property. **%ObjectSelectMode** will [swizzle](#) the collection. If **%SelectMode = 1** or **2**, the system converts the collection serial value into Logical mode form before swizzling. The resulting OREF supports the full collection interface.

8.4 Preparing an SQL Statement

Preparing an SQL statement validates the statement, prepares it for subsequent execution, and generates metadata about the SQL statement.

There are three ways to prepare an SQL statement using the %SQL.Statement class:

- **%Prepare()**, which prepares an SQL statement (a query, for example) for a subsequent **%Execute()**.
- **%PrepareClassQuery()**, which prepares a call statement to an existing query. Once prepared, this query can be executed using a subsequent **%Execute()**.
- **%ExecDirect()**, which both prepares and executes an SQL statement. **%ExecDirect()** is described in “Executing an SQL Statement”.
- **%ExecDirectNoPriv()**, which prepares and executes an SQL statement and does not perform privilege checking. **%ExecDirectNoPriv()** is described in “Executing an SQL Statement”.

You can also prepare an SQL statement without creating an object instance by using the **\$\$SYSTEM.SQL.Prepare()** method. The **Prepare()** method is shown in the following Terminal example:

Terminal

```
USER>set topnum=5
USER>set prep=$$SYSTEM.SQL.Prepare("SELECT TOP :topnum Name, Age FROM Sample.Person WHERE Age=?")
USER>do prep.%Display()
```

Preparing an SQL statement creates a cached query. Using a cached query allows the same SQL query to be executed multiple times without the need to re-prepare the SQL statement. A cached query can be executed one or more times by any process; it can be executed with different input parameter values.

Each time you prepare an SQL statement, InterSystems IRIS searches the query cache to determine if the same SQL statement has already been prepared and cached. (Two SQL statements are considered “the same” if they differ only in the values of literals and input parameters.) If the prepared statement does not already exist in the query cache, InterSystems IRIS creates a cached query. If the prepared statement already exists in the query cache, no new cached query is created. For this reason, it is important not to code a prepare statement within a loop structure.

8.4.1 %Prepare()

You can prepare an SQL statement using the **%Prepare()** instance method of the %SQL.Statement class. The **%Prepare()** method takes, as its first argument, the SQL statement. This can be specified as a quoted string or a variable that resolves to a quoted string, as shown in the following example:

ObjectScript

```
set qStatus = tStatement.%Prepare("SELECT Name, Age FROM Sample.Person")
```

More complex queries can be specified using a subscripted array passed by reference, as shown in the following example:

ObjectScript

```
set myquery = 3
set myquery(1) = "SELECT %ID AS id, Name, DOB, Home_State"
set myquery(2) = "FROM Person WHERE Age > 80"
set myquery(3) = "ORDER BY 2"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(.myquery)
```

A query can contain [duplicate field names](#) and [field name aliases](#).

A query supplied to **%Prepare()** can contain [input host variables](#), as shown in the following example:

ObjectScript

```
set minage = 80
set myquery = 3
set myquery(1) = "SELECT %ID AS id, Name, DOB, Home_State"
set myquery(2) = "FROM Person WHERE Age > :minage"
set myquery(3) = "ORDER BY 2"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(.myquery)
```

InterSystems IRIS substitutes the defined literal value for each input host variable when the SQL statement is executed. Note however, that if this code is called as a method, the *minage* variable must be made Public. By default, methods are ProcedureBlocks; this means that a method (such as **%Prepare()**) cannot see variables defined by its caller. You can either override this default by specifying the class as [[Not ProcedureBlock](#)], specifying the method as [[ProcedureBlock = 0](#)], or by specifying [[PublicList = minage](#)].

Note: It is good program practice to always confirm that an input variable contains an appropriate value before inserting it into SQL code.

You can also supply literal values to a query using [? input parameters](#). InterSystems IRIS substitutes a literal value for each ? input parameter using the corresponding parameter value you supply to the **%Execute()** method. Following a **%Prepare()**, you can use the **%GetImplementationDetails()** method to list the input host variables and the ? input parameters in the query.

The **%Prepare()** method returns a [%Status](#) value: Success returns a status of 1 (the query string is valid; referenced tables exist in the current namespace). Failure returns an object expression that begins with 0, followed by encoded error information. For this reason, you cannot perform a `status=0` test for failure; you can perform a `$$$ISOK(status)=0` [macro](#) test for failure.

The **%Prepare()** method uses the `%SchemaPath` property defined earlier to resolve unqualified names.

Note: Dynamic SQL performance can be significantly improved by using fully qualified names whenever possible.

You can specify [input parameters](#) in the SQL statement by using the “?” character:

ObjectScript

```
set myquery="SELECT TOP ? Name, Age FROM Sample.Person WHERE Age > ?"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(myquery)
```

You specify the value for each ? input parameter in the **%Execute()** instance method when you execute the query. An input parameter must take a literal value or an expression that resolves to a literal value. An input parameter cannot take a field name value or a field name alias. An input parameter must be declared PUBLIC for a **SELECT** statement to reference it directly.

A query can contain field aliases. In this case, the Data property accesses the data using the alias, not the field name.

You are not limited to **SELECT** statements within Dynamic SQL: you can use the **%Prepare()** instance method to prepare other SQL statements, including the **CALL**, **INSERT**, **UPDATE**, and **DELETE** statements.

You can display information about the currently prepared statement using the **%Display()** instance method, as shown in the following example:

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New(,"Sample")
set myquery = 3
set myquery(1) = "SELECT TOP ? Name,DOB,Home_State"
set myquery(2) = "FROM Person"
set myquery(3) = "WHERE Age > 60 AND Age < 65"
set qStatus = tStatement.%Prepare(.myquery)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
do tStatement.%Display()
write !,"End of %Prepare display"
```

This information consists of the Implementation Class, the Arguments (a comma-separated list of the actual arguments, either literal values or ? input parameters), and the Statement Text.

%Prepare() takes an optional second argument, `checkPriv`, which is a logical value that determines whether InterSystems IRIS checks privileges on the statement. If `checkPriv` is 0, no privileges are checked. Disabling privilege checking gives applications more control over the execution of dynamic queries but increases security risk. The default value is 1 (privileges are checked). For example:

ObjectScript

```
set statement = ##class(%SQL.Statement).%New()
set status =statement.%Prepare("DELETE FROM T",0) // No privileges checked

set statement2 = ##class(%SQL.Statement).%New()
set status =statement2.%Prepare("DELETE FROM T") // Privilege is checked
```

8.4.2 %PrepareClassQuery()

You can prepare an existing SQL query using the **%PrepareClassQuery()** instance method of the `%SQL.Statement` class. The **%PrepareClassQuery()** method takes three parameters: the class name of the existing query, the query name, and an optional third argument that determines if privileges should be checked (if omitted, privileges are checked). The class name and the query name are specified as a quoted string or a variable that resolves to a quoted string, and the privilege checking option is specified as 0 or 1, as shown in the following example:

ObjectScript

```
set qStatus = tStatement.%PrepareClassQuery("User.queryDocTest","DocTest", 0) // No privileges checked
set qStatus2 = tStatement.%PrepareClassQuery("User.queryDocTest","DocTest", 1) // Privilege is checked
set qStatus3 = tStatement.%PrepareClassQuery("User.queryDocTest","DocTest") // Privilege is checked
```

The **%PrepareClassQuery()** method returns a `%Status` value: Success returns a status of 1. Failure returns an object expression that begins with 0, followed by encoded error information. For this reason, you cannot perform a `qStatus=0` test for failure; you can perform a `$$$ISOK(qStatus)=0` [macro](#) test for failure.

The **%PrepareClassQuery()** method uses the `%SchemaPath` property defined earlier to resolve unqualified names.

%PrepareClassQuery() executes using a [CALL](#) statement. Because of this, the executed class query must have an [SqlProc](#) parameter.

The following example shows **%PrepareClassQuery()** invoking the `ByName` query defined in the `Sample.Person` class, passing a string to limit the names returned to those that start with that string value:

ObjectScript

```
set statemt=##class(%SQL.Statement).%New()
set cqStatus=statemt.%PrepareClassQuery("Sample.Person","ByName")
if cqStatus'=1 {write "%PrepareClassQuery failed:" do $System.Status.DisplayError(cqStatus) quit}
set rset=statemt.%Execute("L")
do rset.%Display()
```

The following example shows **%PrepareClassQuery()** invoking an existing query:

ObjectScript

```

set tStatement=##class(%SQL.Statement).%New()
set cqStatus=tStatement.%PrepareClassQuery("%SYS.GlobalQuery","Size")
if cqStatus'=1 {write "%PrepareClassQuery failed:" do $System.Status.DisplayError(cqStatus) quit}

set install=$SYSTEM.Util.DataDirectory()
set rset=tStatement.%Execute(install_"mgr\User")
do rset.%Display()

```

The following example shows **%Prepare()** preparing a CREATE QUERY statement, and then **%PrepareClassQuery()** invoking this class query:

ObjectScript

```

/* Creating the Query */
set query=4
set query(1)="CREATE QUERY DocTest() SELECTMODE RUNTIME PROCEDURE "
set query(2)="BEGIN "
set query(3)="SELECT TOP 5 Name,Home_State FROM Sample.Person ; "
set query(4)="END"

set statement = ##class(%SQL.Statement).%New()
set qStatus = statement.%Prepare(.query)
if qStatus '= 1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE '= 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
write !,"Created a query",!

/* Calling the Query */
write !,"Calling a class query..."
set cqStatus = statement.%PrepareClassQuery("User.queryDocTest","DocTest")
if cqStatus '= 1 {write "%PrepareClassQuery failed:" do $SYSTEM.Status.DisplayError(cqStatus) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE '= 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

write "Query data:",!,!
while rset.%Next()
{
do rset.%Print()
}
if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
write !,"End of data."

/* Deleting the Query */
&sql(DROP QUERY DocTest)
if SQLCODE < 0 {write !,"Error deleting query:", SQLCODE, " ", %msg quit}
write !,"Deleted the query."

```

To display a row of data retrieved by a stored query you can use the **%Print()** method, as shown in this example. To display specific column data that was retrieved by a stored query you must use either the **%Get("fieldname")** or the **%GetData(colnum)** method. See [Iterating through a Result Set](#).

If the query is defined to accept arguments, you can specify input parameters in the SQL statement by using the “?” character. You specify the value for each ? input parameter in the **%Execute()** method when you execute the query. An input parameter must be declared PUBLIC for a **SELECT** statement to reference it directly.

You can display information about the currently prepared query using the **%Display()** method, as shown in the following example:

ObjectScript

```

/* Creating the Query */
set myquery=4
set myquery(1)="CREATE QUERY DocTest() SELECTMODE RUNTIME PROCEDURE "
set myquery(2)="BEGIN "
set myquery(3)="SELECT TOP 5 Name,Home_State FROM Sample.Person ; "
set myquery(4)="END"

set statement = ##class(%SQL.Statement).%New()

```

```

set qStatus = statement.%Prepare(.myquery)
if qStatus != 1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE != 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
write !,"Created a query",!

/* Preparing and Displaying Info about the Query */
write !,"Preparing a class query..."
set cqStatus = statement.%PrepareClassQuery("User.queryDocTest","DocTest")
if cqStatus != 1 {write "%PrepareClassQuery failed:" do $SYSTEM.Status.DisplayError(cqStatus) quit}

do statement.%Display()
write !,"End of %Prepare display"

/* Deleting the Query */
&sql(DROP QUERY DocTest)
if SQLCODE < 0 {write !,"Error Deleting query:",SQLCODE," ",%msg quit }
write !,"Deleted the query"

```

This information consists of the Implementation Class, the Arguments (a comma-separated list of the actual arguments, either literal values or ? input parameters), and the Statement Text.

For further details, see [Defining and Using Class Queries](#).

8.4.3 Results of a Successful Prepare

Following a successful prepare (**%Prepare()**, **%PrepareClassQuery()**, or **%ExecDirect()**) you can invoke the **%SQL.Statement %Display()** instance method or **%GetImplementationDetails()** instance method to return the details of the currently prepared statement. For example:

%Display():

ObjectScript

```

set myquery = "SELECT TOP 5 Name,Age FROM Sample.Person WHERE Age > 21"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(myquery)
if qStatus!=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
do tStatement.%Display()
set rset = tStatement.%Execute()

```

%GetImplementationDetails():

ObjectScript

```

set myquery = "SELECT TOP ? Name,Age FROM Sample.Person WHERE Age > 21 AND Name=:fname"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(myquery)
if qStatus!=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
set bool = tStatement.%GetImplementationDetails(.pclassname,.ptext,.pargs)
if bool=1 {write "Implementation class= ",pclassname,!
           write "Statement text= ",ptext,!
           write "Arguments= ",listtostring(pargs,! ) // returns "? ,?,c,21,v,fname"
           else {write "%GetImplementationDetails() failed",!}
set rset = tStatement.%Execute()

```

These methods provide the following information:

- **Implementation class:** the class name corresponding to the cached query. For example: %sqlcq.SAMPLES.cls49.
- **Arguments:** A list of the query arguments in the order specified. If an argument is enclosed in double parentheses to suppress literal substitution the argument is not included in the argument list.

%Display() displays a comma-separated list of the query arguments. Each argument can be a literal value, the name of an [input host variables](#) (without the colon), or a question mark (?) for an [input parameter](#). If there are no arguments, this item displays <<none>>. A predicate that specifies multiple values, such as **IN** or **%INLIST** lists each value as a separate argument.

%GetImplementationDetails() returns the query arguments as a %List structure. Each argument is represented by a pair of elements, a type and a value: Type c (constant) is followed by a literal value; Type v (variable) is followed by the name of an [input host variable](#) (without the colon); Type ? is an [input parameter](#), and is followed by a second question mark. If there are no arguments, the arguments list is an empty string. A predicate that specifies multiple values, such as **IN** or **%INLIST** lists each value as a separate type and value pair.

- **Statement Text:** the query text, exactly as specified. Letter case is preserved, host variables and input parameters are shown as written, the default schema is not shown. For **%Prepare()** for example, `SELECT TOP :n Name FROM Clients`. For **%PrepareClassQuery()** for example, call `Sample.SP_Sample_By_Name(?)`.

For other metadata information generated for a prepared query, refer to [SQL Metadata](#).

8.4.4 The `prepare()` Method

You can use the **prepare()** method to return a %List structure of the query arguments without having to prepare the SQL query. The query arguments are returned in the [same format](#) as **%GetImplementationDetails()**.

The **prepare()** method also returns the query text. However, unlike **%Display()** and **%GetImplementationDetails()** which return the query text exactly as specified, the **prepare()** method replaces each query argument with a ? character, removes comments, and normalizes whitespace. It does not supply a default schema name. The **prepare()** method in the following example returns a parsed version of the query text and a %List structure of the query arguments:

ObjectScript

```
set myq=2
set myq(1)="SELECT TOP ? Name /* first name */, Age "
set myq(2)="FROM Sample.MyTable WHERE Name='Fred' AND Age > :years -- end of query"
do ##class(%SQL.Statement).prepare(.myq,.stripped,.args)
write "prepared query text: ",stripped,!
write "arguments list: ",$listtostring(args)
```

8.5 Executing an SQL Statement

There are two ways to execute an SQL statement using the %SQL.Statement class:

- **%Execute()**, which executes an SQL statement previous prepared using **%Prepare()** or **%PrepareClassQuery()**.
- **%ExecDirect()**, which both prepares and executes an SQL statement.
- **%ExecDirectNoPriv()**, which prepares and executes an SQL statement and does not perform privilege checking.

You can also execute an SQL statement without creating an object instance by using the **\$\$SYSTEM.SQL.Execute()** method. This method both prepares and executes the SQL statement. It creates a cached query. The **Execute()** method is shown in the following Terminal example:

```
USER>set topnum=5
USER>set rset=$$SYSTEM.SQL.Execute("SELECT TOP :topnum Name, Age FROM Sample.Person")
USER>do rset.%Display()
```

8.5.1 %Execute()

After preparing a query, you can execute it by calling the **%Execute()** instance method of the %SQL.Statement class. In the case of a non-**SELECT** statement, **%Execute()** invokes the desired operation (such as performing an **INSERT**). In the case of a **SELECT** query, **%Execute()** generates a result set for subsequent traversal and data retrieval. For example:

ObjectScript

```
set rset = tStatement.%Execute()
```

The **%Execute()** method sets the %SQL.StatementResult class properties %SQLCODE and %Message for all SQL statements. Successful execution of the statement sets %SQLCODE to 0. This does not mean that the statement successfully retrieved results. Similarly, **%Execute()** does not set %SQLCODE to 100 if the statement retrieves no results. The check for results, and subsequent setting of %SQLCODE to 0, 100, or a negative error value occurs as you fetch the results one row at a time, such as by using the **%Next()** method.

%Execute() sets other %SQL.StatementResult properties as follows:

- **INSERT**, **UPDATE**, **INSERT OR UPDATE**, **DELETE**, and **TRUNCATE TABLE** statements set %ROWCOUNT to the number of rows affected by the operation. **TRUNCATE TABLE** cannot determine the actual number of rows deleted, so it sets %ROWCOUNT to -1.

INSERT, **UPDATE**, **INSERT OR UPDATE**, and **DELETE** set %ROWID to the [RowID](#) value of the last record inserted, updated, or deleted. If the operation did not insert, update, or delete any records, %ROWID is undefined, or remains set to its prior value. **TRUNCATE TABLE** does not set %ROWID.

- A **SELECT** statement sets the %ROWCOUNT property to 0 when it creates the result set. %ROWCOUNT is incremented when the program iterates through the contents of the result set, for example by using the **%Next()** method. **%Next()** returns 1 to indicate that it is positioned on a row or 0 to indicate that it is positioned after the last row (at the end of the result set). If the cursor is positioned after the last row, the value of %ROWCOUNT indicates the number of rows contained in the result set.

If a **SELECT** query returns only [aggregate functions](#), every **%Next()** sets %ROWCOUNT=1. The first **%Next()** always sets %SQLCODE=0, even when there is no data in the table; any subsequent **%Next()** sets %SQLCODE=100 and sets %ROWCOUNT=1.

A **SELECT** also sets the %CurrentResult and the %ResultColumnCount. **SELECT** does not set %ROWID.

You can use **ZWRITE** to return the values for all of the %SQL.StatementResult class properties.

For further details, see [SQL System Variables](#). If you are executing TSQL code with %Dialect set to Sybase or MSSQL, errors are reported both in the standard protocols for that SQL dialect and in the InterSystems IRIS %SQLCODE and %Message properties.

8.5.1.1 %Execute() with Input Parameters

The **%Execute()** method can take one or more parameters that correspond to the input parameters (indicated by “?”) within the prepared SQL statement. The **%Execute()** parameters correspond to the sequence in which the “?” characters appear within the SQL statement: the first parameter is used for the first “?”, the second parameter for the second “?”, and so on. Multiple **%Execute()** parameters are separated by commas. You can omit a parameter value by specifying the placeholder comma. The number of **%Execute()** parameters must correspond to the “?” input parameters. If there are fewer or more **%Execute()** parameters than corresponding “?” input parameters, execution fails with the %SQLCODE property set to an SQLCODE -400 error.

You can use an input parameter to supply a literal value or an expression to the SELECT list and to the other query clauses, including the TOP clause and the WHERE clause. You cannot use an input parameter to supply a column name or a column name alias to the SELECT list or to the other query clauses.

When an input parameter is used in a greater than or less than comparison, such as in a WHERE clause, the parameter is normalized only if it is a valid number. If the input parameter is not a valid number, the comparison condition is checked using either the [sorts after operator](#) ([]) or the no sorts after operator (['']), depending on the comparison. Note that this operator orders all numbers before any nonnumeric values (such as strings).

The maximum number of input parameters when specified as explicit **%Execute()** parameters is 255. The maximum number of input parameters when specified using a [variable length array](#) **%Execute(vals...)** is 380.

Following a Prepare, you can use [Prepare arguments metadata](#) to return the count and required data types for ? input parameters. You can use the [%GetImplementationDetails\(\)](#) method to return a list of ? input parameters in a prepared query and the query text with the ? input parameters shown in context.

The following ObjectScript example executes a query with two input parameters. It specifies the input parameter values (21 and 26) in the **%Execute()** method.

ObjectScript

```
SET tStatement = ##class(%SQL.Statement).%New(1)
SET tStatement.%SchemaPath = "MyTests,Sample,Cinema"
SET myquery=2
SET myquery(1)="SELECT Name,DOB,Age FROM Person"
SET myquery(2)="WHERE Age > ? AND Age < ? ORDER BY Age"
SET qStatus = tStatement.%Prepare(.myquery)
IF qStatus'=1 {WRITE "%Prepare failed:" DO $System.Status.DisplayError(qStatus) QUIT}
SET rset = tStatement.%Execute(21,26)
WRITE !,"Execute OK: SQLCODE=",rset.%SQLCODE,!!
DO rset.%Display()
WRITE !,"End of data: SQLCODE=",rset.%SQLCODE
```

The following ObjectScript example executes the same query. The **%Execute()** method formal parameter list uses a [variable length array](#) (dynd...) to specify an indefinite number of input parameter values; in this case, the subscripts of the *dynd* array. The *dynd* variable is set to 2 to indicate two subscript values.

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New(1)
set tStatement.%SchemaPath = "MyTests,Sample,Cinema"
set myquery=2
set myquery(1)="SELECT Name,DOB,Age FROM Person"
set myquery(2)="WHERE Age > ? AND Age < ? ORDER BY Age"
set dynd=2,dynd(1)=21,dynd(2)=26
set qStatus = tStatement.%Prepare(.myquery)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
set rset = tStatement.%Execute(dynd...)
write !,"Execute OK: SQLCODE=",rset.%SQLCODE,!!
do rset.%Display()
write !,"End of data: SQLCODE=",rset.%SQLCODE
```

You can issue multiple **%Execute()** operations on a prepared result set. This enables you to run a query multiple times, supplying different input parameter values. It is not necessary to close the result set between **%Execute()** operations, as shown in the following example:

ObjectScript

```
set myquery="SELECT Name,SSN,Age FROM Sample.Person WHERE Name %STARTSWITH ?"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(myquery)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
set rset = tStatement.%Execute("A")
do rset.%Display()
write !,"End of A data",!!
set rset = tStatement.%Execute("B")
do rset.%Display()
write !,"End of B data"
```

8.5.1.2 Handling %Execute Errors Using Try/Catch

You can execute Dynamic SQL within a **TRY** block structure, passing runtime errors to the associated **CATCH** block exception handler. For **%Execute()** errors, you can use the `%Exception.SQL` class to create an exception instance, which you can then **THROW** to the **CATCH** exception handler.

The following example creates an SQL exception instance when an **%Execute()** error occurs. In this case, the error is a cardinality mismatch between the number of ? input parameters (1) and the number of **%Execute()** parameters (3). It throws the `%SQLCODE` and `%Message` property values (as Code and Data) to the **CATCH** exception handler. The exception handler uses the **%IsA()** instance method to test the exception type, then displays the **%Execute()** error:

ObjectScript

```
try {
set myquery = "SELECT TOP ? Name,DOB FROM Sample.Person"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(myquery)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
set rset = tStatement.%Execute(7,9,4)
if rset.%SQLCODE=0 { write !,"Executed query",! }
else { set badSQL=##class(%Exception.SQL).%New(,rset.%SQLCODE,,rset.%Message)
      throw badSQL }
do rset.%Display()
write !,"End of data"
return
}
catch exp { write "In the catch block",!
            if 1=exp.%IsA("%Exception.SQL") {
              write "SQLCODE: ",exp.Code,!
              write "Message: ",exp.Data,! }
            else { write "Not an SQL exception",! }
            return
}
```

8.5.2 %ExecDirect()

The %SQL.Statement class provides the **%ExecDirect()** class method, that both prepares and executes a query in a single operation. It can prepare either a specified query (like **%Prepare()**) or an existing class query (like **%PrepareClassQuery()**).

%ExecDirect() prepares and executes a specified query:

ObjectScript

```
set myquery=2
set myquery(1)="SELECT Name,Age FROM Sample.Person"
set myquery(2)="WHERE Age > 21 AND Age < 30 ORDER BY Age"
set rset = ##class(%SQL.Statement).%ExecDirect(,myquery)
if rset.%SQLCODE=0 { write !,"ExecDirect OK",!! }
else { write !,"ExecDirect SQLCODE=",rset.%SQLCODE,!,rset.%Message quit}
do rset.%Display()
write !,"End of data: SQLCODE=",rset.%SQLCODE
```

%ExecDirect() prepares and executes an existing class query:

ObjectScript

```
set mycallq = "?=CALL Sample.PersonSets('A','NH')"
```

```
set rset = ##class(%SQL.Statement).%ExecDirect(,mycallq)
if rset.%SQLCODE=0 { write !,"ExecDirect OK",!! }
else { write !,"ExecDirect SQLCODE=",rset.%SQLCODE,!,rset.%Message quit}
do rset.%Display()
write !,"End of data: SQLCODE=",rset.%SQLCODE
```

You can specify input parameter values as the third and subsequent parameters of the **%ExecDirect()** class method, as shown in the following example:

ObjectScript

```
set myquery=2
set myquery(1)="SELECT Name,Age FROM Sample.Person"
set myquery(2)="WHERE Age > ? AND Age < ? ORDER BY Age"
set rset = ##class(%SQL.Statement).%ExecDirect(,myquery,12,20)
if rset.%SQLCODE'=0 {write !,"1st ExecDirect SQLCODE=",rset.%SQLCODE,!,rset.%Message quit}
do rset.%Display()
write !,"End of teen data",!!
set rset2 = ##class(%SQL.Statement).%ExecDirect(,myquery,19,30)
if rset2.%SQLCODE'=0 {write !,"2nd ExecDirect SQLCODE=",rset2.%SQLCODE,!,rset2.%Message quit}
do rset2.%Display()
write !,"End of twenties data"
```

The **%ExecDirect()** input parameters correspond to the sequence in which the “?” characters appear within the SQL statement: the third parameter is used for the first “?”, the fourth parameter for the second “?”, and so on. You can omit

a parameter value by specifying a placeholder comma. If there are fewer **%ExecDirect()** input parameters than corresponding “?” input parameters, the default value (if one exists) is used.

In the following example, the first **%ExecDirect()** specifies all three “?” input parameters, the second **%ExecDirect()** specifies only the second ? input parameter, and omits the first and third. It takes the **Sample.PersonSets()** default ('MA') for the third input parameter:

ObjectScript

```
set mycall = "?=CALL Sample.PersonSets(?,?)"
set rset = ##class(%SQL.Statement).%ExecDirect(,mycall,"","A","NH")
if rset.%SQLCODE'=0 {write !,"1st ExecDirect SQLCODE=",rset.%SQLCODE,!,"rset.%Message quit"}
do rset.%Display()
write !,"End of A people data",!!
set rset2 = ##class(%SQL.Statement).%ExecDirect(,mycall,"B")
if rset2.%SQLCODE'=0 {write !,"2nd ExecDirect SQLCODE=",rset2.%SQLCODE,!,"rset2.%Message quit"}
do rset2.%Display()
write !,"End of B people data"
```

%ExecDirect() can invoke the **%SQL.Statement** **%Display()** instance method or **%GetImplementationDetails()** instance method to return the details of the currently prepared statement. Because **%ExecDirect()** can prepare and execute either a specified query or an existing class query, you can use the **%GetImplementationDetails()** *pStatementType* parameter to determine which kind of query was prepared:

ObjectScript

```
set mycall = "?=CALL Sample.PersonSets('A',?)"
set rset = ##class(%SQL.Statement).%ExecDirect(tStatement,mycall,"NH")
if rset.%SQLCODE'=0 {write !,"ExecDirect SQLCODE=",rset.%SQLCODE,!,"rset.%Message quit"}
set bool = tStatement.%GetImplementationDetails(.pclassname,.ptext,.pargs,.pStatementType)
if bool=1 {if pStatementType=1 {write "Type= specified query",!}
elseif pStatementType=45 {write "Type= existing class query",!}
write "Implementation class= ",pclassname,!
write "Statement text= ",ptext,!
write "Arguments= ",$listtostring(pargs),!! }
else {write "%GetImplementationDetails() failed"}
do rset.%Display()
write !,"End of data"
```

For further details, see [Results of a Successful Prepare](#).

8.5.3 %ExecDirectNoPriv()

The **%SQL.Statement** class provides the **%ExecDirectNoPriv()** class method, which, like **%ExecDirect**, prepares and executes a query in a single operation. **%ExecDirectNoPriv()** also disables privilege checking on the statement during query preparation. Disabling privilege checking gives applications more control over the execution of dynamic queries but increases security risk.

While **%ExecDirectNoPriv()** disables privilege checked during query preparation, it does not disable privilege checking at execution time. This behavior particularly applies when attempting to grant users new privileges. For example, if a user attempts to grant another user the **SELECT** privilege on a certain table, but does not have that privilege, the SQL Statement fails during execution because the operation is not allowed.

8.6 Returning the Full Result Set

Executing a statement with either **%Execute()** or **%ExecDirect()** returns an object that implements the **%SQL.StatementResult** interface. This object can be a unitary value, a result set, or a context object that is returned from a **CALL** statement.

8.6.1 %Display() Method

You can display the entire result set (the contents of the result object) by calling the **%Display()** instance method of the **%SQL.StatementResult** class, as shown in the following example:

ObjectScript

```
do rset.%Display()
```

Note that the **%Display()** method does not return a **%Status** value.

When displaying a query result set, **%Display()** concludes by displaying the row count: “5 Rows(s) Affected”. (This is the **%ROWCOUNT** value after **%Display()** has iterated through the result set.) Note that **%Display()** does not issue a line return following this row count statement.

%Display() has two optional arguments:

- **Delimiter:** a string inserted between data columns and data headers. It appears between resultset columns, immediately before the header or data value. The default is no delimiter. If omitted, specify a placeholder comma before the **Column Alignment** flag.
- **Column Alignment:** an integer flag that specifies how whitespace is calculated between data columns and data headers. The available options are:
 - 0: Resultset header/data columns will be aligned based on the standard delimiter (tab). This is the default.
 - 1: Resultset header/data columns will be aligned based on the length of the column header and the standard delimiter (tab).
 - 2: Resultset header/data columns will be aligned based on the precision/length of the column data property and the standard delimiter (tab).

8.6.2 %DisplayFormatted() Method

You can reformat and redirect the result set contents to a generated file by calling the **%DisplayFormatted()** instance method of the **%SQL.StatementResult** class, rather than calling **%Display()**.

You can specify the result set format either by specifying the string option **%DisplayFormatted("HTML")** or the corresponding integer code **%DisplayFormatted(1)**. InterSystems IRIS generates a file of the specified type, appending the appropriate file name extension. This table shows the options you can specify and the files you can generate.

String Option	Integer Code	Extension of Generated File
"XML"	0	.xml
"HTML"	1	.html
"PDF"	2	.pdf
"TXT"	99	.txt
"CSV"	100	.csv

Note that the values in the generated CSV file are separated by tabs, not commas.

If you specify any other number or string, then **%DisplayFormatted()** generates a text (.txt) file. Text files conclude with the row count (for example “5 Rows(s) Affected”). The other formats do not include a row count.

You can specify or omit a result set file name:

- If you specify a destination file (for example, `%DisplayFormatted(99, "myresults")`) a file with that name and the appropriate suffix (file name extension) is generated in the mgr directory in the subdirectory for the current namespace. For example, `C:\InterSystems\IRIS\mgr\user\myresults.txt`. If the specified file with that suffix already exists, InterSystems IRIS overwrites it with new data.
- If you do not specify a destination file (for example, `%DisplayFormatted(99)`) a file with a randomly-generated name and the appropriate suffix (file name extension) is generated in the mgr directory in the Temp subdirectory. For example, `C:\InterSystems\IRIS\mgr\Temp\w4FR2gM7tX2Fjs.txt`. Each time a query is run a new destination file is generated.

These examples show Windows filenames; InterSystems IRIS supports equivalent locations on other operating systems.

If the specified file cannot be opened, this operation times out after 30 seconds with an error message; this commonly occurs when the user does not have write privileges to the specified directory (file folder).

If data cannot be rendered in the specified format, the destination file is created but no result set data is written to it. Instead, an appropriate message is written to the destination file. For example, a stream field OID contains characters that conflict with XML and HTML special formatting characters. This XML and HTML stream field issue can be resolved by using the [XMLELEMENT](#) function on stream fields; for example, `SELECT Name, XMLELEMENT("Para", Notes)`.

You can optionally supply the name of a [translate table](#) that `%DisplayFormatted()` will use when performing the specified format conversion.

In the case of multiple result sets in a result set sequence, the content of each result set is written to its own file.

The optional third `%DisplayFormatted()` argument specifies that messages are stored in a separate result set. Upon successful completion a message like the following is returned:

```
Message
21 row(s) affected.
```

The following Windows example creates two PDF (integer code 2) result set files in `C:\InterSystems\IRIS\mgr\user\`. It creates the *mess* result set for messages, then uses `%Display()` to display messages to the Terminal:

ObjectScript

```
set $NAMESPACE="USER"
set myquery=2
set myquery(1)="SELECT Name, Age FROM Sample.Person"
set myquery(2)="WHERE Age > ? AND Age < ? ORDER BY Age"
set rset = ##class(%SQL.Statement).%ExecDirect(, .myquery, 12, 20)
if rset.%SQLCODE'=0 {write !, "1st ExecDirect SQLCODE=", rset.%SQLCODE, !, rset.%Message quit}
do rset.%DisplayFormatted(2, "Teenagers", .mess)
do mess.%Display()
write !, "End of teen data", !!
set rset2 = ##class(%SQL.Statement).%ExecDirect(, .myquery, 19, 30)
if rset2.%SQLCODE'=0 {write !, "2nd ExecDirect SQLCODE=", rset2.%SQLCODE, !, rset2.%Message quit}
do rset2.%DisplayFormatted(2, "Twenties", .mess)
do mess.%Display()
write !, "End of twenties data"
```

8.6.3 Paginating a Result Set

You can use a [view ID \(%VID\)](#) to paginate a result set. The following example returns pages from the result set, each page containing 5 rows:

ObjectScript

```

set q1="SELECT %VID AS RSRow, * FROM "
set q2="(SELECT Name,Home_State FROM Sample.Person WHERE Home_State %STARTSWITH 'M') "
set q3="WHERE %VID BETWEEN ? AND ?"
set myquery = q1_q2_q3
set tStatement = ##class(%SQL.Statement).%New()
set qStatus=tStatement.%Prepare(myquery)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
for i=1:5:25 {
    write !!,"Next Page",!
    set rset=tStatement.%Execute(i,i+4)
    do rset.%Display()
}

```

Refer to [%GetRows\(\)](#) for another way to return groups of rows (records) from a result set.

8.7 Returning Specific Values from the Result Set

To return specific values from a query result set, you must iterate through the result set one row at a time. To iterate through a result set, use the **%Next()** instance method. You can then either display the results of the whole current row using the **%Print()** method, or retrieve the value of a specified column in the current row.

The **%Next()** method fetches the data for the next row within the query results and places this data in the Data property of the result set object. **%Next()** returns one of these values:

- **%Next() = 1** — Cursor is positioned on a row in the query result.
- **%Next() = 0** — Cursor is positioned after the last row, indicating that there are no more rows to return or that the query returned 0 rows.

Each call to **%Next()** that returns 1 increments the **%ROWCOUNT** property of the result set by 1. If the cursor is positioned after the last row (**%Next()** returns 0), **%ROWCOUNT** indicates the number of rows in the result set.

Each call to **%Next()** also updates the **%SQLCODE** property of the result set. The updated **%SQLCODE** value depends on the fetched results:

- **%SQLCODE = 0** — **%Next()** successfully fetched a row of results.
- **%SQLCODE = 100** — **%Next()** fetched no results. Either the query returned no results or the cursor is positioned after the last row and there are no more results to fetch.
- **%SQLCODE < 0** — **%Next()** failed to perform the fetch. **%Next()** sets **%SQLCODE** to the **SQLCODE** of the error that caused the fetch to fail. It also sets the **%Message** property of the result set to the error message text. When calling **%Next()** iteratively in a loop, to avoid silent errors, check for negative **%SQLCODE** values and display the **%SQLCODE** error and its **%Message** text. For example:

ObjectScript

```

while rset.%Next()
{
    write "%Next succeeded."
}
if (rset.%SQLCODE < 0)
{
    write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
    quit
}

```

If a **SELECT** query returns only [aggregate functions](#), every **%Next()** sets **%ROWCOUNT=1**. The first **%Next()** returns 1 and sets **%SQLCODE=0** and **%ROWCOUNT=1**, even when there is no data in the table. Any subsequent **%Next()** returns 0 and sets **%SQLCODE=100** and **%ROWCOUNT=1**.

After fetching a row from the result set, you can display data from that row using these methods:

- `rset.%Print()` to return all of the data values for the current row from the query result set.
- `rest.%GetRow()` and `rset.%GetRows()` to return the data values for a row as elements in an encoded List structure from the query result set.
- `rset.name` to return a data value by property name, field name, alias property name, or alias field name from a query result set.
- `rset.%Get("fieldname")` to return a data value by field name or alias field name from either a query result set or a stored query.
- `rset.%GetData(n)` to return a data value by column number from either a query result set or a stored query.

8.7.1 %Print() Method

The `%Print()` instance method retrieves the current record from the result set. By default, `%Print()` inserts a blank space delimiter between data field values. `%Print()` does not insert a blank space before the first field value or after the last field value in a record; it issues a line return at the end of the record. If a data field value already contains a blank space, that field value is enclosed in quotation marks to differentiate it from the delimiter. For example, if `%Print()` is returning city names, it would return them as follows: Chicago "New York" Boston Atlanta "Los Angeles" "Salt Lake City" Washington. `%Print()` quotes field values that contain the delimiter as part of the data value even when the `%Print()` delimiter is never used; for example if there is only one field in the result set.

You can optionally specify a `%Print()` parameter that provides a different delimiter to be placed between the field values. Specifying a different delimiter overrides the quoting of data strings that contain blank spaces. This `%Print()` delimiter can be one or more characters. It is specified as a quoted string. It is generally preferable that the `%Print()` delimiter be a character or string not found in the result set data. However, if a field value in the result set contains the `%Print()` delimiter character (or string), that field value is returned enclosed in quotation marks to differentiate it from the delimiter.

If a field value in the result set contains a line feed character, that field value is returned delimited by quotation marks.

The following ObjectScript example iterates through the query result set using `%Print()` to display each result set record, separating values with a `"^|^"` delimiter. Note how `%Print()` displays data from the FavoriteColors field which is an encoded list of elements:

ObjectScript

```
set q1="SELECT TOP 5 Name,DOB,Home_State,FavoriteColors "
set q2="FROM Sample.Person WHERE FavoriteColors IS NOT NULL"
set query = q1_q2
set statement = ##class(%SQL.Statement).%New()

set status = statement.%Prepare(query)
if $$$ISERR(status) {write "%Prepare failed:" do $SYSTEM.Status.DisplayError(status) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE '= 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message quit}

while rset.%Next()
{
    write "Row count ",rset.%ROWCOUNT,!
    do rset.%Print("^|^")
}
if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message quit}
write !,"End of data"
write !,"Total row count=",rset.%ROWCOUNT
```

The following example shows how field values that contain the delimiter are returned enclosed in quotation marks. In this example, the capital letter A is used as the field delimiter; therefore, any field value (name, street address, or state abbreviation) that contains a capital A literal is returned delimited by quotation marks.

ObjectScript

```

set query = "SELECT TOP 25 Name,Home_Street,Home_State,Age FROM Sample.Person"
set statement = ##class(%SQL.Statement).%New()

set status = statement.%Prepare(query)
if $$$ISERR(status) {write "%Prepare failed:" do $SYSTEM.Status.DisplayError(status) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE != 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

while rset.%Next()
{
do rset.%Print("A")
}
if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
write !,"End of data"
write !,"Total row count=",rset.%ROWCOUNT

```

8.7.2 %GetRow() and %GetRows() Methods

The **%GetRow()** instance method retrieves the current row (record) from the result set as an [encoded list](#) of field value elements:

ObjectScript

```

set myquery = "SELECT TOP 17 %ID,Name,Age FROM Sample.Person"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(myquery)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
set rset = tStatement.%Execute()
for {set x=rset.%GetRow(.row,.status)
    if x=1 {write $listtostring(row," | "),! }
    else {write !,"End of data"
        write !,"Total row count=",rset.%ROWCOUNT
        return }
    }
}

```

The **%GetRows()** instance method retrieves a group of rows (records) of a specified size from the result set. Each row is returned as an [encoded list](#) of field value elements.

The following example returns the 1st, 6th, and 11th row in the result set. In this example, the **%GetRows()** first parameter (5) specifies that **%GetRows()** should retrieve successive groups of five rows. **%GetRows()** returns 1 if it successfully retrieves a group of five rows. The `.rows` parameter passes by reference a subscripted array of these five rows, so `rows(1)` returns the first row from each set of five: rows 1, 6, and 11. Specifying `rows(2)` would return rows 2, 7, and 12.

ObjectScript

```

set myquery = "SELECT TOP 17 %ID,Name,Age FROM Sample.Person"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(myquery)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
set rset = tStatement.%Execute()
for {set x=rset.%GetRows(5,.rows,.status)
    if x=1 {write $listtostring(rows(1)," | "),! }
    else {write !,"End of data"
        write !,"Total row count=",rset.%ROWCOUNT
        return }
    }
}

```

Rather than retrieving individual rows by subscript, you can use the **ZWRITE rows** command to return all of the subscripts in the retrieved array. Note that above example **ZWRITE rows** does not return the 16th and 17th row in the result set, because these rows are remainders after the last group of five rows was retrieved.

8.7.3 rset.name Property

When InterSystems IRIS generates a result set, it creates a result set class that contains a unique property corresponding to each field name and [field name alias](#) in the result set.

You can use the `rset.name` property to return a data value by property name, field name, property name alias, or field name alias.

- **Property Name:** If no field alias is defined, specify the field property name as `rset.PropName`. The result set field property name is taken from the corresponding property name in the table definition class.
- **Field Name:** If no field alias is defined, specify the field name (or the property name) as `rset."fieldname"`. This is the [SqlFieldName](#) specified in the table definition. InterSystems IRIS uses this field name to locate the corresponding property name. In many cases, the property name and the field name (`SqlFieldName`) are identical.
- **Alias Property Name:** If a field alias is defined, specify the alias property name as `rset.AliasProp`. An alias property name is generated from the column name alias in the **SELECT** statement. You cannot specify a field property name for a field with a defined alias.
- **Alias Name:** If a field alias is defined, specify this alias name (or the alias property name) as `rset."alias"`. This is the column name alias in the **SELECT** statement. You cannot specify a field name for a field with a defined alias.
- **Aggregate, Expression, or Subquery:** InterSystems IRIS assigns these *select-items* a field name of `Aggregate_n`, `Expression_n`, or `Subquery_n` (where the integer *n* corresponds to the sequence of the *select-item* list specified in the query). You can retrieve these *select-item* values using the field name (`rset."SubQuery_7"` not case-sensitive), the corresponding property name (`rset.Subquery7` case-sensitive), or by a user-defined field name alias. You can also just specify the *select-item* sequence number using [rset.%GetData\(n\)](#).

When specifying a property name, you must use correct letter case; when specifying a field name, correct letter case is not required.

This invocation of `rset.name` using the property name has the following consequences:

- **Letter Case:** Property names are case-sensitive. Field names are not case-sensitive. Dynamic SQL can automatically resolve differences in letter case between a specified field or alias name and the corresponding property name. However, letter case resolution takes time. To maximize performance, you should specify the exact letter case of the property name or the alias.
- **Non-alphanumeric Characters:** A property name can only contain alphanumeric characters (except for an initial % character). If the corresponding SQL field name or field name alias contains non-alphanumeric characters (for example, `Last_Name`) you can do either of the following:
 - Specify the field name delimited with quotation marks. For example, `rset."Last_Name"`). This use of delimiters does not require that delimited identifiers be enabled. Letter case resolution is performed.
 - Specify the corresponding property name, eliminating the non-alphanumeric characters. For example, `rset.LastName` (or `rset."LastName"`). You must specify the correct letter case for the property name.
- **% Property Names:** Generally, property names beginning with a % character are reserved for system use. If a field property name or alias begins with a % character and that name conflicts with a system-defined property, the system-defined property is returned. For example, for `SELECT Notes AS %Message`, invoking `rset.%Message` will not return the `Notes` field values; it returns the `%Message` property defined for the statement result class. You can use [rset.%Get\("%Message"\)](#) to return the field value.
- **Column Alias:** If an alias is specified, Dynamic SQL always matches the alias rather than matching the field name or field property name. For example, for `SELECT Name AS Last_Name`, the data can only be retrieved using `rset.LastName` or `rset."Last_Name"`, not by using `rset.Name`.

- **Duplicate Names:** Names are duplicate if they resolve to the same property name. Duplicate names can be multiple references to the same field in a table, alias references to different fields in a table, or references to fields in different tables. For example `SELECT p.DOB, e.DOB` specifies two duplicate names, even though those names refer to fields in different tables.

If the **SELECT** statement contains multiple instances of the same field name or field name alias, `rset.PropName` or `rset.fieldname` always return the first one specified in the **SELECT** statement. For example, for `SELECT c.Name, p.Name FROM Sample.Person AS p, Sample.Company AS c using rset.Name` retrieves the company name field data; `SELECT c.Name, p.Name AS Name FROM Sample.Person AS p, Sample.Company AS c using rset.name` also retrieves the company name field data. If there are duplicate Name fields in the query the last character of the field name (Name) is replaced by a character (or characters) to create a unique property name. Thus a duplicate Name field name in a query has a corresponding unique property name, beginning with `Nam0` (for the first duplicate) through `Nam9` and continuing with capital letters `NamA` through `NamZ`.

For a user-specified query prepared using **%Prepare()** you can use the property name by itself. For a stored query prepared using **%PrepareClassQuery()**, you must use the **%Get("fieldname")** method.

The following example returns the values of three fields specified by property names: two field values by property name and the third field value by alias property name. In these cases, the specified property name is identical to the field name or field alias:

ObjectScript

```
set query = "SELECT TOP 5 Name,DOB AS bdate,FavoriteColors FROM Sample.Person"
set statement = ##class(%SQL.Statement).%New(1)

set status = statement.%Prepare(query)
if $$$ISERR(status) {write "%Prepare failed:" do $SYSTEM.Status.DisplayError(status) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE != 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

while rset.%Next()
{
    write "Row count ",rset.%ROWCOUNT,!
    write rset.Name
    write " prefers ",rset.FavoriteColors
    write ", Birth date ",rset.bdate,!
}
if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
write !,"End of data"
write !,"Total row count=",rset.%ROWCOUNT
```

In the above example, one of the fields returned is the `FavoriteColors` field, which contains `%List` data. To display this data, the **%New(1)** class method sets the **%SelectMode** property parameter to 1 (ODBC), causing this program to display `%List` data as a comma-separated string and the birth date in ODBC format:

The following example returns the `Home_State` field. Because a property name cannot contain an underscore character, this example specifies the field name (the **SqlFieldName**) delimited with quotation marks ("`Home_State`"). You could also specify the corresponding generated property name without quotation marks (`HomeState`). Note that the delimited field name ("`Home_State`") is not case-sensitive, but the generated property name (`HomeState`) is case-sensitive:

ObjectScript

```

set query = "SELECT TOP 5 Name,Home_State FROM Sample.Person"
set statement = ##class(%SQL.Statement).%New(2)

set status = statement.%Prepare(query)
if $$$ISERR(status) {write "%Prepare failed:" do $SYSTEM.Status.DisplayError(status) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE != 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

while rset.%Next()
{
    write "Row count ",rset.%ROWCOUNT,!
    write rset.Name
    write " lives in ",rset."Home_State",!
}
if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
write !,"End of data"
write !,"Total row count=",rset.%ROWCOUNT

```

8.7.3.1 Swizzling a Fieldname Property with %ObjectSelectMode=1

The following example is prepared with `%ObjectSelectMode=1`, which causes fields whose type class is a swizzleable type (a persistent class, a serial class, or a stream class) to automatically [swizzle](#) when returning a value using the field name property. The result of swizzling a field value is the corresponding object reference (OREF). InterSystems IRIS does not perform this swizzling operation when accessing a field using the `%Get()` or `%GetData()` methods. In this example, `rset.Home` is swizzled, while `rset.%GetData(2)`, which refers to the same field, is not swizzled:

ObjectScript

```

set query = "SELECT TOP 5 Name,Home FROM Sample.Person"
set statement = ##class(%SQL.Statement).%New(0)
set statement.%ObjectSelectMode = 1
write !,"set ObjectSelectMode=",statement.%ObjectSelectMode,!

set status = statement.%Prepare(query)
if $$$ISERR(status) {write "%Prepare failed:" do $SYSTEM.Status.DisplayError(status) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE != 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

while rset.%Next()
{
    write "Row count: ",rset.%ROWCOUNT,!
    write rset.Name,!
    write " ",rset.Home,!
    write rset.%GetData(1)
    write " ",$listtostring(rset.%GetData(2)),!!
}
if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
write !,"End of data"
write !,"Total row count=",rset.%ROWCOUNT

```

The following example uses `%ObjectSelectMode=1` to derive `Home_State` values for the selected records from the unique record ID (%ID). Note that the `Home_State` field is not selected in the original query:

ObjectScript

```

set query = "SELECT TOP 5 %ID AS MyID,Name,Age FROM Sample.Person"
set statement = ##class(%SQL.Statement).%New()
set statement.%ObjectSelectMode=1

set status = statement.%Prepare(query)
if $$$ISERR(status) {write "%Prepare failed:" do $SYSTEM.Status.DisplayError(status) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE != 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

while rset.%Next()
{
    write rset.Name
    write " Home State:",rset.MyID.Home.State,!
}
if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
write !,"End of data"
write !,"Total row count=",rset.%ROWCOUNT

```

If configured, the system generates a <SWIZZLE FAIL> error if the swizzled property is defined but cannot be referenced. This can occur if the referenced property has been unexpectedly deleted from disk or is locked by another process. To determine the cause of the swizzle failure look in %objlasterror immediately after the <SWIZZLE FAIL> error and decode this %Status value.

By default, <SWIZZLE FAIL> is not configured. You can set this behavior globally by setting set ^%SYS("ThrowSwizzleError")=1, or by using the InterSystems IRIS Management Portal. From **System Administration**, select **Configuration**, then **SQL and Object Settings**, then **Objects**. On this screen you can set the <SWIZZLE FAIL> option.

8.7.4 %Get("fieldname") Method

You can use the %Get("fieldname") instance method to return a data value by field name or field name alias. Dynamic SQL resolves letter case as needed. If the specified field name or field name alias does not exist, the system generates a <PROPERTY DOES NOT EXIST> error.

The following example returns values for the Home_State field and the Last_Name alias from the query result set.

ObjectScript

```

set query = "SELECT TOP 5 Home_State,Name AS Last_Name FROM Sample.Person"
set statement = ##class(%SQL.Statement).%New(2)

set status = statement.%Prepare(query)
if $$$ISERR(status) {write "%Prepare failed:" do $SYSTEM.Status.DisplayError(status) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE != 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

while rset.%Next()
{
    write rset.%Get("Home_State")," : ",rset.%Get("Last_Name"),!
}
if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
write !,"End of data"
write !,"Total row count=",rset.%ROWCOUNT

```

You must use the %Get("fieldname") instance method to retrieve individual data items by field property name from an existing query prepared using %PrepareClassQuery(). If the field property name does not exist, the system generates a <PROPERTY DOES NOT EXIST> error.

The following example returns the Nsp (namespace) field values by field property name from a built-in query. Because this query is an existing stored query, this field retrieval requires the use of the %Get("fieldname") method. Note that because "Nsp" is a property name, it is case-sensitive:

ObjectScript

```
set statement = ##class(%SQL.Statement).%New(2)
set status = statement.%PrepareClassQuery("%SYS.Namespace", "List")
if $$$ISERR(status) {write "%Prepare failed:" do $SYSTEM.Status.DisplayError(status) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE '= 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

while rset.%Next()
{
    write "Namespace: ", rset.%Get("Nsp"), !
}
if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
write !, "End of data"
write !, "Total row count=", rset.%ROWCOUNT
```

Duplicate Names: Names are duplicate if they resolve to the same property name. Duplicate names can be multiple references to the same field, references to different fields in a table, or references to fields in different tables. If the **SELECT** statement contains multiple instances of the same field name or field name alias, **%Get("fieldname")** always returns the last instance of a duplicate name as specified in the query. This is the opposite of **rset.PropName**, which returns the first instance of a duplicate name as specified in the query. This is shown in the following example:

ObjectScript

```
set query = "SELECT c.Name,p.Name FROM Sample.Person AS p,Sample.Company AS c"
set statement = ##class(%SQL.Statement).%New()

set status = statement.%Prepare(query)
if $$$ISERR(status) {write "%Prepare failed:" do $SYSTEM.Status.DisplayError(status) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE '= 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

while rset.%Next()
{
    write "Prop=", rset.Name, " Get=", rset.%Get("Name"), !
}
if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
write !, rset.%ROWCOUNT, " End of data"
```

8.7.5 %GetData(n) Method

The **%GetData(n)** instance method returns data for the current row indexed by the integer count column number of the result set. You can use **%GetData(n)** with either a specified query prepared using **%Prepare()** or a stored query prepared using **%PrepareClassQuery()**.

The integer *n* corresponds to the sequence of the *select-item* list specified in the query. The [RowID](#) field is not given an integer *n* value, unless explicitly specified in the *select-item* list. If *n* is higher than the number of *select-items* in the query, or 0, or a negative number, Dynamic SQL returns no value and issues no error.

%GetData(n) is the only way to return a specific duplicate field name or duplicate alias; **rset.Name** returns the first duplicate, **%Get("Name")** returns the last duplicate.

ObjectScript

```

set query = "SELECT TOP 5 Name,SSN,Age FROM Sample.Person"
set statement = ##class(%SQL.Statement).%New()

set status = statement.%Prepare(query)
if $$$ISERR(status) {write "%Prepare failed:" do $SYSTEM.Status.DisplayError(status) quit}

set rset = statement.%Execute()
if (rset.%SQLCODE '= 0) {write "%Execute failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}

while rset.%Next()
{
  write "Years:",rset.%GetData(3)," Name:",rset.%GetData(1),!
}
if (rset.%SQLCODE < 0) {write "%Next failed:", !, "SQLCODE ", rset.%SQLCODE, ": ", rset.%Message
quit}
write "End of data"
write !,"Total row count=",rset.%ROWCOUNT

```

8.8 Returning Multiple Result Sets

A [CALL](#) statement can return multiple dynamic result sets as a collection referred to as a *result set sequence* (RSS).

The following example uses the `%NextResult()` method to return multiple result sets separately:

ObjectScript

```

set mycall = "CALL Sample.CustomSets()"
set rset = ##class(%SQL.Statement).%ExecDirect(,mycall)
if rset.%SQLCODE'=0 {write !,"ExecDirect SQLCODE=",rset.%SQLCODE,!,rset.%Message quit}

set rset1 = rset.%NextResult()
do rset1.%Display()
write !,"End of 1st Result Set data",!!

set rset2 = rset.%NextResult()
do rset2.%Display()
write !,"End of 2nd Result Set data"

```

8.9 SQL Metadata

Dynamic SQL provides the following types of metadata:

- After a Prepare, [metadata describing the type of query](#).
- After a Prepare, [metadata describing the select-items in the query](#) (**Columns** and **Extended Column Info**).
- After a Prepare, [metadata describing the query arguments](#): ? parameters, :var parameters, and constants. (**Statement Parameters**, **Formal Parameters**, and **Objects**)
- After an Execute, [metadata describing the query result set](#).

%SQL.StatementMetadata property values are available following a Prepare operation (`%Prepare()`, `%PrepareClassQuery()`, or `%ExecDirect()`).

- You can return %SQL.StatementMetadata properties directly for the most recent `%Prepare()`.
- You can return the %SQL.Statement %Metadata property containing the OREF for the %SQL.StatementMetadata properties. This enables you to return metadata for multiple Prepare operations.

A **SELECT** or **CALL** statement returns all of this metadata. An **INSERT**, **UPDATE**, or **DELETE** returns Statement Type Metadata and the **Formal Parameters**.

8.9.1 Statement Type Metadata

Following a Prepare using the %SQL.Statement class, you can use the %SQL.StatementMetadata statementType property to determine what type of SQL statement was prepared, as shown in the following example. This example uses the %SQL.Statement %Metadata property to preserve and compare the metadata for two Prepare operations:

ObjectScript

```
set tStatement = ##class(%SQL.Statement).%New()
set myquery1 = "SELECT TOP ? Name, Age, AVG(Age), CURRENT_DATE FROM Sample.Person"
set myquery2 = "CALL Sample.SP_Sample_By_Name(?)"
set qStatus = tStatement.%Prepare(myquery1)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
set meta1 = tStatement.%Metadata
set qStatus = tStatement.%Prepare(myquery2)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
set meta2 = tStatement.%Metadata
write "Statement type query 1: ", meta1.statementType, !
write "Statement type query 2: ", meta2.statementType, !
write "End of metadata"
```

The Class Reference entry for the *statementType* property lists the statement type integer codes. The most common codes are 1 (a SELECT query) and 45 (a CALL to a stored query).

You can return the same information using the **%GetImplementationDetails()** instance method, as described in [Results of a Successful Prepare](#).

After executing a query, you can return the [statement type name](#) (for example, SELECT) from the result set.

8.9.2 Select-item Metadata

Following a Prepare of a **SELECT** or **CALL** statement using the %SQL.Statement class, you can return metadata about each select-item column specified in the query, either by displaying all of the metadata or by specifying individual metadata items. This column metadata includes ODBC data type information, as well as client type and InterSystems Objects property origins and class type information.

The following example returns the number of columns specified in the most recently prepared query:

ObjectScript

```
set myquery = "SELECT %ID AS id, Name, DOB, Age, AVG(Age), CURRENT_DATE, Home_State FROM Sample.Person"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(myquery)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
write "Number of columns=", tStatement.%Metadata.columnCount, !
write "End of metadata"
```

The following example returns the column name (or column alias), [ODBC data type](#), maximum data length (precision), and scale for each select-item field:

ObjectScript

```

set $NAMESPACE="SAMPLES"
set myquery=2
set myquery(1)="SELECT Name AS VendorName,LastPayDate,MinPayment,NetDays,"
set myquery(2)="AVG(MinPayment),$HOROLOG,%TABLENAME FROM Sample.Vendor"
set rset = ##class(%SQL.Statement).%New()
set qStatus = rset.%Prepare(.myquery)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
set x=rset.%Metadata.columns.Count()
set x=1
while rset.%Metadata.columns.GetAt(x) {
    set column=rset.%Metadata.columns.GetAt(x)
    write !,x," ",column.colName," is data type ",column.ODBCType
    write " with a size of ",column.precision," and scale = ",column.scale
    set x=x+1 }
write !,"End of metadata"

```

The following example displays all of the column metadata using the %SQL.StatementMetadata **%Display()** instance method:

ObjectScript

```

set query = "SELECT %ID AS id,Name,DOB,Age,AVG(Age),CURRENT_DATE,Home_State FROM Sample.Person"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(query)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
do tStatement.%Metadata.%Display()
write !,"End of metadata"

```

This returns two table listings of the selected fields. The first columns metadata table lists column definition information:

Display Header	%SQL.StatementColumn Property	Description
Column Name	colName	The SQL name of the column. If the column is given an alias, the column alias, not the field name, is listed here. Names and aliases are truncated to 12 characters. For an expression, aggregate, literal, host variable, or subquery, the assigned “Expression_n”, “Aggregate_n”, “Literal_n”, “HostVar_n”, or “Subquery_n” label is listed (with <i>n</i> being the SELECT item sequence number). If you have assigned an alias to an expression, aggregate, literal, host variable, or subquery, the alias is listed here.
Type	ODBCType	The integer code for the ODBC data type. These codes are listed in Integer Codes for Data Types . Note that these ODBC data type codes are not the same as the CType data type codes.
Prec	precision	The precision or maximum length , in characters. Precision and scale metadata for TIME data types are described in Date, Time, PosixTime, and TimeStamp Data Types .
Scale	scale	The maximum number of fractional decimal digits . Returns 0 for integer or non-numeric values. Precision and scale metadata for TIME data types are described in Date, Time, PosixTime, and TimeStamp Data Types .
Null	isNullable	An integer value that indicates whether the column is defined as Non-NULL (0), or if NULL is permitted (1). The RowID returns 0. If the SELECT item is an aggregate or subquery that could result in NULL, or if it specifies the NULL literal, this item is set to 1. If the SELECT item is an expression or host variable, this item is set to 2 (cannot be determined).
Label	label	The column name or column alias (same as Column Name).
Table	tableName	The SQL table name . The actual table name is always listed here, even if you have given the table an alias. If the SELECT item is an expression or an aggregate no table name is listed. If the SELECT item is a subquery, the subquery table name is listed.
Schema	schemaName	The table’s schema name . If no schema name was specified, returns the system-wide default schema . If the SELECT item is an expression or an aggregate no schema name is listed. If the SELECT item is a subquery no schema name is listed.
CType	clientType	The integer code for the client data type. See the %SQL.StatementColumn <i>clientType</i> property for a list of values.

The second columns metadata table lists extended column information. The Extended Column Info table lists each column with twelve boolean flags (SQLRESULTCOL), specified as Y (Yes) or N (No):

Boolean Flag	%SQL.StatementColumn Property	Description
1: AutoIncrement	isAutoIncrement	The RowID and IDENTITY fields returns Y.
2: CaseSensitive	isCaseSensitive	A string data type field with %EXACT collation returns Y. A property that references a %SerialObject embedded object returns Y.
3: Currency	isCurrency	A field defined with a data type of %Library.Currency, such as the MONEY data type.
4: ReadOnly	isReadOnly	An Expression, Aggregate, Literal, HostVar, or Subquery returns Y. The RowID, IDENTITY, and RowVersion fields returns Y.
5: RowVersion	isRowVersion	The RowVersion field returns Y.
6: Unique	isUnique	A field defined as having a unique value constraint . The RowID and IDENTITY fields returns Y.
7: Aliased	isAliased	The system supplies an alias to a non-field select-item. Therefore, an Expression, Aggregate, Literal, HostVar, or Subquery returns Y, whether or not the user replaced the system alias by specifying a column alias. This flag is not affected by user-specified column aliases.
8: Expression	isExpression	An Expression returns Y.
9: Hidden	isHidden	If the table is defined with %PUBLICROWID or SqlRowIdPrivate=0 (the default), the RowID field returns N. Otherwise, the RowID field returns Y. A property that references a %SerialObject embedded object returns Y.
10: Identity	isIdentity	A field defined as an IDENTITY field returns Y. The RowID field if the RowID is not hidden returns Y.

Boolean Flag	%SQL.StatementColumn Property	Description
11: KeyColumn	isKeyColumn	A field defined as a primary key field or the target of a foreign key constraint . The RowID field returns Y.
12: RowID	isRowId	The RowID and IDENTITY fields returns Y.
13: isList	isList	A field defined as data type %Library.List or %Library.ListOfBinary , or a field that is a list or array collection returns Y. CType (client data type)=6. An expression using the \$LISTBUILD or \$LISTFROMSTRING function to generate a list returns Y.

The Extended Column Info metadata table lists the **Column Name** (the SQL name or column alias), the **Linked Prop** (linked persistent class property) and **Type Class** (data type class) for each of the selected fields. Note that the **Linked Prop** lists the persistent class name (not the SQL table name) and the property name (not the column alias).

- For an ordinary table field (SELECT Name FROM Sample.Person): **Linked Prop**=Sample.Person.Name, **Type Class**=%Library.String.
- For the table's RowID (SELECT %ID FROM Sample.Person): **Linked Prop**= [none], **Type Class**=Sample.Person.
- For an Expression, Aggregate, Literal, HostVar, or Subquery (SELECT COUNT(Name) FROM Sample.Person): **Linked Prop**= [none], **Type Class**=%Library.BigInt.
- For a referenced [%SerialObject embedded object](#) property (SELECT Home_State FROM Sample.Person). **Linked Prop**=Sample.Address.State, **Type Class**=%Library.String.
- For a field referencing a [%SerialObject embedded object](#) (SELECT Home FROM Sample.Person). **Linked Prop**=Sample.Person.Home, **Type Class**=Sample.Address.

In this example, the Home_State field in Sample.Person references the State property of the %SerialObject class Sample.Address.

The following example returns the metadata for a called stored procedure with one formal parameter, which is also a statement parameter:

ObjectScript

```
set mysql = "CALL Sample.SP_Sample_By_Name(?)"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(.mysql)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
do tStatement.%Metadata.%Display()
write !,"End of metadata"
```

It returns not only column (field) information, but also values for Statement Parameters, Formal Parameters, and Objects.

The following example returns the metadata for a with three formal parameters. One of these three parameters is designated with a question mark (?) making it a statement parameter:

ObjectScript

```
set mycall = "CALL personsets(?, 'MA')"
```

```
set tStatement = ##class(%SQL.Statement).%New(0, "sample")
```

```
set qStatus = tStatement.%Prepare(mycall)
```

```
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
```

```
do tStatement.%Metadata.%Display()
```

```
write !,"End of metadata"
```

Note that this metadata returns no column information, but the Statement Parameters, Formal Parameters lists contain the column names and data types.

8.9.3 Query Arguments Metadata

Following a Prepare using the %SQL.Statement class, you can return metadata about query arguments: [input parameters](#) (specified as a question mark (?)), [input host variables](#) (specified as :varname), and constants (literal values). The following metadata can be returned:

- Count of ? parameters: *parameterCount* property
- ODBC data types of ? parameters: %SQL.StatementMetadata **%Display()** instance method Statement Parameters list.
- List of ?, v (:var), and c (constant) parameters: **%GetImplementationDetails()** instance method, as described in [Results of a Successful Prepare](#).
- ODBC data types of ?, v (:var), and c (constant) parameters: *formalParameters* property.
%SQL.StatementMetadata **%Display()** instance method Formal Parameters list.
- Text of query showing these arguments: **%GetImplementationDetails()** instance method, as described in [Results of a Successful Prepare](#).

The statement metadata **%Display()** method lists the Statement Parameters and Formal parameters. For each parameter it lists the sequential parameter number, ODBC data type, precision, scale, whether it is nullable (2 means that a value is always supplied), and its corresponding property name (colName), and column type.

Note that some ODBC data types are returned as negative integers. For a table of [ODBC data type integer codes](#), see [Data Types](#).

The following example returns the ODBC data types of each of the query arguments (?, :var, and constants) in order. Note that the TOP argument is returned as data type 12 (VARCHAR) rather than 4 (INTEGER) because it is possible to specify TOP ALL:

ObjectScript

```
set myquery = 4
```

```
set myquery(1) = "SELECT TOP ? Name,DOB,Age+10 "
```

```
set myquery(2) = "FROM Sample.Person"
```

```
set myquery(3) = "WHERE %ID BETWEEN :startid :endid AND DOB=?"
```

```
set myquery(4) = "ORDER BY $PIECE(Name,','?)"
```

```
set tStatement = ##class(%SQL.Statement).%New()
```

```
set qStatus = tStatement.%Prepare(.myquery)
```

```
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
```

```
set prepmeta = tStatement.%Metadata
```

```
write "Number of ? parameters=",prepmeta.parameterCount,!
```

```
set formalobj = prepmeta.formalParameters
```

```
set i=1
```

```
while formalobj.GetAt(i) {
```

```
    set prop=formalobj.GetAt(i)
```

```
    write prop.colName," type= ",prop.ODBCType,!
```

```
    set i=i+1 }
```

```
write "End of metadata"
```

Following an Execute, arguments metadata is not available from the [query result set metadata](#). In a result set all parameters are resolved. Therefore parameterCount = 0, and formalParameters contains no data.

8.9.4 Query Result Set Metadata

Following an Execute using the %SQL.Statement class, you can return result set metadata by invoking:

- %SQL.StatementResult class properties.
- %SQL.StatementResult %GetMetadata() method, accessing %SQL.StatementMetadata class properties.

8.9.4.1 %SQL.StatementResult Properties

Following an Execute query operation, %SQL.StatementResult returns:

- The %StatementType property returns an integer code that corresponds to the SQL statement most recently executed. The following is a partial list of these integer codes: 1 = **SELECT**; 2 = **INSERT**; 3 = **UPDATE**; 4 = **DELETE** or **TRUNCATE TABLE**; 9 = **CREATE TABLE**; 15 = **CREATE INDEX**; 45 = **CALL**. For a complete list of these values, see %SQL.StatementResult.
- The %StatementTypeName calculated property returns the command name of the SQL statement most recently executed, based on the %StatementType. This name is returned in uppercase letters. Note that a **TRUNCATE TABLE** operation is returned as DELETE. An **INSERT OR UPDATE** is returned as INSERT, even when it performed an update operation.
- The %ResultColumnCount property returns the number of columns in the result set rows.

The following example shows these properties:

ObjectScript

```
set myquery = "SELECT TOP ? Name,DOB,Age FROM Sample.Person WHERE Age > ?"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(myquery)
    if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
set rset = tStatement.%Execute(10,55)
if rset.%SQLCODE=0 {
write "Statement type=",rset.%StatementType,!
write "Statement name=",rset.%StatementTypeName,!
write "Column count=",rset.%ResultColumnCount,!
write "End of metadata" }
else { write !,"SQLCODE=",rset.%SQLCODE," ",rset.%Message }
```

8.9.4.2 %SQL.StatementResult %GetMetadata()

Following an Execute, you can use the %SQL.StatementResult %GetMetadata() method to access the %SQL.StatementMetadata class properties. These are the same properties accessed by the %SQL.Statement %Metadata property following a Prepare.

The following example shows the properties:

ObjectScript

```

set myquery=2
set myquery(1)="SELECT Name AS VendorName,LastPayDate,MinPayment,NetDays,"
set myquery(2)="AVG(MinPayment),$HOROLOG,%TABLENAME FROM Sample.Vendor"
set tStatement = ##class(%SQL.Statement).%New()
set qStatus = tStatement.%Prepare(.myquery)
if qStatus'=1 {write "%Prepare failed:" do $System.Status.DisplayError(qStatus) quit}
set rset = tStatement.%Execute()
if rset.%SQLCODE=0 {
set rsmeta=rset.%GetMetadata()
set x=rsmeta.columns.Count()
set x=1
while rsmeta.columns.GetAt(x) {
set column=rsmeta.columns.GetAt(x)
write !,x," ",column.colName," is data type ",column.ODBCType
write " with a size of ",column.precision," and scale = ",column.scale
set x=x+1 }
}
else { write !,"SQLCODE=",rset.%SQLCODE," ",rset.%Message }
write !,"End of metadata"

```

Note that the result set metadata does not provide arguments metadata. This is because the Execute operation resolves all parameters. Therefore, in a result set, parameterCount = 0, and formalParameters contains no data.

8.10 Auditing Dynamic SQL

InterSystems IRIS supports optional [auditing](#) of Dynamic SQL statements. Dynamic SQL auditing is performed when the %System/%SQL/DynamicStatement [system audit event](#) is enabled. By default, this system audit event is not enabled.

If you enable %System/%SQL/DynamicStatement, the system automatically audits every %SQL.Statement dynamic statement that is executed system-wide. Auditing records information in the Audit Database.

To view the Audit Database, go to the Management Portal, **System Administration**, select **Security**, then **Auditing**, then **View Audit Database**. You can set the **Event Name** filter to DynamicStatement to limit the **View Audit Database** to Dynamic SQL statements. The Audit Database lists Time (a local timestamp), User, PID (process ID), and the Description of the event. The Description specifies the type of Dynamic SQL statement. For example, SQL SELECT Statement (%SQL.Statement) or SQL CREATE VIEW Statement (%SQL.Statement).

By selecting the **Details** link for an event you can list additional information, including the **Event Data**. The Event Data includes the SQL statement executed and the values of any arguments to the statement. For example:

```

SELECT TOP ? Name , Age FROM Sample . MyTest WHERE Name %STARTSWITH ?
/*#OPTIONS {"DynamicSQLTypeList":["1"]} */
Parameter values:
%CallArgs(1)=5
%CallArgs(2)="Fred"

```

The total length of Event Data, which includes the statement and parameters, is 3,632,952 characters. If the statement and parameters are longer than 3632952, the Event Data will be truncated.

InterSystems IRIS also supports auditing of ODBC and JDBC statements (**Event Name**=XDBCStatement), and auditing of Embedded SQL statements (**Event Name**=EmbeddedStatement).

9

Using the SQL Shell Interface

One way to test SQL statements is to execute them from the Terminal using the SQL Shell. This interactive SQL Shell allows you to execute SQL statements dynamically. The SQL Shell uses Dynamic SQL, which means that queries are prepared and executed at runtime. It accesses resources and performs operations within the current namespace.

Unless otherwise indicated, SQL Shell commands and SQL code are not case-sensitive.

9.1 Invoking the SQL Shell

You can use the `$$SYSTEM.SQL.Shell()` method to invoke the SQL Shell from the Terminal prompt using any of the following four options:

ObjectScript

```
DO $$SYSTEM.SQL.Shell()
```

ObjectScript

```
:sql
```

ObjectScript

```
DO ##class(%SQL.Shell).%Go("IRIS")
```

ObjectScript

```
SET sqlsh=##class(%SQL.Shell).%New()  
DO sqlsh.%Go("IRIS")
```

Regardless of how you invoke it, the SQL Shell returns the SQL Shell prompt, which displays as follows:

```
[SQL]termprompt>>
```

Where [SQL] is a literal indicating that you are in the SQL Shell, *termprompt* is the [configured terminal prompt](#), and >> is a literal indicating the SQL command line. By default, the SQL Shell prompt appears as follows `[SQL]nsp>>`, where *nsp* is the name of the current namespace.

At this prompt you can use either of the following Shell modes:

- Single line mode: at the prompt type a line of SQL code. To end the SQL statement, press **Enter**. By default, this both prepares and executes the SQL code (this is known as Immediate execute mode). For a query, the result set is displayed

on the terminal screen. For other SQL statements, the SQLCODE and row count values are displayed on the terminal screen.

- **Multiline mode:** at the prompt press **Enter**. This puts you in multiline mode. You can type multiple lines of SQL code, each new line prompt indicating the line number. (A blank line does not increment the line number.) To conclude a multiline SQL statement, type **GO** and press **Enter**. By default, this both prepares and executes the SQL code. For a query, the result set is displayed on the terminal screen. For other SQL statements, the SQLCODE and row count values are displayed on the terminal screen.

Multiline mode provides the following commands, which you type at the multiline prompt and then press **Enter**: **L** or **LIST** to list all SQL code entered thus far. **C** or **CLEAR** to delete all SQL code entered thus far. **C n** or **CLEAR n** (where *n* is a line number integer) to delete a specific line of SQL code. **G** or **GO** to prepare and execute the SQL code and return to single line mode. **Q** or **QUIT** to delete all SQL code entered thus far and return to single line mode. These commands are not case-sensitive. Issuing a command does not increment the line number of the next multiline prompt. Typing **?** at the multiline prompt lists these multiline commands.

To prepare an SQL statement, the SQL Shell first validates the statement, including confirming that the specified tables exist in the current namespace and the specified fields exist in the table. If not, it displays the appropriate SQLCODE.

The SQL Shell performs SQL privilege checking; you must have the appropriate privileges to access or modify a table, field, etc. For further details, refer to [SQL Users, Roles, and Privileges](#).

If the statement is valid and you have appropriate privileges, the SQL Shell echoes your SQL statement, assigning a sequential number to it. These numbers are assigned sequentially for the duration of the terminal session, regardless of whether you change namespaces and/or exit and re-enter the SQL Shell. These assigned statement numbers permit you to recall prior SQL statements, as described below.

To list all the available SQL Shell commands, enter **?** at the SQL prompt.

To terminate an SQL Shell session and return to the Terminal prompt, enter either the **Q** or **QUIT** command or the **E** or **EXIT** command at the SQL prompt. SQL Shell commands are not case-sensitive. The **Ctrl-C** command is disabled while in the SQL Shell.

The following is a sample SQL Shell session using the default parameter settings:

```
USER>DO $SYSTEM.SQL.Shell()
SQL Command Line Shell
-----
The command prefix is currently set to: <<nothing>>.
Enter q to quit, ? for help.
[SQL]USER>>SELECT TOP 5 Name,Home_State FROM Sample.Person ORDER BY Home_State
1. SELECT TOP 5 Name,Home_State FROM Sample.Person ORDER BY Home_State

Name                Home_State
Djokovic,Josephine W. AK
Klingman,Aviel P.    AK
Quine, Sam X.        AK
Xiang,Robert C.      AL
Roentgen,Alexandria Q. AR

5 Row(s) Affected
-----
[SQL]USER>>SELECT GETDATE()
2. SELECT GETDATE()

Expression_1
2009-09-29 11:41:42

1 Row(s) Affected
-----
[SQL]USER>>QUIT
USER>
```

The following is a multiline SQL Shell session using the default parameter settings:

```

USER>DO $SYSTEM.SQL.Shell()
SQL Command Line Shell
-----
The command prefix is currently set to: <<nothing>>.
Enter q to quit, ? for help.
[SQL]USER>> << entering multiline statement mode >>
      1>>SELECT TOP 5
      2>>Name,Home_State
      3>>FROM Sample.Person
      4>>ORDER BY Home_State
      5>>GO

1. SELECT TOP 5
   Name,Home_State
   FROM Sample.Person
   ORDER BY Home_State

Name                Home_State
Djokovic,Josephine W.  AK
Klingman,Aviel P.      AK
Quine, Sam X.          AK
Xiang,Robert C.        AL
Roentgen,Alexandria Q. AR

5 Row(s) Affected
[SQL]USER>>

```

9.1.1 GO Command

The SQL Shell **GO** command executes the most recent SQL statement. In single line mode, **GO** re-executes the SQL statement most recently executed. When in multiline mode, the **GO** command is used to execute the multiline SQL statement and exit multiline mode. A subsequent **GO** in single line mode re-executes the prior multiline SQL statement.

9.1.2 Input Parameters

The SQL Shell supports the use of input parameters using the “?” character in the SQL statement. Each time you execute the SQL statement, you are prompted to specify values for these input parameters. You must specify these values in the same sequence that the “?” characters appear in the SQL statement: the first prompt supplies a value to the first “?”, the second prompt supplies a value to the second “?”, and so on.

There is no limit on the number of input parameters. You can use input parameters to supply values to the TOP clause, the WHERE clause, and to supply expressions to the SELECT list; you cannot use input parameters to supply column names to the SELECT list.

You can specify a host variable as an input parameter value. At the input parameter prompt, specify a value prefaced by a colon (:). This value may be a public variable, an ObjectScript special variable, a numeric literal, or an expression. The SQL Shell then prompts you with “is this a literal (Y/N)?”. Specifying N (No) at this prompt (or just pressing Enter) means that the input value is parsed as a host variable. For example, `:myval` would be parsed as the value of the local variable `myval`; `^myval` would be parsed as the value of the global variable `^myval`; `:$HOROLOG` would be parsed as the value of the `$HOROLOG` special variable; `:3` would be parsed as the number 3; `:10-3` would be parsed as the number 7. Specifying Y (Yes) at this prompt means that the input value, including the colon preface, is supplied to the input parameter as a literal.

9.1.3 Executing ObjectScript Commands

Within the SQL Shell, you may wish to issue an [ObjectScript command](#). For example, to change the InterSystems IRIS namespace by using the **SET \$NAMESPACE** command to the namespace containing the SQL table or stored procedure you wish to reference. You can use the SQL Shell **!** command or **OBJ** command to issue an [ObjectScript command line](#),

consisting of one or more ObjectScript commands. (**OBJ** is an abbreviation for **OBJECTSCRIPT**.) The **!**, **OBJ**, and **OBJECTSCRIPT** commands are synonyms. Use of these commands is shown in the following example:

```
%SYS>DO $SYSTEM.SQL.Shell()
SQL Command Line Shell
-----
The command prefix is currently set to: <<nothing>>.
Enter q to quit, ? for help.
[SQL]%SYS>>! SET oldns=$NAMESPACE SET $NAMESPACE="USER" WRITE "changed the namespace"
changed the namespace
[SQL]USER>>OBJ SET $NAMESPACE=oldns WRITE "reverted to old namespace"
reverted to old namespace
[SQL]%SYS>>
```

The rest of the command line following the **OBJ** command is treated as ObjectScript code. A space is not required between the **!** and the ObjectScript command line. You can specify an **OBJ** command while in SQL Shell single-line mode or in SQL Shell multiline mode. The following example executes a **SELECT** query on a table defined in the USER namespace:

```
%SYS>DO $SYSTEM.SQL.Shell()
SQL Command Line Shell
-----
The command prefix is currently set to: <<nothing>>.
Enter q to quit, ? for help.
[SQL]%SYS>> << entering multiline statement mode >>
1>>OBJ SET $NAMESPACE="USER"

1>>SELECT TOP 5 Name,Home_State
2>>FROM Sample.Person
3>>GO
/* SQL query results */
[SQL]USER>>
```

Note that the **OBJ** statement does not advance the SQL line count.

In SQL Shell multiline mode, an **OBJ** command is executed upon line return, but an SQL statement is not issued until you specify **GO**. Thus, the following example is functionally identical to the previous example:

```
%SYS>DO $SYSTEM.SQL.Shell()
SQL Command Line Shell
-----
The command prefix is currently set to: <<nothing>>.
Enter q to quit, ? for help.
[SQL]%SYS>> << entering multiline statement mode >>
1>>SELECT TOP 5 Name,Home_State
2>>FROM Sample.Person
3>>OBJ SET $NAMESPACE="USER" WRITE "changed namespace"
changed namespace
3>>GO
/* SQL query results */
[SQL]USER>>
```

The following example uses an **OBJ** command to define a host variable:

```
USER>DO $SYSTEM.SQL.Shell()
SQL Command Line Shell
-----
The command prefix is currently set to: <<nothing>>.
Enter q to quit, ? for help.
[SQL]USER>> << entering multiline statement mode >>
1>>SELECT TOP :n Name,Home_State
2>>FROM Sample.Person
3>>OBJ SET n=5

3>>GO
```

9.1.4 Browsing the Namespace

The SQL Shell supports a **BROWSE** command that displays the schemas, tables, and views defined in or accessible from the current namespace. The display consists of several levels of prompts. To return to the previous prompt level, press the **Return** key at a prompt. Names are case-sensitive.

1. Type **BROWSE** at the SQL Shell prompt to list the schemas in the current namespace.
2. At the **Schema:** prompt, select a schema by name or by number. This lists the tables and views in the schema.
3. At the **Table/View:** prompt, select a table (T) or view (V) by name or by number. This displays table information then presents a list of options.
4. At the **Option:** prompt, select an option by number. You can use this option to list the fields or maps defined for the table.

Specify option 1 (Fields by name) or option 2 (fields by number) to display the **Field:** prompt. Specify option 3 (maps) to display the **Map:** prompt.
5. At the **Field:** prompt, select a field by number or by name, or specify * to list all fields. This lists detailed field information.

At the **Map:** prompt, select a map by number or by name, or specify * to list all maps. This lists detailed map information.

9.1.5 CALL Command

You can use the SQL Shell to issue the SQL **CALL** statement to call an [SQL stored procedure](#), as shown in the following example:

```
[SQL]USER>>CALL Sample.PersonSets('G','NY')
```

The SQL Shell issues an SQLCODE -428 error if the specified stored procedure does not exist in the current namespace.

The SQL Shell issues an SQLCODE -370 error if you specify more input parameters than are defined in the stored procedure. You can specify parameter values to the stored procedure using any combination of literals ('string'), host variables (:var), and input parameters (?).

- You can use [host variables](#) in a **CALL** statement, as shown in the following example:

```
[SQL]USER>>OBJ SET a="G", b="NY"
[SQL]USER>>CALL Sample.PersonSets(:a, :b)
```

- You can use input parameters ("?" characters) in a **CALL** statement, as shown in the following example:

```
[SQL]USER>>CALL Sample.PersonSets(?,?)
```

The SQL Shell prompts you for a value for each of these input parameters when the **CALL** statement is executed.

9.1.6 Executing an SQL Script File

The SQL Shell **RUN** command executes an SQL script file. The type of script file is determined by the DIALECT setting. The DIALECT default is IRIS (InterSystems SQL). For further details, see [RUN Command](#).

9.2 Storing and Recalling SQL Statements

9.2.1 Recall by Number

The SQL Shell automatically stores each successful SQL statement issued during the terminal session in a local cache and assigns it a sequential number. These numbers are used for recalling prior SQL statements during the current Terminal process. SQL Shell only assigns numbers to SQL statements that are successful; if an error occurs during preparation of

an SQL statement, no number is assigned. These number assignments are not namespace-specific. The following are the available recall by number commands:

- **#**: You can use **#** to list all of the prior cached SQL statements with their assigned numbers.
- **#n**: You can recall and execute a prior SQL statement by specifying **#n** at the SQL Shell prompt, where *n* is an integer that SQL Shell assigned to that statement.
- **#0**: You can recall and execute the most recently prepared SQL statement by specifying **#0** at the SQL Shell prompt. **#0** recalls the most recently prepared SQL statement, not necessarily the most recently executed SQL statement. Therefore, recalling and executing SQL statements has no effect on which SQL statement is recalled by **#0**.

Recalling an SQL statement by number does not assign a new number to the statement. SQL Shell assigns numbers sequentially for the duration of the Terminal session; exiting and re-entering the SQL Shell or changing namespaces have no effect on number assignment or the validity of prior assigned numbers.

To delete all number assignments, use **#CLEAR** and confirm this action at the displayed prompt. This deletes all prior number assignments and restarts number assignment with 1.

9.2.2 Recall by Name

You can optionally assign a name to an SQL statement, then recall the statement by name. These names are used for recalling prior SQL statements issued from any of the current user's Terminal processes. There are two ways to save and recall an SQL statement by name:

- Save to a global using **SAVEGLOBAL**; recall from a global using **OPEN**.
- Save to a file using **SAVE**; recall from a file using **LOAD**.

9.2.2.1 Saving to a Global

To assign a global name to the most recent SQL statement, use the SQL Shell command **SAVEGLOBAL name**, which can be abbreviated as **SG name**. You can then use the SQL Shell command **OPEN name** to recall the SQL statement from the global. If **EXECUTEMODE** is **IMMEDIATE**, the SQL Shell both recalls and executes the statement. If **EXECUTEMODE** is **DEFERRED**, the statement will be prepared but will not be executed until you specify the **GO** command.

Each time you use **OPEN name** to recall an SQL statement by global name, the SQL Shell assigns a new number to the statement. Both the old and new numbers remain valid for recall by number.

A *name* can contain any printable characters except the blank space character. Letters in a *name* are case-sensitive. A *name* can be of any length. A *name* is specific to the current namespace. You can save the same SQL statement multiple times with different names; all of the saved names remain valid. If you attempt to save an SQL statement using a name already assigned, SQL Shell prompts you whether you wish to overwrite the existing name, reassigning it to the new SQL statement.

Global names are assigned for the current namespace. You can list all assigned global names for the current namespace using the SQL Shell **L** (or **LIST**) command. Once assigned, a *name* is available to all of the current user's Terminal processes. An assigned *name* persists after the Terminal process that created it has ended. If there are no *name* assignments, **LIST** returns a “No statements saved” message.

To delete a global *name* assignment, use **CLEAR name**. To delete all global *name* assignments for the current namespace, use **CLEAR** and confirm this action at the displayed prompt.

9.2.2.2 Saving to a File

To assign a file name to the most recent SQL statement, use the SQL Shell command **SAVE name**. You can then use the SQL Shell command **LOAD name** to recall the SQL statement. If **EXECUTEMODE** is **IMMEDIATE**, the SQL Shell both recalls and executes the statement. Each time you use **LOAD name** to recall an SQL statement by file name, the SQL Shell assigns a new number to the statement. Both the old and new numbers remain valid for recall by number.

A *name* can contain any printable characters except the blank space character. Letters in a *name* are case-sensitive. A *name* can be of any length. A *name* is specific to the current namespace. You can save the same SQL statement multiple times with different names; all of the saved names remain valid. If you attempt to save an SQL statement using a name already assigned, SQL Shell prompts you whether you wish to overwrite the existing name, reassigning it to the new SQL statement.

Names are assigned for the current namespace. Once assigned, a *name* is available to all of the current user's Terminal processes. An assigned *name* persists after the Terminal process that created it has ended.

9.3 Purging Cached Queries

The SQL Shell provides a **PURGE** (abbreviated **P**) command to purge all cached queries in the current namespace. This command purges all cached queries in the namespace, not just those generated using the SQL Shell.

The %SYSTEM.SQL.PurgeQueries() method and the [Management Portal Actions drop-down list options](#) provides you with more specific options to purge only selected cached queries or to purge all of the cached queries in the namespace.

9.4 Configuring the SQL Shell

- You can configure the SQL Shell defaults system-wide using the Management Portal.
- You can configure an individual SQL Shell using SQL Shell Parameters. Changing an SQL Shell parameter overrides the system-wide default for the current invocation of the SQL Shell; it does not change the system-wide SQL Shell default value.

The following are the available SQL Shell configuration options, the corresponding shell parameters, and the default settings:

Management Portal Shell Configuration	Shell Parameter	Default
Select Mode	selectmode	Logical
SQL Dialect (TSQL)	dialect (TSQL)	IRIS
Schema Search Path	path	none
Result Column Alignment	colalign	Delimiter
Command Prefix (TSQL)	commandprefix (TSQL)	none
Result Output Display Mode	displaymode	Current Device
Display Path	displaypath	none
Display File	displayfile	none
Display File Translate Table	displaytranslatetable	none
Echo Mode	echo	On
Execute Mode	executemode	Immediate
Messages Mode	messages	On
IF condition to allow execution		1
	log	Off

The parameters labelled (TSQL) are principally used for executing Sybase or MSSQL Transact-SQL code from the SQL Shell. They are described in [Transact-SQL Support](#).

9.4.1 Configuring SQL Shell System-wide Defaults

Go to the Management Portal, select **System Administration, Configuration, SQL and Object Settings, SQL**. Select the **SQL Shell** tab. View and set the current default settings for SQL Shell system-wide.

If you change one or more configuration settings, this is indicated by an asterisk (*) in the upper left-hand corner of the screen immediately following the Management Portal path. For example, **System > Configuration > SQL ***. Press the **Save** button to accept the changes. The changes are activated and the asterisk disappears.

9.4.2 Configuring Parameters for an SQL Shell

SQL Shell configuration parameters are specific to the current SQL Shell invocation on the current Terminal process. Settings apply across namespaces. However, if you exit the SQL Shell, all SQL Shell parameters reset to system-wide default values. InterSystems IRIS provides system default values; you can establish different default values for the current process using **SET SAVE**, as described below.

The SQL Shell **SET** command (with no arguments) displays the current shell configuration parameters, as shown in the following example. In this example, the **SET** shows the system default values, which are the values established when you invoke the SQL Shell:

```
[SQL]USER>>SET
commandprefix = ""
dialect = IRIS
displayfile =
displaymode = currentdevice
displaypath =
displaytranslatetable =
echo = on
executemode = immediate
log = off
messages = on
path = SQLUser
selectmode = logical
[SQL]USER>>
```

To display the current setting for a single configuration parameter, specify **SET param**. For example, **SET SELECTMODE** returns the current selectmode setting.

You can use the SQL Shell **SET** command to set a shell configuration parameter. A set value persists for the duration of the SQL Shell invocation; each time you invoke the SQL Shell, the parameters reset to default values. **SET** can use either of the following syntax forms:

```
SET param value SET param = value
```

Both *param* and *value* are not case-sensitive. Spaces are permitted, but not required, before and after the equal sign.

The SQL Shell **SET SAVE** command saves the current shell configuration parameter settings as the user defaults. These defaults are applied to all subsequent SQL Shell invocations from the current process. They are also applied as SQL shell defaults to any subsequently invoked SQL Shell on a Terminal process invoked by that user. They remain in effect until specifically reset. Using **SET SAVE** does not affect currently running SQL Shell invocations.

The SQL Shell **SET CLEAR** command clears (resets to system defaults) the current shell configuration parameter settings for the current process. InterSystems IRIS applies this reset to defaults to subsequent SQL Shell invocations by the current process, or any new Terminal process invoked by the current user. **SET CLEAR** does not affect currently running SQL Shell invocations.

Neither **SET SAVE** nor **SET CLEAR** change the system-wide SQL Shell default settings configured and displayed using the Management Portal.

9.4.3 Setting COLALIGN

You can use **SET COLALIGN** to specify the whitespace format used to display query resultset data and column headers. The available options are:

- **delimiter**: Resultset header/data columns will be aligned based on the standard delimiter (tab). This is the default.
- **header**: Resultset header/data columns will be aligned based on the length of the column header and the standard delimiter (tab).
- **data**: Resultset header/data columns will be aligned based on the precision/length of the column data property and the standard delimiter (tab).

For further details, see [%Display\(\) method](#).

9.4.4 Setting DISPLAYMODE and DISPLAYTRANSLATETABLE

You can use **SET DISPLAYMODE** to specify the format used to display query data, as shown in the following example:

```
USER>DO $SYSTEM.SQL.Shell()
SQL Command Line Shell
-----
The command prefix is currently set to: <<nothing>>.
Enter q to quit, ? for help.
[SQL]USER>>SET DISPLAYMODE XML

displaymode = xml
[SQL]USER>>
```

The **DISPLAYMODE** default is **CURRENTDEVICE**, which displays the query data on the Terminal in **TXT** format. You can specify **SET DISPLAYMODE = CUR** to restore the **CURRENTDEVICE** default.

The other available options are **TXT**, **HTML**, **PDF**, **XML**, and **CSV**. The selection of a format determines the file type. InterSystems IRIS creates a file of this type, writes the query data to the file, and, when possible, launches the appropriate program to display this query data file. For all options except **TXT**, a second file is created to record result set messages. By default, SQL Shell creates these files in the InterSystems IRIS mgr\Temp\ directory and assigns a randomly generated file name with the appropriate file type suffix. The generated Message file name is the same as the data file name, except for the appended string “Messages”. For the **HTML**, **PDF**, and **XML** options, the Messages file has the same file type suffix as the query data file. For the **CSV** option, the Messages file has the **TXT** file type suffix.

The following is an example of the files created when **DISPLAYMODE = TXT**:

```
C:\InterSystems\IRIS\mgr\Temp\sGm7qLdVZn5VbA.txt
C:\InterSystems\IRIS\mgr\Temp\sGm7qLdVZn5VbAMessages.txt
```

Each time you run a query, the SQL Shell creates a new pair of files with randomly generated file names.

If **DISPLAYMODE** is **TXT** or **CSV**, you can optionally specify the name of a [translate table](#) to apply when performing format conversion. You can specify either **SET DISPLAYTRANSLATE** or **SET DISPLAYTRANSLATETABLE**. Translate table name values are case-sensitive.

If **DISPLAYMODE** is set to a value other than **CURRENTDEVICE**, any query result set data containing a control character results in a generated Warning message. Generally, control characters only appear in query result set data when it is in Logical mode. For example, data in a List structure contains control characters when displayed in Logical mode. For this reason, it is recommended that when you set **DISPLAYMODE** to a value other than **CURRENTDEVICE** that you also set **SELECTMODE** to either **DISPLAY** or **ODBC**.

9.4.4.1 Setting DISPLAYFILE and DISPLAYPATH

If DISPLAYMODE is set to a value other than CURRENTDEVICE, you can specify the target file location using the DISPLAYFILE and DISPLAYPATH parameters:

- **DISPLAYFILE:** set this parameter to a simple file name with no suffix; for example, SET DISPLAYFILE myfile. You can also set this parameter to a partially-qualified path, which InterSystems IRIS appends to the DISPLAYPATH value or the default directory, creating subdirectories as needed; for example, SET DISPLAYFILE mydir\myfile. If DISPLAYPATH is set, the system creates a file with this file name in the specified directory; if DISPLAYPATH is not set, the system creates a file with this file name in the InterSystems IRIS mgr\Temp\ directory.
- **DISPLAYPATH:** set this parameter to an existing fully-qualified directory path structure ending in a slash (“/”) or backslash (“\”), depending on operating system platform. If DISPLAYFILE is set, the system creates a file with the DISPLAYFILE name in this directory; if DISPLAYFILE is not set, the system creates a file with a randomly-generated name in this directory. If the DISPLAYPATH directory does not exist, InterSystems IRIS ignores DISPLAYPATH and DISPLAYFILE settings and instead uses the default directory and default randomly-generated file name.

When necessary, the system automatically adds a slash (or backslash) to the end of your DISPLAYPATH value and/or removes a slash (or backslash) from the beginning of your DISPLAYFILE value to create a valid fully-qualified directory path.

The following example sets DISPLAYMODE, DISPLAYFILE, and DISPLAYPATH:

```
[SQL]USER>>SET DISPLAYMODE XML
displaymode = xml
[SQL]USER>>SET DISPLAYFILE myfile
displayfile = myfile
[SQL]USER>>SET DISPLAYPATH C:\temp\mydir\
displaypath = C:\temp\mydir\
[SQL]USER>>
```

When you execute a query the SQL Shell will generate the following files. The first contains the query data. The second contains any messages resulting from the query execution:

```
C:\temp\mydir\myfile.xml
C:\temp\mydir\myfileMessages.xml
```

If you specify neither DISPLAYFILE or DISPLAYPATH, the system creates files in the Mgr\Temp\ directory for your InterSystems IRIS installation (for example, C:\InterSystems\IRIS\Mgr\Temp\) with a randomly generated file name.

If DISPLAYMODE is not set to CURRENTDEVICE, each time you run a query with DISPLAYFILE set, any existing data in the named file and the corresponding Messages file is replaced by the new query data. Each time you run a query with DISPLAYFILE not set, the SQL Shell creates a new file with a randomly generated file name and a new corresponding Messages file.

If DISPLAYMODE is set to CURRENTDEVICE, the DISPLAYFILE and DISPLAYPATH parameters have no effect.

9.4.5 Setting EXECUTEMODE

The SQL Shell supports immediate and deferred SQL statement execution. Immediate execution prepares and executes the specified SQL statement when you press Enter. Deferred execution prepares the statement when you press Enter, but does not execute it until you specify **GO** at the SQL prompt.

The available options are `SET EXECUTEMODE IMMEDIATE` (the default), `SET EXECUTEMODE DEFERRED`, and `SET EXECUTEMODE` to display the current mode setting. The following example sets the execute mode:

```
USER>DO $SYSTEM.SQL.Shell()
SQL Command Line Shell
-----
The command prefix is currently set to: <<nothing>>.
Enter q to quit, ? for help.
[SQL]USER>>SET EXECUTEMODE DEFERRED

Executemode = deferred
[SQL]USER>>
```

Deferred execution allows you to prepare multiple SQL queries, then recall them by name or number for execution. To execute a prepared SQL statement, recall the desired statement (from the appropriate namespace) then specify **GO**.

The following example shows the preparation of three queries in Deferred mode. The first two are saved and assigned a recall name; the third is not assigned a name, but can be recalled by number:

```
[SQL]USER>>SELECT TOP 5 Name,Home_State FROM Sample.Person
1. SELECT TOP 5 Name,Home_State FROM Sample.Person
[SQL]USER>>SAVE 5sample
Query saved as: 5sample
[SQL]USER>>SELECT TOP 5 Name,Home_State FROM Sample.Person ORDER BY Home_State
2. SELECT TOP 5 Name,Home_State FROM Sample.Person ORDER BY Home_State
[SQL]USER>>SAVE 5ordered
Query saved as: 5ordered
[SQL]USER>>SELECT Name,Home_State FROM Sample.Person ORDER BY Home_State
3. SELECT Name,Home_State FROM Sample.Person ORDER BY Home_State
[SQL]USER>>
```

The following example shows the deferred mode execution of two of the queries defined in the previous example. Note that this example recalls one query by name (upon recall the SQL Shell gives it a new number), and one query by number:

```
[SQL]USER>>OPEN 5ordered
SELECT TOP 5 Name,Home_State FROM Sample.Person ORDER BY Home_State
4. SELECT TOP 5 Name,Home_State FROM Sample.Person ORDER BY Home_State
-----
[SQL]USER>>GO

Name                Home_State
Djokovic,Josephine W.  AK
Klingman,Aviel P.      AK
Quine, Sam X.          AK
Xiang,Robert C.        AL
Roentgen,Alexandria Q. AR

5 Row(s) Affected
-----
[SQL]USER>>#3
SELECT Name,Home_State FROM Sample.Person ORDER BY Home_State
3. SELECT Name,Home_State FROM Sample.Person ORDER BY Home_State
-----
[SQL]USER>>GO
.
.
.
```

9.4.6 Setting ECHO

You can use **SET ECHO** to specify whether to echo the query results to the SQL Shell. If you specify **SET ECHO=OFF**, the query is prepared, a cached query is defined, and the query is executed. No query results are displayed to the Terminal. This is shown in the following example:

```
[SQL]USER>>set echo=off
echo = off
[SQL]USER>>SELECT Name, Age FROM Sample.MyTest
4.      SELECT Name, Age FROM Sample.MyTest

statement prepare time(s)/globals/cmds/disk: 0.0002s/5/155/0ms
execute time(s)/globals/cmds/disk: 0.0001s/0/105/0ms
cached query class: %sqlcq.USER.cls3
-----
[SQL]USER>>
```

If you specify **SET ECHO=ON** (the default) the query results are displayed to the Terminal. This is shown in the following example:

```
[SQL]USER>>set echo=on
echo = on
[SQL]USER>>SELECT Name, Age FROM Sample.MyTest
5.      SELECT Name, Age FROM Sample.MyTest

Name      Age
Fred Flintstone 41
Wilma Flintstone 38
Barney Rubble 40
Betty Rubble 42

4 Rows(s) Affected
statement prepare time(s)/globals/cmds/disk: 0.0002s/5/155/0ms
execute time(s)/globals/cmds/disk: 0.0002s/5/719/0ms
cached query class: %sqlcq.USER.cls3
-----
[SQL]USER>>
```

SET ECHO is only meaningful if **DISPLAYMODE=CURRENTDEVICE** (the default).

SET ECHO and **SET MESSAGES** specify what is displayed on the Terminal; they do not affect the prepare or execution of the query. If both **SET MESSAGES=OFF** and **SET ECHO=OFF**, the query is prepared, a cached query is created, and query execution creates a query result set, but nothing is returned to the Terminal.

9.4.7 Setting MESSAGES

You can use **SET MESSAGES** to specify whether to display the query error message (if unsuccessful), or query execution information (if successful):

- If query execution is unsuccessful: If you specify **SET MESSAGES=OFF**, nothing is displayed to the Terminal. If you specify **SET MESSAGES=ON** (the default) the query error message is displayed, such as the following: **ERROR #5540: SQLCODE: -30 Message: Table 'SAMPLE.NOTABLE' not found.**
- If query execution is successful: If you specify **SET MESSAGES=OFF**, only the query results and the line `n Rows(s) Affected` are displayed to the Terminal. If you specify **SET MESSAGES=ON** (the default) the query results and the line `n Rows(s) Affected` are followed by the statement prepare metrics, the statement execution metrics, and the name of the generated cached query.

Prepare and Execute metrics are measured in elapsed time (in fractional seconds), total number of global references, total number of commands executed, and disk read latency (in milliseconds).

The information displayed when `SET MESSAGES=ON` is not changed by setting `DISPLAYMODE`. Some `DISPLAYMODE` options create both a query result set file and a messages file. This messages file contains result set messages, not the query prepare and execute messages displayed to the Terminal when `SET MESSAGES=ON`.

SET MESSAGES and **SET ECHO** specify what is displayed on the Terminal; they do not affect the prepare or execution of the query. If both `SET MESSAGES=OFF` and `SET ECHO=OFF`, a successful query is prepared, a cached query is created, and query execution creates a query result set, but nothing is returned to the Terminal.

9.4.8 Setting LOG

You can use **SET LOG** to specify whether to log SQL Shell activity to a file. The available options are:

- **SET LOG OFF**: The default. InterSystems IRIS does not log activity for the current SQL Shell.
- **SET LOG ON**: InterSystems IRIS logs SQL Shell activity to the default log file.
- **SET LOG *pathname***: InterSystems IRIS logs SQL Shell activity to the file specified by *pathname*.

`SET LOG ON` creates a log file in `IRIS\mgr\namespace`, where *namespace* is the name of the current namespace for the process. This default log file is named `xsqlnnnn.log`, where *nnnn* is the process ID (pid) number for the current process.

By default, a log file is specific to the current process and the current namespace. To log SQL Shell activity from multiple processes and/or from multiple namespaces in the same log, specify `SET LOG pathname` for each process and/or namespace using the same *pathname*.

A log file can be suspended and resumed. Once a log file has been created, `SET LOG OFF` suspends writing to that log file. `SET LOG ON` resumes writing to the default log file. Log `restarted: date time` is written to the log file when logging resumes. `SET LOG ON` always activates the default log file. Thus, if you suspend writing to a specified *pathname* log file, you must specify `SET LOG pathname` when resuming.

Activating a log file creates a copy of SQL Shell activity displayed on the terminal; it does not redirect SQL Shell terminal output. The SQL Shell log records SQL errors for failed SQL execution and the SQL code and resulting row count for successful SQL execution. The SQL Shell log does not record result set data.

If a log is already active, specifying `SET LOG ON` has no effect. If a log is already active, specifying `SET LOG pathname` suspends the current log and activates the log specified by *pathname*.

9.4.9 Setting PATH

You can use **SET PATH *schema*** to set the [schema search path](#), which SQL uses to supply the correct schema name for an unqualified table name. *schema* can be a single schema name, or a comma-separated list of schema names, as shown in the following example:

```
[SQL]USER>>SET PATH cinema,sample,user
```

SET PATH with no argument deletes the current schema search path, reverting to the [system-wide default schema name](#).

If **SET PATH *schema*** is not specified, or the table is not found in the specified schemas, SQL Shell uses the [system-wide default schema name](#). For further details on schema search paths, see the [#sqlcompile path](#) macro.

9.4.10 Setting SELECTMODE

You can use **SET SELECTMODE** to specify the mode used to display query data.

```
USER>DO $SYSTEM.SQL.Shell()
SQL Command Line Shell
-----
The command prefix is currently set to: <<nothing>>.
Enter q to quit, ? for help.
[SQL]USER>>SET SELECTMODE DISPLAY

selectmode = display
[SQL]USER>>
```

The available options are DISPLAY, LOGICAL, and ODBC. LOGICAL is the default. To determine the current mode, specify **SET SELECTMODE** without a value:

```
[SQL]USER>>SET SELECTMODE

selectmode = logical
[SQL]USER>>
```

%List data is encoded using non-printing characters. Therefore, when selectmode=logical, SQL Shell displays a %List data value as a **\$LISTBUILD** statement, such as the following: `$lb("white", "Green")`. Time data type data supports fractional seconds. Therefore, when selectmode=odbc, SQL Shell displays fractional seconds, which does not correspond to the ODBC standard. The actual ODBC TIME data type truncates fractional seconds.

For further details on SelectMode options, see [Data Display Options](#).

You can also use **SET SELECTMODE** to specify whether input data will be converted from display format to logical storage format. For this data conversion to occur, the SQL code must have been compiled with a select mode of RUNTIME. At execution time, **SET SELECTMODE** must be set to LOGICAL (the default). For further details, refer to the [INSERT](#) or [UPDATE](#) statement.

9.5 SQL Metadata, Query Plan, and Performance Metrics

9.5.1 Displaying Metadata

The SQL Shell supports the **M** or **METADATA** commands to display metadata information about the current query.

For each result set item, this command lists the following metadata: **Column Name** ([SQL field name](#)), **Type** ([ODBC data type integer code](#)), **Prec** ([precision](#) or [maximum length](#)), **Scale** ([maximum fractional digits](#)), **Null** (boolean: 1=NULL allowed, 0=NULL not allowed), **Label** (header label, see [column alias](#)), **Table** ([SQL table name](#)), **Schema** ([schema name](#)), **CType** (client data type, see the `%SQL.StatementColumn` *clientType* property).

For further details, refer to [Select-item Metadata](#).

For further details on InterSystems SQL Shell commands, enter ? at the SQL prompt, or see `%SYSTEM.SQL.Shell()`.

9.5.2 SHOW STATEMENT

You can executing a query, then issue **SHOW STATEMENT** or **SHOW ST** to display the prepared SQL statement. By default, you must execute the query. You can avoid executing the query by setting `executemode=deferred`, issuing the query, then issuing the **SHOW STATEMENT** SQL Shell command.

SHOW STATEMENT information consists of the Implementation Class (cached query name), the Arguments (a comma-separated list of the actual argument values, such as the TOP clause and WHERE clause literal values), and the Statement Text (the literal text of the SQL command, including letter case and argument values).

9.5.3 EXPLAIN and Show Plan

There are two ways to display the query plan for an SQL query; both can display alternate query plans if desired.

- **EXPLAIN:** preface a SELECT query with the EXPLAIN command. For example:

```
SQLJUSER>>EXPLAIN SELECT Name FROM Sample.MyTable WHERE Name='Fred Rogers'
```

- **SHOW PLAN:** issue a query, then issue the SHOW PLAN Shell command. For example:

```
SQLJUSER>>SELECT Name FROM Sample.MyTable WHERE Name='Fred Rogers'
SQLJUSER>>SHOW PLAN
```

The **EXPLAIN** SQL command displays query plan information about a specified SELECT query without executing the query. EXPLAIN ALT allows you to display alternate queries plans. EXPLAIN STAT returns performance statistics as well as the query plan(s). EXPLAIN can only be used to return a query plan for a SELECT query; it does not return a query plan for other commands such as INSERT, UPDATE, or DELETE statements that perform a query operation.

The **SHOW PLAN** SQL Shell command allows you to display the query plan information for the last query successfully issued by the SQL Shell. **SHOW PLAN** can be used for any SQL command that performs a query operation, including SELECT, INSERT, UPDATE, and DELETE. By default, you must execute the query. You can avoid executing the query by setting **executemode=deferred**, issuing the query, then issuing one of the following SQL Shell commands:

- **SHOW PLAN, SHOW PL** (or simply **SHOW**) to display query plan information about the current query. The query plan can be used for debugging and optimizing the performance of a query. It specifies how the query executes, including the use of indexes and a cost value for the query. A query plan can be returned for the following statements: SELECT, DECLARE, non-cursor UPDATE or DELETE, and INSERT...SELECT. This command has a **V (VERBOSE)** option.
- **SHOW PLANALT** to display alternate show plans for the current query. This command has a **V (VERBOSE)** option. For further details, refer to [Alternate Show Plans](#).

For further details on InterSystems SQL Shell commands, enter ? at the SQL prompt, or see %SYSTEM.SQL.Shell().

You can generate query plans from ObjectScript using the **\$\$SYSTEM.SQL.Explain()** method. For further details refer to [Query Execution Plans](#).

For further details on interpreting a query plan, see [Interpreting an SQL Query Plan](#).

9.5.4 SQL Shell Performance

Following the successful execution of an SQL statement, the SQL Shell displays four statement prepare values (**times(s)/globals/cmds/disk**) and four statement execute values (**times(s)/globals/cmds/disk**):

- The statement **prepare time** is the time it took to prepare the dynamic statement. This includes the time it took to generate and compile the statement. It includes the time it took to find the statement in the statement cache. Thus, if a statement is executed, then recalled by number or recalled by name, the prepare time on the recalled statement is near zero. If a statement is prepared and executed, then re-executed by issuing the GO command, the prepare time on the re-execution is zero.
- The elapsed **execute time** is the elapsed time from the call to **%Execute()** until the return from **%Display()**. It does not include wait time for input parameter values.

The statement **globals** is the count of global references, **cmds** is the count of SQL commands executed, and **disk** is the disk latency time in milliseconds. The SQL Shell keeps separate counts for the Prepare operation and the Execute operation.

These performance values are only displayed when DISPLAYMODE is set to currentdevice, and MESSAGES is set to ON. These are the SQL Shell default settings.

9.6 Transact-SQL Support

By default, the SQL Shell executes InterSystems SQL code. However, the SQL Shell can be used to execute Sybase or MSSQL code.

9.6.1 Setting DIALECT

By default, the SQL Shell parses code as InterSystems SQL. You can use SET DIALECT to configure the SQL Shell to execute Sybase or MSSQL code. To change the current dialect, SET DIALECT to Sybase, MSSQL, or IRIS. The default is Dialect=IRIS. These SET DIALECT options are not case-sensitive.

The following is an example of the executing a MSSQL program from the SQL Shell:

```
USER>DO $SYSTEM.SQL.Shell()
SQL Command Line Shell
-----
The command prefix is currently set to: <<nothing>>.
Enter q to quit, ? for help.
[SQL]USER>>SET DIALECT MSSQL

dialect = MSSQL
[SQL]USER>>SELECT TOP 5 name + '-' + ssn FROM Sample.Person
1.      SELECT TOP 5 name + '-' + ssn FROM Sample.Person

Expression_1
Zweifelhofer, Maria H.-559-20-7648
Vonnegut, Bill A.-552-41-2071
Clinton, Terry E.-757-30-8013
Bachman, Peter U.-775-59-3756
Avery, Emily N.-833-18-9563

5 Rows(s) Affected
statement prepare time: 0.2894s, elapsed execute time: 0.0467s.
-----
[SQL]USER>>
```

The Sybase and MSSQL dialects support a limited subset of SQL statements in these dialects. They support the **SELECT**, **INSERT**, **UPDATE**, and **DELETE** statements. They support the **CREATE TABLE** statement for permanent tables, but not for temporary tables. **CREATE VIEW** is supported. **CREATE TRIGGER** and **DROP TRIGGER** are supported. However, this implementation does not support transaction rollback should the **CREATE TRIGGER** statement partially succeed but then fail on class compile. **CREATE PROCEDURE** and **CREATE FUNCTION** are supported.

9.6.2 Setting COMMANDPREFIX

You can use **SET COMMANDPREFIX** to specify a prefix (usually a single character) that must be appended to subsequent SQL Shell commands. This prefix is not used on SQL statements issued from the SQL Shell prompt. The purpose of this prefix is to prevent ambiguity between SQL Shell commands and SQL code statements. For example, SET is an SQL Shell command; SET is also an SQL code statement in Sybase and MSSQL.

By default, there is no command prefix. To establish a command prefix, SET COMMANDPREFIX=*prefix*, with *prefix* specified without quotation marks. To revert to having no command prefix, SET COMMANDPREFIX="". The following example shows the command prefix / (the slash character) being set, used, and reverted:

```

USER>DO $SYSTEM.SQL.Shell()
SQL Command Line Shell
-----

The command prefix is currently set to: <<nothing>>.
Enter q to quit, ? for help.
[SQL]USER>>SET COMMANDPREFIX=/

commandprefix = /
[SQL]USER>>/SET LOG=ON

log = xsql4148.log
[SQL]USER>> << entering multiline statement mode >>
1>>SELECT TOP 3 Name, Age
2>>FROM Sample.Person
3>>/GO
9.      SELECT TOP 3 Name, Age
        FROM Sample.Person

Name      Age
Frith, Jose M.   13
Finn, William D. 15
Ximines, Uma Y.  44

3 Rows(s) Affected
statement prepare time: 0.0010s, elapsed execute time: 0.0014s.
-----
[SQL]USER>>/SET COMMANDPREFIX

commandprefix = /
[SQL]USER>>/SET COMMANDPREFIX=""

commandprefix = ""
[SQL]USER>>SET COMMANDPREFIX

commandprefix =
[SQL]USER>>

```

When a command prefix is set, the command prefix is required for all SQL Shell commands, except ?, #, and GO; these three SQL Shell commands can be issued with or without the command prefix.

The SQL Shell displays the current command prefix as part of the SQL Shell initialization, when you issue a SET or a SET COMMANDPREFIX command, and at the end of the ? commands option display.

9.6.3 RUN Command

The SQL Shell RUN command executes an SQL script file. You must SET DIALECT before issuing a RUN command to specify either IRIS (InterSystems SQL), Sybase (Sybase TSQL), or MSSQL (Microsoft SQL); the default dialect is IRIS. You can either invoke RUN *scriptname* or just invoke RUN and be prompted for the script file name.

RUN loads the script file, then prepares and executes each statement contained in the file. Statements in the script file must be delimited, usually either with a GO line, or with a semicolon (;). The RUN command prompts you to specify the delimiter.

The SQL script file results are displayed on the current device and, optionally, in a log file. Optionally, a file containing statements that failed to prepare can be produced.

The RUN command returns prompts to specify these options, as shown in the following example:

```

[SQL]USER>>SET DIALECT=Sybase

dialect = Sybase
[SQL]USER>>RUN

Enter the name of the SQL script file to run: SybaseTest

Enter the file name that will contain a log of statements, results and errors (.log): SyTest.log
SyTest.log

```

Many script files contain statements not supported by IRIS SQL.
Would you like to log the statements not supported to a file so they
can be dealt with manually, if applicable? Y=> y
Enter the file name in which to record non-supported statements (_Unsupported.log): SyTest_Unsupported.log

Please enter the end-of-statement delimiter (Default is 'GO'): GO=>

Pause how many seconds after error? 5 => 3

Sybase Conversion Utility (v3)

Reading source from file:

Statements, results and messages will be logged to: SyTest.log

.
.
.

9.6.4 TSQL Examples

The following SQL Shell example creates a Sybase procedure AvgAge. It executes this procedure using the Sybase EXEC command. It then changes the dialect to InterSystems IRIS and executes the same procedure using the InterSystems SQL CALL command.

```
[SQL]USER>>SET DIALECT Sybase
```

```
dialect = Sybase
```

```
[SQL]USER>> << entering multiline statement mode >>
```

```
1>>CREATE PROCEDURE AvgAge
```

```
2>>AS SELECT AVG(Age) FROM Sample.Person
```

```
3>>GO
```

```
12. CREATE PROCEDURE AvgAge
```

```
AS SELECT AVG(Age) FROM Sample.Person
```

```
statement prepare time: 0.1114s, elapsed execute time: 0.4364s.
```

```
[SQL]USER>>EXEC AvgAge
```

```
13. EXEC AvgAge
```

```
Dumping result #1
```

```
Aggregate_1
```

```
44.35
```

```
1 Rows(s) Affected
```

```
statement prepare time: 0.0956s, elapsed execute time: 1.1761s.
```

```
[SQL]USER>>SET DIALECT=IRIS
```

```
dialect = IRIS
```

```
[SQL]USER>>CALL AvgAge()
```

```
14. CALL AvgAge()
```

```
Dumping result #1
```

```
Aggregate_1
```

```
44.35
```

```
1 Rows(s) Affected
```

```
statement prepare time: 0.0418s, elapsed execute time: 0.0040s.
```

```
[SQL]USER>>
```

9.7 Other Ways of Executing SQL

You can execute a single line of SQL code from the Terminal command line without invoking the SQL Shell by using the **\$SYSTEM.SQL.Execute()** method. The following examples show how this method is used from the Terminal prompt:

```
USER>SET result=$SYSTEM.SQL.Execute("SELECT TOP 5 name,dob,ssn FROM Sample.Person")
USER>DO result.%Display()

USER>SET result=$SYSTEM.SQL.Execute("CALL Sample.PersonSets('M','MA')")
USER>DO result.%Display()
```

If the SQL statement contains an error, the **Execute()** method completes successfully; the **%Display()** method returns the error information, such as the following:

```
USER>DO result.%Display()
[SQLCODE: <-29>:<Field not found in the applicable tables>]
[%msg: < Field 'GAME' not found in the applicable tables^ SELECT TOP ? game ,>]
0 Rows Affected
USER>
```

The **Execute()** method also provides optional [SelectMode](#), [Dialect](#), and [ObjectSelectMode](#) parameters.

InterSystems IRIS supports numerous other ways to write and execute SQL code:

- [Embedded SQL](#): SQL code embedded within ObjectScript code.
- [Dynamic SQL](#): using %SQL.Statement class methods to execute SQL statements from within ObjectScript code.
- [Management Portal SQL Interface](#): executing Dynamic SQL from the InterSystems IRIS Management Portal using the **Execute Query** interface.

10

Using the Management Portal SQL Interface

This topic describes how to perform SQL operations from the InterSystems IRIS® data platform Management Portal. The Management Portal interface uses Dynamic SQL, which means that queries are prepared and executed at runtime. The Management Portal interface is intended as an aid for developing and testing SQL code against small data sets. It is not intended to be used as an interface for SQL execution in a production environment.

The Management Portal also provides various options to configure SQL. For further details, refer to [SQL and Object Settings Pages](#).

For general information on using the Management Portal, select the **Help** button found in the upper left corner. You can report an issue with InterSystems software to the [InterSystems Worldwide Response Center \(WRC\)](#) from the Management Portal by using the **Contact** button found in the top right corner. To report an SQL performance issue to the WRC, see [Tools](#).

Note: The Management Portal interface to performing SQL operations described on this page is meant only as a simple interface for consulting your SQL schema objects, statements, and other related data and metadata. As a web interface, it is not well suited for long-running queries or those that return very large datasets. Such queries are better directed through a client tool that connections to InterSystems IRIS, such as JDBC, ODBC, or DB-API.

10.1 Management Portal SQL Facilities

InterSystems IRIS allows you to examine and manipulate data using SQL tools from the InterSystems IRIS Management Portal. The starting point for this is the Management Portal **System Explorer** option. From there you select the **SQL** option. This displays the SQL interface, which allows you to:

- [Execute SQL Query](#) — write and run SQL commands. You can execute an SQL query against existing tables and data, create a table, or insert, update, or delete table data. You can either write the SQL code directly into a text box (including SELECT, INSERT, UPDATE, DELETE, CREATE TABLE and other SQL statements), retrieve a statement from the SQL history into the text box, drag and drop a table into the text box to generate a query (SELECT statement), or compose a query (SELECT statement) using the Query Builder interface.
- [Filtering Schema Contents](#) — on the left side of the screen display the [SQL schemas](#) for the current namespace or a filtered subset of these schemas, with each schema's tables, views, procedures, and cached queries. You can select an individual table, view, procedure, or cached query to display its [Catalog Details](#).

- **Wizards** — execute a wizard to perform [data import](#), [data export](#), or data migration. Execute a wizard to link to tables or views or to link to stored procedures.
- **Actions** — define a view; print out the details of a table definition; improve the performance of a query by running Tune Table and/or rebuilding indexes; or perform clean up by purging unwanted cached queries and/or dropping unwanted table, view, or procedure definitions.
- **Open Table** — display the current data in the table in Display mode. This is commonly not the complete data in the table: both the number of records and the length of data in a column are restricted to provide a manageable display.
- **Tools** — execute one of the following tools: SQL Runtime Statistics, Index Analyzer, Alternate Show Plans, Generate Report, Import Report.
- **Documentation** — Allows you to view the [list of SQL error codes](#) and the [list of SQL reserved words](#). If you select a table, allows you to display Class Documentation.

In addition to the features under the **System Explorer** option, you can view monitor currently running queries with the [SQL Process View](#) under the **System Operation** option.

10.1.1 Selecting a Namespace

All SQL operations occur within a specific namespace. Therefore, you must first specify which namespace you wish to use by clicking the name of the current namespace displayed at the top of the SQL interface page. This displays the list of available namespaces, from which you can make your selection.

You can set your Management Portal default namespace. From the Management Portal select **System Administration**, **Security**, **Users**. Click the name of the desired user. This allows you to edit the user definition. From the General tab, select a **Startup Namespace** from the drop-down list. Click **Save**. If no startup namespace is selected, it defaults to %SYS.

10.1.2 User Customization

Many of the Management Portal SQL operations are automatically customized for each user. If you set a filter, maximum, mode, or other option in the [Execute Query](#) tab or the [SQL Statements](#) tab, this user-specified value is retained for future use. When the same user activates the Management Portal, the user's prior settings are shown. Restarting InterSystems IRIS returns all options to default values.

Namespace selection is not customized. It reverts to the user definition [Startup Namespace](#).

For details on using Filter options, see [Filtering Schema Contents](#).

10.2 Executing SQL Query

From the Management Portal select **System Explorer**, then **SQL**. Select a namespace by clicking the name of the current namespace displayed at the top of the page; this displays the list of available namespaces. To execute an SQL query, there are three options:

- **Execute Query**: write and execute an SQL command. The SQL command can be a **SELECT** query, or it can be an InterSystems SQL DDL or DML statement; the statement is validated on the InterSystems IRIS server when it executes.
- **Show History**: recall a previously run SQL statement, and either re-run it, or modify it and then run it. All executed statements are listed, including those that did not successfully execute.
- **Query Builder**: invoke the SQL Query Builder (which is exclusively for creating **SELECT** statements). Within the SQL Query Builder, create an SQL SELECT query by choosing tables, columns, WHERE clause predicates, and other query components. You can then run the query by clicking **Execute Query**.

10.2.1 Writing SQL Statements

The **Execute Query** text box allows you to write not only **SELECT** and **CALL** queries, but most SQL statements, including DDL statements such as **CREATE TABLE**, and DML statements such as **INSERT**, **UPDATE**, and **DELETE**.

You can specify SQL code in the **Execute Query** text box using the following:

- Type (or paste) the SQL code into the text box. The SQL code area does not colorize SQL text or provide any syntax or existence validation. However, it does provide automatic spelling verification. You can erase the contents of the text box using the **X** icon.
- Use the [Show History](#) list to select a prior SQL statement. The selected statement is copied into the text box. Upon execution, this statement moves to the top of the Show History list. Note that Show History lists all previously executed statements, including those that failed execution.
- Use [Table Drag and Drop](#) to construct SQL code in the text box.
- You can use the **Query Builder**, rather than the **Execute Query** text box, to specify and execute a **SELECT** query. A **SELECT** query executed using **Query Builder** is not shown in **Execute Query** or listed in **Show History**.

SQL code in the **Execute Query** text box can include:

- **Input Parameters.** If you specify input parameters, such as `TOP ?` or `WHERE Age BETWEEN ? AND ?`, the **Execute** button displays the **Enter Parameter Value for Query** window, with entry fields for each input parameter in the order specified in the query. For further details on **input parameters**, refer to [Executing an SQL Statement](#).
- **Whitespace Characters.** You can specify multiple blank spaces, single and multiple line returns. The tab key is disabled; when copying code into the SQL code area, existing tabs are converted to single blank spaces. Line returns and multiple blank spaces are not retained.
- **Comments.** The SQL code area supports single-line and multiline [comments](#). Comments are retained and shown in the [Show History](#) display. Comments are not shown in the [Show Plan](#) Statement Text display or in cached queries.
- **Queries that Return Multiple Result Sets.**

After writing SQL code in the text box, you can click the **Show Plan** button to check the SQL code without executing the SQL code. If the code is valid, **Show Plan** displays a Query Plan. If the code is invalid, **Show Plan** displays an SQLCODE error value and message. You can also use the **Show Plan** button to display this information for the most-recently-executed SQL code.

To [execute the SQL code](#), click the **Execute** button.

10.2.1.1 Table Drag and Drop

You can generate a query by dragging a table (or view) from the Tables list (or Views list) on the left side of the screen and dropping it into the **Execute Query** text box. This generates a **SELECT** with a *select-item* list of all of the [non-hidden](#) fields in the table and a FROM clause specifying the table. You can then further modify this query and execute it using the **Execute** button.

You can also drag and drop a procedure name from the Procedures list on the left side of the screen.

10.2.2 Execute Query Options

The SQL execution interface has the following options:

- The [Select Mode](#) drop-down list with a **SELECT** specifies the format that the query should use to supply data values (for example, in the WHERE clause) and to display data values in the query result set. The options are **Display Mode** (the default), **ODBC Mode**, and **Logical Mode**. For further details on these options, refer to [Data Display Options](#).

The [Select Mode](#) drop-down list with an **INSERT** or **UPDATE** allows you to specify whether input data will be converted from display format to logical storage format. For this data conversion to occur, the SQL code must have been compiled with a select mode of **RUNTIME**. At execution time, the Select Mode drop-down list must be set to **Logical Mode**. For further details, refer to the [INSERT](#) or [UPDATE](#) statements.

Select Mode is meaningful for data types whose Logical storage format differs from the desired display format (Display or ODBC), such as InterSystems IRIS dates and times and ObjectScript %List structured data.

- The **Max** field allows you to limit how many rows of data to return from a query. It can be set to any positive integer, including 0. Once you set **Max**, that value is used for all queries for the duration of the session, unless explicitly changed. The default is 1000. The maximum value is 100,000, which is the default if you enter no value (set **Max** to null), enter a value greater than 100,000, or a non-numeric value. You can also limit the number of rows of data to return by using a TOP clause. **Max** has no effect on other SQL statements, such as **DELETE**.

If you click the **more** option, the SQL execution interface displays the following additional options:

- **Dialect:** the dialect of SQL code. Available values are IRIS, Sybase, and MSSQL. The default is IRIS. Sybase and MSSQL are described in the [InterSystems Transact-SQL \(TSQL\) Migration Guide](#). Note that the dialect you select becomes the [user customized default](#) the next time you access the Management Portal.

If **Dialect** is Sybase or MSSQL, you can specify multiple SQL commands in the **Execute Query** text box. All insert, delete, and update commands are executed first, then **SELECT** commands in the order specified. Multiple result sets are returned in separate tabs.

- **Row Number:** a check box specifying whether to include a row count number for each row in the result set display. Row Number is a sequential integer assigned to each row in the result set. This is simply a numbering of the returned rows, it does not correspond to either the [RowID](#) or the [%VID](#). The row number column header name is #. The default is to not display row numbers.
- **Execute Query in the foreground:** a check box specifying whether or not to run the query in the foreground. Simple queries run in the foreground are often significantly faster than those run in the background. However, long queries run in the foreground may cause the Management Portal to be unresponsive during query execution. The default is to run all queries in the background.

All of these options are [user customized](#).

10.2.3 Show Plan Button

The **Show Plan** button displays the Statement Text and the [Query Plan](#) including the relative cost (overhead) of the current query plan for the query in the page's text box. You can invoke Show Plan from either the **Execute Query** or **Show History** interface.

A query plan is generated when a query is Prepared (compiled); this occurs when you write a query and select the **Show Plan** button. You do not have to execute a query to show its query plan.

When viewing a query plan through the Management Portal, high level descriptions of how the plan processes modules and subqueries are hyperlinked to the sections that explain in detail how these parts of the query are processed, allowing you to easily traverse the plan.

Show Plan displays an SQLCODE and error message when invoked for an invalid query.

10.2.4 SQL Statement Results

After writing SQL code in the **Execute Query** text box, you can execute the code by clicking the **Execute** button. This either successfully executes the SQL statement and displays the results below the code window, or the SQL code fails. If the SQL code fails, it displays an error message (in red) below the code window; pressing the **Show Plan** button displays the SQLCODE error and error message.

Execute Query SQL code execution is performed as a background process. While the code is executing, the **Execute** button is replaced by a **Cancel** button. This allows you to cancel execution of a long-running query.

10.2.4.1 Query Data Display

The result set is returned as a table with a row counter displayed as the first column (#), if the **Row Number box** is checked. The remaining columns are displayed in the order specified. The **RowID** (ID field) may be displayed or hidden. Each column is identified by the column name (or the column alias, if specified). An aggregate, expression, subquery, host variable, or literal SELECT item is either identified by a column alias (if specified), or by the word **Aggregate_**, **Expression_**, **Subquery_**, **HostVar_**, or **Literal_** followed by the SELECT item sequence number (by default).

If a row column contains no data (NULL) the result set displays a blank table cell. Specifying an empty string literal displays a **HostVar_** field with a blank table cell. Specify NULL displays a **Literal_** field with a blank table cell.

If a selected field is a date, time, timestamp, or %List-encoded field, the displayed value depends on the **Display Mode**.

The following display features are unique to the Management Portal SQL interface **Execute Query** results display and **Open Table** data display:

- A **stream field** of data type %Stream.GlobalCharacter displays the actual data (up to 100 characters) as a string. If the data in a stream field is longer than 100 characters, the first 100 characters of the data are displayed followed by an ellipsis (. . .) indicating additional data.
- A stream field of data type %Stream.GlobalBinary displays as <binary>.
- A string data field displays the actual data in full with line wrapping as needed.
- An integer field is right-aligned within the result table cell. RowID, numeric, and all other fields are left-aligned.

These result display features do not occur when the same query is executed using Dynamic SQL code, the SQL Shell, or Embedded SQL code.

If the specified query returns more than one result set, **Execute Query** displays these result sets as named tabs: **Result #1**, **Result #2**, and so forth.

10.2.4.2 Query Execution Metrics

If successful, **Execute Query** displays performance information and the name of the cached query routine. If there is resulting data to display, this appears below the performance information. The execution information includes the Row count, the Performance, the Cached Query showing the cached query name, and Last update specifying the timestamp for the last execution of the query.

- **Row count:** For a DDL statement such as CREATE TABLE, displays Row count : 0 if the operation was successful. For a DML statement such as INSERT, UPDATE, or DELETE, displays the number of rows affected.

For a SELECT, displays the number of rows returned as a result set. Note that the number of rows returned is governed by the **Max** setting, which may be lower than the number of rows which could have been selected. For multiple result sets, the number of rows for each result set are listed, separated by the / character. A query that specifies one or more aggregate functions (and no selected fields) always displays Row count : 1 and returns the results of expressions, subqueries, and aggregate functions, even if the FROM clause table contains no rows. A query that specifies no aggregate functions and selects no rows always displays Row count : 0 and returns no results, even if the query specifies only expressions and subqueries that do not reference the FROM clause table. A query with no FROM clause always displays Row count : 1 and returns the results of expressions, subqueries, and aggregate functions.

- **Performance:** measured in elapsed time (in fractional seconds), total number of global references, total number of commands executed, and disk read latency (in milliseconds). If a cached query exists for the query these performance metrics are for executing the cached query. Therefore, the first execution of a query will have substantially higher performance metrics than subsequent executions. If the specified query returns more than one result set, these performance metrics are totals for all of the queries.

To analyze these performance metrics in greater depth you can run MONLBL (the monitor line-by-line utility) and specify the Routine Name using the asterisk wildcard as %sqlcq*. Refer to [Examining Routine Performance Using ^%SYS.MONLBL](#).

- **Cached Query:** the automatically generated cached query class name. For example, %sqlcq.USER.cls2 indicating the second cached query in the USER namespace. Each new query is assigned a new cached query name with the next consecutive integer. By clicking this cached query name, you can display information about the cached query and further links to display its Show Plan or to Execute the cached query. (For a DDL statement, see SQL Commands That Are Not Cached.)

Closing the Management Portal or stopping InterSystems IRIS does not delete cached queries or reset cached query numbering. To purge cached queries, invoke the %SYSTEM.SQL.PurgeQueries() method.

Not all SQL statements result in a cached query. A query that is the same as an existing cached query, except for literal substitution values (such as the TOP clause value and predicate literals) does not create a new cached query. Some SQL statements are not cached, including DDL statements and privilege assignment statements. Non-query SQL statements, such as **CREATE TABLE**, also display a cached query name. However, this cached query name is created then immediately deleted; the next SQL statement (query or non-query) reuses the same cached query name.

- **Frozen state:** if a query plan is frozen, the [frozen plan state](#) (in parentheses) is displayed after the cached query class name. For example (Frozen/Explicit). If the query plan is not frozen, nothing is displayed here.
- **Last update:** the date and time that the last **Execute Query** (or other SQL operation) was performed. This timestamp is reset each time the query is executed, even when repeatedly executing the identical query.
- Successful execution also provides a **Print** link that displays the Print Query window, which gives you the options to either print or export to a file the query text and/or the query result set. The clickable **Query** and **Result** toggles enable you to display or hide the query text or the query result set. The displayed query result set includes the namespace name, the result set data and row count, a timestamp, and the cached query name. (Note that the timestamp is the time when the Print Query window was invoked, not the time when the query was executed.) The Print Query window **Print** button prints a screenshot of the Print Query window. The Export to File check box displays options to specify an export file format (xml, html, pdf, txt, csv) and an export file pathname. The **Export** option ignores the **Query** and **Result** toggles and always exports only the result set data (by default to: exportQuery.pdf) and the row count (by default to: exportQueryMessages.pdf); the query text, namespace, timestamp, and cached query name are not included.

If unsuccessful, **Execute Query** displays an error message. You can click the [Show Plan](#) button to display the corresponding SQLCODE error value and message.

10.2.5 Show History

Click **Show History** to list prior SQL statements executed during the current session. Show History lists all SQL statements invoked from this interface, both those successfully executed and those whose execution failed. By default, SQL statements are listed by Execution Time, with the most recently executed appearing at the top of the list. You can click on any of the column headings to order the SQL statements in ascending or descending order by column values. Executing an SQL Statement from the **Show History** listing updates its Execution Time (local date and time stamp), and increments its Count (number of times executed).

You can filter the **Show History** listing, as follows: in the **Filter** box specify a string then press the Tab key. Only those history items that contain that string will be included in the refreshed listing. The filter string can either be a string found in the SQL Statement column (such as a table name), or it can be a string found in the Execution Time column (such as a date). The filter string is not case-sensitive. A filter string remains in effect until you explicitly change it.

You can modify and execute an SQL statement from **Show History** by selecting the statement, which causes it to be displayed in the **Execute Query** text box. In **Execute Query** you can modify the SQL code and then click **Execute**. Making any change to an SQL statement retrieved from **Show History** causes it to be stored in **Show History** as a new statement; this include

changes that do not affect execution, such as changing letter case, whitespace, or comments. Whitespace is not shown in **Show History**, but it is preserved when an SQL statement is retrieved from **Show History**.

You can execute (re-run) an unmodified SQL statement directly from the **Show History** list by clicking the **Execute** button found to the right of the SQL statement in the Show History listing.

You can also select buttons for **Plan**, **Print**, or **Delete**. The **Print** button displays the Print Query window, which gives you the options to either print or export to a file the query text and/or the query result set. The **Delete** button deletes the SQL statement from the history; the **Delete All** button at the end of the **Show History** list deletes all the SQL statements in the history.

Note that the **Show History** listing is not the same as the list of cached queries. **Show History** lists all invoked SQL statements from the current session, including those that failed during execution.

10.2.6 Other SQL Interfaces

InterSystems IRIS supports numerous other ways to write and execute SQL code. These include:

- **Embedded SQL**: SQL code embedded within ObjectScript code.
- **Dynamic SQL**: using %SQL.Statement class methods (or other result set class methods) to execute SQL statements from within ObjectScript code.
- **SQL Shell**: executing Dynamic SQL from the Terminal using the SQL Shell interface.

10.3 Filtering Schema Contents

The left side of the Management Portal SQL interface allows you to view the contents of a schema (or multiple schemas that match a filter pattern).

1. Specify which namespace you wish to use by clicking the name of the current namespace displayed at the top of the SQL interface page. This displays the list of available namespaces, from which you can make your selection.
2. Apply a **Filter** or select a schema from the **Schema** drop-down list.

You can use the **Filter** field to filter the lists by typing a search pattern. You can filter for schemas, or for table/view/procedure names (items) within a schema or within multiple schemas. A search pattern consists of the name of a schema, a dot (.), and the name of an item — each name composed of some combination of literals and wildcards. Literals are not case-sensitive. The wildcards are:

- asterisk (*) meaning 0 or more characters of any type.
- underscore (_) meaning a single character of any type.
- an apostrophe (') inversion prefix meaning “not” (everything except).
- a backslash (\) escape character: _ means a literal underscore character.

For example, **S*** returns all schemas that begin with **S**. **S*.Person** returns all **Person** items in all schemas that begin with **S**. ***.Person*** returns all items that begin with **Person** in all schemas. You can use a comma-separated list of search patterns to select all items that fulfil any one of the listed patterns (OR logic). For example, ***.Person*, *.Employee*** selects all **Person** and **Employee** items in all schemas.

To apply a **Filter** search pattern, click the refresh button, or press the Tab key.

A **Filter** search pattern remains in effect until you explicitly change it. The “x” button to the right of the **Filter** field clears the search pattern.

3. Selecting a schema from the **Schema** drop-down list overrides and resets any prior **Filter** search pattern, selecting for a single schema. Specifying a **Filter** search pattern overrides any prior **Schema**.
4. Optionally, use the drop-down “applies to” list to specify which categories of item to list: Tables, Views, Procedures, Cached Queries, or all of the above. The default is All. Any category that was specified in the “applies to” drop-down list is limited by **Filter** or **Schema**. Those categories not specified in “applies to” continue to list all of the items of that category type in the namespace.
5. Optionally, click the **System** check box to include system items (items whose names begin with %). The default is to not include system items.
6. Expand the list for a category to list its items for the specified **Schema** or specified **Filter** search pattern. When you expand a list, any category that contains no items does not expand.
7. Click on an item in an expanded list to display its **Catalog Details** on the right side of the SQL interface.

If the selected item is a Table or a Procedure, the **Catalog Details Class Name** information provides a link to the corresponding class documentation.

Note that Filter settings are [user customized](#), and are retained for future use for that user.

10.3.1 Browse Tab

The Browse tab provides a convenient way to quickly view all the schemas in a namespace, or a filtered subset of the schemas in the namespace. You can select **Show All Schemas** or **Show Schemas with Filter**, which applies the [filter](#) specified on the left side of the Management Portal SQL interface. By clicking on the Schema Name heading, you can list the schemas in ascending or descending alphabetical order.

Each listed schema provides links to lists of its associated Tables, Views, Procedures, and Queries (cached queries). If the schema has no items of that type, a hyphen (rather than a named link) is shown in that schema list column. This enables you to quickly get information about the contents of schemas.

Clicking a Tables, Views, Procedures, or Queries link displays a table of basic information about those items. By clicking on a table heading, you can sort the list by that column’s values in ascending or descending order. The Procedures table always includes Extent procedures, regardless of the Procedures setting on the left side of the Management Portal SQL interface.

You can get more information on individual Tables, Views, Procedures, and Cached Queries using the [Catalog Details](#) tab. Selecting a Table or View from the Browse tab *does not* activate the **Open Table** link for that table.

10.4 Catalog Details

The Management Portal provides **Catalog Details** information for each [Table](#), [View](#), [Procedure](#), and [Cached Query](#). The [filtering schema contents \(left side\) component](#) of the Management Portal SQL interface allows you to select an individual item to display its **Catalog Details**.

10.4.1 Catalog Details for a Table

The following **Catalog Details** options are provided for each table:

- **Table Info:** Table Type: either TABLE, [GLOBAL TEMPORARY](#), or SYSTEM TABLE (system tables are only displayed if the [System check box](#) is selected), Owner name, Last Compiled timestamp, External and [Readonly](#) boolean values, [Class Name](#), [Extent Size](#), the name of the [Child Table\(s\) and/or the Parent Table](#) (if relevant) and one or more [References](#) fields to other tables (if relevant), whether it uses the [%Storage.Persistent](#) default storage class, whether it [Supports](#)

[Bitmap Indexes](#), the [RowID field name](#), a list of the fields that RowId is based on (if relevant), and whether the [table is sharded](#). If there is an explicit shard key, it displays the shard key fields.

Class Name is a link to the corresponding entry in the *InterSystems Class Reference* documentation. The **Class Name** is a unique package.class name derived from the table name by removing punctuation characters, as described in [Identifiers and Class Entity Names](#).

References only appears in Table Info if there is one or more references from a field in the current table to another table. These references to other tables are listed as links to the Table Info for the referenced table.

Sharded: if the table is a shard-master table, the Table Info displays the name of the shard-local class and table with a link to the corresponding entry in the class documentation. If the table is a shard-local table, the Table Info displays the name of the shard-master class and table with a link to the corresponding entry in the class documentation. Shard-local tables are only displayed if the [System check box](#) is selected.

This option also provides a modifiable value for the **Number of rows to load when table is opened**. This sets the maximum number of rows to display in [Open Table](#). The available range is from 1 to 10,000; the default is 100. The Management Portal corrects a value outside the available range to a valid value: 0 corrects to 100; a fractional number rounds up to the next higher integer; a number greater than 10,000 corrects to 10,000.

- **Fields:** a list of the fields (columns) in the table showing: [Field Name](#), [Datatype](#), [Column #](#), [Required](#), [Unique](#), [Collation](#), [Hidden](#), [MaxLen](#), [MaxVal](#), [MinVal](#), [Stream](#), [Container](#), [SQLType](#), [Reference To](#), [Version Column](#), [Selectivity](#), [Outlier Selectivity](#), and [Average Field Size](#).
- **Maps/Indices:** a list of the indexes defined for the table showing: [Index Name](#), [SQL Map Name](#), [Columns](#), [Type](#), [Block Count](#), [Map Inherited](#), and [Global](#).

Index Name is the index property name and follows property naming conventions; when generated from an SQL index name, punctuation characters (such as underscores) in the SQL index name are stripped out. The **SQL Map Name** is the [SQL name for the index](#). A generated **SQL Map Name** is the same as the Constraint Name, and follows the same naming conventions (described below). **Columns** specifies a field or a comma-separated list of fields specified for the index; it may specify the [index collation type](#) and full schema.table.field reference, as in the following example: `$$$SQLUPPER( {Sample.People.Name} )`. **Type** can be one of the following: [Bitmap Extent](#), [Data/Master](#), [Index](#) (standard index), [Bitmap](#), or [Bitslice](#) index, and the [Unique](#) constraint. The **Block Count** contains both the count and how that count was determined: set explicitly by the class author (Defined), computed by [the collected statistics utility](#) (Measured), or estimated by the class compiler (Estimated). If **Map Inherited?** is Yes, this map was inherited from a superclass. **Global** is the name of the subscripted global containing the index data. The naming conventions for index globals are described in [Index Global Names](#). You can supply this global name to [ZWRITE](#) to display the index data.

This option also provides a link for each index to rebuild the index.

- **Triggers:** a list of the triggers defined for the table showing: [Trigger Name](#), [Time Event](#), [Order](#), [Code](#).
- **Constraints:** a list of the constraints for fields of the table showing: [Constraint Name](#), [Constraint Type](#), and [Constraint Data](#) (field name(s) listed in parentheses). Constraints include [primary key](#), [foreign key](#), and [unique](#) constraints. A primary key is, by definition, unique; it is only listed once. This option list constraints by constraint name; a constraint involving multiple fields is listed once with Constraint Data displaying a comma-separated list of the component fields. The Constraint Type can be UNIQUE, PRIMARY KEY, Implicit PRIMARY KEY, FOREIGN KEY, or Implicit FOREIGN KEY.

You can also list constraints by invoking INFORMATION_SCHEMA.CONSTRAINT_COLUMN_USAGE. This list constraints by field name. The following example returns the name of the field and the name of the constraint for all UNIQUE, PRIMARY KEY, FOREIGN KEY and CHECK constraints:

SQL

```
SELECT Column_Name,Constraint_Name FROM INFORMATION_SCHEMA.CONSTRAINT_COLUMN_USAGE WHERE
TABLE_SCHEMA='Sample' AND TABLE_NAME='Person'
```

If the table is defined with `%PUBLICROWID` and no explicit primary key is defined, the RowID field is listed with a Constraint Type of `Implicit PRIMARY KEY` with the Constraint Name `RowIDField_As_PKey`.

For explicit constraints, the Constraint Name is generated as follows:

- **Constraint specified in the field definition:** For example, `FullName VARCHAR(48) UNIQUE` or `FullName VARCHAR(48) PRIMARY KEY`. The Constraint Name value for the field is a generated value with the syntax `TABLERNAME_CTYPE#`, where CTYPE is `UNIQUE`, `PKEY`, or `FKEY`, and # is a sequential integer assigned to unnamed constraints in the order specified in the table definition. For example, if `FullName` has the 2nd unnamed unique constraint (excluding the ID field) in the `MyTest` table, the generated Constraint Name for `FullName` would be `MYTEST_UNIQUE2`; if `FullName` is the primary key and the 3rd unnamed constraint (excluding the ID field) specified in the `MyTest` table, the generated Constraint Name for `FullName` would be `MYTEST_PKEY3`.
- **CONSTRAINT keyword named constraint clause:** For example, `CONSTRAINT UFullName UNIQUE(FirstName, LastName)` or `CONSTRAINT PKName PRIMARY KEY(FullName)`, the Constraint Name is the specified unique constraint name. For example, `FirstName` and `LastName` in the `MyTest` table would each have the Constraint Name `UFullName`; `FullName` would have the Constraint Name `PKName`.
- **Unnamed constraint clause:** For example, `UNIQUE(FirstName, LastName)` or `PRIMARY KEY (FullName)`. The Constraint Name value is a generated value with the syntax `TABLERNAMECType#`, where CType is `Unique`, `PKey`, or `FKey`, and # is a sequential integer assigned to unnamed constraints in the order specified in the table definition. For example, if `FirstName` and `LastName` have the 2nd unnamed unique constraint (excluding the ID field) in the `MyTest` table, the generated Constraint Name for `FirstName` and `LastName` would be `MYTESTUnique2`; if `FullName` is the primary key and the 3rd unnamed constraint (excluding the ID field) specified in the `MyTest` table, the generated Constraint Name for `FullName` would be `MYTESTPKey3`. (Note mixed uppercase/lowercase and absence of an underscore.)

If a field is involved in more than one uniqueness constraint, it is listed separately for each Constraint Name.

- **Cached Queries:** a list of the cached queries for the table showing: Routine name, Query text, Creation Time, Source, Query Type.
- **Table's SQL Statements:** a list of the SQL Statements generated for this table. Same information as namespace-wide [SQL Statements](#) display.

10.4.2 Catalog Details for a View

Management Portal SQL interface also provides **Catalog Details** for views, procedures, and cached queries:

The following **Catalog Details** options are provided for each view:

- **View Info:** **Owner** name, **Last Compiled** timestamp. This timestamp updates when you use the **Edit View** link and save changes.

Defined as Read Only and **View is Updateable** booleans: if view definition included `WITH READ ONLY`, these are set to 1 and 0 respectively. Otherwise, if the view is defined from a single table they are set to 0 and 1; if the view is defined from joined tables they are set to 0 and 0. You can change this option using the **Edit View** link.

Class Name is a unique package.class name derived from the view name by removing punctuation characters, as described in [Identifiers and Class Entity Names](#).

Check Option is only listed if the view definition included the `WITH CHECK OPTION` clause. It can be `LOCAL` or `CASCADDED`. You can change this option using the **Edit View** link.

Class Type is `VIEW`. It provides an **Edit View** link to edit the view definition.

View Text is the `SELECT` statement used to define the view. You can change the view definition using the **Edit View** link.

The list of fields includes the Field Name, Data Type, MAXLEN Parameter, MAXVAL Parameter, MINVAL Parameter, BLOB (%Stream.GlobalCharacter or %Stream.GlobalBinary field), Length, Precision, and Scale.

- **View's SQL Statements:** a list of the SQL Statements generated for this view. Same information as namespace-wide [SQL Statements](#) display.

10.4.3 Catalog Details for a Stored Procedure

The following **Catalog Details** options are provided for each procedure:

- [Stored Procedure Info:](#)

Class Name is a unique package.class name derived from the procedure name by pre-pending a type identifier ('func', 'meth', 'proc', or 'query') to the class name (for example, the SQL function MyProc becomes funcMyProc) and removing punctuation characters, as described in [Identifiers and Class Entity Names](#). **Class Document** is a link to the corresponding entry in the class documentation. **Procedure Type** (for example, function). **Method or Query Name** the name of the generated class method or class query; this name is generated described in [Identifiers and Class Entity Names](#). The **Run Procedure** link provides an option to interactively run the procedure.

- **Stored Procedure's SQL Statements:** a list of the SQL Statements generated for this stored procedure. Same information as namespace-wide [SQL Statements](#) display.

10.4.4 Catalog Details for a Cached Query

Cached Query provides the full text of the query, an option to [show the query execution plan](#), and an option to interactively execute the cached query.

10.5 Wizards

- **Data Import Wizard** — Runs [a wizard to import data](#) from a text file into an InterSystems IRIS class.
- **Data Export Wizard** — Runs [a wizard to export data](#) from an InterSystems IRIS class into a text file.
- **Data Migration Wizard** — Runs a wizard to migrate data from an external source and create an InterSystems IRIS class definition to store it.
- **Link Table Wizard** — Runs a wizard to link to tables or views in external sources as if it were native InterSystems IRIS data.
- **Link Procedure Wizard** — Runs a wizard to link to procedures in external sources.

10.6 Actions

- **Create View** — Displays a page for [creating a view](#).
- **Print Catalog** — Allows you to print complete information about a table definition. Clicking **Print Catalog** displays a print preview. By clicking Indices, Triggers, and/or Constraints on this print preview you can include or exclude this information from the catalog printout.
- **Purge Cached Queries** — Provides three options for purging cached queries: purge all cached queries for the current namespace, purge all cached queries for the specified table, or purge only selected cached queries.

- **Tune Table Information** — Run the [collected statistics facility](#) against the selected table. This calculates the [selectivity](#) of each table column against the current data. A selectivity value of 1 indicates a column that defined as unique (and therefore has all unique data values). A selectivity value of 1.0000% indicates a column not defined as unique for which all current data values are unique values. A percentage value greater than 1.0000% indicates the relative number of duplicate values for that column in the current data. By using these selectivity values you can determine what indexes to define and how to use these indexes to optimize performance.
- **Tune all tables in schema** — Run the [collected statistics facility](#) against all of the tables belonging to a specified schema in the current namespace.
- **Rebuild Table's Indices** — Rebuild all indexes for the specified table.
- **Drop this item** — Drop (delete) the specified table definition, view definition, procedure, or cached query. You must have the appropriate privileges to perform this operation. **Drop** cannot be used on a [table created by defining a persistent class](#), unless the table class definition includes [[DdlAllowed](#)]. Otherwise, the operation fails with an SQLCODE-300 error with the %msg DDL not enabled for class 'Schema.tablename'. **Drop** cannot be used on a table if the corresponding persistent class has a [subclass](#) (a derived class); the operation fails with an SQLCODE -300 error with the %msg Class 'Schema.tablename' has derived classes and therefore cannot be dropped via DDL.

If a class is defined as a linked table, the Drop action drops the linked table on the local system, even if the linked table class is not defined as DdlAllowed. Drop does not drop the actual table this link references that resides on the server.
- **Export All Statements** — [Exports all SQL Statements](#) in the current namespace. SQL Statements are exported in XML format. You can choose to export to a file, or export to a browser display page.
- **Import Statements** — [Imports SQL Statements](#) from an XML file into the current namespace.

10.7 Open Table

If you select a table or view on the left side of the Management Portal SQL interface, the [Catalog Details](#) for that table or view are displayed. The **Open Table** link at the top of the page also becomes active. **Open Table** displays the actual data in the table (or accessed via the view). The data is shown in Display format.

By default, the first 100 rows of data are displayed; this default is modifiable by setting the **Number of rows to load when table is opened** in the [Catalog Details](#) tab Table Info. If there are more rows in the table than this number of rows to load value, the `More data . . .` indicator is shown at the bottom of the data display. If there are fewer rows in the table than this number of rows to load value, the `Complete` indicator is shown at the bottom of the data display.

A column of data type %Stream.GlobalCharacter displays the actual data (up to 100 characters) as a string. Additional data beyond the first 100 characters is indicated by an ellipsis (. . .).

A column of data type %Stream.GlobalBinary displays as <binary>.

10.8 Tools

The **System Explorer**, **SQL**, **Tools** drop-down list provides access to the following tools. These are the same tools available from **System Explorer**, **Tools**, **SQL Performance Tools**:

- [SQL Runtime Statistics](#): user interface to generate SQL Runtime Statistics for a specified query.
- [Index Analyzer](#): user interface for gathering various types of index analysis for a specified schema.

- [Alternative Show Plans](#): user interface to generate Alternate Show Plans for a specified query.
- [Generate Report](#) to submit an SQL query performance report to InterSystems WRC (Worldwide Response Center customer support). To use this reporting tool you must first get a WRC tracking number from the WRC.
- **Import Report** to import an existing WRC report by file name. For InterSystems use only.

10.9 SQL Process View

Within the Management Portal, you can view which queries are currently running on an instance of InterSystems IRIS. To do so, navigate to the view the Current SQL Statements page (**System Operation > SQL Activity**).

The table on this page lists each of the currently running SQL statements, based on the data stored in INFORMATION_SCHEMA.CURRENT_STATEMENTS. The table, as presented in the Management Portal, contains the following columns:

- **Process** — the process ID in which the SQL Statement is running
- **Server** — the instance that the query is running on
- **User** — the user that issued the query
- **Namespace** — the namespace in which the query is running
- **Type** — the type of the currently running query
- **Elapsed time** — the current amount of time that the query has been running
- **Statement** — the text of the query

To view further details about a statement, select an individual row in the table of currently executing SQL statements. These details include the following information:

- **Transaction?** — whether the query is being executed as part of a transaction
- **Parameters** — the parameters that the query is currently using to run, listed in the order in which they have been substituted into the query
- **Cached Query** — the name of the cached query

In addition, on the Current SQL Statements page, you can also view the execution statistics, which provide information about the performance of the selected query. These statistics are presented both for all time and for the previous week. These execution statistics include the following information:

- **Time executed**
- **Average runtime**
- **Runtime standard deviation**
- **Average rowcount** (the average number of rows that the query returns)
- **Average commands** (the average number of commands the system performs as it processes the query)

10.10 Diagnostic Logs

From the home page of the Management Portal, navigate to **System Operation > System Logs > SQL Diagnostics Logs** to view a log that details the performance of **LOAD DATA** commands that have run within the current namespace.

Each command is viewable on a different row. When a row is selected, the details appear on the right. These details include:

- **Start time:** The time that the LOAD DATA command started running.
- **User:** The user that issued the LOAD DATA command.
- **Status:** The current status of the statement.
- **Process ID:** The process ID that the LOAD DATA command ran in.
- **SQLCODE:** The SQLCODE returned when the LOAD DATA command has finished running.
- **Input record count:** The number of input records from the data source.
- **Error count:** The number of errors loading specific rows of the data source.
- **Statement:** The full statement text of the command that was run.
- **Messages:** Any messages logged as the LOAD DATA statement ran, including errors.

To purge older entries, select the check box for the rows you would like to delete, then click the **Delete** button at the top of the screen.

11

Defining Tables

This topic describes how you create tables in InterSystems SQL.

11.1 Table Names and Schema Names

You can create a table either by defining the table (using **CREATE TABLE**) or by defining a persistent class that is projected to a table:

- **DDL:** InterSystems IRIS® data platform uses the table name specified in **CREATE TABLE** to generate a corresponding persistent class name, and uses the specified schema name to generate a corresponding package name.
- **Class Definition:** InterSystems IRIS® data platform uses the persistent class name to generate a corresponding table name, and uses the package name to generate a corresponding schema name.

The correspondence between these two names may not be identical for the following reasons:

- Persistent classes and SQL tables follow different [naming conventions](#). Different valid character and length requirements apply. Schema and table names are not case-sensitive; package and class names are case-sensitive. The system automatically converts a valid supplied name to a valid corresponding name, insuring that the generated name is unique.
- The match between a persistent class name and the corresponding SQL table name is a default. You can use the [SqlTableName](#) class keyword to supply a different SQL table name.
- The [default schema name](#) may not match the default package name. If you specify an unqualified SQL table name or persistent class name, the system supplies a default schema name or package name. The initial default schema name is SQLUser; the initial default package name is User.

11.2 Schema Name

A table, view, or stored procedure name is either qualified (schema . name) or unqualified (name).

- If you specify a schema name (qualified name), the specified table, view, or stored procedure is assigned to that schema. If the schema does not exist, InterSystems SQL creates the schema and assigns the table, view, or stored procedure to it.
- If you do not specify a schema name (unqualified name), InterSystems SQL assigns a schema using either the [default schema name](#) or a [schema search path](#), as described below.

This section describes the following topics:

- [Schema Naming Considerations](#)
- [Reserved Schema Names](#)
- [Default Schema Name](#)
- [Use of `\_CURRENT\_USER` to Specify a Schema](#)
- [Schema Search Path](#)
- [Platform-Specific Schema Name Inclusion](#)
- [Listing Schemas](#)

11.2.1 Schema Naming Considerations

Schema names follow [identifier](#) conventions, with significant considerations concerning the use of non-alphanumeric characters. A schema name should not be specified as a delimited identifier. Attempting to specify “USER” or any other SQL [reserved word](#) as a schema name results in an SQLCODE -312 error. The INFORMATION_SCHEMA schema name and the corresponding INFORMATION.SCHEMA package name are reserved in all namespaces. Users should not create tables/classes within this schema/package.

When you issue a create operation, such as **CREATE TABLE**, that specifies a schema that does not yet exist, InterSystems IRIS creates the new schema. InterSystems IRIS uses the schema name to generate a corresponding package name. Because the naming conventions for schemas and their corresponding packages differ, the user should be aware of name conversion considerations for non-alphanumeric characters. These name conversion considerations are not the same as for tables:

- Initial character:
 - % (percent): Specify % as the first character of a schema name denotes the corresponding package as a system package, and all of its classes as system classes. This usage requires appropriate privileges; otherwise, this usage issues an SQLCODE -400 error with the %msg indicating a <PROTECT> error.
 - _ (underscore): If the first character of a schema name is the underscore character, this character is replaced by a lowercase “u” in the corresponding package name. For example, the schema name `_MySchema` generates the package name `uMySchema`.
- Subsequent characters:
 - _ (underscore): If any character other than the first character of a schema name is the underscore character, this character is replaced by a period (.) in the corresponding package name. Because a period is the class delimiter, an underscore divides a schema into a package and a sub-package. Thus `My_Schema` generates the package `My` containing the package `Schema` (`My.Schema`).
 - @, #, \$ characters: If a schema name contains any of these characters, these characters are stripped from the corresponding package name. If stripping these characters would produce a duplicate package name, the stripped package name is further modified: the final character of the stripped schema name is replaced by a sequential integer (beginning with 0) to produce a unique package name. Thus `My@$Schema` generates package `MySchema`, and subsequently creating `My#Schema` generates package `MySchema0`. The same rules apply to table name corresponding class names.

11.2.2 Reserved Schema Names

The INFORMATION_SCHEMA schema name and the corresponding INFORMATION.SCHEMA package name are reserved in all namespaces. Users should not create tables/classes within this schema/package.

The `IRIS_Shard` schema name is reserved in all namespaces. Users should not create tables, views, or procedures within this schema. Items stored in the `IRIS_Shard` schema are not displayed by catalog queries or `INFORMATION_SCHEMA` queries.

11.2.3 Default Schema Name

- When performing a DDL operation, such as creating or deleting a table, view, trigger, or stored procedure, an unqualified name is supplied the default schema name. Schema search path values are ignored.
- When performing a DML operation, such as a `SELECT`, `CALL`, `INSERT`, `UPDATE`, or `DELETE` to access an existing table, view, or stored procedure, an unqualified name is supplied the schema name from the [schema search path](#) (if provided). If there is no schema search path, or the named item is not located using the schema search path, the default schema name is supplied.

The initial setting is to use the same default schema name for all namespaces (system-wide). You can set the same default schema name for all namespace, or set a default schema name for the current namespace.

If you create a table or other item with an unqualified name, InterSystems IRIS assigns it the default schema name, and the corresponding [persistent class package name](#). If a named or default schema does not exist, InterSystems IRIS creates the schema (and package) and assigns the created item to the schema. If you delete the last item in a schema, InterSystems IRIS deletes the schema (and package). The following description of schema name resolution applies to table names, view names, and stored procedure names.

The initial system-wide default schema name is `SQLUser`. The corresponding [persistent class package name](#) is `User`. Therefore, either the unqualified table name `Employee` or the qualified table name `SQLUser.Employee` would generate the class `User.Employee`.

Because `USER` is a reserved word, attempting to specify a qualified name with the schema name of `User` (or any [SQL Reserved Word](#)) results in an `SQLCODE -1` error.

To return the current default schema name, invoke the `$$SYSTEM.SQL.Schema.Default()` method.

Or use the following pre-processor macro:

ObjectScript

```
#include %occConstant
WRITE $$$DefSchema
```

You can change the default schema name using either of the following:

- Go to the Management Portal. From **System Administration**, select **Configuration**, then **SQL and Object Settings**, then **SQL**. On this screen you can view and edit the current system-wide setting of **Default Schema**. This option sets the default schema name system-wide. This system-wide setting can be overridden by a `SetDefault()` method value for the current namespace.
- The `$$SYSTEM.SQL.Schema.SetDefault()` method. By default, this method sets the default schema name system-wide. However, by setting the Boolean 3rd argument = 1, you can set the default schema for just the current namespace. When different namespaces have different default schema names, the `$$SYSTEM.SQL.CurrentSettings()` method returns the default schema name for the current namespace.

CAUTION: When you change the default SQL schema name, the system automatically purges all cached queries in all namespaces on the system. By changing the default schema name, you change the meaning of all queries that contain unqualified table, view, or stored procedure names. It is strongly recommended that the default SQL schema name be established at InterSystems IRIS installation and not subsequently modified.

The schema name is used to generate the corresponding class package name. Because these names have different naming conventions, they may not be identical.

You can create a schema with the same name as an [SQL reserved word](#) by setting this as the system-wide **Default Schema**, though this is not recommended. A default schema named `User` generates the corresponding class package name `Use0`, following the class naming uniqueness convention.

11.2.3.1 `_CURRENT_USER` Keyword

- As System-wide Default Schema Name: If you specify `_CURRENT_USER` as the default schema name, InterSystems IRIS assigns the user name of the currently logged-in process as the default schema name. The `_CURRENT_USER` value is the first part of the `$USERNAME` ObjectScript special variable value. If `$USERNAME` consists of a name and a system address (Deborah@TestSys), `_CURRENT_USER` contains only the name piece; this means that `_CURRENT_USER` can assign the same default schema name to more than one user. If the process has not logged in, `_CURRENT_USER` specifies `SQLUser` as the default schema name.

If you specify `_CURRENT_USER/name` as the default schema name, where *name* is any string of your choice, then InterSystems IRIS assigns the user name of the currently logged-in process as the default schema name. If the process has not logged in, *name* is used as the default schema name. For example, `_CURRENT_USER/HMO` uses `HMO` as the default schema name if the process has not logged in.

In `$$SYSTEM.SQL.Schema.SetDefault()`, specify `"_CURRENT_USER"` as a quoted string.

- As Schema Name in DDL Command: If you specify `_CURRENT_USER` as the explicit schema name in a DDL statement, InterSystems IRIS replaces it with the current system-wide default schema. For example, if the system-wide default schema is `SQLUser`, the command `DROP TABLE _CURRENT_USER.OldTable` drops `SQLUser.OldTable`. This is a convenient way to qualify a name to explicitly indicate that the system-wide default schema should be used. It is functionally identical to specifying an unqualified name. This keyword cannot be used in DML statements.

11.2.4 Schema Search Path

When accessing an existing table (or view, or stored procedure) for a DML operation, an unqualified name is supplied the schema name from the schema search path. Schemas are searched in the order specified and the first match is returned. If no match is found in the schemas specified in the search path, or no search path exists, the [default schema name](#) is used. (Note that the `#import` macro directive uses a different search strategy and does not “fall through” to the default schema name.)

- In [Embedded SQL](#) you can use the `#sqlcompile path` macro directive or the `#import` macro directive to supply a schema search path that InterSystems IRIS uses to resolve unqualified names. `#sqlcompile path` resolves an unqualified name with the first match encountered. `#import` resolves an unqualified name if there is exactly one match for all the schemas listed in the search path.
- The following example provides a search path containing two schema names:

ObjectScript

```
#sqlcompile path=Customers,Employees
```

For further details, refer to [Macro Preprocessor Directives](#).

- In [Dynamic SQL](#) you can use the `%SchemaPath` property to supply a schema search path that InterSystems IRIS uses to resolve unqualified table names. You can specify the `%SchemaPath` property directly or specify it as the second parameter of the `%SQL.Statement %New()` method. The following example provides a search path containing two schema names:

ObjectScript

```
SET tStatement = ##class(%SQL.Statement).%New(0, "Customers, Employees")
```

- In [SQL Shell](#) you can set the [PATH](#) SQL Shell configuration parameter to supply a schema search path that InterSystems IRIS uses to resolve unqualified names.

If the unqualified name does not match any of the schemas specified in the schema search path or the default schema name, an SQLCODE -30 error is issued, such as the following: SQLCODE: -30 Message: Table 'PEOPLE' not found within schemas: CUSTOMERS, EMPLOYEES, SQLUSER.

11.2.5 Platform-Specific Schema Name Inclusion

When creating an ODBC-based query to run from Microsoft Excel via Microsoft Query on the Mac, if you choose a table from the list of those available, the generated query does not include the table's schema (equivalent to the package for a class). For example, if you choose to return all the rows of the Person table from the Sample schema, the generated query is:

SQL

```
SELECT * FROM Person
```

Because InterSystems IRIS interprets an unqualified table name as being in the SQLUser schema, this statement either fails or returns data from the wrong table. To correct this, edit the query (on the SQL View tab) to explicitly refer to the desired schema. The query should then be:

```
SELECT * FROM Sample.Person
```

11.2.6 Listing Schemas

The INFORMATION.SCHEMA.SCHEMATA persistent class lists all schemas in the current namespace.

The following example returns all non-system schema names in the current namespace:

SQL

```
SELECT SCHEMA_NAME
FROM INFORMATION_SCHEMA.SCHEMATA WHERE NOT SCHEMA_NAME %STARTSWITH '%'
```

The left side of the Management Portal SQL interface allows you to view the contents of a schema (or multiple schemas that match a filter pattern). See [Filtering Schema Contents](#) for further details.

11.3 Table Name

Every table has a unique name within its schema. A table has both an SQL table name and a corresponding persistent class name; these names differ in permitted characters, case-sensitivity, and maximum length. If defined using the SQL **CREATE TABLE** command, you specify an SQL table name that follows [identifier](#) conventions; the system generates a corresponding persistent class name. If defined as a persistent class definition, you must specify a name that contains only alphanumeric characters; this name is used as both the case-sensitive persistent class name and (by default) the corresponding non-case-sensitive SQL table name. The optional [SqlTableName](#) class keyword allows the user to specify a different SQL table name.

When you use the **CREATE TABLE** command to create a table, InterSystems IRIS uses the table name to [generate a corresponding persistent class name](#). Because the naming conventions for tables and their corresponding classes differ, the user should be aware of name conversion considerations for non-alphanumeric characters:

- Initial character:
 - % (percent): % as the first character of a table name is reserved and should be avoided (see [Identifiers](#)). If specified, the % character is stripped from the corresponding persistent class name.
 - _ (underscore): If the first character of a table name is the underscore character, this character is stripped from the corresponding persistent class name. For example, the table name `_MyTable` generates the class name `MyTable`.
 - Numbers: The first character of a table name cannot be a number. If the first character of the table name is a punctuation character, the second character cannot be a number. This results in an SQLCODE -400 error, with a %msg value of “ERROR #5053: Class name 'schema.name' is invalid” (without the punctuation character). For example, specifying the table name `_7A` generates the %msg “ERROR #5053: Class name 'User.7A' is invalid”.
- Subsequent characters:
 - Letters: A table name must include at least one letter. Either the first character of the table name or the first character after initial punctuation characters must be a letter. A character is a valid letter if it passes the **\$ZNAME** test; **\$ZNAME** letter validation differs for different locales. (Note that **\$ZNAME** cannot be used to validate SQL [identifiers](#) because an identifier can contain punctuation characters.)
 - _ (underscore), @, #, \$ characters: If a table name contains any of these characters, these characters are stripped from the corresponding class name and a [unique persistent class name is generated](#). Because generated class names do not include punctuation characters, it is not advisable to create table names that differ only in their punctuation characters.
- A table name must be unique within its schema. Attempting to create a table with a name that differs only in letter case from an existing table generates an SQLCODE -201 error.

A view and a table in the same schema cannot have the same name. Attempting to do so results in an SQLCODE -201 error.

You can determine if a table name already exists using the **\$\$SYSTEM.SQL.Schema.TableExists()** method. You can determine if a view name already exists using the **\$\$SYSTEM.SQL.Schema.ViewExists()** method. These methods also return the class name corresponding to the table or view name. The Management Portal SQL interface [Catalog Details Table Info](#) option displays the Class Name corresponding to the selected SQL table name.

Attempting to specify “USER” or any other SQL [reserved word](#) as a table name or schema name results in an SQLCODE -312 error. To specify an SQL reserved word as a table name or schema name, you can specify the name as a [delimited identifier](#). If you use a delimited identifier to specify a table or schema name that contains non-alphanumeric characters, InterSystems IRIS strips out these non-alphanumeric characters when generating the corresponding class or package name.

The following table name length limits apply:

- Uniqueness: InterSystems IRIS performs uniqueness checking on the first 189 characters of the persistent class name. The corresponding SQL table name may be more than 189 characters long, but, when stripped of non-alphanumeric characters, it must be unique within this 189 character limit. InterSystems IRIS performs uniqueness checking on the first 189 characters of a package name.
- Recommended maximum length: as a general rule, a table name should not exceed 128 characters. A table name may be much longer than 96 characters, but table names that differ in their first 96 alphanumeric characters are much easier to work with.
- Combined maximum length: a package name and its persistent class name (when added together) cannot exceed 220 characters. This includes the default schema (package) name (if no schema name was specified) and the dot character separating the package name and class name. A combined schema and table name can be longer than 220 characters.

when the characters in excess of 220 are stripped out when the table name is converted to the corresponding persistent class name.

For further details on [table names](#), refer to the [CREATE TABLE](#) command. For further details, see [Classes](#).

11.4 RowID Field

In SQL, every record is identified by a unique integer value, known as the RowID. In InterSystems SQL you do not need to specify a RowID field. When you create a table and specify the desired data fields, a RowID field is automatically created. This RowID is used internally, but is not mapped to a class property. By default, its existence is only visible when a persistent class is projected to an SQL table. In this projected table, an additional RowID field appears. By default, this field is named "ID" and is assigned to column 1.

By default, when a table is populated with data, InterSystems IRIS assigns sequential positive integers to this field, starting with 1. The RowID [data type](#) is BIGINT (%Library.BigInt). The values generated for the RowID have the following constraints: Each value is unique. The NULL value is not permitted. Collation is EXACT. By default, values are not modifiable.

By default, InterSystems IRIS names this field "ID". However this field name is not reserved. The RowID field name is re-established each time the table is compiled. If the user defines a field named "ID", when the table is compiled InterSystems IRIS names the RowID as "ID1". If, for example, the user then uses **ALTER TABLE** to define a field named "ID1", the table compile renames the RowID as "ID2", and so forth. In a persistent class definition you can use the [SqlRowIdName](#) class keyword to directly specify the RowID field name for the table to which this class is projected. For these reasons, referencing the RowID field by name should be avoided.

InterSystems SQL provides the **%ID** pseudo-column name (alias) which always returns the RowID value, regardless of the field name assigned to the RowID. (InterSystems TSQL provides the \$IDENTITY pseudo-column name, which does the same thing.)

ALTER TABLE cannot modify or delete the RowID field definition.

When records are inserted into the table, InterSystems IRIS assigns each record an integer ID value. RowID values always increment. They are not reused. Therefore, if records have been inserted and deleted, the RowID values will be in ascending numeric sequence, but may not be numerically contiguous.

- By default, a table defined using **CREATE TABLE** performs ID assignment using [\\$SEQUENCE](#), allowing for the rapid simultaneous populating of the table by multiple processes. When [\\$SEQUENCE](#) is used to populate the table, a sequence of RowID values is allocated to a process, which then assigns them sequentially. Because concurrent processes are assigning RowIDs using their own allocated sequences, records inserted by more than one process cannot be assumed to be in the order of insert.

You can configure InterSystems IRIS to perform ID assignment using [\\$INCREMENT](#) by setting the **SetOption()** method **DDLUseSequence** option; to determine the current setting, call the **\$SYSTEM.SQL.CurrentSettings()** method.

- By default, [a table defined by creating a persistent class](#) performs ID assignment using [\\$INCREMENT](#). In a persistent class definition, the [IdFunction](#) storage keyword can be set to either `sequence` or `increment`; for example, `<IdFunction>sequence</IdFunction>`.

In a persistent class definition, the [IdLocation](#) storage keyword global (for example, for persistent class `Sample.Person`: `<IdLocation>^Sample.PersonD</IdLocation>`) contains the highest assigned value of the RowID counter. (This is the highest integer assigned to a record, not the highest allocated to a process.) Note that this RowID counter value may no longer correspond to an existing record. To determine if record with a specific RowID value exists, invoke the table's **%ExistsId()** method.

The RowID counter is reset by the [TRUNCATE TABLE](#) command. It is not reset by a **DELETE** command, even when the **DELETE** command deletes all rows in the table. If no data has been inserted into the table, or **TRUNCATE TABLE** has been used to delete all table data, the IdLocation storage keyword global value is undefined.

By default, RowID values are not user-modifiable. Attempting to modify a RowID value generates an SQLCODE -107 error. Overriding this default to allow modifying of RowID values can have serious consequences and should only be done in very specific cases and with extreme caution. The *Config.SQL.AllowRowIDUpdate* property allows RowID values to be user-modifiable.

11.4.1 RowID Based on Fields

By [defining a persistent class that projects a table](#), you can define the RowID to have values from a field or a combination of fields. To do this, specify an index with the [IdKey](#) index keyword. For example, a table can have a RowID whose values are the same as the values of the PatientName field by specifying the index definition `IdxId On PatientName [IdKey] ;`, or the combined values of the PatientName and SSN fields by specifying the index definition `IdxId On (PatientName, SSN) [IdKey] ;`.

- A RowID based on fields is less efficient than a RowID that takes system-assigned sequential positive integers.
- On INSERT: The values specified for the field or combination of fields that make up the RowID must be unique. Specifying a non-unique value generates an SQLCODE -119 “UNIQUE or PRIMARY KEY constraint failed uniqueness check upon INSERT”.
- On UPDATE: By default, the values of each of the fields that makes up the RowID are non-modifiable. Attempting to modify the value of one of these fields generates an SQLCODE -107 “Cannot UPDATE RowID or RowID based on fields”.

When a RowID is based on multiple fields, the RowID value is the values of each of its component fields joined by the || operator. For example, `Ross, Betty | 123-45-6789`. InterSystems IRIS attempts to determine the maximum length of a RowID that is based on multiple fields; if it cannot determine the maximum length, the RowID length defaults to 512.

For further details, refer to [Primary Key](#).

11.4.2 RowID Hidden?

- When using **CREATE TABLE** to create a table, the RowID is hidden by default. A hidden field is not displayed by `SELECT *` and is PRIVATE. When you create a table you can specify the [%PUBLICROWID](#) keyword to make the RowID not hidden and public. This optional [%PUBLICROWID](#) keyword can be specified anywhere in the **CREATE TABLE** comma-separated list of table elements. It cannot be specified in **ALTER TABLE**. For further details, refer to [The RowID Field and %PUBLICROWID](#) in the **CREATE TABLE** reference page.
- When creating a persistent class that projects as a table, the RowID is not hidden by default. It is displayed by `SELECT *` and is PUBLIC. You can define a persistent class with a RowID that is hidden and PRIVATE by specifying the class keyword [SqlRowIdPrivate](#).

A RowID used as a foreign key reference must be public.

By default, a table with a public RowID cannot be used as either source or destination table to [copy data into a duplicate table](#) using `INSERT INTO Sample.DupTable SELECT * FROM Sample.SrcTable`.

You can display whether the RowID is hidden using the Management Portal SQL interface **Catalog Details Fields** listing **Hidden** column.

You can use the following program to return whether a specified field (in this example, ID) is hidden:

ObjectScript

```
SET myquery = "SELECT FIELD_NAME,HIDDEN FROM %Library.SQLCatalog_SQLFields(?) WHERE FIELD_NAME='ID'"
SET tStatement = ##class(%SQL.Statement).%New()
SET qStatus = tStatement.%Prepare(myquery)
IF qStatus'=1 {WRITE "%Prepare failed:" DO $System.Status.DisplayError(qStatus) QUIT}
SET rset = tStatement.%Execute()
DO rset.%Display()
WRITE !,"End of data"
```

To list the field names (hidden and non-hidden) in a table, refer to “[Column Names and Numbers](#)”.

11.5 Primary Key

InterSystems IRIS provides two ways to uniquely identify a row in a table: the RowID and the Primary Key.

The optional primary key is a meaningful value an application can use to uniquely identify a row in the table (for example in joins). A primary key can be user-specified data field or can be a combination of more than one data fields. Primary key values must be unique, but are not required to be integer values. The [RowID](#) is an integer value used internally to identify a row in the table. Often the primary key is a value generated by the application, while the RowID is a unique integer value generated by InterSystems IRIS.

The system automatically creates a [Master Map](#) to access rows of data using the RowID field. If you define a primary key field, the system automatically creates and maintains a primary key index.

Obviously, the duality of having two different fields and indexes to identify rows might not necessarily be a good thing. You can resolve to a single row identifier and index in either of two ways:

- Use the application-generated primary key value as the IDKEY. You can do this by identifying the primary key index in the class definition with both keywords [PrimaryKey](#) and [IdKey](#) (you can also do that from DDL if you set the [PKey is IDKey](#) flag for this purpose). This makes the primary key index the table's [Master Map](#). Thus, the primary key will be used as the main internal address for the rows. This can be less efficient if the primary key consists of more than one field, or if the primary key values are not integers.
- Do not use an application-generated primary key value, but instead use the system-generated RowID integer within the application as the application-used primary key (for example in joins). The advantage of doing this is that the integer RowID lends itself to more efficient processing, including use of bitmap indexes.

Depending on the nature of the application, you may wish to resolve to a single row identifier and index or to have separate indexes for the application-generated primary key and the system-generated RowID.

11.6 RowVersion, AutoIncrement, and Serial Counter Fields

InterSystems SQL supports three special-purpose data types for automatically-incrementing counter values. All three data types are subclasses that extend the `%Library.BigInt` data type class.

- `%Library.RowVersion`: Counts inserts and updates to all RowVersion tables namespace-wide. Only inserts and updates in tables that contain a ROWVERSION field increment this counter. ROWVERSION values are unique and non-modifiable. This namespace-wide counter never resets. For further details, see [ROWVERSION Field](#).

- `%Library.Counter` (also known as a `SERIAL` counter field): Counts inserts to the table. By default, this field receives an automatically incremented integer. However, a user can specify a non-zero integer value to this field. A user can specify a duplicate value. If the user-supplied value is greater than the highest system-supplied value, the automatic increment counter is set to increment from the user-specified value. For further details, see [Serial Counter Field](#).
- `%Library.AutoIncrement`: Counts inserts to the table. By default, this field receives an automatically incremented integer. However, a user can specify a non-zero integer value to this field. A user can specify a duplicate value. Specifying a user value has no effect on the automatic increment counter. For further details, see [AutoIncrement Field](#).

All three of these fields, and the [IDENTITY](#) field, return `AUTO_INCREMENT = YES`, as shown in the following example:

SQL

```
SELECT COLUMN_NAME,AUTO_INCREMENT FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME = 'MyTable'
```

11.6.1 RowVersion Field

The `RowVersion` field is an optional user-defined field that provides row-level version control, allowing you to determine the order in which changes were made to the data in each row namespace-wide. InterSystems IRIS maintains a namespace-wide counter, and assigns a unique incremental positive integer to this field each time the row data is modified (insert, update, or `%Save`). Because this counter is namespace-wide, an operation on one table with a `ROWVERSION` field sets the increment point for the `ROWVERSION` counter that is used for all other tables with a `ROWVERSION` field in the same namespace.

You create a `RowVersion` field by specifying a field of data type [ROWVERSION](#) (`%Library.RowVersion`). You can only specify one `ROWVERSION` data type field per table. Attempting to create a table with more than one `ROWVERSION` field results in a 5320 compilation error.

This field can have any name and can appear in any column position. The `ROWVERSION` (`%Library.RowVersion`) [data type](#) maps to `BIGINT` (`%Library.BigInt`).

This field receives a positive integer from an automatic increment counter, starting with 1. This counter increments whenever data in any `ROWVERSION`-enabled table is modified by an insert, update, or `%Save` operation. The incremented value is recorded in the `ROWVERSION` field of the row that has been inserted or updated.

A namespace can contain tables with a `RowVersion` field and tables without this field. Only data changes to tables that have a `RowVersion` field increment the namespace-wide counter.

When a table is populated with data, InterSystems IRIS assigns sequential integers to this field for each inserted row. If you use **ALTER TABLE** to add a `ROWVERSION` field to a table that already contains data, this field is created as `NULL` for pre-existing fields. Any subsequent insert or update to the table assigns a sequential integer to the `RowVersion` field for that row. This field is read-only; attempting to modify a `RowVersion` value generates an `SQLCODE -138` error: `Cannot INSERT/UPDATE a value for a read only field`. Therefore, a `RowVersion` field is defined as unique and non-modifiable, but not required or non-null.

`RowVersion` values always increment. They are not reused. Therefore, inserts and updates assign unique `RowVersion` values in temporal sequence. Delete operations remove numbers from this sequence. Therefore, `RowVersion` values may not be numerically contiguous.

This counter is never reset. Deleting all table data does not reset the `RowVersion` counter. Even dropping all tables in the namespace that contain a `ROWVERSION` field does not reset this counter.

The `RowVersion` field should not be included in a unique key or primary key. The `RowVersion` field cannot be part of an `IDKey` index.

A sharded table cannot include a `RowVersion` field.

The `RowVersion` field is not hidden (it is displayed by `SELECT *`).

This is shown in the following example of three tables in the same namespace.

1. Create Table1 and Table3, each of which has a ROWVERSION field, and Table2 that does not have a ROWVERSION field.
2. Insert ten rows into Table1. The ROWVERSION values of these rows are the next ten counter increments. Since the counter has not previously been used, they are 1 through 10.
3. Insert ten rows into Table2. Because Table2 does not have a ROWVERSION field, the counter is not incremented.
4. Update a row of Table1. The ROWVERSION values for this row is changed to the next counter increment (11 in this case).
5. Insert ten rows into Table3. The ROWVERSION values of these rows are the next ten counter increments (12 through 21).
6. Update a row of Table1. The ROWVERSION values for this row is changed to the next counter increment (22 in this case).
7. Delete a row of Table1. The ROWVERSION counter is unchanged.
8. Update a row of Table3. The ROWVERSION values for this row is changed to the next counter increment (23 in this case).

11.6.2 Serial Counter Field

You can use the SERIAL [data type](#) (`%Library.Counter` in a [persistent class table definition](#)) to specify one or more optional integer counter fields to record the order of inserts of records into a table. Each serial counter field maintains its own independent counter.

A serial counter field receives a positive integer from its automatic increment counter whenever a row is inserted into the table with either no value supplied to this field (NULL) or a value of 0. However, a user can specify a non-zero integer value for this field during an insert, overriding the table counter default.

- If an INSERT does not specify a non-zero integer value for the counter field, the counter field automatically receives a positive integer counter value. Counting starts from 1. Each successive value is an increment of 1 from the highest allocated counter value for this field.
- If an INSERT specifies a non-zero integer value for the counter field, the field receives that value. It can be a positive or negative integer value, can be lower or higher than the current counter value, and can be an integer already assigned to this field. If this value is higher than any assigned counter value, it sets the increment starting point for the automatic increment counter to that value.

Attempting to UPDATE a counter field value results in an SQLCODE -105 error.

This counter is reset to 1 by the [TRUNCATE TABLE](#) command. It is not reset by a **DELETE** command, even when the **DELETE** command deletes all rows in the table.

Tables cannot include a serial counter field if they meet any of the following conditions:

- The table is sharded.
- The table uses custom storage (`%Storage.SQL`).

11.6.3 AutoIncrement Field

You can use the `%Library.AutoIncrement` [data type](#) (or `BIGINT AUTO_INCREMENT`) to specify an integer counter field to record the order of inserts of records into a table. You can only specify one `%AutoIncrement` data type field per table. This field receives a positive integer from an automatic increment counter whenever a row is inserted into the table with either

no value supplied to this field (NULL) or a value of 0. However, a user can specify a non-zero integer value for this field during an insert, overriding the table counter default.

- If an INSERT does not specify a non-zero integer value for the counter field, the counter field automatically receives a positive integer counter value. Counting starts from 1. Each successive value is an increment of 1 from the highest allocated counter value for this field.
- If an INSERT specifies a non-zero integer value for the counter field, the field receives that value. It can be a positive or negative integer value, can be lower or higher than the current counter value, and can be an integer already assigned to this field. User-assigned values have no effect on the automatic increment counter.

Attempting to UPDATE a counter field value results in an SQLCODE -105 error.

This counter is reset to 1 by the [TRUNCATE TABLE](#) command. It is not reset by a **DELETE** command, even when the **DELETE** command deletes all rows in the table.

Tables cannot include an AutoIncrement field if they meet any of the following conditions:

- The table is sharded.
- The table uses custom storage (%Storage.SQL).

11.7 Defining a Table by Using DDL

You can define tables in InterSystems SQL using standard DDL commands:

Table 11-1: Available DDL Commands in InterSystems SQL

ALTER Commands	CREATE Commands	DROP Commands
ALTER TABLE ALTER VIEW	CREATE TABLE CREATE VIEW CREATE INDEX CREATE TRIGGER	DROP TABLE DROP VIEW DROP INDEX DROP TRIGGER

You can execute DDL commands in a variety of ways, including:

- [Using Dynamic SQL](#).
- [Using Embedded SQL](#).
- [Using a DDL script file](#).
- Using ODBC calls.
- Using JDBC calls.

11.7.1 Using DDL in Embedded SQL

Within an ObjectScript class or Python method, you can use [Embedded SQL](#) to invoke DDL commands.

For example, the following method creates a Sample.Employee table:

Class Member

```

ClassMethod CreateTable() As %String
{
    &sql(CREATE TABLE Sample.Employee (
        EMPNUM          INT NOT NULL,
        NAMELAST         CHAR (30) NOT NULL,
        NAMEFIRST        CHAR (30) NOT NULL,
        STARTDATE        TIMESTAMP,
        SALARY            MONEY,
        ACCRUEDVACATION   INT,
        ACCRUEDSICKLEAVE  INT,
        CONSTRAINT EMPLOYEEPK PRIMARY KEY (EMPNUM)))

    IF SQLCODE=0 {WRITE "Table created" RETURN "Success"}
    ELSEIF SQLCODE=-201 {WRITE "Table already exists" RETURN SQLCODE}
    ELSE {WRITE "Serious SQL Error, returning SQLCODE" RETURN SQLCODE_" " "%msg"}
}

```

Python

```

def create_table():
    try:
        print("Creating Sample.Employee table...")
        iris.sql.exec(
            """
            CREATE TABLE Sample.Employee (
                EMPNUM          INT NOT NULL,
                NAMELAST        CHAR(30) NOT NULL,
                NAMEFIRST       CHAR(30) NOT NULL,
                STARTDATE        TIMESTAMP,
                SALARY           MONEY,
                ACCRUEDVACATION  INT,
                ACCRUEDSICKLEAVE INT,
                CONSTRAINT EMPLOYEEPK PRIMARY KEY (EMPNUM)
            )
            """
        )
        print("Table created")
        return "Success"
    except Exception as e:
        sql_code = e.sqlcode
        if sql_code == -201:
            print("Table already exists")
        else:
            print(f"Serious SQL Error, returning SQLCODE: {sql_code} - {str(e)}")
        return sql_code

```

SQL

```

CREATE TABLE IF NOT EXISTS Sample.Employee (
    EMPNUM          INT NOT NULL,
    NAMELAST        CHAR(30) NOT NULL,
    NAMEFIRST       CHAR(30) NOT NULL,
    STARTDATE        TIMESTAMP,
    SALARY           MONEY,
    ACCRUEDVACATION  INT,
    ACCRUEDSICKLEAVE INT,
    CONSTRAINT EMPLOYEEPK PRIMARY KEY (EMPNUM)
)

```

The most common reasons that a DDL command such as this one will fail are:

- **SQLCODE -99 (Privilege Violation):** This error indicates that you do not have permission to execute the desired DDL command. Typically this is because an application has not established who the current user is. You can do this programmatically using the **\$SYSTEM.Security.Login()** method:

ObjectScript

```
DO $SYSTEM.Security.Login("username", "password")
```

Python

```
iris.system.Security.Login("username", "password")
```

- **SQLCODE -201** (Table or view name not unique): This error indicates that you are attempting to create a new table using the name of a table that already exists.

11.7.2 Using a Class Method to Execute DDL

Within ObjectScript you can use the Dynamic SQL `%SQL.Statement` object to prepare and execute DDL commands using [Dynamic SQL](#). Python provides an equivalent `sql.prepare` method that can be used to prepare and execute DDL commands dynamically.

The following example defines a method to create a table using Dynamic SQL:

Class Member

```
ClassMethod DefTable(user As %String, pwd As %String) As %Status [ Language = objectscript ]
{
    Do ##class(%SYSTEM.Security).Login(user, pwd)
    Set myddl=2
    Set myddl(1)="CREATE TABLE Sample.MyTest "
    Set myddl(2)=(NAME VARCHAR(30) NOT NULL, SSN VARCHAR(15) NOT NULL)"
    Set tStatement=##class(%SQL.Statement).%New()
    Set tStatus=tStatement.%Prepare(.myddl)
    If tStatus'=1 {Write "%Prepare failed:" Do $SYSTEM.Status.DisplayError(tStatus) Quit}
    Set rset=tStatement.%Execute()
    If rset.%SQLCODE=0 {Write "Created a table"}
    ElseIf rset.%SQLCODE=-201 {Write "table already exists"}
    Else {Write "Unexpected error SQLCODE=", rset.%SQLCODE}
}
```

Python

```
def def_table(user, pwd):
    try:
        iris.system.Security.Login(user, pwd)
        myddl = "CREATE TABLE Sample.MyTest (NAME VARCHAR(30) NOT NULL, SSN VARCHAR(15) NOT NULL)"
        stmt = iris.sql.prepare(myddl)
        stmt.execute()
        print("Created a table")
    except Exception as e:
        sql_code = e.sqlcode
        if sql_code == -201:
            print("Table already exists")
        else:
            print(f"Unexpected error SQLCODE: {sql_code} - {str(e)}")
```

SQL

```
CREATE TABLE IF NOT EXISTS Sample.MyTest (
    NAME          VARCHAR(30) NOT NULL,
    SSN           VARCHAR(11) NOT NULL
)
```

This method is invoked as follows:

ObjectScript

```
DO ##class(Sample.NewT).DefTable("username", "password")
```

Python

```
deftable("username", "password")
```

As with the embedded SQL example, this method will fail if there is no current user logged in.

11.7.3 Defining Tables by Importing and Executing a DDL Script

You can import InterSystems SQL DDL script files using either the `$SYSTEM.SQL.Schema.Run()` method interactively from a Terminal session, or the `$SYSTEM.SQL.Schema.ImportDDL("IRIS")` method as a background job. This method

can import and execute multiple SQL commands, enabling you to use a txt script file to define tables and views and populate them with data. For further details, see [Importing SQL Code](#).

If you are migrating tables from another vendor's relational database to InterSystems IRIS, you may have one or more DDL scripts within text files. InterSystems IRIS provides several %SYSTEM.SQL.Schema methods to help load such tables into InterSystems IRIS. You can use the general-purpose **ImportDDL()** method or the %SYSTEM.SQL.Schema Load method for the specific vendor. The vendor-specific SQL is converted to InterSystems SQL and executed. Errors and unsupported features are recorded in log files. For further details, see [Code Migration: Importing non-InterSystems SQL](#).

For example, to load an Oracle DDL file from the ObjectScript command line:

1. Start a Terminal session using the **Terminal** command in the InterSystems IRIS launcher menu.
2. Switch to the namespace in which you wish to load the table definitions:

ObjectScript

```
SET $namespace = "MYNAMESPACE"
```

3. Invoke the desired DDL import method:

ObjectScript

```
DO $SYSTEM.SQL.Schema.LoadOracle()
```

and follow the directions displayed at the terminal.

11.8 Defining a Table by Creating a Persistent Class

Although the standard way to define SQL tables is through using the CREATE TABLE DDL command, either from a SQL prompt or over a JDBC or ODBC connection, you can also create a persistent class table definition through an IDE such as VS Code. A class must either be defined as %Persistent or inherit from a superclass that is defined as %Persistent. When these classes are saved and compiled within the InterSystems IRIS database, they automatically project to a relational table that corresponds to the class definition: each class represents a table; each property represents a column, and so on. The maximum number of properties (columns) definable for a class (table) is 1000.

For example, the following defines the persistent class MyApp.Person:

Class Definition

```
Class MyApp.Person Extends %Persistent
{
  Parameter USEEXTENTSET = 1;

  Property Name As %String(MAXLEN=50) [Required];
  Property SSN As %String(MAXLEN=15) [InitialExpression = "Unknown"];
  Property DateOfBirth As %Date;
  Property Sex As %String(MAXLEN=1);

  Index BitmapExtent [ Extent, Type = bitmap ];
}
```

The class has these characteristics:

- When compiled, this definition creates the MyApp.Person persistent class and the corresponding SQL table, Person within the MyApp schema. For details on how to perform these operations, see [Defining Classes](#).
- The class definition includes a package name, MyApp. When defining a persistent class, an unspecified package name defaults to User. This name corresponds to the [default SQL schema name](#), SQLUser. For example, defining a table

named `Students` as a persistent class creates the class `User.Students`, and the corresponding SQL schema.table name `SQLUser.Students`.

- The persistent class name, `Person`, is used as the SQL table name. To supply a different SQL table name, you can use the [SqlTableName](#) class keyword.
- The `USEEXTENTSET` class parameter is defined and set to 1. This parameter organizes table storage into a more efficient set of globals. As a best practice, specify the `USEEXTENTSET` parameter for all persistent classes that project to SQL tables.
- The bitmap extent index creates an index of all IDs in the extent set. This settings makes counting and other operations more efficient. As a best practice, create a bitmap extent index for all persistent classes that project to SQL tables.

The same `MyApp.Person` table could have been defined using the DDL **CREATE TABLE** statement, specifying the SQL schema.table name. Successful execution of this SQL statement generates a corresponding persistent class with package name `MyApp` and class name `Person`:

SQL

```
CREATE TABLE MyApp.Person (  
    Name VARCHAR(50) NOT NULL,  
    SSN VARCHAR(15) DEFAULT 'Unknown',  
    DateOfBirth DATE,  
    Sex VARCHAR(1)  
)
```

When defining a table using DDL commands, you do not need to specify `USEEXTENTSET` or create a bitmap extent index. InterSystems SQL applies these settings automatically and includes them in the projected persistent class.

By default, **CREATE TABLE** specifies the [Final](#) class keyword in the corresponding class definition, indicating that it cannot have subclasses.

To learn how the object view of the database corresponds to the relational view, see [Introduction to the Default SQL Projection](#).

A persistent class definition such as the one shown above creates the corresponding table when it is compiled, but this table definition cannot be modified or deleted using [SQL DDL commands](#) (or by using the [Management Portal Drop action](#)), which give you the message “DDL not enabled for class 'schema.name' . . .”). You must specify [\[DdlAllowed\]](#) in the table class definition to permit these operations:

```
Class MyApp.Person Extends %Persistent [DdlAllowed]
```

11.8.1 Defining Property Parameters

When defining a table in a persistent class, the properties you define project to columns of the table. Every property definition must specify a data type class, which specifies the class that the property is based on. A specified [data type](#) limits a property's allowed values to that data type. When defining a persistent class that projects to a table you must specify this data type using a class in the `%Library` package. This class can be specified as either `%Library.Datatype` or as `%Datatype`.

Many data type classes provide parameters that allow you to further define allowed property values. These parameters are specific to individual data types. The following are some of the more common data definition parameters:

- [Property Value Limits](#)
- [Permitted Property Values](#)
- [Unique Property Values](#)
- [Computed Property Values](#)

When data is inserted or updated in a field, InterSystems SQL automatically validates the data, enforcing data type and referential integrity constraints. If you use other means to populate a table with data, you should [validate the table data](#).

11.8.1.1 Property Value Limits

For numeric data types, you can specify MAXVAL and MINVAL parameters to limit the range of allowed values. By definition, a numeric data type has a maximum supported values (positive and negative). You can use MAXVAL and MINVAL to further limit the allowed range.

For string data types, you can specify a MAXLEN and MINLEN parameters to limit the allowed length (in characters). By definition, a string data type has a maximum supported length. You can use MAXLEN and MINLEN to further limit the allowed range. The length limits imposed by the MAXLEN and MINLEN parameters are applied only when data is being stored in the database, as queries assume all data in the database is valid. Specifically, the limits are enforced when using [INSERT](#) or [UPDATE](#) (or %Save() in ObjectScript) to add or edit data in the database. By default, a property value that exceeds MAXLEN generates a validation error: SQLCODE -104 for **INSERT** or SQLCODE -105 for **UPDATE**. You can specify TRUNCATE=1 to permit string values that exceed MAXLEN; the specified string is truncated to the MAXLEN length. The default maximum length of a string is 4096; this limit can be configured with the [ODBCVarcharMaxlen](#) field in the CPF.

11.8.1.2 Permitted Property Values

You can limit the actual property values in two ways:

- A list of allowed values (Enumerated Values with VALUelist and DISPLAYLIST).
- A match pattern for allowed values (PATTERN).

Enumerated Property Values

Defining a table as a persistent class allows you to define properties (columns) that can only contain certain specified values. This is done by specifying the [VALUelist parameter](#). VALUelist (which specifies a list of logical storage values) is commonly used with DISPLAYLIST (which specifies a list of corresponding display values). Both lists begin with the list delimiter character. Several data types can specify VALUelist and DISPLAYLIST. The following example defines two properties with enumerated values:

Class Definition

```
Class Sample.Students Extends %Persistent
{
Parameter USEEXTENTSET = 1;

Property Name As %String(MAXLEN=50) [Required];
Property DateOfBirth As %Date;
Property ChoiceStr As %String(VALUelist="0,1,2", DISPLAYLIST="NO, YES, MAYBE");
Property ChoiceODBCStr As %EnumString(VALUelist="0,1,2", DISPLAYLIST="NO, YES, MAYBE");

Index BitmapExtent [ Extent, Type = bitmap ];
}
```

If VALUelist is specified, an **INSERT** or **UPDATE** can only specify one of the values listed in VALUelist, or be provided with no value (NULL). VALUelist valid values are case-sensitive. For required properties, specifying a value that doesn't match the VALUelist values results in a validation error: SQLCODE -104 for **INSERT** or SQLCODE -105 for **UPDATE**. For non-required properties, non-matching VALUelist values are converted to NULL values.

The %String and the %EnumString data types behave differently when displayed in ODBC mode. Using the example above, when displayed in Logical mode, both ChoiceStr and ChoiceODBCStr display their VALUelist values. When displayed in Display mode, both ChoiceStr and ChoiceODBCStr display their DISPLAYLIST values. When displayed in ODBC mode, ChoiceStr displays VALUelist values; ChoiceODBCStr displays DISPLAYLIST values.

Pattern Matching for Property Values

Several data types can specify a **PATTERN** parameter. **PATTERN** restricts allowed values to those that match the specified [ObjectScript pattern](#), specified as a quoted string with the leading question mark omitted. The following example defines a property with a pattern:

Class Definition

```
Class Sample.Students Extends %Persistent
{
  Parameter USEEXTENTSET = 1;

  Property Name As %String(MAXLEN=50) [Required];
  Property DateOfBirth As %Date;
  Property Telephone As %String(PATTERN = "3N1"-"-"3N1"-"-"4N");

  Index BitmapExtent [ Extent, Type = bitmap ];
}
```

Because a pattern is specified as a quoted string, literals specified in the pattern need to have their enclosing quotes doubled. Note that pattern matching is applied before **MAXLEN** and **TRUNCATE**. Therefore, if you are specifying a pattern for a string that may exceed **MAXLEN** and be truncated, you may wish to end the pattern with “.E” (an unlimited number of trailing characters of any type).

A value that does not match **PATTERN** generates a validation error: **SQLCODE -104** for **INSERT** or **SQLCODE -105** for **UPDATE**.

11.8.1.3 Unique Property Values

CREATE TABLE allows you to define a column as [UNIQUE](#). This means that every field value is a unique (non-duplicate) value.

Defining a table as a persistent class does not support a corresponding uniqueness property keyword. Instead, you must define both the property and a unique index on that property. The following example provides for a unique Num value for each record:

Class Definition

```
Class Sample.CaveDwellers Extends %Persistent [ DdlAllowed ]
{
  Parameter USEEXTENTSET = 1;

  Property Num As %Integer;
  Property Troglodyte As %String(MAXLEN=50);

  Index UniqueNumIdx On Num [ Type=index,Unique ];
  Index BitmapExtent [ Extent, Type = bitmap ];
}
```

The index name follows the naming conventions for properties. The optional [Type keyword](#) specifies the index type. The [Unique keyword](#) defines the property (column) as unique.

Having a unique value column is necessary for using the [INSERT OR UPDATE](#) statement.

11.8.1.4 Computed Property Values

The following class definition example defines a table containing a column (Birthday) that uses [SqlComputed](#) to compute its value when you initially set the DateOfBirth value and [SqlComputeOnChange](#) to recompute its value when you update the DateOfBirth value. The Birthday value includes the current timestamp to record when this value was computed/recomputed:

Class Definition

```
Class Sample.MyStudents Extends %Persistent [DdlAllowed]
{
  Parameter USEEXTENTSET = 1;

  Property Name As %String(MAXLEN=50) [Required];
  Property DateOfBirth As %Date;
  Property Birthday As %String
    [ SqlComputeCode = {SET {Birthday}=$PIECE($ZDATE({DateOfBirth},9),",")_
      " changed: "_$ZTIMESTAMP},
      SqlComputed, SqlComputeOnChange = DateOfBirth ];

  Index BitmapExtent [ Extent, Type = bitmap ];
}
```

Note that an **UPDATE** to DateOfBirth that specifies the existing DateOfBirth value does not recompute the Birthday value.

The [SqlComputeCode](#) property keyword contains the ObjectScript code used to compute the value. Alternatively, you can specify the compute code in a *PropertyComputation* method, where *Property* is the name of the property you want to compute. This method enables you to specify the compute code in languages other than ObjectScript, such as Python.

In this class, the *AgeComputation* method computes the Age property based on the DOB (date of birth) property. The *cols* input argument is a %Library.PropertyHelper object. You can use the *getField* method of this object to reference other properties.

Class/ObjectScript

```
Class Sample.MyStudents Extends %Persistent [ DdlAllowed ]
{
  Parameter USEEXTENTSET = 1;

  Property Name As %String(MAXLEN = 50) [ Required ];
  Property DOB As %Date;
  Property Age As %Integer [ Calculated, SqlComputed, SqlComputeOnChange = DOB ];

  Index BitmapExtent [ Extent, Type = bitmap ];

  ClassMethod AgeComputation(cols As %Library.PropertyHelper) As %Integer
  {
    set today = $zdate($horolog,8)
    set bdate = $zdate(cols.getField("DOB"), 8)
    return $select(bdate = "": "", 1:(today - bdate) \ 10000)
  }
}
```

Class/Python

```
Class Sample.MyStudents Extends %Persistent [ DdlAllowed ]
{
  Parameter USEEXTENTSET = 1;

  Property Name As %String(MAXLEN = 50) [ Required ];
  Property DOB As %Date;
  Property Age As %Integer [ Calculated, SqlComputed, SqlComputeOnChange = DOB ];

  Index BitmapExtent [ Extent, Type = bitmap ];

  ClassMethod AgeComputation(cols As %Library.PropertyHelper) As %Integer [ Language = python ]
  {
    import datetime as d
    iris_date_offset = d.date(1840,12,31).toordinal()
    bdate = d.date.fromordinal(cols.getField("DOB") + iris_date_offset).strftime("%Y%m%d")
    today = d.date.today().strftime("%Y%m%d")
    return str((int(today) - int(bdate)) // 10000) if bdate else ""
  }
}
```

If [ComputeLocalOnly](#) is specified in addition to *SqlComputed* and *SqlComputeCode*, then the compute code is only executed when the data is stored on the same server that the query is issued from. This option is used primarily in sharded environments to return only locally-stored computed data.

For more details on specifying compute code, see [Computed Columns](#).

For more details on class property keywords, see [Property Keywords](#).

11.8.2 Embedded Object (%SerialObject)

You can simplify the structure of a persistent table by referencing an embedded serial object class that defines properties. For example, you want the `MyData.Person` to contain address information, consisting of street, city, state, and postal code. Rather than specifying these properties in `MyData.Person`, you can define a serial object (`%SerialObject`) class that defines these properties, and then in `MyData.Person` specify a single *Home* property that references that embedded object. This is shown in the following class definitions:

Class Definition

```
Class MyData.Person Extends (%Persistent) [ DdlAllowed ]
{
  Parameter USEEXTENTSET = 1;
  Property Name As %String(MAXLEN=50);
  Property Home As MyData.Address;
  Property Age As %Integer;
  Index BitmapExtent [ Extent, Type = bitmap ];
}
```

Class Definition

```
Class MyData.Address Extends (%SerialObject)
{
  Property Street As %String;
  Property City As %String;
  Property State As %String;
  Property PostalCode As %String;
}
```

You cannot access the data in a serial object property directly, you must access them through a persistent class/table that references it:

- To refer to an individual serial object property from the persistent table, use an underscore. For example, `SELECT Name, Home_State FROM MyData.Person` returns the State serial object property value as a string. Serial object property values are returned in the order specified in the query.
- To refer to all of the serial object properties from the persistent table, specify the referencing field. For example, `SELECT Home FROM MyData.Person` returns values of all of the `MyData.Address` properties as a `%List` structure. Serial object property values are returned in the order specified in the serial object: `Home_Street`, `Home_City`, `Home_State`, `Home_PostalCode`. In the Management Portal SQL interface [Catalog Details](#), this referencing field is referred to as a **Container** field. It is a **Hidden** field, and therefore not returned by `SELECT *` syntax.
- A `SELECT *` for a persistent class returns all of the serial object properties individually, including nested serial objects. For example, `SELECT * FROM MyData.Person` returns `Age`, `Name`, `Home_City`, `Home_PostalCode`, `Home_State`, and `Home_Street` values (in that order); it does not return the `Home %List` structure value. Serial object property values are returned in collation sequence. `SELECT *` first lists all of the fields in the persistent class in collation sequence (commonly alphabetical order), followed by the nested serial object properties in collation sequence.

Note that an embedded serial object does not have to be in the same package as the persistent table that references it. The [SqlCategory](#) for `%Library.SerialObject` (and all sub-classes of `%SerialObject` that do not define the `SqlCategory` explicitly) is `STRING`.

Defining embedded objects can simplify persistent table definitions:

- A persistent table can contain multiple properties that reference different records in the same embedded object. For example, the `MyData.Person` table can contain a *Home* and an *Office* property, both of which reference the `MyData.Address` serial object class.
- Multiple persistent tables can reference instances of the same embedded object. For example, the `MyData.Person` table *Home* property and the `MyData.Employee` *WorkPlace* property can both reference the `MyData.Address` serial object class.

- An embedded object can reference another embedded object. For example, the MyData.Address embedded object contains the *Phone* property that references the MyData.Telephone embedded object, containing CountryCode, AreaCode, and PhoneNum properties. From the persistent class you use multiple underscores to refer to a nested serial object property, for example Home_Phone_AreaCode.

Compiling a serial object class generate a data specification in the storage definition. The compiler assigns this specification a data name by appending the word “State” to the serial object class name. Therefore, MyData.Address is assigned <Data name="AddressState">. If this name (AddressState in this example) is already used as a property name, the compiler appends an integer to create a unique data name: <Data name="AddressState1">.

Refer to [Introduction to Serial Objects](#).

For information on creating an index for a serial object property, refer to [Indexing an Embedded Object \(%SerialObject\) Property](#).

11.8.3 Class Methods

You can specify [class methods](#) as part of a table definition, as shown in the following example:

Class Definition

```
Class MyApp.Person Extends %Persistent
{
  Parameter USEEXTENTSET = 1;
  Property Name As %String(MAXLEN=50) [Required];
  Property SSN As %String(MAXLEN=15) [InitialExpression = "Unknown"];
  Property DateOfBirth As %Date;
  Property Sex As %String(MAXLEN=1);
  Index BitmapExtent [ Extent, Type = bitmap ];
  ClassMethod Numbers() As %Integer [ SqlName = Numbers, SqlProc ]
  {
    QUIT 123
  }
}
```

The SqlProc keyword is required to project the class method to a SQL procedure. In a **SELECT** query, you can call this method by using the defined SqlName value you defined for the method. For example:

SQL

```
SELECT Name, SSN, MyApp.Numbers() FROM MyApp.Person
```

11.8.4 Defining a Sharded Table by Creating a Persistent Class

Before you can define a persistent class that projects as a sharded table, you must establish a [sharding environment](#). Then, to define a sharded persistent class, specify the class keyword Sharded=1, along with any optional shard-related class attributes.

CAUTION: Define sharded tables only for new persistent classes that contain no data. Applying sharding to an existing class can make your data inaccessible.

This class defines a sample sharded persistent class with optional shard-related class attributes set:

Class Definition

```
Class Sample.MyShardT Extends %Persistent [ Sharded = 1 ]
{
  Parameter DEFAULTCONCURRENCY As BOOLEAN = 0;
  Parameter USEEXTENTSET = 1;
  Index BitmapExtent [ Extent, Type = bitmap ];
}
```


The following class attributes apply:

- The `Sharded = 1` keyword defines the projected table for this class as sharded. With this setting, the sharding infrastructure manages the sharded table storage, including data distribution. Therefore, you cannot customize the default storage definition generated and maintained by InterSystems IRIS. Any changes made to the storage definition in this class definition are ignored.
- The `DEFAULTCONCURRENCY` class parameter is set to 0 (no locking). Due to the distributed nature of sharded tables, the parameter value of 0 is required. This value is used as the default in object methods such as `%Open` and `%OpenId`, so that you do not have to pass the concurrency argument to every method call.
- The `USEEXTENTSET` class parameter is set to 1, which organizes table storage into a more efficient set of globals. When defining a sharded table using DDL commands, InterSystems SQL applies this setting automatically.
- The bitmap extent index creates an index of all IDs in the extent set. This settings makes counting and other operations more efficient. When defining a sharded table using DDL commands, InterSystems SQL applies this setting automatically.

You can then define the shard key index. When you create a sharded table, an [abstract shard key index](#) is generated automatically. The shard key index determines the shard in which a row resides. For more details on defining a shard key index and creating sharded tables, see [Creating Sharded Tables and Loading Data](#).

11.8.4.1 Sharded Class Restrictions

- Class parameters that are not supported for sharded classes: `CONNECTION`, `DEFAULTGLOBAL`, `DSINTERVAL`, `DSTIME`, `IDENTIFIEDBY`, `OBJJOURNAL`.
- Class keywords that are not supported for sharded classes: `Language`, `ViewQuery`.
- Super classes that are not supported for sharded classes: `%Library.IndexBuilder`, `%DocDB.Document`.
- Property data types not supported for sharded classes: `%Library.Text`.
- Relationship properties are not supported for sharded classes.
- Projections are not supported for sharded classes.
- Any methods with a Language other than "objectscript" are not supported for sharded classes.
- Any class queries not of Type `%SQLQuery` are not supported for sharded classes.

An attempt to compile a sharded class using any of these features results in a compile-time error.

11.9 Defining a Sharded Table

There are three requirements for creating a sharded table.

1. The License Key must support sharding. Use the Management Portal, **System Administration, Licensing, License Key** to display the current licence or activate a new license.
2. You must enable sharding on your InterSystems IRIS instance. You must have the `%Admin_Secure` privilege to enable sharding. Use the Management Portal, **System Administration, Configuration, System Configuration, Sharding Configuration** to select the **Enable Sharding** button. This enables the current InterSystems IRIS instance for use in a sharded cluster. Select either **Enable this instance for any role** or **Enable this instance for the shard master role only**. Press **OK**. Restart your InterSystems IRIS instance.
3. You must deploy a sharded cluster on your InterSystems IRIS instance. This sharded cluster contains a Shard Master namespace. If the current namespace is not configured for sharding, attempting to define a shard table fails with `ERROR`

#9319: Current namespace %1 has no shards configured. For details, refer to [Deploy the Cluster Using the API or the Management Portal](#).

You can then define a sharded table within the Shard Master namespace that has been defined as part of the sharded cluster. You can use **CREATE TABLE** to define a sharded table by [specifying a shard key](#). Alternately, you can [create a persistent class that projects to a sharded table](#).

For further details about defining a sharded table, refer to [Create Target Sharded Tables](#).

11.9.1 Convert a Non-Sharded Table to Sharded

The **ALTER TABLE** command offers a **CONVERT** option that allows you to convert any non-sharded table to the sharded. Before issuing such a command, you must first enable sharding on your instance and deploy a sharded cluster. After defining a non-sharded table, you may then use the **ALTER TABLE** command with the **CONVERT** option to start rebalancing data across your multiple shards.

Note that such conversions are never triggered by a manual change to the class definition.

11.10 Defining a Table by Querying an Existing Table

You can define and populate a new table based on an existing table (or tables or views). You specify a query and a new table name. The existing table name and/or the new table name can be [qualified or unqualified](#). The query can contain **JOIN** syntax. The query can supply [column name aliases](#) that become the column names in the new table.

You can perform this operation using either the **CREATE TABLE AS SELECT** command or the **\$SYSTEM.SQL.Schema.QueryToTable()** method, as described below:

1. **QueryToTable()** copies the DDL definition of a existing table and assigns it the specified new table name. It copies the definitions of the fields specified in the query, including the data type, maxlength, and minval/maxval. It does not copy field data constraints, such as default value, required value, or unique value. It does not copy references from a field to another table.

If the query specifies **SELECT *** or **SELECT %ID**, the RowID field of the original table is copied as a non-required, non-unique data field of data type integer. **QueryToTable()** generates a unique RowID field for the new table. If the copied RowID is named ID, the generated RowID is named ID1.

QueryToTable() creates a corresponding persistent class for this new table. The persistent class is defined as [DdlAllowed](#). The owner of the new table is the current user.

The new table is defined with Default Storage = YES and Supports Bitmap Indices = YES, regardless of these settings in the source table.

The only index created for the new table is the IDKEY index. No bitmap extent index is generated. Index definitions for the copied fields are not copied into the new table.

2. **QueryToTable()** then populates the new table with data from the fields selected by the query. It sets the table's Extent Size to 100,000. It estimates the IDKEY Block Count. The [automatic collected statistics utility](#) sets the actual Average Field Size, Block Count, Extent Size, and the Selectivity values for each field.

QueryToTable() both creates a table definition and populates the new table with data. If you wish to only create a table definition, specify a condition in the query WHERE clause that selects for no data rows. For example, **WHERE Age < 20 AND Age > 20**.

The following example copies the Name, and Age, fields from Sample.Person and creates an AVG(Age) field. These field definitions are used to create a new table named Sample.Youth. The method then Populates Sample.Youth with the Sam-

ple.Person data for those records where Age < 21. The AvgInit field contains the aggregate value for the selected records at the time that the table was created.

ObjectScript

```
DO $SYSTEM.SQL.Schema.QueryToTable("SELECT Name, Age, AVG(Age) AS AvgInit FROM Sample.Person WHERE Age < 21", "Sample.Youth", 1, .errors)
```

11.11 Listing Tables

The INFORMATION.SCHEMA.TABLES persistent class displays information about all tables (and views) in the current namespace. It provides a number of properties including the schema and table names, the owner of the table, and whether you can insert new records. The TABLETYPE property indicates whether it is a base table or a [view](#).

The following example returns the table type, schema name, table name, and owner for all tables and views in the current namespace:

SQL

```
SELECT Table_Type, Table_Schema, Table_Name, Owner FROM INFORMATION_SCHEMA.TABLES
```

The INFORMATION.SCHEMA.CONSTRAINTTABLEUSAGE persistent class displays one row for each [Primary Key](#) (explicit or implicit), Foreign Key, or [Unique](#) constraint defined for each table in the current namespace.

INFORMATION.SCHEMA.KEYCOLUMNUSAGE displays one row for each field defined as part of one of these constraints for each table in the current namespace.

You can display much of the same information for a single table using the [Catalog Details tab](#) in the Management Portal SQL Interface.

11.12 Listing Column Names and Numbers

You can list all of the column names (field names) for a specified table in four ways:

- The **GetAllColumns()** method. This lists all column names and column numbers, including hidden columns. The **GetVisibleColumns()** method lists all non-hidden columns. The [ID \(RowID\) field may or may not be hidden](#). The x__classname column is always hidden; it is automatically defined unless the persistent class is defined with the [Final](#) class keyword.
- The Management Portal SQL interface (**System Explorer, SQL**) schema contents [Catalog Details](#) tab. This lists all column names and column numbers (including hidden columns) and other information, including [data types](#) and a flag indicating if a column is hidden.
- `SELECT TOP 0 * FROM tablename`. This lists all non-hidden column names in column number order. Note that because hidden columns can appear anywhere in the column number order, you cannot determine the column number by counting these non-hidden column names. For further details on Asterisk Syntax, refer to the [SELECT](#) command.
- The INFORMATION.SCHEMA.COLUMNS persistent class lists a row for each non-hidden column in each table or view in the current namespace. INFORMATION.SCHEMA.COLUMNS provides a large number of properties for listing characteristics of table and view columns. Note that ORDINALPOSITION is not the same as column number, because hidden fields are not counted. The **GetAllColumns()** method counts both hidden and non-hidden fields.

The following example uses INFORMATION.SCHEMA.COLUMNS to list some of the column properties:

SQL

```
SELECT TABLE_NAME, COLUMN_NAME, ORDINAL_POSITION, DATA_TYPE, CHARACTER_MAXIMUM_LENGTH,
       COLUMN_DEFAULT, IS_NULLABLE, UNIQUE_COLUMN, PRIMARY_KEY
FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA='Sample'
```

11.12.1 The Get Columns Methods

To list the names of the columns in a table in column number order, you can use either the **GetAllColumns()** or **GetVisibleColumns()** method, as follows:

ObjectScript

```
SET stat=##class(%SYSTEM.SQL.Schema).GetAllColumns("Sample.Person",.byname,.bynum)
IF stat=1 {
    SET i=1
    WHILE $DATA(bynum(i)) { WRITE "name is ",bynum(i),"    col num is ",i,!
                           SET i=i+1 }
}
ELSE { WRITE "GetAllColumns() cannot locate specified table" }
```

GetAllColumns() lists all defined columns, including hidden columns. If a table references an [embedded %SerialObject class](#), **GetAllColumns()** first lists all of the columns in the persistent class, including the property that references the %SerialObject, then lists all of the %SerialObject properties. This is shown in the following **GetAllColumns()** results:

```
name is ID    col num is 1
name is Age   col num is 2
name is Home  col num is 3
name is Name  col num is 4
name is x__classname  col num is 5
name is Home_City  col num is 6
name is Home_Phone  col num is 7
name is Home_Phone_AreaCode  col num is 8
name is Home_Phone_Country  col num is 9
name is Home_Phone_TNum  col num is 10
name is Home_PostalCode  col num is 11
name is Home_State  col num is 12
name is Home_Street  col num is 13
```

You can also use this method to determine the column number for a specified column name, as follows:

ObjectScript

```
SET stat=##class(%SYSTEM.SQL.Schema).GetAllColumns("Sample.Person",.byname)
IF stat=1 {
    WRITE "Home_State is column number ",byname("Home_State"),! }
ELSE { WRITE "GetAllColumns() cannot locate specified table" }
```

11.13 Listing Constraints

The INFORMATION_SCHEMA.TABLECONSTRAINTS persistent class lists the table name, constraint type, and constraint name. Constraint types include UNIQUE, PRIMARY KEY, and FOREIGN KEY. If the table definition did not specify a name for a constraint, a constraint name is generated from the table name, constraint type, and table column number — for example, MYTABLE_UNIQUE3. This is shown in the following example:

SQL

```
SELECT Table_Schema, Table_Name, Constraint_Type, Constraint_Name FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS
```

The INFORMATION_SCHEMA.CONSTRAINTCOLUMNUSAGE persistent class lists the table name, column name, and constraint name. If a constraint involves multiple columns a separate item is listed for each column. If the table definition did

not specify a name for a constraint, a constraint name is generated from the table name, constraint type, and table column number — for example, MYTABLE_UNIQUE3. This is shown in the following example:

SQL

```
SELECT Table_Schema, Table_Name, Column_Name, Constraint_Name FROM INFORMATION_SCHEMA.CONSTRAINT_COLUMN_USAGE
```

The INFORMATION_SCHEMA.REFERENTIALCONSTRAINTS persistent class lists foreign key constraints including the referencing table (CONSTRAINT_SCHEMA, CONSTRAINT_TABLE_NAME), the referenced table (UNIQUE_CONSTRAINT_SCHEMA, UNIQUE_CONSTRAINT_TABLE), the foreign key name (CONSTRAINT_NAME), and the UPDATE and DELETE referential actions (UPDATE_RULE, DELETE_RULE) with the values NO ACTION, SET DEFAULT, SET NULL, or CASCADE. This is shown in the following example:

SQL

```
SELECT Constraint_Table_Name, Unique_Constraint_Table, Constraint_Name, Update_Rule, Delete_Rule FROM INFORMATION_SCHEMA.REFERENTIAL_CONSTRAINTS
```

12

Defining and Using Views

A view is a virtual table consisting of data retrieved at execution time from one or more physical tables by means of a **SELECT** statement or a **UNION** of several **SELECT** statements. The **SELECT** can access data by specifying any combination of tables or other views. Thus a view is a stored query that provides all of the flexibility and security privileges of a physical table.

The same [Selectivity](#) value is used for a field specified in a **SELECT** query and the same field specified in the **SELECT** clause of a view. Note that a view may have a different distribution of rows than the source table. This can affect the accuracy of view field selectivity.

InterSystems SQL on InterSystems IRIS® data platform supports the ability to define and execute queries on views. All views are either [updateable](#) or [read-only](#).

Note: You cannot create a view on data stored in a database that is mounted read-only.

You cannot create a view on data stored in an Informix table linked through an ODBC or JDBC gateway connection. This is because InterSystems IRIS query conversion uses subqueries in the FROM clause for this type of query; Informix does not support FROM clause subqueries. See the [ISQL Migration Guide](#) for InterSystems support for Informix SQL.

12.1 Creating a View

You can define views in several ways:

- Using the SQL [CREATE VIEW](#) command (either in a DDL script or via JDBC or ODBC).
- Using the Management Portal [Create View interface](#).

A view name may be unqualified or qualified. An unqualified view name is a simple [identifier](#): MyView. A qualified view name consists of two simple identifiers, a schema name and a view name, separated by a period: MySchema.MyView. View names and table names follow the same [naming conventions](#) and perform the same [schema name resolution](#) for unqualified names. A view and a table in the same schema cannot have the same name.

You can determine if a view name already exists using the `$$SYSTEM.SQL.Schema.ViewExists()` method. This method also returns the class name that projected the view. You can determine if a table name already exists using the `$$SYSTEM.SQL.Schema.TableExists()` method.

A view can be used to create a restricted subset of a table. The following Embedded SQL example creates a view that restricts both the rows (through the WHERE clause) and columns (assuming that Sample.Person contains more than two columns) of the original table that can be accessed through the view:

ObjectScript

```
&sql(CREATE VIEW Sample.VSrStaff
      AS SELECT Name AS Vname, Age AS Vage
      FROM Sample.Person WHERE Age>75)
IF SQLCODE=0 {WRITE "Created a view",!}
ELSEIF SQLCODE=-201 {WRITE "View already exists",!}
ELSE {WRITE "Serious SQL problem: ",SQLCODE," ",%msg,! }
```

SQL

```
SELECT * FROM Sample.VSrStaff ORDER BY Vage
```

The following Embedded SQL example creates a view based on the SalesPeople table, creating a new calculated value column TotalPay:

ObjectScript

```
&sql(CREATE VIEW Sample.VSalesPay AS
      SELECT Name, (Salary + Commission) AS TotalPay
      FROM Sample.SalesPeople)
IF SQLCODE=0 {WRITE "Created a view",!}
ELSEIF SQLCODE=-201 {WRITE "View already exists",!}
ELSE {WRITE "Serious SQL problem: ",SQLCODE," ",%msg,! }
```

12.1.1 Management Portal Create View Interface

You can create a view from the Management Portal. Go to the InterSystems IRIS Management Portal. From **System Explorer**, select **SQL**. Select a namespace by clicking the name of the current namespace displayed at the top of the page; this displays the list of available namespaces. Once you have selected a namespace, click the **Actions** drop-down list and select **Create View**.

This displays the Create a View window with the following fields:

- **Schema:** You can decide to include the view within an existing schema, or create a new schema. If you opt to select an existing schema, a drop-down list of existing schemas is provided. If you opt to create a new schema, you enter a schema name. In either case, if you omit the schema, InterSystems IRIS uses the [default schema name](#).
- **View Name:** a valid [view name](#). You cannot use the same name for a table and a view in the same schema.
- **With Check Option:** the options are [READONLY](#), [LOCAL](#), [CASCADED](#).
- **Grant all privilege on the view to _PUBLIC:** if selected, this option gives all users execution privileges for this view. The default is to not give all users access to the view.
- **View Text:** you can specify the View Text in any of the following three ways:
 - Type a **SELECT** statement into the View Text area.
 - Use the Query Builder to create a **SELECT** statement, then press **OK** to supply this query to the View Text area.
 - If you select a Cached Query name (for example %sqlcq.USER.cls4) on the left side of the Management Portal SQL interface, then invoke **Create View**, this cached query is provided to the View Text area. Note that in the View Text area you must replace host variable references (question marks) with actual values before saving the view text.

12.1.2 Views and Corresponding Classes

When you define a view, InterSystems IRIS generates a corresponding class. An SQL view name is used to [generate a corresponding unique class name](#), following name translation rules. The Management Portal SQL interface displays **Catalog Details** for existing views, including this class name. Refer to [Catalog Details for a View](#).

12.2 Altering a View

In the Management Portal SQL interface you can select an existing view to display **Catalog Details** for that view. The **Catalog Details View Info** option displays an **Edit View** link that provides an interface to edit the view text (the **SELECT** statement for the view). It also provides a drop-down list to select the **With Check Option** as none, READONLY, LOCAL, or CASCADED.

12.3 Updateable Views

An updateable view is one on which you can perform **INSERT**, **UPDATE**, and **DELETE** operations. A view is considered updateable only if the following conditions are true:

- The FROM clause of the view's query contains only *one* table reference. This table reference must identify either an updateable base table or an updateable view.
- The value expressions within the SELECT list of the view's query must all be column references.
- The view's query must not specify GROUP BY, HAVING, or SELECT DISTINCT.
- The view is not a class query projected as a view.
- The view's class does not contain the class parameter *READONLY=1* (true if the view definition contains a WITH READ ONLY clause).

12.3.1 The WITH CHECK Option

In order to prevent an INSERT or UPDATE operation on a view which would result in a row in the underlying base table which is not part of the derived view table, InterSystems SQL supports the WITH CHECK OPTION clause within a View definition. This clause can only be used with updateable views.

The WITH CHECK OPTION clause specifies that any INSERT or UPDATE operations on an updateable view must validate the resulting row against the WHERE clause of the view definition to make sure the inserted or modified row will be part of the derived view table.

For example, the following DDL statement defines an updateable GoodStudent view containing all Students with a high GPA (grade point average):

SQL

```
CREATE VIEW GoodStudent AS
  SELECT Name, GPA
  FROM Student
  WHERE GPA > 3.0
  WITH CHECK OPTION
```

Because the view contains a WITH CHECK OPTION, any attempt to INSERT or UPDATE a row in the GoodStudent view with a GPA value of 3.0 or less will fail (such a row would not represent a “good student”).

There are two flavors of WITH CHECK OPTION:

- WITH LOCAL CHECK OPTION means that only the WHERE clause of the view specified in the INSERT or UPDATE statement is checked.

- **WITH CASCADED CHECK OPTION** (and **WITH CASCADE CHECK OPTION**) means that the **WHERE** clause of the view specified in the **INSERT** or **UPDATE** statement as well as **ALL** views on which that view is based are checked, regardless of the appearance or absence of other **WITH LOCAL CHECK OPTION** clauses in those view definitions.

The default is **CASCADED** if just **WITH CHECK OPTION** is specified.

During an **UPDATE** or **INSERT**, the **WITH CHECK OPTION** conditions are checked after all default values and triggered computed fields have been calculated for the underlying table's fields and before the regular table's validation (required fields, data type validation, constraints, and so on).

After the **WITH CHECK OPTION** validation passes, the **INSERT** or **UPDATE** operation continues as if the **INSERT** or **UPDATE** was performed on the base table itself. All constraints are checked, triggers pulled, and so on.

If the **%NOCHECK** option is specified on the [INSERT](#) or [UPDATE](#) statement, the **WITH CHECK OPTION** validation is not checked.

There are two **SQLCODE** values related to the **WITH CHECK OPTION** validation (the **INSERT/UPDATE** would have resulted in a row not existing in the derived view table):

- **SQLCODE -136**—View's **WITH CHECK OPTION** validation failed in **INSERT**.
- **SQLCODE -137**—View's **WITH CHECK OPTION** validation failed in **UPDATE**.

12.4 Read-only Views

A read-only view is one on which you cannot perform **INSERT**, **UPDATE**, and **DELETE** operations. Any view that does not meet the criteria for [updateable views](#) is a read-only view.

A view definition may specify a **WITH READ ONLY** clause to force it to be a read-only view.

If you attempt to compile/prepare an **INSERT**, **UPDATE**, or **DELETE** statement against a read-only view an **SQLCODE -35** error is generated.

12.5 View ID: %VID

InterSystems IRIS assigns an integer view ID (**%VID**) to each row returned by a view or by a [FROM clause subquery](#). Like table row ID numbers, these view row ID numbers are system-assigned, unique, non-null, non-zero, and non-modifiable. This **%VID** is commonly invisible to the user, and is only returned when explicitly specified. It is returned as data type **INTEGER**. Because **%VID** values are sequential integers, they are far more meaningful if the view returns ordered data; a view can only use an [ORDER BY](#) clause when it is paired with a [TOP](#) clause. The following Embedded SQL example creates a view named **VSrStaff**:

ObjectScript

```
&sql(CREATE VIEW Sample.VSrStaff
      AS SELECT TOP ALL Name AS Vname, Age AS Vage
      FROM Sample.Person WHERE Age>75
      ORDER BY Name)
IF SQLCODE=0 {WRITE "Created a view",!}
ELSEIF SQLCODE=-201 {WRITE "View already exists",!}
ELSE {WRITE "Serious SQL problem: ",SQLCODE," ",%msg,! }
```


The following example returns all of the data defined by the VSrStaff view (using `SELECT *`) and also specifies that the view ID for each row should be returned. Unlike the table row ID, the view row ID is not displayed when using asterisk syntax; it is only displayed when explicitly specified in the `SELECT`:

SQL

```
SELECT *,%VID AS ViewID FROM Sample.VSrStaff
```

The `%VID` can be used to further restrict the number of rows returned by a `SELECT` from a view, as shown in the following example:

SQL

```
SELECT *,%VID AS ViewID FROM Sample.VSrStaff WHERE %VID BETWEEN 5 AND 10
```

Thus `%VID` can be used instead of [TOP](#) (or in addition to `TOP`) to restrict the number of rows returned by a query. Generally, a `TOP` clause is used to return a small subset of the data records; `%VID` is used to return most or all of the data records, returning records in small subsets. This feature may be useful, especially for porting Oracle queries (`%VID` maps easily to Oracle `ROWNUM`). However, the user should be aware of some performance limitations in using `%VID`, as compared to `TOP`:

- `%VID` does not perform time-to-first-row [optimization](#). `TOP` optimizes to return the first row of data as quickly as possible. `%VID` optimizes to return the full data set as quickly as possible.
- `%VID` does not perform a limited sort (which is a special optimization performed by `TOP`) if the query specifies sorted results. The query first sorts the full data set, then restricts the return data set using `%VID`. `TOP` is applied before sorting, so the `SELECT` performs a limited sort involving only a restricted subset of rows.

To preserve time to first row optimization and limited sort optimization, you can use a [FROM clause subquery](#) with a combination of `TOP` and `%VID`. Specify the upper bound (in this case, 10) in the `FROM` subquery as the value of `TOP`, rather than using `TOP ALL`. Specify the lower bound (in this case, `>4`) in the `WHERE` clause with `%VID`. The following example uses this strategy to return the same results as the previous view query:

SQL

```
SELECT *,%VID AS SubQueryID
FROM (SELECT TOP 10 Name, Age
      FROM Sample.Person
      WHERE Age > 75
      ORDER BY Name)
WHERE %VID > 4
```

[Parallel execution](#) cannot be performed on a query that specifies a `%VID`, even when the `%PARALLEL` keyword is explicitly specified.

12.6 Listing View Properties

The `INFORMATION.SCHEMA.VIEWS` persistent class displays information about all views in the current namespace. It provides a number of properties including the view definition, the owner of the view, and the timestamps when the view was created and last modified. These properties also include whether the [view is updateable](#) and if so, whether it was defined with a [check option](#).

When specified in [Embedded SQL](#), `INFORMATION.SCHEMA.VIEWS` requires the `#include %occInclude` macro preprocessor directive. This directive is not required for Dynamic SQL.

The `VIEWDEFINITION` property (`SqlFieldName = VIEW_DEFINITION`) returns as a string the view field names and the view's query expression for all views in the current namespace. For example,

SQL

```
SELECT View_Definition FROM INFORMATION_SCHEMA.VIEWS
```

returns strings such as: "(vName, vAge) SELECT Name, Age FROM Sample.Person WHERE Age > 21". When issued from the Management Portal SQL Execute Query interface, returns a string with a space between the view fields list with whitespace and line breaks removed and (if necessary) an appended ellipsis (...) indicating truncated content. Otherwise, issuing this query returns a string of up to 1048576 characters for each view, with a line break between the view fields list and the query text, with the whitespace specified in the view's query expression preserved, and (if necessary) an appended ellipsis (...) indicating truncated content.

The following example returns the view name (`Table_Name` field) and owner name for all views in the current namespace:

SQL

```
SELECT Table_Name, Owner FROM INFORMATION_SCHEMA.VIEWS
```

The following example returns all information for all non-system views in the current namespace:

SQL

```
SELECT * FROM INFORMATION_SCHEMA.VIEWS WHERE Owner != '_SYSTEM'
```

The `INFORMATION_SCHEMA.VIEWCOLUMNUSAGE` persistent class displays the names of the source table fields for each of the views in the current namespace:

SQL

```
SELECT * FROM INFORMATION_SCHEMA.VIEW_COLUMN_USAGE WHERE View_Name='MyView'
```

You can display much of the same information as `INFORMATION_SCHEMA.VIEWS` for a single view using the [Catalog Details tab](#) in the Management Portal SQL Interface. The Catalog Details for a view include the definition of each view field (data type, max length, minval/maxval, etc.), details that are not provided by the `INFORMATION_SCHEMA` view classes. The Catalog Details View Info display also provides an option to edit the view definition.

12.7 Listing View Dependencies

The `INFORMATION_SCHEMA.VIEWTABLEUSAGE` persistent class displays all views in the current namespace and the tables they depend on. This is shown in the following example:

SQL

```
SELECT View_Schema, View_Name, Table_Schema, Table_Name FROM INFORMATION_SCHEMA.VIEW_TABLE_USAGE
```

You can invoke the `%Library.SQLCatalog.SQLViewDependsOn` class query to list the tables that a specified view depends upon. You specify `schema.viewname` to this class query. If you specify only `viewname`, it uses the [system-wide default schema name](#). The caller must have privileges for the specified view to execute this class query. This is shown in the following example:

ObjectScript

```
SET statemt=##class(%SQL.Statement).%New()  
SET cqStatus=statemt.%PrepareClassQuery("%Library.SQLCatalog","SQLViewDependsOn")  
    IF cqStatus'=1 {WRITE "%PrepareClassQuery failed:" DO $System.Status.DisplayError(cqStatus) QUIT}  
SET rset=statemt.%Execute("vschema.vname")  
DO rset.%Display()
```

This SQLViewDependsOn query lists the tables that the view depends upon, listing the table schema followed by the table name. If the caller does not have privileges for a table that the view depends upon, that table and its schema are listed as <NOT PRIVILEGED>. This allows a caller without table privileges to determine how many tables the view depends upon, but not the names of the tables.

13

Relationships Between Tables

To enforce referential integrity between tables you can define foreign keys. When a table containing a foreign key constraint is modified, the foreign key constraints are checked.

13.1 Defining a Foreign Key

There are several ways to define foreign keys in InterSystems SQL:

- You can define a [relationship](#) between two classes. Defining a relationship automatically projects a foreign key constraint to SQL. For more information on relationships, see [Defining and Using Classes](#).
- You can add an explicit foreign key definition to a class definition (for cases not covered by relationships). For information, see [Foreign Key Definitions](#).
- You can add a foreign key using the [CREATE TABLE](#) or [ALTER TABLE](#) command. You can remove a foreign key using the [ALTER TABLE](#) command. These commands are described in the [InterSystems SQL Reference](#).

A RowID field used as a foreign key reference must be public. Refer to [RowID Hidden?](#) for how to define a table with a public (or private) RowID field.

The maximum number of foreign keys for a table (class) is 400.

13.2 Foreign Key Referential Integrity Checking

A foreign key constraint can specify a referential action on update or on delete. Defining this referential action using DDL is described in **CREATE TABLE** [Referential Action Clause](#). When creating a sharded table, these referential actions must be set to NO ACTION.

Note: When using foreign key constraints with the CASCADE referential action in a mapped environment, such as having code and data in separate namespaces, it is possible for the referential action to not be triggered due to an out-of-date [extent index](#).

To ensure the trigger of the CASCADE referential action, it is recommended to [rebuild the extent index](#) in a namespace after mapping in additional packages.

By default, InterSystems IRIS® data platform performs foreign key referential integrity checking on [INSERT](#), [UPDATE](#) and [DELETE](#) operations. If the operation would violate referential integrity, it is not performed; the operation issues an SQLCODE -121, -122, -123, or -124 error. A failed referential integrity check generates an error such as the following:

```
ERROR #5540: SQLCODE: -124 Message: At least 1 Row exists in table 'HealthLanguage.FKey2'
which references key NewIndex1 - Foreign Key Constraint 'NewForeignKey1' (Field 'Pointer1')
failed on referential action of NO ACTION [Execute+5*IRISql16:USER]
```

If you use other means to populate a table with data, referential integrity constraints may not be enforced. If this is a concern you should [validate the table data](#).

By default, when a row with a foreign key is deleted, InterSystems IRIS acquires a long term (until the transaction ends) shared lock on the corresponding referenced table's row. This prevents an update or delete of the referenced row until the **DELETE** transaction on the referencing row completes. This prevents the situation where the referenced row is deleted and then the delete of the referencing row is rolled back. If that happened, the foreign key would reference a non-existent row. This lock is not acquired if the foreign key is defined with [NoCheck](#), or if the **DELETE** of the referencing row is specified with `%NOCHECK` or `%NOLOCK`.

When using a persistent class definition to define a table, you can define a foreign key with the [NoCheck](#) keyword to suppress future checking of that foreign key. **CREATE TABLE** does not provide this keyword option.

You can suppress checking for a specific operation by using the `%NOCHECK` keyword option.

By default, InterSystems IRIS also performs foreign key referential integrity checking on the following operations. If the specified action violates referential integrity, the command is not executed:

- [ALTER TABLE DROP COLUMN](#).
- [ALTER TABLE DROP CONSTRAINT](#). Issues SQLCODE -317. Foreign Key integrity checking can be suppressed using `SET OPTION COMPILEMODE=NOCHECK`.
- [DROP TABLE](#). Issues SQLCODE -320. Foreign Key integrity checking can be suppressed using `SET OPTION COMPILEMODE=NOCHECK`.
- [TRUNCATE TABLE](#) (same considerations as [DELETE](#)).
- [Trigger events](#), including BEFORE events. For example, a BEFORE DELETE trigger is not executed if the DELETE operation would not be performed because it violates foreign key referential integrity.

In a parent/child relationship there is no defined ordering of the children. Application code must not rely on any particular ordering.

13.2.1 Defining Referential Integrity Checking with Persistent Classes

Defining a referential action on update or delete using a persistent class that projects to a table is defined in the [onDelete](#) and [onUpdate](#) foreign key keywords.

Referential integrity checking can be suppressed system-wide using the `$$SYSTEM.SQL.Util.SetOption()` method, as follows: `SET status=$$SYSTEM.SQL.Util.SetOption("FilerRefIntegrity", 0, .oldval)`. The default is 1 (referential integrity checking performed). To determine the current setting, call `$$SYSTEM.SQL.CurrentSettings()`.

13.3 Parent and Child Tables

This section provides a brief overview on defining and working with parent/child relationships through persistent classes. For further details, see [Defining and Using Relationships](#).

13.3.1 Defining Parent and Child Tables

When defining persistent classes that project to tables you can specify a parent/child relationship between two tables using the [Relationship property](#).

The following example defines the parent table:

Class Definition

```
Class Sample.Invoice Extends %Persistent
{
  Property Buyer As %String(MAXLEN=50) [Required];
  Property InvoiceDate As %TimeStamp;
  Relationship Pchildren AS Sample.LineItem [ Cardinality = children, Inverse = Cparent ];
}
```

The following example defines a child table:

Class Definition

```
Class Sample.LineItem Extends %Persistent
{
  Property ProductSKU As %String;
  Property UnitPrice As %Numeric;
  Relationship Cparent AS Sample.Invoice [ Cardinality = parent, Inverse = Pchildren ];
}
```

In the Management Portal SQL interface [Catalog Details tab](#), the **Table Info** provides the name of the Child Table(s) and/or the Parent Table. If a child table, it provides references to the parent table, such as Cparent ->Sample.Invoice.

A child table can itself be the parent of a child table. (This child of a child is known as a “grandchild” table.) In this case, the **Table Info** provides the names of both the Parent Table and the Child Table.

13.3.2 Inserting Data into Parent and Child Tables

You must insert each record into the parent table before inserting the corresponding records in the child table. To find the ROWID of the record in the parent table, select the %ID column and use a WHERE clause to specify the newly added record. For example:

SQL

```
INSERT INTO Sample.Invoice (Buyer,InvoiceDate) VALUES ('Fred',CURRENT_TIMESTAMP)
```

SQL

```
SELECT %ID FROM Sample.Invoice WHERE Buyer = 'Fred' -- returns 1
```

SQL

```
INSERT INTO Sample.LineItem (Cparent,ProductSKU,UnitPrice) VALUES (1,'45-A7',99.95)
INSERT INTO Sample.LineItem (Cparent,ProductSKU,UnitPrice) VALUES (1,'22-A1',0.75)
```

Attempting to insert a child record for which no corresponding parent record ID exists generates an SQLCODE -104 error with a %msg Child table 'Sample.LineItem' references non-existent row in parent table.

During an **INSERT** operation on a child table, a shared lock is acquired on the corresponding row in the parent table. This row is locked while inserting the child table row. The lock is then released (it is not held until the end of the transaction). This ensures that the referenced parent row is not changed during the insert operation.

13.3.3 Identifying Parent and Child Tables

In Embedded SQL, you can use a [host variable array](#) to identify parent and child tables. In a child table, Subscript 0 of the host variable array is set to the parent reference (Cparent), with the format `parent ref`, Subscript 1 is set to the child record ID with the format `parent ref || child ref`. In a parent table, Subscript 0 is undefined. This is shown in the following examples:

ObjectScript

```
KILL tflds,SQLCODE,C1
&sql(DECLARE C1 CURSOR FOR
    SELECT *,%TABLENAME INTO :tflds(),:tname
    FROM Sample.Invoice)
&sql(OPEN C1)
    IF SQLCODE<0 {WRITE "Serious SQL Error:",SQLCODE," ",%msg  QUIT}
    &sql(FETCH C1)
        IF SQLCODE=100 {WRITE "The ",tname," table contains no data",!  QUIT}
        WHILE $DATA(tflds(0)) {
            WRITE tname," is a child table",!,"parent ref: ",tflds(0)," %ID:
",tflds(1),!
            &sql(FETCH C1)
            IF SQLCODE=100 {QUIT}
        }
        IF $DATA(tflds(0))=0 {WRITE tname," is a parent table",!}
    &sql(CLOSE C1)
    IF SQLCODE<0 {WRITE "Error closing cursor:",SQLCODE," ",%msg  QUIT}
```

ObjectScript

```
KILL tflds,SQLCODE,C1
&sql(DECLARE C1 CURSOR FOR
    SELECT *,%TABLENAME INTO :tflds(),:tname
    FROM Sample.LineItem)
&sql(OPEN C1)
    IF SQLCODE<0 {WRITE "Serious SQL Error:",SQLCODE," ",%msg  QUIT}
    &sql(FETCH C1)
        IF SQLCODE=100 {WRITE "The ",tname," table contains no data",!  QUIT}
        WHILE $DATA(tflds(0)) {
            WRITE tname," is a child table",!,"parent ref: ",tflds(0)," %ID:
",tflds(1),!
            &sql(FETCH C1)
            IF SQLCODE=100 {QUIT}
        }
        IF $DATA(tflds(0))=0 {WRITE tname," is a parent table",!}
    &sql(CLOSE C1)
    IF SQLCODE<0 {WRITE "Error closing cursor:",SQLCODE," ",%msg  QUIT}
```

For a child table, `tflds(0)` and `tflds(1)` return values such as the following:

```
parent ref: 1 %ID: 1|1
parent ref: 1 %ID: 1|2
parent ref: 1 %ID: 1|3
parent ref: 1 %ID: 1|9
parent ref: 2 %ID: 2|4
parent ref: 2 %ID: 2|5
parent ref: 2 %ID: 2|6
parent ref: 2 %ID: 2|7
parent ref: 2 %ID: 2|8
```

For a “grandchild” table (a table that is the child of a child table), `tflds(0)` and `tflds(1)` return values such as the following:

```
parent ref: 1|1 %ID: 1|1|1
parent ref: 1|1 %ID: 1|1|7
parent ref: 1|1 %ID: 1|1|8
parent ref: 1|2 %ID: 1|2|2
parent ref: 1|2 %ID: 1|2|3
parent ref: 1|2 %ID: 1|2|4
parent ref: 1|2 %ID: 1|2|5
parent ref: 1|2 %ID: 1|2|6
```


14

Using Triggers

This topic describes how you can define triggers in InterSystems SQL. Triggers are lines of code that are executed in response to certain SQL events.

14.1 Defining Triggers

There are several ways to define a trigger for a specific table:

- Include a trigger definition in the persistent class definition that projects to an SQL table. For example, this definition of the MyApp.Person class includes a definition of the LogEvent trigger, which is invoked after each successful [INSERT](#) of data into the MyApp.Person table:

Class/ObjectScript

```
Class MyApp.Person Extends %Persistent [DdlAllowed]
{
    // ... Class Property Definitions

    Trigger LogEvent [ Event = INSERT, Time = AFTER ]
    {
        // Trigger code to log an event
    }
}
```

Class/Python

```
Class MyApp.Person Extends %Persistent [DdlAllowed]
{
    // ... Class Property Definitions

    Trigger LogEvent [ Event = INSERT, Time = AFTER, Language = python ]
    {
        // Trigger code to log an event
    }
}
```

For further details, refer to [Syntax of Trigger Definitions](#).

- Use the SQL [CREATE TRIGGER](#) command to create a trigger. This generates a trigger object definition in the corresponding persistent class. SQL trigger names follow [identifier](#) naming conventions. InterSystems IRIS® data platform uses the SQL trigger name to [generate a corresponding trigger class entity name](#).

You must have the [%CREATE_TRIGGER administrative-level privilege](#) to create a trigger. You must have the [%DROP_TRIGGER administrative-level privilege](#) to drop a trigger.

The maximum number of user-defined triggers for a class is 200.

Note: InterSystems IRIS does not support triggers on tables projected by collections. A user cannot define such a trigger, and the projection of a collection as a child table does not consider triggers involving that base collection.

InterSystems IRIS does not support triggers that modify the Security.Roles and Security.Users tables.

14.2 Types of Triggers

A trigger is defined by the following:

- The type of event that causes it to execute. A trigger may be either a single-event trigger or a multiple-event trigger. A single-event trigger is defined to execute when an [INSERT](#), an [UPDATE](#), or a [DELETE](#) event occurs on the specified table. A multiple-event trigger is defined to execute when any one of the multiple specified events occurs on the specified table. You can define an INSERT/UPDATE, an UPDATE/DELETE, or an INSERT/UPDATE/DELETE multiple-event trigger using either a class definition or the **CREATE TRIGGER** command. The type of event is specified in a class definition by the required [Event](#) trigger keyword.
- The time that the trigger executes: Before or After the event occurs. This is specified in a class definition by the optional [Time](#) trigger keyword. The default is Before.
- You can associate multiple triggers with the same event and time; in this case, you can control the order in which multiple triggers are fired using the [Order](#) trigger keyword. Triggers with a lower Order value are fired first. If multiple triggers have the same Order value, then the order in which they are fired is not specified.
- The optional [Foreach](#) trigger keyword provides additional granularity. This keyword controls whether the trigger is fired once per row (Foreach = row), once per row or object access (Foreach = row/object), or once per statement (Foreach = statement). A trigger defined with no [Foreach](#) trigger keyword is fired once per row. If a trigger is defined with Foreach = row/object, then the trigger is also called at specific points during object access, as described [later on this page](#). [You can list the Foreach value for each trigger](#) using the ACTIONORIENTATION property of INFORMATION.SCHEMA.TRIGGERS

Note: Python supports only the Foreach = row/object option.

For a full list of trigger keywords, see the [Class Definition Reference](#).

This table shows the available triggers and their corresponding [callback methods](#):

Trigger	Corresponding Callback Method
BEFORE INSERT	%OnBeforeSave()
AFTER INSERT	%OnAfterSave()
BEFORE UPDATE	%OnBeforeSave()
AFTER UPDATE	%OnAfterSave()
BEFORE UPDATE OF (specified columns)	None
AFTER UPDATE OF (specified columns)	None
BEFORE DELETE	%OnDelete()
AFTER DELETE	%OnAfterDelete()

Triggers and callback methods are similar, but they can execute at different times:

- Triggers execute either right before or right after modification of stored data.
- Callback methods execute before or after the **%SaveData()** or **%DeleteData()** methods, depending on the type of callback. Therefore, the **%OnBeforeSave()** method before the object locks or is modified by a referential action.

Note: When a trigger is executed, it cannot directly modify the value of a property in the table that is being processed. This is because InterSystems IRIS executes trigger code after field (property) value validation code. For example, a trigger cannot set a LastModified field to the current timestamp in the row being processed. However, the trigger code can issue an **UPDATE** to a field value in the table. The **UPDATE** performs its own field value validation.

For further details, refer to [CREATE TRIGGER](#).

14.2.1 AFTER Triggers

An AFTER trigger executes after an INSERT, UPDATE, or DELETE event occurs:

- If SQLCODE=0 (event completed successfully) InterSystems IRIS executes the AFTER trigger.
- If SQLCODE is a negative number (event failed) InterSystems IRIS does not execute the AFTER trigger.
- If SQLCODE=100 (no row was found to insert, update, or delete) InterSystems IRIS executes the AFTER trigger.

14.2.2 Recursive Triggers

Trigger execution can be recursive. For example, if table T1 has a trigger that performs an insert into table T2 and table T2 has a trigger that performs an insert into table T1. Recursion can also occur when table T1 has a trigger that calls a routine/procedure and that routine/procedure performs an insert into T1. Handling of trigger recursion depends on the [type of trigger](#):

- Row and Row/Object triggers: InterSystems IRIS *does not* prevent row triggers and row/object triggers from being executed recursively. It is the programmer's responsibility to handle trigger recursion. A runtime <FRAMESTACK> error may occur if the trigger code does not handle recursive execution.
- Statement triggers: InterSystems IRIS prevents an AFTER statement trigger from being executed recursively. InterSystems IRIS will not issue an AFTER trigger if it detects that the trigger has been called previously in the execution stack. No error is issued; the trigger is simply not executed a second time.

InterSystems IRIS *does not* prevent a BEFORE statement trigger from being executed recursively. It is the programmer's responsibility to handle BEFORE trigger recursion. A runtime <FRAMESTACK> error may occur if the BEFORE trigger code does not handle recursive execution.

14.3 ObjectScript Trigger Code

Each trigger contains one or more lines of code that perform a triggered action. This code is invoked by the SQL Engine whenever the event associated with the trigger occurs. If the trigger is defined using [CREATE TRIGGER](#), this action code can be written in either ObjectScript or SQL. (InterSystems IRIS converts code written in SQL to ObjectScript in the class definition.) If the trigger is defined in an IDE such as VS Code, this action code must be written in ObjectScript.

Because the code for a trigger is not generated as a procedure, all local variables in a trigger are public variables. With the exception of the system variables %ok, %msg, and %oper (described in the next section), all variables in triggers should

therefore be explicitly declared with a **NEW** statement; this protects them from conflicting with variables in the code that invokes the trigger.

14.3.1 %ok, %msg, and %oper System Variables

- **%ok:** A variable used only in trigger code. If trigger code succeeds, it sets %ok=1. If trigger code fails, it sets %ok=0. If during trigger execution an SQLCODE error is issued, InterSystems IRIS sets %ok=0. When %ok=0, the trigger code aborts and the trigger operation and the operation that invoked the trigger are rolled back. If INSERT or UPDATE trigger code fails and there is a foreign key constraint defined for the table, InterSystems IRIS releases the lock on the corresponding row in the foreign key table.

Trigger code can explicitly set %ok=0. This creates a runtime error that aborts execution of the trigger and rolls back the operation. Commonly, before setting %ok=0, trigger code explicitly sets the %msg variable to a user-specified string describing this user-defined trigger code error.

The %ok variable is unchanged from its prior value upon the completion of a non-trigger code **SELECT**, **INSERT**, **UPDATE**, or **DELETE** statement. %ok is only defined by the execution of trigger code. It is passed by reference to the trigger code to allow the trigger code to update it; therefore, it must not be explicitly declared with a **NEW** statement within the trigger code.

- **%msg:** Trigger code can explicitly set the %msg variable to a string describing the cause of the runtime error. SQLCODE errors set the %msg variable. Similar to the %ok variable, it is passed by reference to the trigger code and must not be explicitly declared with a **NEW** statement within the trigger code.
- **%oper:** A variable used only in trigger code. Trigger code can refer to the variable %oper, which contains the name of the event that fired the trigger (INSERT, UPDATE, or DELETE).

14.3.2 {fieldname} Syntax

Within trigger code, you can refer to field values (for the fields belonging to the table the trigger is associated with) using a special {fieldname} syntax. For example, the following definition of the LogEvent trigger in the MyApp.Person class includes a reference to the ID field, as {ID}:

Class Definition

```
Class MyApp.Person Extends %Persistent [DdlAllowed]
{
    // ... Definitions of other class members

    /// This trigger updates the LogTable after every insert
    Trigger LogEvent [ Event = INSERT, Time = AFTER ]
    {
        // get row id of inserted row
        NEW id,SQLCODE
        SET id = {ID}

        // INSERT value into Log table
        &sql(INSERT INTO LogTable
            (TableName, IDValue)
            VALUES ('MyApp.Person', :id))
        IF SQLCODE<0 {SET baderr="SQLCODE ERROR: "_SQLCODE_" "%msg
            SET %ok=0
            RETURN baderr }

    }
    // ... Definitions of other class members
}
```

This {fieldname} syntax supports unitary fields. It does not support %SerialObject collection properties. For example, if a table references the embedded serial object class Address, which contains the property City, the trigger syntax {Address_City} is a valid reference to a field. The trigger syntax {Address} is a reference to a collection property, and cannot be used.

14.3.3 Macros within Trigger Code

Your trigger code can contain a macro definition that references a field name (using `{fieldname}` syntax). However, if your trigger code contains a `#include` preprocessor directive for a macro that references a field name (using `{fieldname}` syntax), the field name cannot be accessed. This is because InterSystems IRIS translates `{fieldname}` references in the trigger code *before* the code is passed to the macro preprocessor. If a `{fieldname}` reference is in the `#include` file, it is not “seen” in the trigger code, and is therefore not translated.

The workaround for this situation is to define the macro with an argument, then pass the `{fieldname}` in to the macro in the trigger. For example, the `#include` file could contain a line such as the following:

ObjectScript

```
#define dtThrowTrigger(%val) SET x=$GET(%val, "?")
```

And then within the trigger code, refer to the macro, supplying the `{fieldname}` syntax as an argument:

ObjectScript

```
$$$dtThrowTrigger({%ID})
```

14.3.4 {name*O}, {name*N}, and {name*C} Trigger Code Syntax

Three ObjectScript syntax shortcuts are available in **UPDATE** trigger code.

You can reference the old (pre-update) value using the following syntax:

```
{fieldname*O}
```

where *fieldname* is the name of the field and the character after the asterisk is the letter “O” (for Old). For an **INSERT** trigger, `{fieldname*O}` is always the empty string (“”).

You can reference the new (post-update) value using the following syntax:

```
{fieldname*N}
```

where *fieldname* is the name of the field and the character after the asterisk is the letter “N” (for New). This `{fieldname*N}` syntax can be used only to reference a value to be stored; it cannot be used to change the value. You cannot set `{fieldname*N}` in trigger code. Computing the value of a field on **INSERT** or **UPDATE** should be achieved by other means, such as [SqlComputeOnChange](#).

You can test whether a field value has been changed (updated) using the following syntax:

```
{fieldname*C}
```

where *fieldname* is the name of the field and the character after the asterisk is the letter “C” (for Changed). `{fieldname*C}` evaluates to 1 if the field has been changed and 0 if it has not been changed. For an **INSERT** trigger, InterSystems IRIS sets `{fieldname*C}` to 1.

For a class with stream properties, an SQL trigger reference to the stream property `{Stream*N}` and `{Stream*O}` returns the OID for the stream, if the SQL statement (**INSERT** or **UPDATE**) did not insert/update the stream property itself. However, if the SQL statement did insert/update the stream property, `{Stream*O}` remains the OID, but the `{Stream*N}` value is set to one of the following:

- **BEFORE** trigger returns the value of the stream field in whatever format it was passed to the **UPDATE** or **INSERT**. This could be the literal data value that was entered into the stream property, or the OREF or OID of a temporary stream object.

- AFTER trigger returns the Id of the stream as the {Stream*N} value. This is the Id value InterSystems IRIS stored in the ^classnameD global for the stream field. This value is in the appropriate Id format based on the CLASSNAME type parameter for the stream property.

If a stream property is updated using InterSystems IRIS objects, the {Stream*N} value is always an OID.

Note: For a trigger for child tables created by an array collection of serial objects, trigger logic works with object access/save but does not work with SQL access (**INSERT** or **UPDATE**).

14.3.5 Additional ObjectScript Trigger Code Syntax

Trigger code written in ObjectScript can contain the pseudo-field reference variables {%%CLASSNAME}, {%%CLASSNAMEQ}, {%%OPERATION}, {%%TABLENAME}, and {%%ID}. These pseudo-fields are translated into a specific value at class compilation time. For further details, refer to [CREATE TRIGGER](#).

You can use class methods from within trigger code, SQL computed code, and SQL map definitions since class methods do not depend on having an open object. You must use the `##class(classname).Methodname()` syntax to invoke a method from within trigger code. You cannot use the `..Methodname()` syntax, because this syntax requires a current open object.

You can pass the value of a field of the current row as an argument of the class method, but the class method itself cannot use field syntax.

14.4 Python Trigger Code

Within a trigger block containing embedded Python, you can access a `trigger` object that contains attributes and methods related to that trigger. You can access these attributes, which are described in more detail in [Types of Triggers](#).

- `trigger.type` — Valid values are "row/object" or "row".
- `trigger.operation` — Valid values are "BEFORE", "UPDATE", and "DELETE".
- `trigger.time` — Valid values are "before" and "after".

Additionally, you can access `trigger.ok` and `trigger.msg` to check the `%ok` and `%msg` variables, respectively.

The trigger object also provides a `getfield()` method that enables you to access the names of table fields. You can specify these options:

- **`trigger.getfield(fieldName)`** returns the value of the specified field, *fieldName*. If the trigger operation changes the field value, then **`getfield()`** returns the new value.
- **`trigger.getfield(fieldName, new)`** uses a boolean value, *new*, to determine whether to return the new or original field value.
 - If *new* = 1 (default), then **`getfield()`** returns the new value resulting from the trigger operation. This option applies to INSERT and UPDATE operations. This is analogous to the {fieldname*O} syntax in ObjectScript.
 - If *new* = 0, then **`getfield()`** returns the value prior to the trigger operation. This option applies to UPDATE and DELETE operations. This is analogous to the {fieldname*N} syntax in ObjectScript.
 - There is no immediate Python equivalent to {fieldname*C}. Instead, you should compare the values of `trigger.getfield(fieldName, 0)` and `trigger.getfield(fieldName, 1)`.

This example uses a `trigger` object to write the results of a trigger operation, the renaming of the `Name` field, to a log file. Both the Python code and the equivalent ObjectScript code are shown.

Python

```
Trigger LogRename [ Event = UPDATE, Foreach = row/object, Language = python, Time = AFTER ]
{
    with open('C:/temp/log.txt', 'a+') as file:
        file.write("Rename Event Occurred")
        file.write("\nID: " + str(trigger.getfield("ID")))
        file.write("\nOperation: " + trigger.operation)
        oldname = trigger.getfield("Name",0)
        newname = trigger.getfield("Name",1)
        changed = (oldname != newname)
        file.write("\nOld Name: " + oldname)
        file.write("\nNew Name: " + newname)
        file.write("\nChange in Name: " + str(oldname != newname) + "\n\n")
}
```

ObjectScript

```
Trigger LogRename [ Event = UPDATE, Foreach = row/object, Time = AFTER ]
{
    set file = "C:/temp/log.txt"
    open file:("EWA") use file
    write "Rename Event Occurred"
    write !,"ID: "_{ID}
    write !,"Operation: "_{%OPERATION}
    write !,"Old Name: "_{Name*O}
    write !,"New Name: "_{Name*N}
    write !,"Change in Name: "_{Name*C},!!
    close file
}
```

14.5 Pulling Triggers

A defined trigger is “pulled” (executed) if the corresponding DML command for that table is invoked.

A Row or Row/Object trigger is pulled for each row that the DML command successfully inserts, updates, or deletes.

A Statement trigger is pulled once for each INSERT, UPDATE, or DELETE statement that successfully executes, regardless of whether the statement actually changes any rows of the table data.

- An **INSERT** statement pulls the corresponding INSERT trigger. An **INSERT** can prevent pulling of this corresponding trigger by specifying the `%NOTRIGGER` keyword. An **INSERT** that specifies the `%NOJOURN` keyword does not journal the insert or the corresponding INSERT trigger. This means that rollback is not possible for either the insert event or the trigger event.
- An **UPDATE** statement pulls the corresponding UPDATE trigger. An **UPDATE** can prevent pulling of this corresponding trigger by specifying the `%NOTRIGGER` keyword. An **UPDATE** that specifies the `%NOJOURN` keyword does not journal the update or the corresponding UPDATE trigger. This means that rollback is not possible for either the update event or the trigger event.
- An **INSERT OR UPDATE** statement pulls the corresponding INSERT trigger or UPDATE trigger, depending on the type of DDL operation performed. To prevent pulling of either type of trigger, specify the `%NOTRIGGER` keyword.
- A **DELETE** statement pulls the corresponding DELETE trigger. A **DELETE** can prevent pulling of this corresponding trigger by specifying the `%NOTRIGGER` keyword. A **DELETE** that specifies the `%NOJOURN` keyword does not journal the delete or the corresponding DELETE trigger. This means that rollback is not possible for either the delete event or the trigger event.
- A **TRUNCATE TABLE** statement does not pull a DELETE trigger.

By default, DDL statements and the corresponding triggered actions are journaled. The %NOJOURN keyword prevents journaling of both the DDL command and the triggered action.

14.6 Triggers and Object Access

If a trigger is defined with `Foreach = row/object`, then the trigger is also called at specific points during object access, depending on the Event and Time keywords of the trigger definition, as follows:

Event	Time	Trigger is also called at this time
INSERT	BEFORE	Just before %Save() for a new object
INSERT	AFTER	Just after %Save() for a new object
UPDATE	BEFORE	Just before %Save() for an existing object
UPDATE	AFTER	Just after %Save() for an existing object
DELETE	BEFORE	Just before %DeleteId() for an existing object
DELETE	AFTER	Just after %DeleteId() for an existing object

As a consequence, it is not necessary to also implement callback methods in order to keep SQL and object behavior synchronized,

For information on the `Foreach` trigger keyword, see [Foreach](#).

14.6.1 Not Pulling Triggers During Object Access

By default, SQL objects are stored using [%Storage.Persistent](#). InterSystems IRIS also supports [%Storage.SQL](#) storage.

When saving or deleting objects in a class that uses [%Storage.SQL](#) storage, all statement (`Foreach = statement`), row (`Foreach = row`), and row/object (`Foreach = row/object`) triggers are pulled. A trigger defined with no [Foreach](#) trigger keyword is a row trigger. Pulling all triggers is the default behavior.

However, when saving or deleting objects in a class using [%Storage.SQL](#), you can specify that only triggers defined as `Foreach = row/object` should be pulled. Triggers defined as `Foreach = statement` or `Foreach = row` are not pulled. This is done by specifying the class parameter `OBJECTSPULLTRIGGERS = 0`. The default is `OBJECTSPULLTRIGGERS = 1`.

This parameter only applies to classes defined as using [%Storage.SQL](#).

14.7 Triggers and Transactions

A trigger executes trigger code within a [transaction](#). It sets the [transaction level](#), then executes the trigger code. Upon successful completion of trigger code, the trigger commits the transaction.

Note: A consequence of triggers using transactions is that if a trigger invokes code that commits a transaction, completion of the trigger fails because the transaction level has already been decremented to 0. This situation can occur when invoking a Business Service of a Production.

With an AFTER INSERT statement-level ObjectScript trigger, if the trigger sets %ok=0, the insert of the row fails with an SQLCODE -131 error. Depending on your automatic transaction setting, transaction rollback might also occur. In SQL, the [SET TRANSACTION](#) %COMMITMODE *commitmode* option controls this setting. In ObjectScript, the "AutoCommit" option of the **SetOption()** method controls this setting, using this syntax: `SET status=$SYSTEM.SQL.Util.SetOption("AutoCommit",intval,.oldval)`. This table shows the effect the automatic transaction setting has on rollback. It also shows the corresponding *commitmode* option to set for SQL and "AutoCommit" integer value to set for ObjectScript.

Automatic Transaction Setting	%COMMITMODE Option (SQL)	"AutoCommit" Value (ObjectScript)	Transaction Rollback Result
No automatic transaction processing	NONE	0	No transaction was started, so the INSERT cannot be rolled back.
Automatic transaction commitment is on (default)	IMPLICIT	1	The transaction for the INSERT is rolled back.
Automatic transaction commitment is off	EXPLICIT	2	It is up to the application to either rollback or commit the transaction for the INSERT.

The trigger can set an error message in the %msg variable in the trigger. This message will be returned to the caller, giving information why the trigger failed.

The %ok and %msg system variables are described in [System Variables](#).

14.8 Listing Triggers

Triggers defined for a specified table are listed in the Management Portal SQL interface [Catalog Details](#). This lists basic information for each trigger. To list more detailed information, use INFORMATION.SCHEMA.TRIGGERS.

The INFORMATION.SCHEMA.TRIGGERS class lists the defined triggers in the current namespace. For each trigger INFORMATION.SCHEMA.TRIGGERS lists various properties, including the name of the trigger, the associated schema and table name, the EVENTMANIPULATION property (INSERT, UPDATE, DELETE, INSERT/UPDATE, INSERT/UPDATE/DELETE), the ACTIONTIMING property (BEFORE, AFTER), the CREATED property (trigger creation timestamp), and the ACTIONSTATEMENT property, which is the generated SQL trigger code.

The CREATED property derives the trigger creation timestamp from when the class definition was last modified. Therefore, subsequent use of this class (for example, to define other triggers) may result in unintended updating of the CREATED property value.

You can access this INFORMATION.SCHEMA.TRIGGERS information from an SQL query, as shown in the following example:

SQL

```
SELECT
TABLE_NAME, TRIGGER_NAME, CREATED, EVENT_MANIPULATION, ACTION_TIMING, ACTION_ORIENTATION, ACTION_STATEMENT
FROM INFORMATION_SCHEMA.TRIGGERS WHERE TABLE_SCHEMA='Sample'
```


15

Collation

Collation specifies how values are ordered and compared. There are two fundamental collations: numeric and string.

- Numeric collation orders numbers based on the complete number in the following order: null, then negative numbers from largest to smallest, zero, then positive numbers from smallest to largest. This creates a sequence such as the following: -210, -185, -54, -34, -.02, 0, 1, 2, 10, 17, 100, 120.
- String collation orders strings by collating on each sequential character. This creates an order such as the following: null, A, AA, AAA, AAB, AB, B. For numbers, this creates an order such as the following: -.02, -185, -210, -34, -54, 0, 1, 10, 100, 120, 17, 2.

The default string collation is SQLUPPER; this default is set for each namespace. SQLUPPER collation converts all letters to uppercase (for the purpose of collation), and prepends a space character to the beginning of the string. This conversion is for the purposes of collation only; in InterSystems SQL strings are usually displayed in uppercase and lowercase letters, regardless of the collation applied, and the length of a string does not include the prepended space character.

A timestamp is a string, and therefore follows the current string collation. However, because a timestamp is in ODBC format, the string collation is the same as chronological sequence, if leading zeros are specified.

- A string expression (such as those using the scalar string functions **LEFT** or **SUBSTR**) makes its result collation EXACT.
- Any comparison of two literals uses EXACT collation.

You can use the ObjectScript [Sorts After operator](#) to determine the relative collation sequence order of two values.

You can specify collation as follows:

- [Namespace default](#)
- [Table field/property definition](#)
- [Index definition](#)
- [Query SELECT item](#)
- [Query DISTINCT and GROUP BY clause](#)

Also see [SQL Collation and NLS Collations](#).

15.1 Collation Types

Collation can be specified as a keyword in the definition of a field/property or the definition of an index.

Collation can be specified by applying a collation function to a field name in a query clause. The % prefix is required when specifying a collation function.

Collation is in ascending ASCII/Unicode sequence, with the following transformations:

- **EXACT** — Enforces case sensitivity for string data without adding or stripping characters. Collates [canonical numbers](#) in numeric order, then collates strings in character-by-character ASCII order. String collation is case sensitive. String collation includes non-canonical numbers (such as 088) and mixed numeric strings (such as 22 Elm Street). Because canonical numbers collate before strings, including single characters and non-canonical numeric strings, EXACT is generally not recommended for string data that may contain values in canonical numeric format (for example 123 or - . 57).
- **SQLSTRING** — Strips trailing whitespace (spaces, tabs, and so on), and adds one leading blank space to the beginning of the string. It collates any value containing only whitespace (spaces, tabs, and so on) as the SQL empty string. SQLSTRING supports an optional *maxlen* integer value.
- **SQLUPPER** — Converts all alphabetic characters to uppercase, strips trailing whitespace (spaces, tabs, and so on), and then adds one leading space character to the beginning of the string. The reason this space character is prepended is to force numeric values to be collated as strings (because the space character is not a valid numeric character). This transformation also causes SQL to collate the SQL empty string (") value and any value containing only whitespace (spaces, tabs, and so on) as a single space character. SQLUPPER supports an optional *maxlen* integer value. Note that the SQLUPPER transform is *not* the same as the result of the SQL function [UPPER](#).
- **TRUNCATE** — Enforces case sensitivity for string data and (unlike EXACT) allows you to specify a length at which to truncate the value. This is useful when indexing exact data that is longer than what is supported for use in a subscript. It takes a positive integer argument, in the form %TRUNCATE(*string*,*n*), to truncate the string to the first *n* characters, which improves indexing and sorting on long strings. If you do not specify a length for TRUNCATE, it behaves identically to EXACT; while this behavior is supported, your definitions and code may be easier to maintain if you use TRUNCATE only when you have a length defined and EXACT when you do not.
- **PLUS** — Makes the value numeric. A non-numeric string value is returned as 0.
- **MINUS** — Makes the value numeric and changes its sign. A non-numeric string value is returned as 0.

Note: There are also various [legacy collation types](#), the use of which is not recommended.

In an SQL query, you can specify a collation function without parentheses %SQLUPPER Name or with parentheses %SQLUPPER(Name). If the collation function specifies truncation, the parentheses are required %SQLUPPER(Name, 10).

Three collation types: SQLSTRING, SQLUPPER, and TRUNCATE support an optional *maxlen* integer value. If specified, *maxlen* truncates parsing of the string to the first *n* characters. This can be used to improve performance when indexing and sorting long strings. You can use *maxlen* in a query to sort on, group by, or return a truncated string value.

You can also perform collation type conversions using the %SYSTEM.Util.Collation() method.

15.2 Namespace-wide Default Collation

Each namespace has a current string collation setting. This string collation is defined for the data type in %Library.String. The default is SQLUPPER. This default can be changed.

You can define the collation default on a per-namespace basis. By default, namespaces have no assigned collation, which means they use SQLUPPER collation. You can assign a different default collation to a namespace. This namespace default collation applies to all processes, and persists across InterSystems IRIS restarts until explicitly reset.

ObjectScript

```
SET stat=$$GetEnvironment^%apiOBJ("collation", "%Library.String", .collval)
WRITE "initial collation for ", $NAMESPACE, !
ZWRITE collval
SetNamespaceCollation
DO SetEnvironment^%apiOBJ("collation", "%Library.String", "SQLstring")
SET stat=$$GetEnvironment^%apiOBJ("collation", "%Library.String", .collnew)
WRITE "user-assigned collation for ", $NAMESPACE, !
ZWRITE collnew
ResetCollationDefault
DO SetEnvironment^%apiOBJ("collation", "%Library.String", .collval)
SET stat=$$GetEnvironment^%apiOBJ("collation", "%Library.String", .collreset)
WRITE "restored collation default for ", $NAMESPACE, !
ZWRITE collreset
```

Note that if you have never set the namespace collation default, `$$GetEnvironment` returns an undefined collation variable, such as `.collval` in this example. This undefined collation defaults to SQLUPPER.

Note: If your data contains German text, uppercase collation may not be a desirable default. This is because the German *eszett* character (`$CHAR(223)`) has only a lowercase form. The uppercase equivalent is the two letters “SS”. SQL collations that convert to uppercase do not convert *eszett*, which remains unchanged as a single lowercase letter.

15.3 Table Field/Property Definition Collation

Within SQL, collation can be assigned as part of field/property definition. The data type used by a field determines its default collation. The default collation for string data types in tables that use row storage is SQLUPPER; however, in tables that use columnar storage, the default collation is EXACT. Non-string data types do not support collation assignment.

You can specify collation for a field in **CREATE TABLE** and **ALTER TABLE**:

SQL

```
CREATE TABLE Sample.MyNames (
  LastName CHAR(30),
  FirstName CHAR(30) COLLATE SQLstring)
```

Note: When specifying collation for a field using **CREATE TABLE** and **ALTER TABLE**, the % prefix is optional: `COLLATE SQLstring` or `COLLATE %SQLstring`.

You can specify collation for a property when defining a table using a persistent class definition:

Class Definition

```
Class Sample.MyNames Extends %Persistent [DdlAllowed]
{
  Property LastName As %String;
  Property FirstName As %String(COLLATION = "SQLstring");
}
```

Note: When specifying collation for class definitions and class methods do not use the % prefix for collation type names.

In these examples, the `LastName` field takes default collation (SQLUPPER, which is not case-sensitive), the `FirstName` field is defined with SQLSTRING collation, which is case-sensitive.

If you change the collation for a class property and you already have stored data for that class, any indexes on the property become invalid. You must rebuild all indexes based on this property.

15.4 Index Definition Collation

The **CREATE INDEX** command cannot specify an index collation type. The index uses the same collation as the field being indexed.

An index defined as part of class definition can specify a collation type. By default, an index on a given property (or properties) uses the collation type of the property data. For example, suppose you have defined a property Name of type %String:

Class Definition

```
Class MyApp.Person Extends %Persistent [DdlAllowed]
{
  Property Name As %String;
  Index NameIDX On Name;
}
```

The collation for Name is SQLUPPER (the default for %String). Suppose that the Person table contains the following data:

ID	Name
1	Jones
2	JOHNSON
3	Smith
4	jones
5	SMITH

Then an index on Name will contain the following entries:

Name	ID(s)
JOHNSON	2
JONES	1, 4
SMITH	3, 5

The SQL Engine can use this index directly for ORDER BY or comparison operations using the Name field.

You can override the default collation used for an index by adding an As clause to the index definition:

Class Definition

```
Class MyApp.Person Extends %Persistent [DdlAllowed]
{
  Property Name As %String;
  Index NameIDX On Name As SQLstring;
}
```

In this case the NameIDX index will now store values in SQLSTRING (case-sensitive) form. Using the data from the above example:

Name	ID(s)
JOHNSON	2
Jones	1
jones	4
SMITH	5
Smith	3

In this case, the SQL Engine can take advantage of this index for any queries requiring case-sensitive collation.

In general, you should not have to change the collations of indexes. If you want to use a different collation, it is better to define it at the property level and let any indexes on the property pick up the correct collation.

If you are performing a property comparison using an indexed property, the property as specified in the comparison should have the same collation type as the corresponding index. For example, the Name property in the WHERE clause of a SELECT or in the ON clause of a JOIN should have the same collation as the index defined for the Name property. If there is a mismatch between the property collation and the index collation, the index may be less effective or may not be used at all. For further details, refer to [Index Collation](#).

If your index is defined to use multiple properties, you can specify the collation of each individually:

Class Member

```
Index MyIDX On (Name As SQLstring, Code As Exact);
```

15.5 Query Collation

InterSystems SQL provides collation functions that can be used to change the collation or display of a field.

15.5.1 select-item Collation

Applying a collation function to a query select-item changes the display of that item.

- Letter Case: By default, a query displays strings with uppercase and lowercase letters. The exceptions to this are the DISTINCT or GROUP BY operations on a field of collation type SQLUPPER. These operations display that field in all uppercase letters. You can use the %EXACT collation function to reverse this letter case transformation and display the field in uppercase and lowercase letters. You should not use an %SQLUPPER collation function in the *select-item* list to display a field in all uppercase letters. This is because %SQLUPPER adds a space character to the length of the string. Use the UPPER function instead:

SQL

```
SELECT TOP 5 Name,$LENGTH(Name) AS NLen,
              %SQLUPPER(Name) AS UpCollN,$LENGTH(%SQLUPPER(Name)) AS UpCollLen,
              UPPER(Name) AS UpN,$LENGTH(UPPER(Name)) AS UpLen
FROM Sample.Person
```

- String Truncation: You can use the %TRUNCATE collation function to limit the length of the string data you wish to display. %TRUNCATE is preferable to %SQLUPPER, which adds a space character to the length of the string.

SQL

```
SELECT TOP 5 Name,$LENGTH(Name) AS NLen,  
             %TRUNCATE(Name,8) AS TruncN,$LENGTH(%TRUNCATE(Name,8)) AS TruncLen  
FROM Sample.Person
```

Note that you cannot nest collation functions or case-transformation functions.

- WHERE clause comparisons: Most WHERE clause predicate condition comparisons use the collation type of the field/property. Because string fields default to SQLUPPER, these comparisons are commonly not case-sensitive. You can use the %EXACT collation function to make them case-sensitive:

The following example returns Home_City string matches regardless of letter case:

SQL

```
SELECT Home_City FROM Sample.Person WHERE Home_City = 'albany'
```

The following example returns Home_City string matches that are case-sensitive:

SQL

```
SELECT Home_City FROM Sample.Person WHERE %EXACT(Home_City) = 'albany'
```

The SQL [Follows operator \(\)](#) uses the field/property collation type.

However, the SQL [Contains operator \(\[\] \)](#) uses EXACT collation, regardless of the collation type of the field/property:

SQL

```
SELECT Home_City FROM Sample.Person WHERE Home_City [ 'c'  
ORDER BY Home_City
```

The [%MATCHES](#) and [%PATTERN](#) predicate conditions use EXACT collation, regardless of the collation type of the field/property. The [%PATTERN](#) predicate provides both case-sensitive wildcards and a wildcard ('A') which is not case-sensitive.

- ORDER BY clause: The ORDER BY clause uses the namespace default collation to order string values. Therefore, ORDER BY does not order based on lettercase. You can use %EXACT collation to order strings based on lettercase.

15.5.2 DISTINCT and GROUP BY Collation

By default, these operation use the current namespace collation. The default namespace collation is SQLUPPER.

- DISTINCT: The DISTINCT keyword uses the namespace default collation to eliminate duplicate values. Therefore, DISTINCT Name returns values in all uppercase letters. You can use EXACT collation to return values in mixed uppercase and lowercase. DISTINCT eliminates duplicates that differ only in letter case. To preserve duplicates that differ in case, but eliminate exact duplicates, use EXACT collation. The following example eliminates exact duplicates (but not lettercase variants) and returns all values in mixed uppercase and lowercase:

SQL

```
SELECT DISTINCT %EXACT(Name) FROM Sample.Person
```

A [UNION](#) involves an implicit DISTINCT operation.

- GROUP BY: The GROUP BY clause uses the namespace default collation to eliminate duplicate values. Therefore, GROUP BY Name returns values in all uppercase letters. You can use EXACT collation to return values in mixed uppercase and lowercase. GROUP BY eliminates duplicates that differ only in letter case. To preserve duplicates that

differ in case, but eliminate exact duplicates, you must specify the %EXACT collation function on the GROUP BY clause, not the select-item.

The following example returns values in mixed uppercase and lowercase; the GROUP BY eliminates duplicates, including those that differ in lettercase:

SQL

```
SELECT %EXACT(Name) FROM Sample.Person GROUP BY Name
```

The following example returns values in mixed uppercase and lowercase; the GROUP BY eliminates exact duplicates (but not lettercase variants):

SQL

```
SELECT Name FROM Sample.Person GROUP BY %EXACT(Name)
```

15.6 Legacy Collation Types

InterSystems SQL supports several legacy collation types. These are deprecated and not recommended for use with new code, as their purpose is to provide continued support for legacy systems. They are:

- **%ALPHAUP** — Removes all punctuation characters except question marks (“?”) and commas (“,”), and translates all the lowercase letters to uppercase. Used mostly for mapping legacy globals. Replaced by SQLUPPER.
- **%STRING** — Converts a logical value to uppercase, strips all punctuation and white space (except for commas), and adds one leading blank space to the beginning of the string. It collates any value containing only whitespace (spaces, tabs, and so on) as the SQL empty string. Replaced by SQLUPPER.
- **%UPPER** — Translates all lowercase letters into uppercase letters. Used mostly for mapping legacy globals. Replaced by SQLUPPER.
- **SPACE** — SPACE collation prepends a single leading space to a value, forcing it to be evaluated as a string. To establish SPACE collation, [CREATE TABLE](#) provides a SPACE collation keyword, and ObjectScript provides a SPACE option in the **Collation()** method of the %SYSTEM.Util class. There is no corresponding SQL collation function.

Note: If a string data type field is defined with EXACT, UPPER, or ALPHAUP collation, and a query applies a [%STARTSWITH](#) condition on this field, inconsistent behavior may result. If the *substring* you specify to **%STARTSWITH** is a canonical number (especially a negative and/or fractional number), **%STARTSWITH** may give different results depending on whether the field is indexed. The **%STARTSWITH** should perform as expected if the column is not indexed. If the column is indexed, unexpected results may occur.

15.7 SQL and NLS Collations

The SQL collations described above should not be confused with the InterSystems IRIS NLS collation feature, which provides subscript-level encoding that adhere to particular national language collation requirements. These are two separate systems of providing collations, and they work at different levels of the product.

InterSystems IRIS NLS collations can have a *process-level collation* for the current process, and different collations for specific globals.

To ensure proper functioning when using InterSystems SQL, it is a requirement that the process-level NLS collation matches exactly the NLS collation of all globals involved, including globals used by the tables and globals used for temporary files such as process private globals and for [IRISTEMP globals](#). If this is not the case, the Query Processor might give different results and ignores parallel processing. In situations where sorting occurs, such as an [ORDER BY](#) clause or a range condition, the Query Processor selects the most efficient sorting strategy. It may use an index, use a temporary file in a process-private global, sort within a local array, or use a "JJ" ([Sorts After](#)) comparison. All these are subscript-type comparisons that adhere to the InterSystems IRIS NLS collation that is in effect, which is why it is necessary that all these types of globals use the exact same NLS collation.

The system creates a global with the data base default collation. You can use the **Create()** method of the `%Library.GlobalEdit` class to create a global with a different collation. The only requirement is that the specified collation be either built-in (such as the InterSystems IRIS standard) or one of the national collations available in the current locale. See [Using %Library.GlobalEdit to Set Collation For A Global](#).

16

Federated Tables

Important: In this release, federated tables are an experimental feature.

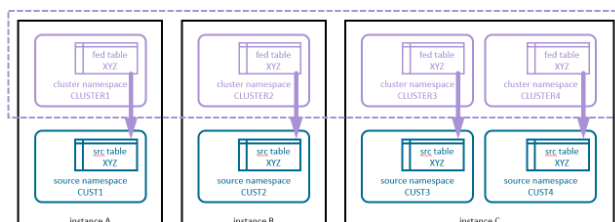
A federated table offers query access to data that lives in disparate instances of InterSystems IRIS® data platform hosting the identical or similar schemas. Unlike a [sharded table](#), federated tables are read-only and do not manage or distribute data among the sources based on a shard key. The source tables for a federated table, which physically store the its data and live in the distinct instances or namespaces, are managed exclusively by their host instance. As such, federated tables are read only. Federated tables offer a solution to use cases that require analytics or querying across different deployments of the same application or schema, such as multi-tenant environments.

16.1 Requirements for Creating a Federated Table

To connect source tables together as a federated table, you must first configure a sharded cluster that spans all the instances containing the relevant source data. For more information about configuring a sharded cluster, see “[Deploying the Sharded Cluster](#).” Users that are new to sharding may find it useful to familiarize themselves with the terminology and concepts in “[Horizontally Scaling for Data Volume with Sharding](#).”

A cluster namespace must be created for each namespace that contains one or more source tables, called a source namespace. This requirement applies to both cases where each namespace resides on a separate instance and to cases where multiple source namespaces reside on the same instance; in the latter case, multiple cluster namespaces on the same instance need to be configured so that each source namespace has a corresponding cluster namespace.

The following image shows the relationships between federated tables, source tables, cluster namespaces, and source namespaces.



16.2 Creating a Federated Table

Once you have configured a sharded cluster, creating a federated table consists of two parts: defining the federated table and connecting source tables to the federated table.

16.2.1 Defining a Federated Table

From the master namespace of the sharded cluster, you can create a federated table by using `$SYSTEM.Sharding.CreateFederatedTable()`. See complete documentation on how to use this method in the [Class Reference](#).

CreateFederatedTable() registers the federated table definition in the cluster namespace that represents the federated table and defines how columns from a source table are projected into the federated table. As part of the API call, columns from the source table may be left out of the federated table. This method also attaches the initial source table from the specified source namespace, which is stored on the same instance. A source namespace is never impacted by the existence of a federated table that any of its tables projects to.

There are a few notes about federated table definitions:

- Private properties in a source table are not projected to a federated table unless explicitly listed as an argument to **CreateFederatedTable()**.
- The types of source fields do not need to match the types of the corresponding fields in the federated tables. Implicit type conversions are identical to those used in [UNION clauses](#) and return a data type with the highest precedence as follows: VARCHAR, DOUBLE, NUMERIC, BIGINT, INTEGER, SMALLINT, TINYINT.
- The `SqlRowIdPrivate` class keyword is determined by the source table and the ID is projected with the name from the source table. If this keyword is defined, the `RowID` is not projected to the federated table. Note that tables created via DDL specify `SqlPrivateRowId` by default.

The newly created federated table is immediately accessible from any cluster namespace in the sharded cluster.

At this point, the newly created federated table only projects data from the source table in the source namespace that was referred to in the call to **CreateFederatedTable()**. See [Connecting Source Tables to a Federated Table](#) below for information about including data from source table in other source namespaces.

16.2.1.1 Examples

The following example demonstrates a call to **CreateFederatedTable()** from the terminal that creates a federated table, `Hospital.Employees`, in the `IRISCLUSTER` namespace (the default name for a cluster namespace). This federated table uses `Employees.Doctors` from the `USER` namespace as an initial source table. The federated table's columns are identical to those of the source table.

ObjectScript

```
do $SYSTEM.Sharding.CreateFederatedTable("Hospital.Employees", "USER", "Employees.Doctors")
```

The following example creates a federated table, `Hospital.DiagnosisLog`, in the `IRISCLUSTER` cluster namespace. This federated table uses `Hospital.Patient` from the `HOSPITAL` namespace as an initial source table and projects the `Diagnosis` and `DateAdmitted` columns from the source table. In this case, the `DateAdmitted` column is projected as `AdmissionDate` to the federated table. The resulting federated table in this example has only two columns.

ObjectScript

```
do $SYSTEM.Sharding.CreateFederatedTable("IRISCLUSTER", "Hospital.DiagnosisLog", "HOSPITAL",  
"Hospital.Patient", $lb($lb("Diagnosis"), $lb("DateAdmitted", "AdmissionDate")))
```

16.2.2 Connecting Source Tables to a Federated Table

After creating the federated table, connect source tables from other source namespaces to it. To do so, use the `$SYSTEM.Sharding.ConnectFederatedTable()` method in each source namespace. See complete documentation for this method in the Class Reference.

When a source table is connected to the federated table, any table statistics that have been collected by [TUNE TABLE](#) are reported to the federated table. For optimal query performance, you should run `TUNE TABLE` on a source table before connecting it to a federated table, so the table statistics are most accurate. If the source table needs to be re-tuned and you wish to propagate the updated statistics to the federated table, tune the source table, then re-connect it to the federated table, specifying 1 for the *Force* argument to `ConnectFederatedTable()`.

If the source table is altered or dropped, the federated table definition is not automatically updated. This is a consequence of how federated tables treat the source namespace as read-only and cannot be triggered by any events in it. When a source table is altered, you will need to call `ConnectFederatedTable()` in the cluster namespace corresponding to the source namespace that has changed and specify the alterations, specifying 1 for the *Force* argument to `ConnectFederatedTable()`. For example, changing a column name in the source table requires you to re-specify the column names that are projected into the federated table. Note that this step is not required for any compatible changes, such as adding a column to the source table that does not need to be projected to the federated table.

When the source tables' structure matches exactly between different source namespaces, calls to `ConnectFederatedTable()` are easy to script, as you can leverage the `%SYSTEM.ShardWorkMgr.Broadcast()` instance method to execute them once in each cluster namespace. See the [Examples](#) for further explanation.

16.2.2.1 Examples

The following example uses `ConnectFederatedTable()` to connect the `Employees.Nurses` source table in the `USER` namespace to the `Hospital.Employees` federated table in the `IRISCLUSTER` namespace.

ObjectScript

```
do $SYSTEM.Sharding.CreateFederatedTable("IRISCLUSTER", "Hospital.Employees", "USER", "Employees.Nurses")
```

The following example employs the `%SYSTEM.ShardWorkMgr.Broadcast()` instance method to connect source tables, called `Employees.Nurses` in the `HOSPITAL` namespace, to a newly created federated table, also called `Employees.Nurses`. Note that this approach requires each instance to have identically named source tables and source namespaces, due to the same arguments being broadcast across instances. The cluster namespace argument to `ConnectFederatedTable()` should be left blank.

ObjectScript

```
zn "IRISCLUSTER"
set status=$SYSTEM.Sharding.CreateFederatedTable("Employees.Nurses", "HOSPITAL", "Employees.Nurses")
set shardManager=$SYSTEM.ShardWorkMgr.%New()
set
status=shardManager.Broadcast("DS", "##class(%SYSTEM.Sharding).ConnectFederatedTable", "Employees.Nurses", "HOSPITAL", "Employees.Nurses")
```

16.3 Querying a Federated Table

Querying a federated table is transparently similar to querying a standard table. Federated tables can be queried with [Dynamic SQL](#), [Embedded SQL](#), or with a database driver, such as JDBC or ODBC.

16.4 Dropping or Disconnecting a Federated Table

To drop a federated table from your sharded cluster, use `$SYSTEM.Sharding.DropFederatedTable()`. This method, called from the cluster namespace of any instance, drops the federated table definition from all the cluster namespaces. Further attempts to query the federated table will fail, as it no longer exists. Dropping a federated table does not affect the source tables or their data.

To remove a single source table from a federated table, call `$SYSTEM.Sharding.DisconnectFederatedTable()` from the cluster namespace corresponding to the source namespace that contains the source table to be disconnected. This method disconnects a source table from the federated table. When a source table is disconnected from a federated table, queries on the federated table will no longer return data from that source table. The source table can later be re-connected to the federated table with **`ConnectFederatedTable()`**.

17

Foreign Tables

InterSystems SQL enables you to define a foreign table, which projects data from an external data source into InterSystems IRIS® data platform and allows you to query such external data alongside data stored in InterSystems IRIS.

Foreign tables are an evolution of linked tables, removing many query processing limitations, and are completely integrated into the InterSystems SQL interface. If you currently use linked tables, consider migrating to foreign tables instead.

17.1 Introduction to Foreign Tables

For various reasons, it may not be feasible or reasonable for you to load data directly into InterSystems IRIS. For example, perhaps a data file is extremely large and will not be queried often enough to justify the storage cost of loading it into an InterSystems IRIS table. In such cases, you can define a foreign table to extend your schema. A foreign table is a projection of data managed by another system that can be queried and accessed alongside data that is managed and stored within an instance of InterSystems IRIS.

Foreign tables are defined exclusively through SQL commands executed through your application ([Dynamic SQL](#), [Embedded SQL](#), or with a [database driver](#)), the [SQL Shell](#), or in the [Management Portal](#).

Foreign tables can be exported and imported with the `%SYSTEM.SQL.Schema.ExportDDL()` and `%SYSTEM.SQL.Schema.ImportDDL()` APIs, respectively. Unlike standard tables, you currently cannot simply import the underlying class definitions of the foreign table or server.

Note: Foreign tables do not currently support accessing [stream-typed data](#) from an external data source.

17.2 Creating a Foreign Table

Prior to creating a foreign table, you must define a foreign server to determine how InterSystems IRIS will interact with the external data source. Once you have defined a foreign server, you can then define one or more foreign tables that represent data in the external source by specifying column names and types, as well as any other details required to map fields in the external data source to columns within InterSystems IRIS.

17.2.1 Step 1: Define a Foreign Server

Before you can define a foreign table, you must define a foreign server and specify which foreign data wrapper to use. To do so, use the [CREATE FOREIGN SERVER](#) command to define and compile a foreign server.

The **CREATE FOREIGN SERVER** command requires you to specify a foreign data wrapper. Foreign data wrappers determine how InterSystems IRIS interacts with a specific type of data source. Within a **CREATE FOREIGN SERVER** command, you will need to specify both a foreign data wrapper and the metadata that the foreign data wrapper requires. At present, the InterSystems SQL supports two foreign data wrappers: CSV and JDBC. The CSV foreign data wrapper requires you to specify a path to a folder in a local file system. The JDBC foreign data wrapper requires you to name a JDBC connection to connect with an external database.

There is no limit on the amount of foreign tables you can define on a foreign server.

A user that creates a foreign server must have the %MANAGE_FOREIGN_SERVER administrative privilege.

The following example demonstrates how to create a foreign server that uses the CSV foreign data wrapper.

SQL

```
CREATE FOREIGN SERVER Sample.TestFile
  FOREIGN DATA WRAPPER CSV HOST '\path\to\file'
```

The following example demonstrates how to create a foreign server that uses the JDBC foreign data wrapper.

SQL

```
CREATE FOREIGN SERVER Sample.PostgresDB
  FOREIGN DATA WRAPPER JDBC CONNECTION 'postgresConnection'
```

17.2.2 Step 2: Define a Foreign Table

After defining and compiling a foreign server, you can use the **CREATE FOREIGN TABLE** command to define a foreign table. This table may have the same column names as the data from the external source, or you may choose to refer to the columns by new names within InterSystems IRIS. The syntax for creating a foreign table is similar to the **LOAD DATA** command.

SQL

```
CREATE FOREIGN TABLE Sample.AccountTeam (
  TeamID BIGINT,
  Name VARCHAR(50),
  CountryCode VARCHAR(10)
) SERVER Sample.PostgresDB TABLE 'Sample.Teams'
```

Creating a foreign table through a data definition language statement will create a corresponding class, which has a [ClassType](#) of “view”. You should not manually edit this class; furthermore, you must define a foreign table with the **CREATE FOREIGN TABLE** command and cannot create one by creating a class definition.

The user that creates a foreign table must have the %CREATE_TABLE administrative privilege and the USE object privilege on the foreign server that the table is created on.

17.3 Querying a Foreign Table

Querying a foreign table is exactly like querying a native table:

SQL

```
SELECT Name, CountryCode FROM Sample.AccountTeam ORDER BY Name
```

Your queries can also take advantage of more advanced syntaxes:

SQL

```
SELECT t.Name, COUNT(m.*)
FROM Sample.AccountManager m JOIN Sample.AccountTeam t
    ON m.TeamID = t.TeamID
WHERE t.CountryCode = 'UK' AND m.Salary > 100000
GROUP BY t.Name
```

Where possible, InterSystems SQL will send, or push down, certain clauses within a query to the remote database, limiting the amount of data transferred over the network and taking advantage of any remote optimizations. The clauses include simple predicates in a **WHERE** clause and many **JOINS** between foreign tables. However, there are some cases where **JOINS** are not pushed down, for example:

- When a query contains an implicit join, no **JOIN** in the corresponding **FROM** clause, including explicit joins with **ON** clauses, is pushed down.
- No **JOIN** with a **USING** clause is pushed down.
- No **NATURAL JOIN** is pushed down.
- No **CROSS JOIN** is pushed down.
- No **JOIN** with conditions in an **ON** clause that use functions supported by a [higher pushdown level](#) than the foreign server has is pushed down.
- No **JOIN** between foreign tables from different foreign servers, or involving local tables or non-database foreign tables, is pushed down.
- No **JOIN** where one of the items is a subquery is pushed down. However, a **JOIN** within a subquery may still be pushed down if the outer subquery is valid.

Some clauses, such as a **GROUP BY**, are processed within InterSystems IRIS after the external data has been retrieved and are not pushed down. [View the query plan](#) to see the statement that is sent to the remote server and see which clauses from the original query are pushed down to the remote database.

The user that issues a query on the external data source must have the %Gateway_Object:USE privilege.

Note: Issuing a query against a foreign table makes use of an underlying Java-based engine that requires a Java Virtual Machine (JVM) installation on your server. If you have a JVM set up and accessible in your PATH environment variable, then the first time you issue a query, InterSystems IRIS automatically uses that JVM to start an External Language Server. To customize your External Language Server to use a specific JVM, or to use a remote server, see Managing External Server Connections.

Important: By default, the system applies EXACT collation to all foreign tables which may cause wrong results when querying a foreign table that uses an instance of InterSystems IRIS with the default SQLUPPER collation as the external data source. Read more about collation in InterSystems SQL in [“Collation.”](#)

17.3.1 Issuing a Passthrough Query

Using the **THROUGH** command, you can send a **SELECT** query straight to the external data source for execution through a foreign server. Such a query is called a *passthrough query*. The external data source is exclusively responsible for the preparing and executing a passthrough query, which can be used to explore the data stored in an external data source on the fly, prior to creating a foreign table for more convenient access.

To issue a passthrough query, you must create a foreign server that uses a foreign data wrapper that supports passthrough queries, such as JDBC. There is no need to create a foreign table before issuing a passthrough query.

17.4 Deleting a Foreign Table

To delete a foreign table, use the **DROP FOREIGN TABLE** command:

SQL

```
DROP FOREIGN TABLE Example.MyForeignTable
```

Alternatively, you may use the **DROP FOREIGN SERVER** command with the **CASCADE** option to drop a foreign server and all the foreign tables defined on that foreign server:

SQL

```
DROP FOREIGN SERVER Example.PostgresDB CASCADE
```

The user that issues the **DROP FOREIGN SERVER** command must have the `%MANAGE_FOREIGN_SERVER` administrative privilege.

18

Partitioned Tables

With a partitioned table, you can divide data stored in a single table across multiple partitions, possibly stored in different databases, based on a logical scheme or on ranges of RowIDs. Any DML commands (**SELECT**, **INSERT**, **UPDATE**, and **DELETE**) work the same on partitioned tables as they do on any other SQL table, with only minor exceptions. As a result, partitioned tables can:

- [Improve query performance](#).
- Allow you to [move subsets of your data](#) to different tiers of storage.

18.1 Create a Partitioned Table

Use **CREATE TABLE** with the **PARTITION** option to create a partitioned table.

There are two main types of partitioned tables: *default-partitioned tables* and *key-partitioned tables*. In general, the term “partitioned table” is a catch-all term to refer to a default-partitioned table or a key-partitioned table, and these terms are used only when the distinctions between them make a difference. InterSystems recommends defining a key-partitioned table rather than a default-partitioned table in most circumstances.

A default-partitioned table merely splits table data into physical *buckets* (which transparently increase query efficiency for very large tables) according to a row’s [RowID](#). A key-partitioned table maps a logical scheme over these physical buckets to create *partitions* as specified by a *partition key*, which configures how rows are distributed among the partitions based on a partition key field and a [partition key type](#).

Your choice of partition key field and type depends on your dominant query patterns (such as querying based on a range of values for a `DateCreated` column) and the relative business value different partitions have (for example, data added over a year ago may be of lower value than more recent data). There are three distinct partition key types: `RANGE`, `LIST`, and `HASH`. You may [define a multi-field partition key](#) to define multiple, nested levels of partitions.

Specifying a field as part of a partition key causes the field to be [ReadOnly](#). Attempting to update a field that is part of the partition key returns an error.

A partitioned table must have system-assigned RowIDs.

Note: Currently, [sharded tables](#) and tables that use [columnar storage](#), VECTOR-typed fields, computed fields, or functional indexes cannot be partitioned. These limitations will be lifted in a future release. See [Known Issues and Notes](#) for more information.

18.1.1 Partition Key Types

Partition By Range

When partitioning data by range, specify the size of the interval that should separate partitions. All the partition key field values within a single partition will be within the same specific interval (for example, the same year or set of three months). The partition key field used for this partition key type must be a date type (for example, `DATE` or `TIMESTAMP`) or an exact numeric type (for example, `INTEGER` or `DOUBLE`). If the field is of a date type, you must include a unit of time (such as `YEARS` or `MONTHS`) when defining the interval size.

For example, to create a partition key on the `TransactionDate` field in intervals of three months, define a partitioned table as follows:

SQL

```
CREATE TABLE Store.Transactions (  
    CustomerName VARCHAR(50),  
    TransactionDate DATE,  
    StoreLocation VARCHAR(25)  
) PARTITION BY RANGE (TransactionDate) INTERVAL 3 MONTHS
```

The resulting table partitions any rows added into the table based on which three-month interval it belongs to.

As a second example, to create a partition key on the `CustomerID` field in intervals of 100, define a partitioned table as follows:

SQL

```
CREATE TABLE Store.Orders (  
  
    OrderID INTEGER,  
  
    CustomerID INTEGER,  
  
    OrderTotal NUMERIC(12,2)  
) PARTITION BY RANGE (CustomerID) INTERVAL 100
```

Partition By List

When partitioning data by list, each unique value in the partition key field corresponds to a unique partition. All rows stored in a given partition have the same value for the partitioned field. The partition key field used for this partition key type must employ `EXACT` [collation](#).

As the system creates a new partition for each distinct value, consider how many distinct values you expect to appear in the field. The query performance benefits of this partition key type degrade if used on a field containing many distinct values. Instead, in such situations, consider using the `HASH` partition key type.

To create a partition key for the unique values in the `StoreLocation` field, define a partitioned table as follows:

SQL

```
CREATE TABLE Store.Transactions (  
    CustomerName VARCHAR(50),  
    TransactionDate DATE,  
    StoreLocation VARCHAR(25) COLLATE %EXACT  
) PARTITION BY LIST (StoreLocation)
```

The above table partitions rows based on the value of the `StoreLocation` field.

Partition By Hash

When partitioning data by hash, define a fixed number of partitions. Rows added to the table are distributed equally across those partitions by dividing a generated hash value for the partition key field (n) by the number of partitions (m) and taking the remainder (equivalent to the operation $n \bmod m$). All rows with identical values for this modulo operation are stored in the same partition.

If the data in the field you are partitioning has many distinct values, then the data in your table is evenly distributed across the number of partitions you have defined. However, if there are only a few distinct values, you should consider using the `LIST` partition type.

The following example creates a partition key on the `TransactionGUID` field, defining a partitioned table with exactly four partitions and distributing its data evenly across the partitions:

SQL

```
CREATE TABLE Store.Transactions (
    TransactionGUID VARCHAR(128),
    TransactionDate DATE,
    StoreLocation VARCHAR(25)
) PARTITION BY HASH (TransactionGUID) IN 4 PARTITIONS
```

The above table partitions rows based on a generated hash of the value in the `TransactionGUID` field.

18.1.2 Multi-Field Partition Keys

When defining a partitioned table, you may choose to define a partition key that includes multiple fields to finely tune how your data is distributed across your partitions. To do this, simply add another partition key type clause to the end of your **CREATE TABLE** statement, separated by a comma. You can add any number of fields to a partition key, but most key-partitioned tables perform adequately with one or two, as is considered useful based on expected query patterns.

The following table defines a two-field partition key on the `Store.Transactions` table. The table is first partitioned according to the value in the `StoreLocation` field; this is the outermost partition level. Then, each `StoreLocation` partition is further divided into sub-partitions in intervals of three months according to the value in the `TransactionDate` field; this is the inner partition level. The resulting partitions contain transactions from a single store location during a certain three-month period.

SQL

```
CREATE TABLE Store.Transactions (
    CustomerName VARCHAR(50),
    TransactionDate DATE,
    StoreLocation VARCHAR(25) COLLATE %EXACT
) PARTITION BY LIST (StoreLocation), RANGE (TransactionDate) INTERVAL 3 MONTHS
```

18.1.3 Improved Query Performance Through Partition Pruning

One of the benefits of partitioned tables is their ability to improve query performance through partition pruning, which is employed on queries that use a `WHERE` clause condition on a partition key field. Each partition in a table is identified with a partition ID, which is derived from a combination of the partition key field and partition key type. The query optimizer looks at the `WHERE` clause of a query, identifies whether it involves a partition key field, and, where possible, uses the constraints of the `WHERE` clause condition to prune scans of irrelevant partition IDs from the query plan. As a result, partition pruning can dramatically shrink the amount of data that the query needs to look at in order to return the proper rows and greatly improve query performance.

Partition pruning is employed as an automatic, standard optimization. Before defining a partition key, consider how frequently you expect to employ `WHERE` clauses on certain fields. If, for example, you anticipate frequently executing queries that would benefit from optimizations of range conditions, you may decide to define a `RANGE` partition key on a field frequently

referenced in a `WHERE` clause. These planning decisions must be made before defining a partition key, as you cannot change a partition key once it is defined.

18.2 View Partition Metadata

Once you have created a partitioned table, information about your partitions, partition keys, and partition mappings is cataloged for you to view at any time through the [Catalog Details](#) in the Management Portal or the tables in the `INFORMATION_SCHEMA` schema. The `INFORMATION_SCHEMA.TABLES` table contains a column, `IS_PARTITIONED`, that you can query to find out which tables are or are not partitioned.

- `INFORMATION_SCHEMA.TABLE_PARTITIONS`: Contains information about each partition in the current namespace, including the partition ID, the schema and table that the partition belongs to, the number of rows and buckets stored, the estimated size of the data, and the database location to which the partition is mapped. This table does not catalog any information for default-partitioned tables, as they only divide data into buckets, not key-based partitions; query `INFORMATION_SCHEMA.TABLE_PARTITION_BUCKETS` instead.
- `INFORMATION_SCHEMA.TABLE_PARTITION_BUCKETS`: Contains information about each bucket in the current namespace, including the number of rows, the estimated size of the data stored in each bucket, and the database location to which the partition is mapped. Buckets are used for both default-partitioned and key-partitioned tables, so `INFORMATION_SCHEMA.TABLE_PARTITION_BUCKETS` contains information about both default-partitioned and key-partitioned tables.
- `INFORMATION_SCHEMA.TABLE_PARTITION_KEY`: Contains information about specific partition keys, including the table and schema the partition key applies to, the partition key field, and the partition key type. There is one entry in this table per partition key field; thus, if a table has a multi-field partition key, there is one row per each field in the partition key. This table includes a `PARTITION_LEVEL` column that indicates the order of nested partitions, with level 1 representing the outermost partition level.
- `INFORMATION_SCHEMA.TABLE_PARTITION_MAPPINGS`: Contains information about each partition mapping in the namespace, including the rule that defines the mapping, the table and schema, the number of rows and buckets affected by the mapping, the database location that the partition is mapped to, and the estimated size of the data. Each entry in the table corresponds with a partition that is mapped to a database that is different from the namespace's default database.

18.3 Move Partitions Between Databases

One of the crucial advantages of a partitioned table is the ability to move partitions to different databases, where they can still be queried as normal. Some benefits of this approach include separating a very large table running up against database size limits into smaller pieces stored in different databases or storing infrequently queried data on a cheaper tier of storage.

The `ALTER TABLE` command with the `MOVE PARTITION` option supports moving a partition or range of partitions to a specified location, creating mappings to the new location. This command can be used either before or after the table has been populated with data. Any data already in the specified partition or range of partitions is moved to the newly specified location. Any data subsequently added to the table is stored according to the locations specified in previous `ALTER TABLE <tablename> MOVE PARTITION` commands. The system transparently uses the low-level [DataMove](#) utility to move stored data to the new location defined by the mapping. No direct use of `DataMove` by the user is required to move partitions, but familiarity with the concepts may help with interpreting log and error messages.

Note: Moving partitions between databases with `DataMove` may result in significant additional journaling.

Prerequisites for Moving Partitions

1. Plan out the databases you wish to use. You can use preexisting databases to store partitions, and any database used for table partitioning can be used for other purposes as well. However, depending on performance and space issues for your application or configuration, you may prefer to define new databases. You may define such additional databases with the [CREATE DATABASE FILE command](#), [through the Management Portal](#), or with the [^DATABASE utility](#).
2. Configure access to these databases by granting appropriate users access to the corresponding [database resources](#).
3. Make sure that “Journal freeze on error” is enabled. From the Management Portal, navigate to System Administration > Configuration > System Configuration > Journal Settings. Then, verify that the “Freeze on error” checkbox is selected.

The partition or range of partitions to be moved can be specified by partition ID or by range of partition key field values or partition IDs.

Move Partitions by Range of Partition Key Field Values

The easiest way to move partitions is by specifying values for the partition key fields, allowing the system to derive the corresponding partitions. Identify the range of values that you would like to move to a different database by using **ALTER TABLE** with the **MOVE PARTITION BETWEEN** option. The range you provide must be of the type of the partition key field and is inclusive of both the starting and the ending value. You may not specify a range of values for a table partitioned with a HASH-typed partition key.

For example, on a table that is key-partitioned on a date field, the following command identifies all partitions from the year 2023 and moves them (along with their data) to a database named `data-2023`.

SQL

```
ALTER TABLE demo.log MOVE PARTITION BETWEEN '2023-01-01' AND '2023-12-31' TO data-2023
```

A range moves all partitions that it encompasses. Additionally, partitions are always moved in full, so it is not necessary to identify the exact endpoints of your range. If your data is partitioned, for example, in intervals of one year, the ranges **BETWEEN '2025-01-01' AND '2025-12-31'** (which identifies the entire calendar year) and **BETWEEN '2025-01-01' AND '2025-01-02'** (which only identifies a span of two days within the same year) both move all the data for the year 2025 to the specified database. On the same table, the range **BETWEEN '2025-01-01' AND '2027-01-01'** moves the partitions for the years 2025, 2026, and 2027 to the specified database.

Move Partitions by a Partition ID or a Range of Partition IDs

Partitions are assigned a partition ID implicitly, based on their partition key, which can be used as an alternative means of moving individual partitions. There are multiple ways to determine the partition IDs you would like to move:

- On tables with a **LIST** partition key type, the partition key field value is identical to the partition ID, so you can specify which partition to move by using the field value.
- Issue a query on the `x__partitionid1` column with a **WHERE** clause that identifies the range of partition key field values to be moved.

Note: On a table with a multi-field partition key, `x__partitionid1` indicates the outermost partition ID. The inner partition ID fields are stored in `x__partitionid2`, `x__partitionid3`, and so on, with each higher number indicating another inner layer of partitioning.

For example, the following query would return the distinct partition IDs of rows that have a `TransactionDate` in the years 2023, 2024, or 2025:

SQL

```
SELECT DISTINCT(x__partitionid1) from Demo.Inventory
WHERE TransactionDate >= TO_DATE('01 JAN 2023')
AND TransactionDate <= TO_DATE('01 JAN 2026')
```

- [View your partition schema](#) by querying `INFORMATION_SCHEMA.TABLE_PARTITIONS` or look in the [Catalog Details](#) and identify possible partition IDs.

Once you have identified the partition ID you would like to move, use **ALTER TABLE** with the `MOVE PARTITION ID` option to move a partition ID or a range of partitions IDs to the specified database. For example, the following command moves the partition with ID `202411010000` to the database named `data-2024`.

SQL

```
ALTER TABLE Demo.Inventory MOVE PARTITION ID 202411010000 TO data-2024
```

As a second example, the following command, for a table with a `LIST` partition key, moves the partition with the value `'alabama'` in its partition key field to the `us-south` database.

SQL

```
ALTER TABLE Demo.Inventory MOVE PARTITION ID 'alabama' TO us-south
```

You may also specify a range of partition IDs to move. Each partition within the range, including the two values specified, is moved to the specified database. For example, the following command moves all partitions with a partition ID between `202411010000` and `202506010000` to a database named `prerelease`.

SQL

```
ALTER TABLE Demo.Inventory MOVE PARTITION ID BETWEEN 202411010000 AND 202506010000 TO prerelease
```

On both default-partitioned and key-partitioned tables, you can move specific buckets between different databases by identifying the bucket ID, which is an integer. To do so, use `<partition id>||<bucket id>`. For example, the following command moves the bucket with bucket ID `3` within the partition with partition ID `202411010000` to the `prerelease` database.

SQL

```
ALTER TABLE Demo.Inventory MOVE PARTITION ID '202411010000||3' TO prerelease
```

You may also move a range of bucket IDs from a single partition to a new database. For example, the following example moves buckets with a bucket ID between `3` and `5` from the partition with partition ID `202610140000` to the `test-db` database.

SQL

```
ALTER TABLE Demo.Inventory MOVE PARTITION ID 202610140000 BETWEEN 3 AND 5 TO test-db
```

Move Inner Partitions on a Table with a Multi-Field Partition Key

You can move the inner partitions of a table with a multi-field partition key to a different database without moving the outer partitions. To do so, identify the partition ID for the inner partition you would like to move, which comes in the form `<outer partition ID>||<inner partition ID>`. The following query returns the correctly formatted multi-field partition keys for each row in the `Demo.Inventory` table.

SQL

```
SELECT x__partitionid1 || '||' || x__partitionid2 FROM Demo.Inventory
```

On a table that has an outer partition of type **LIST** on a string-typed field and an inner partition of type **RANGE** on a date-typed field, the following command moves the inner partition with partition ID 'Massachusetts||202601010000' to the test-db database.

SQL

```
ALTER TABLE Demo.Inventory MOVE PARTITION ID 'Massachusetts||202601010000' TO test-db
```

You may also move a range of inner partition IDs to a different database. To do so, specify the outer partition ID and then use **BETWEEN** to identify the range of inner partition IDs you would like to move. For example, on a table that has an outer partition of type **RANGE** on a date-typed field and an inner partition of type **HASH**, the following command moves the inner partitions with partition IDs between 5 and 8 to the test-db database.

SQL

```
ALTER TABLE Demo.Inventory MOVE PARTITION ID 202504030000 BETWEEN 5 AND 8 TO test-db
```

18.4 Partitioned Table Conversions

You may convert a table between a partitioned and nonpartitioned format using DDL. The **ALTER TABLE** command supports options for turning partitioning on or off and for defining partition keys. You may continue querying the table and using **INSERT**, **UPDATE**, and **DELETE** commands to change its data while the table conversion command is running.

A table cannot be converted between a partitioned and nonpartitioned state if it is defined with **USEEXTENTSET=0**.

Important: When you convert a nonpartitioned or default-partitioned table to a key-partitioned table, the conversion process changes the RowIDs in your table. To enable [foreign keys](#) and references from other tables to continue working, the system creates a [surrogate ID](#) field (to preserve the old RowID values) and a surrogate ID index (to translate the old RowIDs to the new IDs). This can cause query and update performance to be slower than if the same table was originally created with the same partition key.

Convert a Nonpartitioned Table to a Default-Partitioned Table

To convert a nonpartitioned table to a default-partitioned table, use **ALTER TABLE** with the **CONVERT PARTITION ON** option. Turning partitioning on for a table does not define a partition key, but rather stores the rows of the table in buckets based on ranges of RowIDs. The example below demonstrates the use of this option.

SQL

```
ALTER TABLE Store.Transactions CONVERT PARTITION ON
```

You can subsequently use **ALTER TABLE** with the **CONVERT PARTITION BY** option to define a partition key; however, InterSystems strongly recommends performing such a conversion in a single step.

Note: A nonpartitioned table cannot be converted to a partitioned table if it is sharded, uses column-based storage, does not have a system assigned [IDKey](#), or has **VECTOR**-typed fields, computed fields, or functional indexes.

Convert a Nonpartitioned or Default-Partitioned Table to a Key-Partitioned Table

To convert a nonpartitioned or default-partitioned table to a key-partitioned table, use **ALTER TABLE** with the **CONVERT PARTITION BY** option. However, if you have already defined a partition key on a table, you cannot change the partition key, convert it to a multi-field partition key, or convert the table to a default-partitioned table.

SQL

```
ALTER TABLE Store.Transactions CONVERT  
PARTITION BY RANGE (TransactionDate) INTERVAL 3 MONTHS
```

Convert a Partitioned Table to a Nonpartitioned Table

To convert a partitioned table to a nonpartitioned table, use **ALTER TABLE** with the **CONVERT PARTITION OFF** option to turn off the partitioning mechanism. Performing such a conversion moves the data back to the default globals database for the namespace, removes all storage mappings, and removes the partition key. The example below demonstrates the use of this option.

SQL

```
ALTER TABLE Store.Transactions CONVERT PARTITION OFF
```

18.4.1 Surrogate IDs

To maintain [RowID](#) consistency and support partitioned table conversions, converting to or from a key-partitioned table results in the creation of a surrogate ID field and a surrogate ID index.

When a table is partitioned with a partition key, the RowIDs in the table are changed from their initial form (a default, system-assigned integer) to a string identifying the partition containing the row and a unique identifier within the partition. To ensure that previously defined foreign keys or other references to this table from other tables still work, the system stores the preconversion RowID value as a *surrogate ID*, and creates a surrogate ID index that maps the surrogate ID values to the new RowIDs, ensuring referential integrity among all entities in your schema.

There is a performance cost associated with querying and maintaining the surrogate ID field and surrogate ID index. Queries that explicitly refer to the RowID must perform a conversion between the surrogate ID and the current RowID before returning results, and so perform more slowly after conversion. Inserting data into a table with a surrogate ID index also performs slower after conversion, because of the need to update the index. If you are certain that there are no foreign keys or references from other tables referring to a table with a surrogate ID and surrogate ID index, you can drop the surrogate ID and its index after the conversion is completed to avoid the performance costs.

Note: A nonpartitioned table that has a surrogate ID (which only occurs when a key-partitioned table is converted to a nonpartitioned table) cannot be converted to a key-partitioned table unless you first drop the surrogate ID.

18.4.2 Limitations of Partitioned Table Conversions

Due to the structure of surrogate IDs, some conversions between partitioned and nonpartitioned tables are not possible. These restrictions are:

- You cannot convert a nonpartitioned table to a key-partitioned table if the table has an auto-increment column.
- You cannot convert a key-partitioned table to a default-partitioned table.
- You cannot convert a nonpartitioned table to a key-partitioned table if the table already has a surrogate ID field and surrogate ID index. This path only arises as a *reconversion*, where the nonpartitioned table was originally a key-partitioned table. To perform this reconversion, first verify that there are no foreign keys or references to the old RowIDs from other tables that must be preserved. Then, drop the surrogate ID field and surrogate ID index and you will be able to perform the conversion.

Important: If there are foreign keys or references from other tables referring to this table, they will no longer work after dropping the surrogate ID field and surrogate ID index.

18.5 Drop or Truncate a Partition

You may drop or truncate a single partition from a partitioned table using **ALTER TABLE**, either by individual ID or by range of either IDs or partition key value field values.

Dropping a partition deletes the data in the partition and deletes any mappings.

Truncating a partition deletes the data in the partition, keeping intact any mappings to a non-default database you created when [moving the partition](#) so that any data subsequently inserted into the partition is stored in the same non-default database.

Drop or Truncate Partition by Range of Partition Key Field Values

SQL

```
ALTER TABLE Demo.Inventory DROP PARTITION BETWEEN '2023-01-01' AND '2023-12-31'
```

SQL

```
ALTER TABLE Demo.Inventory TRUNCATE PARTITION BETWEEN '2023-01-01' AND '2023-12-31'
```

Drop or Truncate Partition by Individual ID

For a standard key-partitioned table:

SQL

```
ALTER TABLE Demo.Inventory DROP PARTITION ID 202301010000
```

SQL

```
ALTER TABLE Demo.Inventory TRUNCATE PARTITION ID 202301010000
```

For a multi-field key-partitioned table:

SQL

```
ALTER TABLE Demo.Inventory DROP PARTITION ID '202301010000||4'
```

SQL

```
ALTER TABLE Demo.Inventory TRUNCATE PARTITION ID '202301010000||4'
```

Drop or Truncate Partition by Range of Partition IDs

SQL

```
ALTER TABLE Demo.Inventory DROP PARTITION ID BETWEEN '202301010000' AND '202312310000'
```

SQL

```
ALTER TABLE Demo.Inventory TRUNCATE PARTITION ID BETWEEN '202301010000' AND '202312310000'
```

To drop an entire partitioned table, use the **DROP TABLE** command. Dropping a partitioned table deletes the data in the partition and any mappings.

To truncate an entire partitioned table, use the **TRUNCATE TABLE** command. Truncating a partitioned table deletes the data in the partition, but keeps all mappings intact.

19

Modifying the Database

You can use either SQL statements against an existing table or ObjectScript operations on the corresponding persistent class to modify the contents of an InterSystems IRIS® data platform database. You cannot modify a persistent class (table) that is defined as *READONLY*.

Using SQL commands provides automatic support for maintaining the integrity of the data. An SQL command is an atomic (all or nothing) operation. If there are indexes defined on the table, SQL will automatically update them to reflect the changes. If there are any data or referential integrity constraints defined, SQL will automatically enforce them. If there are any defined triggers, performing these actions will pull the corresponding [insert](#), [update](#), or [delete trigger](#).

19.1 Inserting Data

You can insert data into a table either by using the SQL statements or by setting and saving persistent class properties.

19.1.1 Insert Data Using SQL

The [INSERT](#) statement inserts a new record into an SQL table. You can insert a single record or multiple records.

The following example inserts a single record. It is one of several available syntax forms to insert a single record:

SQL

```
INSERT INTO MyApp.Person  
  (Name, HairColor)  
VALUES ('Fred Rogers', 'Black')
```

The following example inserts multiple records by querying data from an existing table:

SQL

```
INSERT INTO MyApp.Person  
  (Name, HairColor)  
SELECT Name, Haircolor FROM Sample.Person WHERE Haircolor IS NOT NULL
```

For more details on how to insert various types of data using the **INSERT** command, see the [INSERT](#) command reference page.

You can also issue an [INSERT OR UPDATE](#) statement. This statement inserts a new record into an SQL table if the record does not already exist. If the record exists, this statement updates the record data with the supplied field values.

19.1.2 Insert Data Using Object Properties

You can use ObjectScript to insert one or more records of data. Create an instance of an existing persistent class, set one or more property values, then use %Save() to insert the data record:

The following example inserts a single record:

ObjectScript

```
SET oref=##class(MyApp.Person).%New()  
SET oref.Name="Fred Rogers"  
SET oref.HairColor="Black"  
DO oref.%Save()
```

The following example inserts multiple records:

ObjectScript

```
SET nom=$LISTBUILD("Fred Rogers","Fred Astore","Fred Flintstone")  
SET hair=$LISTBUILD("Black","Light Brown","Dark Brown")  
FOR i=1:1:$LISTLENGTH(nom) {  
    SET oref=##class(MyApp.Person).%New()  
    SET oref.Name=$LIST(nom,i)  
    SET oref.HairColor=$LIST(hair,i)  
    SET status = oref.%Save() }  
}
```

19.2 UPDATE Statements

The [UPDATE](#) statement modifies values in one or more existing records within an SQL table:

SQL

```
UPDATE MyApp.Person  
SET HairColor = 'Red'  
WHERE Name %STARTSWITH 'Fred'
```

19.3 Computed Field Values on INSERT or UPDATE

When you define a [computed field](#), you can specify code to compute a data value for that field. This data value can be computed when the row is inserted, updated, both inserted and updated, or when queried. The following table shows the keywords required for each type of compute operation and a field/property definition example:

Compute Type	DDL SQL Keywords to Specify	Persistent Class Keywords to Specify
On INSERT only	COMPUTECODE Birthday VARCHAR(50) COMPUTECODE {SET {Birthday}=\$PIECE(\$ZDATE({DOB},9),"")_"} changed: "\$_SZTIMESTAMP }	SqlComputeCode and SqlComputed Property Birthday As %String(MAXLEN = 50) [SqlComputeCode = {SET {Birthday}=\$PIECE(\$ZDATE({DOB},9),"")_"} changed: "\$_SZTIMESTAMP"}, SqlComputed];
On UPDATE only	DEFAULT, COMPUTECODE, and COMPUTEONCHANGE Birthday VARCHAR(50) DEFAULT ' ' COMPUTECODE {SET {Birthday}=\$PIECE(\$ZDATE({DOB},9),"")_"} changed: "\$_SZTIMESTAMP } COMPUTEONCHANGE (DOB)	n/a
On both INSERT and UPDATE	COMPUTECODE and COMPUTEONCHANGE Birthday VARCHAR(50) COMPUTECODE {SET {Birthday}=\$PIECE(\$ZDATE({DOB},9),"")_"} changed: "\$_SZTIMESTAMP } COMPUTEONCHANGE (DOB)	SqlComputeCode , SqlComputed , and SqlComputeOnChange Property Birthday As %String(MAXLEN = 50) [SqlComputeCode = {SET {Birthday}=\$PIECE(\$ZDATE({DOB},9),"")_"} changed: "\$_SZTIMESTAMP"}, SqlComputed, SqlComputeOnChange = DOB];
On query	COMPUTECODE and CALCULATED or TRANSIENT Birthday VARCHAR(50) COMPUTECODE {SET {Birthday}=\$PIECE(\$ZDATE({DOB},9),"")_"} changed: "\$_SZTIMESTAMP } CALCULATED	SqlComputeCode , SqlComputed , and Calculated or Transient Property Birthday As %String(MAXLEN = 50) [SqlComputeCode = {SET {Birthday}=\$PIECE(\$ZDATE({DOB},9),"")_"} changed: "\$_SZTIMESTAMP"}, SqlComputed, Calculated];

The DDL [DEFAULT](#) keyword takes precedence over computing a data value upon insert. DEFAULT must take a data value, such as an empty string; it cannot be NULL. In a persistent class definition, the InitialExpression property keyword does not override an SqlComputed data value upon insert.

The DDL [COMPUTEONCHANGE](#) keyword can take a single field name or a comma-separated list of field names. These field names specify the fields which when updated will trigger the compute of this field; the listed field names must exist in the table, but they do not have to appear in the compute code. You must specify actual field names; you cannot specify asterisk syntax.

You can use the [ON UPDATE](#) keyword phrase to set a field to a literal or a system variable (such as the current timestamp) when a record is modified, rather than using COMPUTECODE and COMPUTEONCHANGE. The ON UPDATE phrase modifies on both INSERT and UPDATE; to modify only on UPDATE, use the DEFAULT phrase and the ON UPDATE phrase.

The DDL [CALCULATED](#) or [TRANSIENT](#) keyword computes a data value each time the field is accessed by a query. The field does not need to be specified in the select list. For example, `SELECT Name FROM MyTable WHERE LENGTH(Birthday)=36` computes the Birthday field before evaluating the condition expression. The Management Portal [Open Table](#) option performs a query, and therefore computes CALCULATED and TRANSIENT data values.

Computed field restrictions:

- UPDATE that doesn't update: An UPDATE that supplies the same values to the fields in a record as their prior values does not actually update the record. COMPUTEONCHANGE is not invoked if no real update is performed for a record. ON UPDATE is invoked on an update operation even if no real update is performed for a record. If you wish to always recalculate a computed field upon update regardless of whether the record was actually updated, use an [update trigger](#).
- User-specified explicit value for a computed field:
 - INSERT: On INSERT you can always supply an explicit value to a COMPUTEONCHANGE, DEFAULT, or ON UPDATE field. InterSystems SQL always takes the explicit value rather than the generated value.
 - UPDATE COMPUTEONCHANGE: An UPDATE operation can supply an explicit value to a COMPUTEONCHANGE field. InterSystems SQL always takes the explicit value rather than the computed value.
 - UPDATE ON UPDATE: An UPDATE operation cannot supply an explicit value to an ON UPDATE field. InterSystems SQL ignores the user-supplied value and takes the ON UPDATE generated value. However, InterSystems SQL does perform field validation on the explicit value and can, for example, generate an SQLCODE -104 error if the supplied value is longer than the maximum data size.
 - CALCULATED or TRANSIENT: An INSERT or UPDATE operation cannot supply an explicit value to a CALCULATED or TRANSIENT field, because a CALCULATED or TRANSIENT field does not store data. However, InterSystems SQL does perform field validation on the explicit value and can, for example, generate an SQLCODE -104 error if the supplied value is longer than the maximum data size.

19.4 Validating Data

The insert and update operations described on this page automatically perform data validation. They prevent invalid data being stored in a table. Data that is stored in a table by other means may not be validated. You can use the **\$SYSTEM.SQL.Schema.ValidateTable()** method to validate the data in a table. The table name can be qualified ("`schema . table`"), or unqualified ("`table`"). An unqualified table name takes the [default schema name](#); schema search path values are not used.

ValidateTable() returns a result set containing a row for each validation issue found in the table's data. This method performs the following data validations:

- Validates each data value against the field's data type, using the data type **IsValid()** method.
- Validates that any field with a Required constraint does not have a null value.
- Validates that any field with a Unique constraint does not have a duplicate value.
- Validates that any Foreign Key field references a valid row in the referenced table.

The validation result set is held in the %sqlcontext object. DO %sqlcontext.%Display() will display the data validation results to the current device, as shown in the following Terminal example:

```
USER>DO $SYSTEM.SQL.Schema.ValidateTable("Sample.MyTable")
USER>DO %sqlcontext.%Display()

Dumping result #1
Row(s) With an Issue      Field Or Constraint Name      Error
%ID 3   Home_City          Cannot be null
%ID 14  Home_City          Cannot be null
%ID 10  Home_City          Cannot be null
%ID 13  Home_City          Cannot be null
%ID 6   Home_PostalCode      Value is invalid: BadZip
%ID 8   Home_PostalCode      Value is invalid: WhoKnows
%ID 9   Home_PostalCode      Value is invalid: BadZip
%ID 10  Home_PostalCode      Value is invalid: WhoKnows
%ID 11  Home_PostalCode      Value is invalid: BadZip
%ID 9   Home_State          Cannot be null
%ID 3   Home_State          Cannot be null
%ID 10  Home_State          Cannot be null

12 Rows(s) Affected
USER>
```

Fields are listed in alphabetical order. In this example, Home_City and Home_State fields failed required value validation; Home_PostalCode (%Integer data type) fields failed data type validation.

You can also invoke this data validation operation from SQL by calling the ValidateTable stored procedure, as shown in the following example:

SQL

```
CALL %SYSTEM.ValidateTable('Sample.MyTable')
```

If the table is [sharded](#), **ValidateTable()** should be called on the shard master table.

ValidateTable() does not perform locking. Therefore, if it is run against a table on a live system with concurrent operations you could receive false-positive error reports.

19.5 DELETE Statements

The **DELETE** statement removes one or more existing records from an SQL table:

SQL

```
DELETE FROM MyApp.Person
WHERE HairColor = 'Aqua'
```

You can issue a **TRUNCATE TABLE** command to delete all records in a table. You can also delete all records in a table using **DELETE**. **DELETE** (by default) pulls delete triggers; **TRUNCATE TABLE** does not pull delete triggers. Using **DELETE** to delete all records does not reset table counters; **TRUNCATE TABLE** resets these counters.

19.6 Transaction Processing

A transaction is a series of **INSERT**, **UPDATE**, **DELETE**, **INSERT OR UPDATE**, and **TRUNCATE TABLE** data modification statements that comprise a single unit of work.

The **SET TRANSACTION** command can be used to set the transaction parameters for the current process. The same parameters can also be set using the **START TRANSACTION** command. These transaction parameters continue in effect across multiple transactions until explicitly changed.

A **START TRANSACTION** command explicitly starts a transaction. This command is generally optional; if transaction **%COMMITMODE** is either **IMPLICIT** or **EXPLICIT**, a transaction begins automatically with the first database modification operation. If transaction **%COMMITMODE** is **NONE**, you must explicitly specify **START TRANSACTION** to initiate transaction processing.

If a transaction succeeds, committing its changes can be implicit (automatic) or explicit; the **%COMMITMODE** value determines whether you need to explicitly use the **COMMIT** statement to permanently add the data modifications to the database and release resources.

If a transaction fails, you can use the **ROLLBACK** statement to undo its data modifications so that these do not go into the database.

Note: SQL transaction statements are *not* supported when running SQL through the Management Portal **Execute SQL Query** interface. This interface is intended as a test environment for developing SQL code, not for modifying actual data.

19.6.1 Transactions and Savepoints

In InterSystems SQL, you can perform two kinds of transaction processing: full transaction processing and transaction processing using savepoints. With full transaction processing, a transaction begins with **START TRANSACTION** statement (explicit or implicit) and continues until either a **COMMIT** statement (explicit or implicit) concludes the transaction and commits all work, or a **ROLLBACK** statement reverses all work done during the transaction.

With savepoints, InterSystems SQL supports levels within a transaction. You begin a transaction with a **START TRANSACTION** statement (explicit or implicit). Then during the transaction you use **SAVEPOINT** to specify one or more named savepoints within the program. You can specify a maximum of 255 named savepoints in a transaction. Adding a savepoint increments the **\$TLEVEL** transaction level counter.

- A **COMMIT** commits all work performed during the transaction. Savepoints are ignored.
- A **ROLLBACK** rolls back all work performed during the transaction. Savepoints are ignored.
- A **ROLLBACK TO SAVEPOINT pointname** rolls back all work performed since the **SAVEPOINT** specified by *pointname* and decrements an internal transaction level counter by the appropriate number of savepoint levels. For example, if you established two savepoints, svpt1 and svpt2, and then rolled back to svpt1, the **ROLLBACK TO SAVEPOINT svpt1** reverse the work done since svpt1 and, in this case, decrements the transaction level counter by 2.

19.6.2 Non-transaction Operations

While a transaction is in effect, the following operations are not included in the transaction and therefore cannot be rolled back:

- The IDKey counter increment is not a transaction operation. The IDKey is automatically generated by **\$INCREMENT** (or **\$SEQUENCE**), which maintains a count independent of the SQL transaction. For example, if you insert records with IDKeys of 17, 18, and 19, then rollback this insert, the next record to be inserted will have an IdKey of 20.
- Cached query creation, modification, and purging are not transaction operations. Therefore, if a cached query is purged during a transaction, and that transaction is then rolled back, the cached query will remain purged (will not be restored) following the rollback operation.

- A DDL operation, a [COLLECT STATISTICS](#) operation, or a [TUNE TABLE](#) operation that occurs within a transaction may create and run a temporary routine. This temporary routine is treated the same as a cached query. That is, the creation, compilation, and deletion of a temporary routine are not treated as part of the transaction. The execution of the temporary routine is considered part of the transaction.

For non-SQL items rolled back or not rolled back, refer to the ObjectScript [TROLLBACK](#) command.

19.6.3 Transaction Locks

A transaction uses locks to safeguard unique data values. For example, if a process deletes a unique data value, this value is locked for the duration of the transaction. Therefore, another process could not insert a record using this same unique data value until the first transaction completed. This prevents a rollback resulting in a duplicate value for a field with a uniqueness constraint. These locks are automatically applied by the **INSERT**, **UPDATE**, **INSERT OR UPDATE**, and **DELETE** statements, unless the statement includes a **%NOLOCK** restriction argument.

19.6.4 Transaction Size Limitations

There is no limitation on the number of operations you can specify in a transaction, other than space availability for journal files. The size of the lock table does not normally impose a limit, because InterSystems IRIS provides automatic lock escalation.

There is a default lock threshold of 1000 locks per table. A table can have 1000 unique data value locks for the current transaction. The 1001st lock operation escalates the locking for that table to a table lock for the duration of the transaction.

This lock threshold value is configurable using either of the following:

- Invoke the `$SYSTEM.SQL.Util.SetOption("LockThreshold")` method. This method changes both the current system-wide value and the configuration file setting. To determine the current lock escalation threshold, use the `$SYSTEM.SQL.Util.GetOption("LockThreshold")` method.
- Go to the Management Portal. From **System Administration**, select **Configuration**, then **SQL and Object Settings**, then **SQL**. On this screen you can view and edit the current setting of **Lock Threshold**.

There is no limit on the number of subnodes (child tables) that can be killed. All subnode kills are journaled, and thus can be rolled back.

19.6.5 Reading Uncommitted Data

You can specify the read isolation level by setting [SET TRANSACTION](#) or [START TRANSACTION](#) for the process issuing the query.

- **ISOLATION LEVEL READ UNCOMMITTED:** Uncommitted inserts, updates, and deletes to data are visible for query (read only) access by other users. This is the default if no transaction is specified.
- **ISOLATION LEVEL READ VERIFIED:** Uncommitted inserts, updates, and deletes to data are visible for query (read only) access by other users. Provides re-checking of data used by query conditions and displayed by the query.
- **ISOLATION LEVEL READ COMMITTED:** Changes made to the data by uncommitted inserts and updates are not shown in the query result set. The query result set only contains inserts and updates that have been committed. However, changes made to the data by uncommitted deletes are shown in the query result set.

The following [SELECT](#) command clauses always return uncommitted data, regardless of the current isolation level: an aggregate function, a **DISTINCT** clause, a **GROUP BY** clause, or a **SELECT** with the **%NOLOCK** keyword. For further details, refer to [Isolation Level](#).

19.6.6 ObjectScript Transaction Commands

ObjectScript and SQL transaction commands are fully compatible and interchangeable, with the following exception:

ObjectScript **TSTART** and SQL **START TRANSACTION** both start a transaction if no transaction is current. However, **START TRANSACTION** does not support nested transactions. Therefore, if you need (or may need) nested transactions, it is preferable to start the transaction with **TSTART**. If you need compatibility with the SQL standard, use **START TRANSACTION**.

ObjectScript transaction processing provides limited support for nested transactions. SQL transaction processing supplies support for savepoints within transactions.

20

Querying the Database

This page discusses how to query data on InterSystems IRIS® data platform.

20.1 Types of Queries

A query is a statement which performs data retrieval and generates a result set. A query can consist of any of the following:

- A simple **SELECT** statement that accesses the data in a specified table or view.
- A **SELECT** statement with **JOIN** syntax that accesses the data from several tables or views.
- A **UNION** statement that combines the results of multiple **SELECT** statements.
- A subquery that uses a **SELECT** statement to supply a single data item to an enclosing **SELECT** query.
- In Embedded SQL, a **SELECT** statement that uses an SQL cursor to access multiple rows of data using a **FETCH** statement.

20.2 Using a SELECT Statement

A **SELECT** statement selects one or more rows of data from one or more tables or views. A simple **SELECT** is shown in the following example:

SQL

```
SELECT Name,DOB FROM Sample.Person WHERE Name %STARTSWITH 'A' ORDER BY DOB
```

In this example, Name and DOB are columns (data fields) in the Sample.Person table.

The order that clauses must be specified in a **SELECT** statement is: **SELECT** **DISTINCT** **TOP** ... *selectItems* **INTO** ... **FROM** ... **WHERE** ... **GROUP BY** ... **HAVING** ... **ORDER BY**. This is the command syntax order. All of these clauses are optional, except **SELECT** *selectItems*. (The optional **FROM** clause is required to perform any operations on stored data, and therefore is almost always required in a query.) Refer to the **SELECT** statement syntax for details on the required order for specifying **SELECT** clauses.

20.2.1 SELECT Clause Order of Execution

The operation of a **SELECT** statement can be understood by noting its semantic processing order (which is *not* the same as the **SELECT** syntax order). The clauses of a **SELECT** are processed in the following order:

1. **FROM clause** — specifies a table, a view, multiple tables or views using **JOIN** syntax, or a subquery.
2. **WHERE clause** — restricts what data is selected using various criteria.
3. **GROUP BY clause** — organizes the selected data into subsets with matching values; only one record is returned for each value.
4. **HAVING clause** — restricts what data is selected from groups using various criteria.
5. **selectItem** — selects a data fields from the specified table or view. A *selectItem* can also be an expression which may or may not reference a specific data field.
6. **DISTINCT clause** — applied to the **SELECT** result set, it limits the rows returned to those that contain a distinct (non-duplicate) value.
7. **ORDER BY clause** — applied to the **SELECT** result set, it sorts the rows returned in collation order by the specified field(s).

This semantic order shows that a table alias (which is defined in the **FROM** clause) can be recognized by all clauses, but a column alias (which is defined in the **SELECT** *selectItems*) can only be recognized by the **ORDER BY** clause.

To use a column alias in other **SELECT** clauses you can use a subquery, as shown in the following example:

SQL

```
SELECT Interns FROM  
  (SELECT Name AS Interns FROM Sample.Employee WHERE Age<21)  
WHERE Interns %STARTSWITH 'A'
```

In this example, Name and Age are columns (data fields) in the Sample.Person table, and Interns is a column alias for Name.

20.2.2 Selecting Fields

When you issue a **SELECT**, InterSystems SQL attempts to match each specified *selectItem* field name to a property defined in the class corresponding to the specified table. Each class property has both a property name and a **SqlFieldName**. If you defined the table using SQL, the field name specified in the **CREATE TABLE** command is the **SqlFieldName**, and InterSystems IRIS generated the property name from the **SqlFieldName**.

Field names, class property names, and **SqlFieldName** names have different naming conventions:

- Field names in a **SELECT** statement are not case-sensitive. **SqlFieldName** names and property names are case-sensitive.
- Field names in a **SELECT** statement and **SqlFieldName** names can contain certain non-alphanumeric characters following **identifier** naming conventions. Property names can only contain alphanumeric characters. When generating a property name, InterSystems IRIS strips out non-alphanumeric characters. InterSystems IRIS may have to append a character to create a unique property name.

The translation between these three names for a field determine several aspects of query behavior. You can specify a *selectItem* field name using any combination of letter case and InterSystems SQL will identify the appropriate corresponding property. The data column header name in the result set display is the **SqlFieldName**, not the field name specified in the *selectItem*. This is why the letter case of the data column header may differ from the *selectItem* field name.

You can specify a [column alias](#) for a *selectItem* field. A column alias can be in any mix of letter case, and can contain non-alphanumeric characters, following [identifier](#) naming conventions. A column alias can be referenced using any combination of letter case (for example, in the ORDER BY clause) and InterSystems SQL resolves to the letter case specified in the *selectItem* field. InterSystems IRIS always attempts to match to the list of column aliases before attempting to match to the list of properties corresponding to defined fields. If you have defined a column alias, the data column header name in the result set display is the column alias in the specified letter case, not the `SqlFieldName`.

When a **SELECT** query completes successfully, InterSystems SQL generates a result set class for that query. The result set class contains a property corresponding to each selected field. If a **SELECT** query contains duplicate field names, the system generates unique property names for each instance of the field in the query by appending a character. For this reason, you cannot include more than 36 instances of the same field in a query.

The generated result set class for a query also contains properties for column aliases. To avoid the performance cost of letter case resolution, you should use the same letter case when referencing a column alias as the letter case used when specifying the column alias in the **SELECT** statement.

In addition to user-specified column aliases, InterSystems SQL also automatically generates up to three aliases for each field name, aliases which correspond to common letter case variants of the field name. These generated aliases are invisible to the user. They are provided for performance reasons, because accessing a property through an alias is faster than resolving letter case through letter case translation. For example, if **SELECT** specifies `FAMILYNAME` and the corresponding property is `familyname`, InterSystems SQL resolves letter case using a generated alias (`FAMILYNAME AS familyname`). However, if **SELECT** specifies `fAmILyNaMe` and the corresponding property is `familyname`, InterSystems SQL must resolve letter case using the slower letter case translation process.

A *selectItem* item can also be an expression, an [aggregate function](#), a subquery, a [user-defined function](#), an asterisk, or some other value. For further details on *selectItem* items other than field names, refer to the [selectItem](#) argument of the **SELECT** command reference page.

20.2.3 The JOIN Operation

A [JOIN](#) provides a way to link data in one table with data in another table and are frequently used in defining reports and queries. Within SQL, a **JOIN** is an operation that combines data from two tables to produce a third, subject to a restrictive condition. Every row of the resulting table must satisfy the restrictive condition.

InterSystems SQL supports five types of joins (some with multiple syntactic forms): CROSS JOIN, INNER JOIN, LEFT OUTER JOIN, RIGHT OUTER JOIN, and FULL OUTER JOIN. Outer joins support the ON clause with a full range of conditional expression predicates and logical operators. There is partial support for NATURAL outer joins and outer joins with a USING clause. For definitions of these join types and further details, see [JOIN](#).

If a query contains a join, all of the field references within that query must have an appended [table alias](#). Because InterSystems IRIS does not include the table alias in the data column header name, you may wish to provide [column aliases](#) for *selectItem* fields to clarify which table is the source of the data.

The following example uses a join operation to match the “fake” (randomly-assigned) zip codes in `Sample.Person` with the real zip codes and city names in `Sample.USZipCode`. A WHERE clause is provided because `USZipCode` does not include all possible 5-digit zip codes:

SQL

```
SELECT P.Home_City,P.Home_Zip AS FakeZip,Z.ZipCode,Z.City AS ZipCity,Z.State
FROM Sample.Person AS P LEFT OUTER JOIN Sample.USZipCode AS Z
ON P.Home_Zip=Z.ZipCode
WHERE Z.ZipCode IS NOT NULL
ORDER BY P.Home_City
```

20.2.4 Queries Selecting Large Numbers of Fields

A query cannot select more than 1,000 *selectItem* fields.

A query selecting more than 150 *selectItem* fields may have the following performance consideration. InterSystems IRIS automatically generates result set column aliases. These generated aliases are provided for field names without user-defined aliases to enable rapid resolution of letter case variations. Letter case resolution using an alias is significantly faster than letter case resolution by letter case translation. However, the number of generated result set column aliases is limited to 500. Because commonly InterSystems IRIS generates three of these aliases (for the three most common letter case variations) for each field, the system generates aliases for roughly the first 150 specified fields in the query. Therefore, a query referencing less than 150 fields commonly has better result set performance than a query referencing significantly more fields. This performance issue can be avoided by specifying an exact column alias for each field *selectItem* in a very large query (for example, `SELECT FamilyName AS FamilyName`) and then making sure that you use the same letter case when referencing the result set item by column alias.

20.3 Defining and Executing Named Queries

You can define and execute a named query as follows:

- Define the query using [CREATE QUERY](#). This query is defined as a stored procedure, and can be executed using [CALL](#).
- Define a [class query](#) (a query defined in a class definition). A class query is projected as a stored procedure. It can be executed using [CALL](#). A class query can also be prepared using the `%SQL.Statement.%PrepareClassQuery()` method, and then executed using the `%Execute()` method. See [Using Dynamic SQL](#).

20.3.1 CREATE QUERY and CALL

You can define a query using [CREATE QUERY](#), and then execute it by name using [CALL](#). In the following example, the first is an SQL program that defines the query `AgeQuery`, the second is Dynamic SQL that executes the query:

SQL

```
CREATE QUERY Sample.AgeQuery(IN topnum INT DEFAULT 10,IN minage INT 20)
  PROCEDURE
  BEGIN
    SELECT TOP :topnum Name,Age FROM Sample.Person
    WHERE Age > :minage
    ORDER BY Age ;
  END
```

ObjectScript

```
SET mycall = "CALL Sample.AgeQuery(11,65)"
SET tStatement = ##class(%SQL.Statement).%New()
SET qStatus = tStatement.%Prepare(mycall)
IF qStatus'=1 {WRITE "%Prepare failed:" DO $System.Status.DisplayError(qStatus) QUIT}
SET rset = tStatement.%Execute()
DO rset.%Display()
```

SQL

```
DROP QUERY Sample.AgeQuery
```


20.3.2 Class Queries

You can define a query in a class. The class may be a %Persistent class, but does not have to be. This [class query](#) can reference data defined in the same class, or in another class in the same namespace. The tables, fields, and other data entities referred to in a class query must exist when the class that contains the query is compiled.

A class query is not compiled when the class that contains it is compiled. Instead, compilation of a class query occurs upon the first execution of the SQL code (runtime). This occurs when the query is prepared in Dynamic SQL using the [%Prepare-ClassQuery\(\) method](#). First execution defines an executable cached query.

The following class definition example defines a class query:

```
Class Sample.QClass Extends %Persistent [DdlAllowed]
{
  Query MyQ(Myval As %String) As %SQLQuery (CONTAINID=1,ROWSPEC="Name,Home_State") [SqlProc]
  {
    SELECT Name,Home_State FROM Sample.Person
    WHERE Home_State = :Myval ORDER BY Name
  }
}
```

The following example executes the MyQ query defined in the Sample.QClass in the previous example:

ObjectScript

```
SET Myval="NY"
SET stmt=##class(%SQL.Statement).%New()
SET status = stmt.%PrepareClassQuery("Sample.QClass", "MyQ")
IF status'=1 {WRITE "%Prepare failed:" DO $System.Status.DisplayError(status) QUIT}
SET rset = stmt.%Execute(Myval)
DO rset.%Display()
WRITE !,"End of data"
```

The following Dynamic SQL example uses %SQL.Statement to execute the ByName query defined in the Sample.Person class, passing a string to limit the names returned to those that start with that string value:

ObjectScript

```
SET statemt=##class(%SQL.Statement).%New()
SET cqStatus=statemt.%PrepareClassQuery("Sample.Person", "ByName")
IF cqStatus'=1 {WRITE "%PrepareClassQuery failed:" DO $System.Status.DisplayError(cqStatus) QUIT}
SET rs=statemt.%Execute("L")
DO rs.%Display()
```

For further details, refer to [Defining and Using Class Queries](#).

20.4 Queries Invoking User-defined Functions

InterSystems SQL allows you to invoke class methods within SQL queries. This provides a powerful mechanism for extending the syntax of SQL.

To create a user-defined function, define a class method within a persistent InterSystems IRIS class. The method must have a literal (non-object) return value. This has to be a *class* method because there will not be an object instance within an SQL query on which to invoke an instance method. It also has to be defined as being an SQL stored procedure.

For example, we can define a **Cube()** method within the class MyApp.Person:

Class Definition

```
Class MyApp.Person Extends %Persistent [DdlAllowed]
{
  /// Find the Cube of a number
  ClassMethod Cube(val As %Integer) As %Integer [SqlProc]
  {
    RETURN val * val * val
  }
}
```

You can create SQL functions with the [CREATE FUNCTION](#), [CREATE METHOD](#) or [CREATE PROCEDURE](#) statements.

To call an SQL function, specify the name of the SQL procedure. A SQL function may be invoked in SQL code anywhere where a scalar expression may be specified. The function name may be qualified with its schema name, or unqualified. Unqualified function names take either a user-supplied [schema search path](#) or the [default schema name](#). A function name may be a [delimited identifier](#).

An SQL function must have a parameter list, enclosed in parentheses. The parameter list may be empty, but the parentheses are mandatory. All specified parameters act as input parameters. Output parameters are not supported.

An SQL function must return a value.

For example, the following SQL query invokes a user-defined SQL function as a method, just as if it was a built-in SQL function:

SQL

```
SELECT %ID, Age, MyApp.Person_Cube(Age) FROM MyApp.Person
```

For each value of Age, this query will invoke the **Cube()** method and place its return value within the results.

SQL functions may be nested.

If the specified function is not found, InterSystems IRIS issues an SQLCODE -359 error. If the specified function name is ambiguous, InterSystems IRIS issues an SQLCODE -358 error.

20.5 Querying Serial Object Properties

A serial object property that is projected as a child table to SQL from a class using default storage (%Storage.Persistent) is also projected as a single column in the table projected by the class. The value of this column is the serialized value of the serial object properties. This single column property is projected as an SQL %List field.

For example, the column Home in Sample.Person is defined as `Property Home As Sample.Address;` It is projected to `Class Sample.Address Extends (%SerialObject)`, which contains the properties Street, City, State, and PostalCode. See [Embedded Object \(%SerialObject\)](#) for details on defining a serial object.

The following example returns values from individual serial object columns:

SQL

```
SELECT TOP 4 Name,Home_Street,Home_City,Home_State,Home_PostalCode
FROM Sample.Person
```

The following example returns the values for all of the serial object columns (in order) as a single %List format string, with the value for each column as an element of the %List:

SQL

```
SELECT TOP 4 Name,$LISTTOSTRING(Home, '^')
FROM Sample.Person
```

By default, this Home column is hidden and is not projected as a column of Sample.Person.

20.6 Querying Collections

Collections may be referenced from the SQL WHERE clause, as follows:

```
WHERE FOR SOME %ELEMENT(collectionRef) [AS label] (predicate)
```

The **FOR SOME %ELEMENT** clause can be used for list collections and arrays that specify `STORAGEDEFAULT="list"`. The *predicate* may contain one reference to the pseudo-columns %KEY, %VALUE, or both. A few examples should help to clarify how the FOR SOME %ELEMENT clause may be used. The following returns the name and the list of FavoriteColors for each person whose FavoriteColors include 'Red'.

SQL

```
SELECT Name,FavoriteColors FROM Sample.Person
WHERE FOR SOME %ELEMENT(FavoriteColors) (%Value = 'Red')
```

Any SQL predicate may appear after the %Value (or %Key), so for example the following is also legal syntax:

SQL

```
SELECT Name,FavoriteColors FROM Sample.Person
WHERE FOR SOME %ELEMENT(Sample.Person.FavoriteColors)
(%Value IN ('Red', 'Blue', 'Green'))
```

A list collection is considered a special case of an array collection that has sequential numeric keys 1, 2, and so on. Array collections may have arbitrary non-null keys:

```
FOR SOME (children) (%Key = 'betty' AND %Value > 5)
```

In addition to the built-in list and array collection types, generalized collections may be created by providing a **BuildValueArray()** class method for any property. The **BuildValueArray()** class method transforms the value of a property into a local array, where each subscript of the array is a %KEY and the value is the corresponding %VALUE.

In addition to simple selections on the %KEY or %VALUE, it is also possible to logically connect two collections, as in the following example:

```
FOR SOME %ELEMENT(flavors) AS f
(f.%VALUE IN ('Chocolate', 'Vanilla') AND
FOR SOME %ELEMENT(toppings) AS t
(t.%VALUE = 'Butterscotch' AND
f.%KEY = t.%KEY))
```

This example has two collections: flavors and toppings, that are positionally related through their key. The query qualifies a row that has chocolate or vanilla specified as an element of flavors, and that also has butterscotch listed as the corresponding topping, where the correspondence is established through the %KEY.

You can change this default system-wide using the `CollectionProjection` option of the `$$SYSTEM.SQL.Util.SetOption()` method. `SET status=$$SYSTEM.SQL.Util.SetOption("CollectionProjection",1,.oldval)` to project a collection as a column if the collection is projected as a child table; the default is 0. Changes made to this system-wide setting takes effect for each class when that class is compiled or recompiled. You can use `$$SYSTEM.SQL.Util.GetOption("CollectionProjection")` to return the current setting.

For information on indexing a collection, refer to [Indexing Collections](#).

20.6.1 Usage Notes and Restrictions

- FOR SOME %ELEMENT may only appear in the WHERE clause.
- %KEY and/or %VALUE may only appear in a FOR predicate.
- Any particular %KEY or %VALUE may be referenced only once.
- %KEY and %VALUE may not appear in an outer join.
- %KEY and %VALUE may not appear in a value expression (only in a predicate).

20.7 Queries Invoking Free-text Search

InterSystems IRIS supports what is called “free-text search,” which includes support for:

- Wildcards
- Stemming
- Multiple-word searches (also called *n-grams*)
- Automatic classification
- Dictionary management

This feature enables SQL to support full text indexing, and also enables SQL to index and reference individual elements of a collection without projecting the collection property as a child table. While the underlying mechanisms that support collection indexing and full text indexing are closely related, text retrieval has many special properties, and therefore special classes and SQL features have been provided for text retrieval.

For further details refer to [Using InterSystems SQL Search](#).

20.8 Pseudo-Field Variables

InterSystems SQL queries support the following pseudo-field values:

- %ID — returns the [RowID field](#) value, regardless of the actual name of the RowID field.
- %TABLENAME — returns the qualified name of an existing table that is specified in the FROM clause. The qualified table name is returned in the letter case used when defining the table, not the letter case specified in the FROM clause. If the FROM clause specifies an unqualified table name, %TABLENAME returns the qualified table name (schema.table), with the schema name supplied from either a user-supplied [schema search path](#) or the [system-wide default schema name](#). For example, if the FROM clause specified mytable, the %TABLENAME variable might return SQLUser.MyTable.
- %CLASSNAME — returns the qualified class name (package.class) corresponding to an existing table specified in the FROM clause. For example, if the FROM clause specified SQLUser.mytable, the %CLASSNAME variable might return User.MyTable.

Note: The %CLASSNAME pseudo-field value should not be confused with the %ClassName() instance method. They return different values.

Pseudo-field variables can only be returned for a table that contains data.

If multiple tables are specified in the FROM clause you must use table aliases, as shown in the following Embedded SQL example:

ObjectScript

```
&sql(SELECT P.Name,P.%ID,P.%TABLENAME,E.%TABLENAME
      INTO :name,:rid,:ptname,:etname
      FROM Sample.Person AS P,Sample.Employee AS E)
IF SQLCODE<0 {WRITE "SQLCODE error ",SQLCODE," ",%msg  QUIT}
ELSEIF SQLCODE=100 {WRITE "Query returns no results"  QUIT}

WRITE "Person Name is: ",name,!
WRITE "Person RowId is: ",rid,!
WRITE "P alias TableName is: ",ptname,!
WRITE "E alias TableName is: ",etname,!
```

Python

```
try:
    result = iris.sql.exec("""
SELECT P.Name, P.%ID, P.%TABLENAME, E.%TABLENAME
FROM Sample.Person AS P, Sample.Employee AS E
""")

    row = next(result, None)

    if row:
        pname, pid, ptname, etname = row
        print(f"Person Name is: {pname}")
        print(f"RowId is: {pid}")
        print(f"P alias TableName is: {ptname}")
        print(f"E alias TableName is: {etname}")
    else:
        print("No results found.")
except Exception as e:
    sql_code = e.sqlcode

    if sql_code < 0:
        print(f"SQLCODE error {sql_code} {str(e)}")
    elif sql_code == 100:
        print("Query returns no results")
```

The %TABLENAME and %CLASSNAME columns are assigned the default column name `Literal_n`, where *n* is the `selectItem` position of the pseudo-field variable in the SELECT statement.

20.9 Query Metadata

You can use Dynamic SQL to return metadata about the query, such as the number of columns specified in the query, the name (or alias) of a column specified in the query, and the data type of a column specified in the query.

The following Dynamic SQL example returns the column name and an integer code for the column's ODBC data type for all of the columns in Sample.Person:

ObjectScript

```
SET myquery="SELECT * FROM Sample.Person"
SET rset = ##class(%SQL.Statement).%New()
SET qStatus = rset.%Prepare(myquery)
IF qStatus'=1 {WRITE "%Prepare failed:" DO $System.Status.DisplayError(qStatus) QUIT}
SET x=rset.%Metadata.columns.Count()
WHILE x>0 {
    SET column=rset.%Metadata.columns.GetAt(x)
    WRITE !,x," ",column.colName," ",column.ODBCType
    SET x=x-1 }
WRITE !,"end of columns"
```

Python

```
myquery = "SELECT * FROM Sample.Person"
try:
    rset = iris.sql.prepare(myquery)
    meta = rset.Statement._Metadata

    columns = meta.columns
    x = columns.Count()

    while x > 0:
        col = columns.GetAt(x)
        try:
            print(f"{x} {col.colName} {col.ODBCType}")
        except AttributeError:
            print(f"{x} {dir(col)}")
        x -= 1
    print("end of columns")
except Exception as e:
    print(f"Error: {str(e)}")
```

In this example, columns are listed in reverse column order. Note that the FavoriteColors column, which contains list structured data, returns a data type of 12 (VARCHAR) because ODBC represents an InterSystems IRIS list data type value as a string of comma-separated values.

For further details, see [Dynamic SQL](#) and see the %SQL.Statement class in the *InterSystems Class Reference*.

20.10 Queries and Enterprise Cache Protocol (ECP)

InterSystems IRIS implementations that use Enterprise Cache Protocol (ECP), such as distributed cache clusters, can synchronize query results. ECP is a distributed data caching architecture that manages the distribution of data and locks among a heterogeneous network of server systems.

If ECP synchronization is active, each time a **SELECT** statement is executed InterSystems IRIS forces all pending ECP requests to the dataserver. On completion this guarantees that the client cache is in sync. This synchronization occurs in the Open logic of the query. This is in the [OPEN cursor](#) execution if this is a cursor query.

To activate ECP synchronization system-wide, use the `$$SYSTEM.SQL.Util.SetOption()` method, as follows: `SET status=$$SYSTEM.SQL.Util.SetOption("ECPSync", 1, .oldval)`; the default is 0. To determine the current setting, call `$$SYSTEM.SQL.CurrentSettings()`.

For further details, refer to [Horizontally Scaling Systems for User Volume with InterSystems Distributed Caching](#).

20.11 Cached Queries

The system automatically maintains a cache of prepared SQL statements (“queries”). This permits the re-execution of an SQL query without repeating the overhead of optimizing the query and developing a Query Plan; this improves performance of the query on subsequent executions. A cached query is created when certain SQL statements are [prepared](#). Preparing a query occurs at runtime, not when the routine containing the SQL query code is compiled. Commonly, a prepare is immediately followed by the first execution of the SQL statement, though in [Dynamic SQL](#) it is possible to prepare a query without executing it.

Subsequent executions ignore the prepare statement and instead access the cached query. To force a new prepare of an existing query it is necessary to purge the cached query.

All invocations of SQL create cached queries, whether invoked in an ObjectScript routine or a class method.

- [Dynamic SQL](#), ODBC, JDBC, and the `$$SYSTEM.SQL.DDLImport()` method create a cached query when the query is prepared. The [Management Portal execute SQL interface](#), the [InterSystems SQL Shell](#), and the `%SYSTEM.SQL.Execute()` method use Dynamic SQL, and thus use a prepare operation to create cached queries.

They are listed in the Management Portal general **Cached Queries** listing for the namespace (or specified schema), the Management Portal Catalog Details **Cached Queries** listings for each table being accessed, and the [SQL Statements](#) listings.

- [Class Queries](#) create a cached query upon prepare (`%PrepareClassQuery()` method) or first execution (`CALL`).

They are listed in the Management Portal general **Cached Queries** listing for the namespace. If the class query is defined in a Persistent class, the cached query is also listed in the Catalog Details **Cached Queries** for that class. It is not listed in the Catalog Details for the table(s) being accessed. It is not listed in the [SQL Statements](#) listings.

- [Embedded SQL](#) creates a cached query upon first execution of the SQL code, or the initiation of code execution by invoking the OPEN command for a declared cursor. Embedded SQL cached queries are listed in the Management Portal **Cached Queries** listings with a **Query Type** of Embedded cached SQL, and the [SQL Statements](#) listings.

SQL query statements that generate a cached query are **SELECT**, **CALL**, **INSERT**, **UPDATE**, **INSERT OR UPDATE**, **DELETE**, **TRUNCATE TABLE**, **SET TRANSACTION**, **START TRANSACTION**, **%INTRANSACTION**, **COMMIT**, **ROLLBACK**.

A cached query is created when you [prepare the query](#). For this reason, it is important not to put a `%Prepare()` method in a loop structure. A subsequent `%Prepare()` of the same query (differing only in specified literal values) uses the existing cached query rather than creating a new cached query.

Changing the `SetMapSelectability()` value for a table invalidates all existing cached queries that reference that table. A subsequent prepare of an existing query creates a new cached query and removes the old cached query from the listing.

Comments are stripped from the text of the query when it is cached. A cached query can include [comment options](#) following the query text, such as `/*#OPTIONS {"optionName":value} */`.

All comments in queries issued over a JDBC or ODBC connection are stripped from the corresponding cached query, unless it begins with `/*#TAG` and ends with a backslash (`/`).

The creation of a cached query is not part of a transaction. The creation of a cached query is not journaled.

20.11.1 Listing Cached Queries

You can list (and manage) the contents of the query cache using the InterSystems IRIS Management Portal. From **System Explorer**, select **SQL**. Select a namespace by clicking the name of the current namespace displayed at the top of the page; this displays the list of available namespaces. On the left side of the screen open the **Cached Queries** folder. Selecting one of these cached queries displays the details.

The **Query Type** can be one of the following values:

- `%SQL.Statement` Dynamic SQL: a Dynamic SQL query using `%SQL.Statement`.
- Embedded cached SQL: an [Embedded SQL](#) query.
- ODBC/JDBC Statement: a dynamic query from either ODBC or JDBC.

The **Statement** is displayed for a Dynamic SQL cached query. This consists of the [statement hash](#), which is a selectable link that takes you to the SQL Statement Details, and the [plan state](#), such as (Unfrozen) or (Frozen/Explicit).

When you successfully prepare an SQL statement, the system generates a new class that implements the statement. If you have set the **Retain cached query source** system-wide configuration option, the source code for this generated class is retained and can be opened for inspection using your [IDE](#). To do this, go to the InterSystems IRIS Management Portal. From **System Administration**, select **Configuration**, then **SQL and Object Settings**, then **SQL**. On this screen you can set the

Retain cached query source option. If this option is not set (the default), the system generates and [deploys the class](#) and does not save the source code. You can set this system-wide behavior with the `CachedQuerySaveSource` [SQL setting](#).

20.11.2 Purge Cached Queries

A cached query is deleted when you purge cached queries. There are four methods of purging cached queries:

- [PURGE CACHED QUERIES](#)
- [PURGE](#) command issued from the SQL Shell
- `$SYSTEM.SQL.PurgeQueries()`
- One of the [Purge Cached Queries](#) options in the Management Portal

In addition, modifying a table definition automatically purges any queries that reference that table. Issuing a Prepare or Purge automatically requests an exclusive system-wide lock while the query cache metadata is updated. To turn these automatic requests off, modify the `CachedQueryLockTimeout` [SQL setting](#).

21

Defining and Using Stored Procedures

This topic describes how to define and use stored procedures in InterSystems SQL on InterSystems IRIS® data platform.

21.1 Overview

An SQL routine is an executable unit of code that can be invoked by the SQL query processor. There are two types of SQL routines: functions and stored procedures. Functions are invoked from any SQL statement that supports `functionname()` syntax. Functions accept some number of input directed arguments and return a single result value. Stored procedures accept some number of input, input-output, and output arguments. A stored procedure can be a user-defined function, returning a single value. A function can also be invoked by a **CALL** statement.

Like most relational database systems, InterSystems IRIS allows you to create SQL stored procedures. A Stored Procedure (SP) provides a callable routine that is stored in the database and can be invoked within an SQL context (for example, by using the **CALL** statement or via ODBC or JDBC).

Unlike relational databases, InterSystems IRIS enables you to define stored procedures as methods of classes. In fact, a stored procedure is nothing more than a class method that is made available to SQL. Within a stored procedure, you can use the full range of InterSystems IRIS object-based features.

- You can define a stored procedure as a query that returns a single result set of data by querying the database.
- You can define a stored procedure as a function procedure that can serve as a user-defined function, returning a single value.
- You can define a stored procedure as a method that can modify the database data and return either a single value or one or more result sets.

You can determine if a procedure already exists using the **\$\$SYSTEM.SQL.Schema.ProcedureExists()** method. This method also returns the procedure type: “function” or “query”.

21.2 Defining a Stored Procedure Using DDL

InterSystems SQL supports the following commands to create a query:

- **CREATE PROCEDURE** can create a query that is always projected as a stored procedure. A query can return a single result set.

- **CREATE QUERY** creates a query that can optionally be projected as a stored procedure. A query can return a single result set.

Note: In InterSystems SQL, a table-valued function is the same as a class query projected as a stored procedure. You can therefore use either **CREATE PROCEDURE** or **CREATE QUERY** to create a table-valued function.

InterSystems SQL supports the following commands to create a **method** or function:

- **CREATE PROCEDURE** can create a method that is always projected as a stored procedure. A method can return a single value, or one or more result sets.
- **CREATE METHOD** can create a method that can optionally be projected as a stored procedure. A method can return a single value, or one or more result sets.
- **CREATE FUNCTION** can create a function procedure that can optionally be projected as a stored procedure. A function can return a single value.

The block of executable code specified within these commands can be written either in InterSystems SQL or ObjectScript. You can include Embedded SQL within an ObjectScript code block.

21.3 SQL to Class Name Transformations

When you use DDL to create a stored procedure, the name you specify is transformed into a class name. If the class does not exist, the system creates it.

- If the name is unqualified and no FOR clause is provided: the **system-wide default schema name** is used as the package name, followed by a dot, followed by a generated class name consisting of the string 'func', 'meth', 'proc', or 'query', followed by the SQL name stripped of punctuation characters. For example, the unqualified procedure name `Store_Name` results in a class name such as the following: `User.procStoreName`. This procedure class contains the method **StoreName()**.
- If the name is qualified and no FOR clause is provided: the name of the schema is converted to a package name, followed by a dot, followed by the string 'func', 'meth', 'proc', or 'query', followed by the SQL name stripped of punctuation characters. If necessary, the specified package name is converted to a valid package name.

If the name is qualified and a FOR clause is provided: the qualified class name specified in the FOR clause overrides the schema name specified in the function, method, procedure, or query name.

- SQL stored procedure names follow **identifier** naming conventions. InterSystems IRIS strips punctuation characters from the SQL name to **generate unique class entity names** for the procedure class and its class methods.

The following rules govern the transformation of a schema name to valid package name:

- If the schema name contains an underscore, this character is converted to a dot, denoting a subpackage. For example, the qualified name `myprocs.myname` creates the package `myprocs`. The qualified name `my_procs.myname` creates the package `my` containing the subpackage `procs`.

The following example shows how the punctuation differs in a class name and its SQL invocation. It defines a method with a class name containing two dots. When invoked from SQL, the example replace the first dot with an underscore character:

Class Definition

```
Class Sample.ProcTest Extends %RegisteredObject
{
  ClassMethod myfunc(dummy As %String) As %String [ SqlProc ]
  { /* method code */
    Quit "abc" }
}
```

SQL

```
SELECT Sample.ProcTest_myfunc(Name)
FROM Sample.Person
```

21.4 Defining a Method Stored Procedure using Classes

Class methods can be exposed as Stored Procedures. These are ideal for actions that do not return data, such as a Stored Procedure that calculates a value and stores it in the database. Almost all classes can expose methods as Stored Procedures; the exception is generator classes, such as a data type class ([ClassType = datatype]). Generator classes do not have a runtime context. It is only valid to use a datatype context within the runtime of some other entity, such as a property.

To define a method stored procedure, simply define a class method and set its [SqlProc](#) keyword:

Class Definition

```
Class MyApp.Person Extends %Persistent [DdlAllowed]
{
  /// This procedure finds total sales for a territory
  ClassMethod FindTotal(territory As %String) As %Integer [SqlProc]
  {
    // use embedded sql to find total sales
    &sql(SELECT SUM(SalesAmount) INTO :total
        FROM Sales
        WHERE Territory = :territory
    )
    Quit total
  }
}
```

After this class is compiled, the **FindTotal()** method will be projected to SQL as the stored procedure **MyApp.Person_FindTotal()**. You can change the name that SQL uses for the procedure using the [SqlName](#) keyword of the method.

The method uses a procedure context handler to pass the procedure context back and forth between the procedure and its caller (for example, the ODBC server). This procedure context handler is automatically generated by InterSystems IRIS (as %qHandle:%SQLProcContext) using the %sqlcontext object.

%sqlcontext consists of properties for the SQLCODE error status, the SQL row count, an error message, and so forth, which are set using the corresponding SQL variables, as follows:

```
SET %sqlcontext.%SQLCode=SQLCODE
SET %sqlcontext.%ROWCOUNT=%ROWCOUNT
SET %sqlcontext.%Message=%msg
```

There is no need to do anything with these values, but their values will be interpreted by the client. The %sqlcontext object is reset before each execution.

The method should return no value.

The maximum number of user-defined methods for a class is 2000.

For instance, suppose there is a **CalcAvgScore()** method:

```
ClassMethod CalcAvgScore(firstname As %String, lastname As %String) [sqlproc]
{
    New SQLCODE,%ROWID
    &sql(UPDATE students SET avgscore =
        (SELECT AVG(sc.score)
         FROM scores sc, students st
         WHERE sc.student_id=st.student_id
              AND st.lastname=:lastname
              AND st.firstname=:firstname)
        WHERE students.lastname=:lastname
              AND students.firstname=:firstname)

    IF ($GET(%sqlcontext)!='') {
        SET %sqlcontext.%SQLCODE = SQLCODE
        SET %sqlcontext.%ROWCOUNT = %ROWCOUNT
    }
    QUIT
}
```

21.5 Defining a Query Stored Procedure using Classes

Many Stored Procedures that return data from the database can be implemented through the standard query interface. This approach works well as long as the procedure can be written in embedded SQL. Note the use of the Embedded SQL [host variable](#) to supply a value to the WHERE clause in the following example:

Class Definition

```
Class MyApp.Person Extends %Persistent [DdlAllowed]
{
    /// This procedure result set is the persons in a specified Home_State, ordered by Name
    Query ListPersons(state As %String = "") As %SQLQuery [ SqlProc ]
    {
        SELECT ID,Name,Home_State
        FROM Sample.Person
        WHERE Home_State = :state
        ORDER BY Name
    }
}
```

To expose a query as a Stored Procedure, add the following “[SqlProc]” string to the query definition:

```
Query QueryName() As %SQLQuery( ... query definition ... )
[ SqlProc ]
```

After this class is compiled, the **ListPersons** query will be projected to SQL as the stored procedure **MyApp.Person_ListPersons**. You can change the name that SQL uses for the procedure using the [SqlName](#) keyword of the query.

When **MyApp.Person_ListPersons** is called from SQL, it will automatically return the result set defined by the query’s SQL statement.

The following example is a stored procedure using a result set:

Class Definition

```
Class apc.OpiLLS.SpCollectResults1 [ Abstract ]
{
    /// This SP returns a number of rows (pNumRecs) from WebService.LLSResults, and updates a property for
    each record
    Query MyQuery(pNumRecs As %Integer) As %Query(ROWSPEC = "Name:%String,DOB:%Date") [ SqlProc ]
    {
    }

    /// You put initial code here in the Execute method
}
```

```

ClassMethod MyQueryExecute(ByRef qHandle As %Binary, pNumRecs As %Integer) As %Status
{
    SET mysql="SELECT TOP ? Name,DOB FROM Sample.Person"
    SET rset=##class(%SQL.Statement).%ExecDirect(,mysql,pNumRecs)
    IF rset.%SQLCODE'=0 {QUIT rset.%SQLCODE}
    SET qHandle=rset
    QUIT $$$OK
}

/// This code is called by the SQL framework for each row, until no more rows are returned
ClassMethod MyQueryFetch(ByRef qHandle As %Binary, ByRef Row As %List,
    ByRef AtEnd As %Integer = 0) As %Status [ PlaceAfter = NewQuery1Execute ]
{
    SET rset=qHandle
    SET tsc=$$$OK

    FOR {
        ///Get next row, quit if end of result set
        IF 'rset.%Next() {
            SET Row = "", AtEnd = 1
            SET tsc=$$$OK
            QUIT
        }
        SET name=rset.Name
        SET dob=rset.DOB
        SET Row = $LISTBUILD(name,dob)
        QUIT
    }
    QUIT tsc
}

ClassMethod MyQueryClose(ByRef qHandle As %Binary) As %Status [ PlaceAfter = NewQuery1Execute ]
{
    KILL qHandle //probably not necessary as killed by the SQL Call framework
    QUIT $$$OK
}
}

```

If it is possible to write the query as a simple SQL statement and create it through the Query Wizard, it is not necessary to know anything about the underlying methods that implement the query.

Behind the scenes, for each query the class compiler generates methods based on the name of the Stored Procedure, including:

- *stored-procedure-nameExecute()*
- *stored-procedure-nameFetch()*
- *stored-procedure-nameFetchRows()*
- *stored-procedure-nameGetInfo()*
- *stored-procedure-nameClose()*

If the query is of type %SQLQuery, the class compiler automatically inserts some embedded SQL into the generated methods. **Execute()** declares and opens a stored cursor for the SQL. **Fetch()** is called repeatedly until it returns an empty row (SET Row=""). You can, optionally, also have **Fetch()** return an AtEnd=1 boolean flag to indicate that the current Fetch constitutes the last row and the next Fetch is expected to return an empty row. However, an empty row (Row="") should always be used as the test to determine when the result set has ended; Row="" should always be set when setting AtEnd=1.

FetchRows() is logically equivalent to repeated calls to **Fetch()**. **GetInfo()** is called to return details of the signature for the Stored Procedure. **Close()** closes the cursor.

All these methods are called automatically when a Stored Procedure is invoked from a client, but could in theory be called directly from ObjectScript running on the server.

To pass an object from the **Execute()** to a **Fetch()**, or from a **Fetch()** to the next invocation of **Fetch()**, you can set the query handler to the object reference (OREF) of the object you wish to pass. To pass multiple objects, you can set qHandle as an array:

ObjectScript

```
SET qHandle(1)=oref1,qHandle(2)=oref2
```

It is possible to create a result set stored procedure that is based on custom-written code (not an SQL statement).

The maximum number of user-defined queries for a class is 200.

21.6 Customized Class Queries

For complex queries, or for Stored Procedures that do not fit the query model, it is often necessary to customize the query by replacing some or all of its methods. You can use %Library.Query, as described in this section.

It is often easier to implement the query if you choose type %Query (%Library.Query) instead of %SQLQuery (%Library.SQLQuery). This generate the same five methods, but now the **FetchRows()** is simply a repeated invocation of **Fetch()** (%SQLQuery has some optimization that causes other behavior). **GetInfo()** simply gets information from the signature, so it is very unlikely that the code will need to be changed. This reduces the problem to creating class methods for each of the other three. Note that when the class is compiled, the compiler detects the presence of these methods, and does not overwrite them.

The methods need specific signatures: They all take a Qhandle (query handler) of type %Binary. This is a pointer to a structure holding the nature and state of the query. This is passed by reference to **Execute()** and **Fetch()** and by value to **Close()**:

```
ClassMethod SP1Close(qHandle As %Binary) As %Status
{
    // ...
}

ClassMethod SP1Execute(ByRef qHandle As %Binary,
    p1 As %String) As %Status
{
    // ...
}

ClassMethod SP1Fetch(ByRef qHandle As %Binary,
    ByRef Row As %List, ByRef AtEnd As %Integer=0) As %Status
{
    // ...
}

Query SP1(p1 As %String)
    As %Query(CONTAINID=0,ROWSPEC="lastname:%String") [sqlproc ]
{
}
```

The code usually includes declaration and use of an SQL cursor. Cursors generated from queries of type %SQLQuery automatically have names such as Q14. You must ensure that your queries are given distinct names.

The class compiler must find a cursor declaration, before making any attempt to use the cursor. Therefore the DECLARE statement (usually in Execute) must be in the same MAC routine as the Close and Fetch and must come before either of them. Editing the source directly, use the method keyword PLACEAFTER in both the Close and the Fetch definitions to make sure this happens.

Error messages refer to the internal cursor name, which typically has an extra digit. Therefore an error message for cursor Q140 probably refers to Q14.

21.7 Using Stored Procedures

You can use stored procedures in two distinct ways:

- You can invoke a stored procedure using the SQL **CALL** statement; see [CALL](#) for more details.
- You can use a stored function (that is, a method-based stored procedure that returns a single value) as if it were a built-in function within an SQL query.

Note: When executing a stored procedure that takes an SQL function as an argument, invoke the stored procedure using [CALL](#), as in the following example:

SQL

```
CALL sp.MyProc(CURRENT_DATE)
```

A **SELECT** query does not support executing a stored procedure with an SQL function argument. **SELECT** does support executing a [stored function](#) with an SQL function argument.

You cannot execute a stored procedure with an SQL function argument using either **SELECT** or **CALL** over a driver connection.

21.7.1 Stored Functions

A stored function is a method-based stored procedure that returns a single value. For example, the following class defines a stored function, **Square**, that returns the square of a given value:

Class Definition

```
Class MyApp.Utils Extends %Persistent [DdlAllowed]
{
  ClassMethod Square(val As %Integer) As %Integer [SqlProc]
  {
    Quit val * val
  }
}
```

A stored function is simply a class method with the [SqlProc](#) keyword specified.

Note: For a stored function, the [ReturnResultsets](#) keyword must either be not specified (the default) or prefaced by the keyword **Not**.

You can use a stored function within an SQL query as if it were a built-in SQL function. The name of the function is the SQL name of the stored function (in this case “Square”) qualified by the schema (package) name in which it was defined (in this case “MyApp”).

The following query uses the **Square** function:

SQL

```
SELECT Cost, MyApp.Utils_Square(Cost) As SquareCost FROM Products
```

If you define multiple stored functions within the same package (schema), you must make sure that they have unique SQL names.

The following example defines a table named Sample.Wages that has two defined data fields (properties) and two defined stored functions, TimePlus and DTime:

Class Definition

```
Class Sample.Wages Extends %Persistent [ DdlAllowed ]
{
  Property Name As %String(MAXLEN = 50) [ Required ];
  Property Salary As %Integer;
  ClassMethod TimePlus(val As %Integer) As %Integer [ SqlProc ]
  {
    QUIT val * 1.5
  }
  ClassMethod DTime(val As %Integer) As %Integer [ SqlProc ]
  {
    QUIT val * 2
  }
}
```

The following query uses these stored procedures to return the regular salary, time-and-a-half, and double time salary rates for each employee in the same table, Sample.Wages:

SQL

```
SELECT Name,Salary,
       Sample.Wages_TimePlus(Salary) AS Overtime,
       Sample.Wages_DTime(Salary) AS DoubleTime FROM Sample.Wages
```

The following query uses these stored procedures to return the regular salary, time-and-a-half, and double time salary rates for each employee in a different table, Sample.Employee:

SQL

```
SELECT Name,Salary,
       Sample.Wages_TimePlus(Salary) AS Overtime,
       Sample.Wages_DTime(Salary) AS DoubleTime FROM Sample.Employee
```

21.7.2 Privileges

To execute a procedure, a user must have EXECUTE privilege for that procedure. Use the [GRANT](#) command or the `$SYSTEM.SQL.Security.GrantPrivilege()` method to assign EXECUTE privilege for a specified procedure to a specified user.

You can determine if a specified user has EXECUTE privilege for a specified procedure by invoking the `$SYSTEM.SQL.Security.CheckPrivilege()` method.

For more information about which class queries check privileges, refer to [SQL Users, Roles, and Privileges](#).

To list all the procedures for which a user has EXECUTE privilege, go to the Management Portal. From **System Administration** select **Security**, then select either **Users** or **Roles**. Select **Edit** for the desired user or role, then select the **SQL Procedures** tab. Select the desired **Namespace** from the drop-down list.

21.8 Listing Procedures

The INFORMATION.SCHEMA.ROUTINES persistent class displays information about all routines and procedures in the current namespace.

When specified in [Embedded SQL](#), INFORMATION.SCHEMA.ROUTINES requires the `#include %occInclude` macro pre-processor directive. This directive is not required for Dynamic SQL.

The following example returns the routine name, method or query name, routine type (PROCEDURE or FUNCTION), routine body (SQL=class query with SQL, EXTERNAL=not a class query with SQL), the return data type, and the routine definition for all routines in the schema “Sample” in the current namespace:

SQL

```
SELECT ROUTINE_NAME, METHOD_OR_QUERY_NAME, ROUTINE_TYPE, ROUTINE_BODY, SQL_DATA_ACCESS, IS_USER_DEFINED_CAST,
DATA_TYPE||' '||CHARACTER_MAXIMUM_LENGTH AS Returns, NUMERIC_PRECISION||':'||NUMERIC_SCALE AS
PrecisionScale,
ROUTINE_DEFINITION
FROM INFORMATION_SCHEMA.ROUTINES WHERE ROUTINE_SCHEMA='Sample'
```

The INFORMATION_SCHEMA.PARAMETERS persistent class displays information about input and output parameters for all routines and procedures in the current namespace.

The following example returns the routine name, parameter name, whether it is an input or output parameter, and the parameter data type information for all routines in the schema “Sample” in the current namespace:

SQL

```
SELECT SPECIFIC_NAME, PARAMETER_NAME, PARAMETER_MODE, ORDINAL_POSITION,
DATA_TYPE, CHARACTER_MAXIMUM_LENGTH AS MaxLen, NUMERIC_PRECISION||':'||NUMERIC_SCALE AS PrecisionScale
FROM INFORMATION_SCHEMA.PARAMETERS WHERE SPECIFIC_SCHEMA='Sample'
```

You can display much of the same information for a single procedure using the [Catalog Details tab](#) in the Management Portal SQL Interface. The Catalog Details for a procedure include the procedure type (query or function), class name, method or query name, the description, and the number of input and output parameters. The Catalog Details Stored Procedure Info display also provides an option to run the stored procedure.

22

Storing and Using Stream Data (BLOBs and CLOBs)

InterSystems SQL supports the ability to store stream data as either BLOBs (Binary Large Objects) or CLOBs (Character Large Objects) within an InterSystems IRIS® data platform database.

22.1 Stream Fields and SQL

InterSystems SQL supports two kinds of stream fields:

- Character streams, used for large quantities of text.
- Binary streams, used for images, audio, or video.

22.1.1 BLOBs and CLOBs

InterSystems SQL supports the ability to store BLOBs (Binary Large Objects) and CLOBs (Character Large Objects) within the database as stream objects. BLOBs are used to store binary information, such as images, while CLOBs are used to store character information. BLOBs and CLOBs can store up to 4 Gigabytes of data (the limit imposed by the JDBC and ODBC specifications). The default data types that store BLOBs and CLOBs in InterSystems SQL are automatically compressed.

The operation of the BLOBs and CLOBs is identical in every respect except how they handle character encoding conversion (such as Unicode to multibyte) when accessed via an ODBC or JDBC client: the data in a BLOB is treated as binary data and is never converted to another encoding, while the data in a CLOB is treated as character data and is converted as necessary.

If a binary stream file (BLOB) contains the single non-printing character `$CHAR(0)`, it is considered to be an empty binary stream. It is equivalent to the "" empty binary stream value: it exists (is not null), but has a length of 0.

From the object point of view, BLOBs and CLOBs are represented as stream objects. For more information, see [Working with Streams](#).

22.1.2 Defining Stream Data Fields

InterSystems SQL supports a variety of [data type names](#) for stream fields. These InterSystems data type names are synonyms that correspond to the following:

- Character streams: data type LONGVARCHAR, which maps to the %Stream.GlobalCharacter class and the ODBC/JDBC data type -1.
- Character streams: data type LONGVARBINARY, which maps to the %Stream.GlobalBinary class and the ODBC/JDBC data type -4.

Some InterSystems stream data types allow you to specify a data precision value. This value is a no-op and has no effect on the permitted size of the stream data. It is provided to allow the user to document the anticipated size of future data.

For data type mappings of stream data types, refer to the [SQL Data Types](#) reference page.

For how to define fields of a table, refer to [Defining a Table by Using DDL](#). The following example defines a table containing two stream fields:

SQL

```
CREATE TABLE Sample.MyTable (  
    Name VARCHAR(50) NOT NULL,  
    Notes LONGVARCHAR,  
    Photo LONGVARBINARY)
```

Equivalently, for how to define properties of a persistent class, refer to [Defining a Table by Creating a Persistent Class](#). When defining a stream property of a persistent class, you can optionally specify the LOCATION parameter; see [Declaring Stream Properties](#).

22.1.2.1 Stream Field Constraints

The definition of a stream field is subject to the following [field data constraints](#):

A stream field can be defined as NOT NULL.

A stream field can take a DEFAULT value, an ON UPDATE value, or a COMPUTECODE value.

A stream field cannot be defined as UNIQUE, a primary key field, or an [IdKey](#). Attempting to do so results in an SQLCODE -400 fatal error with a %msg such as the following: ERROR #5414: Invalid index attribute: Sample.MyTable::MYTABLEUNIQUE2::Notes, Stream property is not allowed in a unique/primary key/idkey index > ERROR #5030: An error occurred while compiling class 'Sample.MyTable'.

A stream field cannot be defined with a specified COLLATE value. Attempting to do so results in an SQLCODE -400 fatal error with a %msg such as the following: ERROR #5480: Property parameter not declared: Sample.MyTable:Photo:COLLATION > ERROR #5030: An error occurred while compiling class 'Sample.MyTable'.

22.1.3 Inserting Data into Stream Data Fields

To [INSERT](#) data into stream fields, if the field is a %Stream.GlobalCharacter, you can insert character stream data directly. For example,

SQL

```
INSERT INTO Sample.MyTable (Name,Notes)  
VALUES ('Fred','These are extensive notes about Fred')
```

In ObjectScript, there are alternative approaches, depending on the field type:

- %Stream.GlobalCharacter and %Stream.GlobalBinary fields: you can insert stream data using an OREF. You can use the **Write()** method to append a string to the character stream, or the **WriteLine()** method to append a string with a line terminator to the character stream. By default, the line terminator is \$CHAR(13,10) (carriage return/line feed); you can change the line terminator by setting the LineTerminator property. In the following example, the first part of the example creates a character stream consisting of two strings and their terminators, then inserts it into a stream field

using Embedded SQL. The second part of the example returns the character stream length and displays the character stream data showing the terminators:

ObjectScript

```
CreateAndInsertCharacterStream
SET gcoref=##class(%Stream.GlobalCharacter).%New()
DO gcoref.WriteLine("First Line")
DO gcoref.WriteLine("Second Line")
&sql(INSERT INTO Sample.MyTable (Name,Notes)
VALUES ('Fred',:gcoref))
IF SQLCODE<0 {WRITE "SQLCODE ERROR:"_SQLCODE_" "_msg QUIT}
ELSE {WRITE "Insert successful",!}
DisplayTheCharacterStream
KILL ^CacheStream
WRITE gcoref.%Save(),!
ZWRITE ^CacheStream
```

- %Stream.GlobalCharacter and %Stream.GlobalBinary fields: you can insert stream data by reading it from a file. For example,

ObjectScript

```
SET myf="C:\InterSystems\IRIS\mgr\temp\IMG_0190.JPG"
OPEN myf:("RF"):10
USE myf:0
READ x(1):10
&sql(INSERT INTO Sample.MyTable (Name,Photo) VALUES ('George',:x(1)))
IF SQLCODE<0 {WRITE "INSERT Failed:"_SQLCODE_" "_msg QUIT}
ELSE {WRITE "INSERT successful",!}
CLOSE myf
```

For further details, refer to [Sequential File I/O](#).

String data that is inserted as a DEFAULT value or a computed value is stored in the format appropriate for the stream field.

22.1.4 Querying Stream Field Data

A query *select-item* that selects a stream field returns the fully formed OID (object ID) value of the stream object, as shown in the following example:

SQL

```
SELECT Name,Photo,Notes
FROM Sample.MyTable WHERE Photo IS NOT NULL
```

An OID is a %List formatted data address such as the following:

```
$lb("1", "%Stream.GlobalCharacter", "^EW3K.Cn9X.S").
```

- The first element of the OID is a sequential positive integer (starting with 1) that is assigned to each inserted stream data value in a table. For example, if Row 1 is inserted with values for the stream fields Photo and Notes, these are assigned 1 and 2. If Row 2 is inserted with a value for Notes, that is assigned 3. If Row 3 is inserted with a value for Photo and Notes, those are assigned 4 and 5. The assignment sequence is the order that the fields are listed in the table definition, not the order they are specified in the **INSERT** command. By default, a single integer sequence is used which corresponds to the stream location global counter. However, a table may have multiple stream counters, as described below.

An **UPDATE** operation does not change the initial integer value. A **DELETE** operation may create gaps in the integer sequence, but does not change these integer values. Using **DELETE** to delete all records does not reset this integer counter. Using [TRUNCATE TABLE](#) to delete all records resets this integer counter if all of the table stream fields use the default StreamLocation value. **TRUNCATE TABLE** cannot be used to reset the stream integer counter for a embedded object (%SerialObject) class.

- The second element of the OID is the stream data type, either `%Stream.GlobalCharacter` or `%Stream.GlobalBinary`.
- The third element of the OID is a global variable. By default, its name is generated from the package name and the persistent class name that correspond to the table. An “S” (for Stream) is appended.
 - If the table was created using the SQL **CREATE TABLE** command, these package and persistent class names are hashed to four characters each (for example, `^EW3K.Cn9X.S`). This global contains the most recently assigned value of the stream data inserts counter. If no stream field data has been inserted, or **TRUNCATE TABLE** has been used to delete all table data, this global is undefined.
 - If the table was created as a persistent class, these package and persistent class names are not hashed (for example, `^Sample.MyTableS`). By default, this is the [StreamLocation](#) storage keyword `<StreamLocation>^Sample.MyTableS</StreamLocation>` value.

The default stream location is a global such as `^Sample.MyTableS`. This global is used to count the inserts to all stream properties (fields) that do not have a custom `LOCATION`. For example, if all stream properties in `Sample.MyTable` use the default stream location, when ten stream data values have been inserted into stream properties of `Sample.MyTable`, the `^Sample.MyTableS` global contains the value 10. This global contains the most recently assigned value of the stream data inserts counter. If no stream field data has been inserted, or **TRUNCATE TABLE** has been used to delete all table data, this global is undefined.

When defining a stream field property, you can define a custom `LOCATION`, such as the following: `Property Note2 As %Stream.GlobalCharacter (LOCATION="^MyCustomGlobalS");`. In this situation, the `^MyCustomGlobalS` global serves as the stream data inserts counter for the stream property (or properties) that specify this `LOCATION`; stream properties that do not specify a `LOCATION` use the default stream location global (`^Sample.MyTableS`) as the stream data inserts counter. Each global counts the inserts for the stream properties associated with that location. If no stream field data has been inserted the location global is undefined. **TRUNCATE TABLE** does not reset stream counters if one or more stream properties defined a `LOCATION`.

Subscripts of these stream location global variables contain the data for each stream field. For example, `^EW3K.Cn9X.S(3)` represents the third inserted stream data item. `^EW3K.Cn9X.S(3,0)` is the length of the data. `^EW3K.Cn9X.S(3,1)` is the actual stream data value.

Note: The OID for a stream field is not the same as the OID returned for a RowID or a reference field. The [%OID](#) function returns the OID for a RowID or a reference field; **%OID** cannot be used with a stream field. Attempting to use a stream field as an argument to **%OID** results in an SQLCODE -37 error.

Use of a stream field in the [WHERE clause](#) or [HAVING clause](#) of a query is highly restricted. You cannot use an equality condition or other [relational operator](#) (`=`, `!=`, `<`, `>`), or a [Contains operator](#) (`[]`) or [Follows operator](#) (`[]`) with a stream field. Attempting to use these operators with a stream field results in an SQLCODE -313 error. Refer to [Predicate Conditions and Streams](#) for valid predicates using a stream field.

22.1.4.1 Result Set Display

- Dynamic SQL executed from a program returns the OID in the format `$lb("6", "%Stream.GlobalCharacter", "^EW3K.Cn9X.S")`.
- The SQL Shell executes as Dynamic SQL and returns the OID in the format `$lb("6", "%Stream.GlobalCharacter", "^EW3K.Cn9X.S")`.
- Embedded SQL returns the same OID, but as an encoded `%List`. You can use the [\\$LISTTOSTRING](#) function to display the OID as a string with its elements separated by commas: `6,%Stream.GlobalBinary,^EW3K.Cn9X.S`.

When a query is run from the Management Portal SQL [Execute](#) interface, the OID is not returned. Instead:

- A character stream field returns the first 100 characters of character stream data. If the character stream data is longer than 100 characters, this is indicated by an ellipsis (...) following the 100th character. This is equivalent to SUBSTRING(cstreamfield,1,100).
- A binary stream field returns the string <binary>.

The same values are shown in the Management Portal SQL interface [Open Table](#) display of table data.

To display the OID value from the Management Portal SQL **Execute** interface, concatenate an empty string to a stream value, as shown in the following: `SELECT Name, ''||Photo, ''||Notes FROM Sample.MyTable`.

22.1.5 DISTINCT, GROUP BY, and ORDER BY

Every stream data field OID value is unique, even when the data itself contains duplicates. These **SELECT** clauses operate on the stream OID value, not the data value. Therefore, when applied to a stream field in a query:

- A **DISTINCT** clause has no effect on duplicate stream data values. A **DISTINCT** clause reduces the number records where the stream field is NULL to one NULL record.
- A **GROUP BY** clause has no effect on duplicate stream data values. A **GROUP BY** clause reduces the number records where the stream field is NULL to one NULL record.
- An **ORDER BY** clause orders stream data values by their OID value, not their data value. An **ORDER BY** clause lists records where the stream field is NULL before listing records with a stream field data value.

22.1.6 Predicate Conditions and Streams

The **IS [NOT] NULL** predicate can be applied to the data value of a stream field, as shown in the following example:

SQL

```
SELECT Name,Notes
FROM Sample.MyTable WHERE Notes IS NOT NULL
```

The **BETWEEN**, **EXISTS**, **IN**, **%INLIST**, **LIKE**, **%MATCHES**, and **%PATTERN** predicates can be applied to the OID value of the stream object, as shown in the following example:

SQL

```
SELECT Name,Notes
FROM Sample.MyTable WHERE Notes %MATCHES '*1[0-9]*GlobalChar*'
```

Attempting to use any other predicate condition on a stream field results in an SQLCODE -313 error.

22.1.7 Aggregate Functions and Streams

The **COUNT** aggregate function takes a stream field and counts the rows containing non-null values for the field, as shown in the following example:

SQL

```
SELECT COUNT(Photo) AS PicRows,COUNT(Notes) AS NoteRows
FROM Sample.MyTable
```

However, **COUNT(DISTINCT)** is not supported for stream fields.

No other aggregate functions are supported for stream fields. Attempting to use a stream field with any other aggregate function results in an SQLCODE -37 error.

22.1.8 Scalar Functions and Streams

InterSystems SQL cannot apply any function to a stream field, except the **%OBJECT**, **CHARACTER_LENGTH** (or **CHAR_LENGTH** or **DATALength**), **SUBSTRING**, **CONVERT**, **XMLCONCAT**, **XMLELEMENT**, **XMLFOREST**, and **%INTERNAL** functions. Attempting to use a stream field as an argument to any other SQL function results in an SQLCODE -37 error.

- The **%OBJECT** function opens a stream object (takes an OID) and returns the OREF (object reference), as shown in the following example:

SQL

```
SELECT Name,Notes,%OBJECT(Notes) AS NotesOref
FROM Sample.MyTable WHERE Notes IS NOT NULL
```

- The **CHARACTER_LENGTH**, **CHAR_LENGTH**, and **DATALength** functions take a stream field and return the actual data length, as shown in the following example:

SQL

```
SELECT Name,DATALength(Notes) AS NotesNumChars
FROM Sample.MyTable WHERE Notes IS NOT NULL
```

- The **SUBSTRING** function takes a stream field and returns the specified substring of the stream field's actual data value, as shown in the following example:

SQL

```
SELECT Name,SUBSTRING(Notes,1,10) AS Notes1st10Chars
FROM Sample.MyTable WHERE Notes IS NOT NULL
```

When issued from the Management Portal SQL **Execute** interface, a **SUBSTRING** function returns up to a 100 character substring of the stream field data. If the specified substring of the stream data is longer than 100 characters, this is indicated by an ellipsis (...) following the 100th character.

- The **CONVERT** function can be used to convert a stream data type to VARCHAR, as shown in the following example:

SQL

```
SELECT Name,CONVERT(VARCHAR(100),Notes) AS NotesTextAsStr
FROM Sample.MyTable WHERE Notes IS NOT NULL
```

CONVERT(datatype, expression) syntax supports stream data conversion. If the VARCHAR precision is less than the length of the actual stream data, it truncates the returned value to the VARCHAR precision. If the VARCHAR precision is greater than the length of the actual stream data, the returned value has the length of the actual stream data. No padding is performed.

{fn **CONVERT**(expression,datatype)} syntax does not support stream data conversion; it issues an SQLCODE -37 error.

- The **%INTERNAL** function can be used on a stream field, but performs no operation.

22.2 Stream Field Concurrency Locking

InterSystems IRIS protects stream data values from concurrent operations by another process by taking out a lock on the stream data.

The system takes an exclusive lock before performing a write operation; the exclusive lock is released immediately after the write operation completes.

Additionally, the system takes a shared lock when the stream is opened to be read and is released when the object reference to the stream is destroyed or the Read() method is called.

22.3 Using Stream Fields within InterSystems IRIS Methods

You cannot use a BLOB or CLOB value using Embedded SQL or Dynamic SQL directly within an InterSystems IRIS method; instead you use SQL to find the stream identifier for a BLOB or CLOB and then create an instance of the %AbstractStream object to access the data.

22.4 Using Stream Fields from ODBC

The ODBC specification does not provide for any recognition or special handling for BLOB and CLOB fields. InterSystems SQL represents CLOB fields within ODBC as having type LONGVARCHAR (-1). BLOB fields are represented as having type LONGVARBINARY (-4). For ODBC/JDBC data type mappings of stream data types, refer to [Integer Codes for Data Types](#) in the [SQL Data Types](#) reference page.

The ODBC driver/server uses a special protocol to access BLOB and CLOB fields. Typically you have to write special code within ODBC application to use CLOB and BLOB fields; the standard reporting tools typically do not support them.

22.5 Using Stream Fields from JDBC

Within a Java program you can retrieve or set data from a BLOB or CLOB using the standard JDBC BLOB and CLOB interfaces. For example:

```
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery("SELECT MyCLOB, MyBLOB FROM MyTable");
rs.next();      // fetch the Blob/Clob

java.sql.Clob clob = rs.getClob(1);
java.sql.Blob blob = rs.getBlob(2);

// Length
System.out.println("Clob length = " + clob.length());
System.out.println("Blob length = " + blob.length());

// ...
```

Note: When finished with a BLOB or CLOB, you must explicitly call the **free()** method to close the object in Java and send a message to the server to release stream resources (objects and locks). Just letting the Java object go out of scope does not send a message to clean up the server resources.

23

SQL Users, Roles, and Privileges

InterSystems IRIS® data platform has both system-level security, and an additional set of SQL-related security features. The InterSystems SQL security provides an additional level of security capabilities beyond its database-level protections. Some of the key differences between SQL and system-level security are:

- SQL protections are more granular than system-level protections. You can define privileges for tables, views, and stored procedures.
- SQL privileges can be granted to users as well as to roles. System-level privileges are only assigned to roles.
- Holding an SQL privilege implicitly grants any related system privileges that are required to perform the SQL action. (Conversely, system-level privileges do not imply table-level privileges.) The different types of privileges are described in “SQL Privileges and System Privileges”.

When SQL security is enabled, privilege-based table/view/procedure security is active. A user can only view or perform actions on a table for which that user has been granted privilege. When this parameter is disabled, a user can view or perform actions on non-security tables even if that user lacks the necessary privilege.

You can enable (1, default) or disable (0) SQL security in several ways:

- In the Management Portal, set the [system-wide security parameter](#) **Enable SQL security**.
- **\$SYSTEM.SQL.Util.SetOption()**:

```
do ##class(%SYSTEM.SQL.Util).SetOption("SQLSecurity", 1)
```

- **Security.System.Modify()**:

```
set properties("SQLSecurity")=1  
do ##class(Security.System).Modify(, .properties)
```

Security tables are unique in that access to them is fully restricted even if SQL security is disabled; these tables cannot be queried directly and can instead only be accessed through the predefined List and Detail query APIs, which require the [standard security API privileges](#). For example, to get a list of roles, you cannot directly query the `Security.Roles` table. Instead, you should use the **Security.Roles_List()** query:

```
SELECT Name,Description FROM Security.Roles_List()
```

Note: InterSystems SQL enforces privilege checking for ODBC, JDBC, [Dynamic SQL](#), and the [SQL Shell](#) interface on InterSystems IRIS data platform.

[Embedded SQL](#) statements do not perform privilege checking; it is assumed that applications using Embedded SQL will check for privileges before using Embedded SQL statements. Similarly, direct invocation of class queries that do not involve %SQL.Statement objects is considered application access and does not check for SQL privileges.

23.1 SQL Privileges and System Privileges

To manipulate tables or other SQL entities through SQL-specific mechanisms, a user must have the appropriate SQL privileges. System-level privileges are not sufficient. A user may be granted SQL privileges directly, or the user may belong to a role that has SQL privileges.

Note: Roles are shared by SQL and system level security: a single role can include both system and SQL privileges.

Consider the following example for an instance of InterSystems IRIS on a Windows machine:

- There is a persistent class in the USER namespace called User.MyPerson. This class is projected to SQL as the SQLUser.MyPerson table.
- There is a user called Test, who belongs to no roles (and therefore has no system privileges) and who has all privileges on the SQLUser.MyPerson table (and no other SQL privileges).
- There is a second user, called Test2. This user is assigned to the following roles: **%DB_USER** (and so can read or write data on the USER database); **%SQL** (and so has SQL access through the [%Service_Bindings](#) service); and, through a custom role, has privileges for using the Console and **%Development**.

If the Test user attempts to read or write data in the SQLUser.MyPerson table through any SQL-specific mechanism (such as one that uses ODBC), the attempt succeeds. This is because InterSystems IRIS makes the Test user a member of the **%SQL** role (which includes the **%Service_SQL:Use** privilege) and the **%DB_USER** role, so the user has the necessary privileges to establish the connection; this is visible in audit events that the connection generates, such as the %System/%Login/Login event. (If the Test user attempts to use Terminal object mechanisms, these attempts fail, because the user lacks sufficient privilege for these.)

If the Test2 user attempts to read or write data in the SQLUser.MyPerson table through any SQL-specific mechanism (such as one that uses ODBC), the attempt fails because the user does not have sufficient privileges for the table. (If the Test2 user attempts to view the same data in the Terminal using object mechanisms, the attempt succeeds — because the user is sufficiently privileged for this type of connection.)

For more information about SQL privileges, see [SQL privileges](#).

InterSystems IRIS persistent classes also support privileges for [row-level security](#).

23.2 %Admin_Secure Permission

If you are a user with the [%Admin_Secure](#) administrative resource with USE permission, you can perform the following operations:

- Create, modify, or delete a [user](#).
- Create, modify, or delete a [role](#).
- See the privileges granted to a [user](#).
- See the privileges granted to a [role](#).
- Revoke [SQL privileges](#) that were granted by another user.

%Admin_Secure allows a user to perform these operations without being given full security privileges on the system.

23.3 %Admin_RoleEdit Permission

Users with the [%Admin_RoleEdit](#) administrative resource with USE permission can perform the following operations:

- Create or delete a [role](#).

[%Admin_RoleEdit](#) allows a user to perform these operations without being given full security privileges on the system. Users with [%Admin_RoleEdit](#), but not [%Admin_Secure](#), can delete any non-system role and system roles that they have created themselves.

23.4 %Admin_UserEdit Permission

Users with the [%Admin_UserEdit](#) administrative resource with USE permission can perform the following operations:

- Create, modify, or delete a [user](#).

[%Admin_UserEdit](#) allows a user to perform these operations without being given full security privileges on the system. Users with [%Admin_UserEdit](#) but not [%Admin_Secure](#) can drop other users they have created, as well as any user created by a user they have created. Such a user cannot drop system users or themselves.

23.5 Users

An InterSystems SQL user is the same as a [user](#) defined for InterSystems security. You can define a user using either SQL commands or the Management Portal.

- In SQL you use the [CREATE USER](#) statement to create a user. This simply creates a user name and user password. The newly created user has no roles. You must use the [GRANT](#) statement to assign privileges and roles to the user. You can use the [ALTER USER](#) and [DROP USER](#) statements to modify existing user definitions.
- In the Management Portal Select **System Administration** select **Security**, then select **Users**. Click the **Create New User** button at the top of the page. This takes you to the **Edit User** page where you can specify the user name, user password, and other parameters. Once you create a user, the other tabs become available, where you can specify which roles a user holds, which general [SQL privileges](#) the user holds, which table-level privileges the user holds, which views are available, and which stored procedures can be executed.

If a user has SQL table privileges, or general SQL privileges, then roles granted or revoked on the user's **Roles** tab do not affect a user's access to tables through SQL-based services, such as ODBC. This is because, in the SQL-based services, table-based privileges take precedence over resource-based privileges.

23.5.1 User Name as Schema Name

Under some circumstances, a [username can be implicitly used as an SQL schema name](#). This may pose problems if the username contains characters that are forbidden in an SQL identifier. For example, in a multiple domain configuration the username contains the "@" character.

InterSystems IRIS handles this situation differently depending on the setting of the *Delimited Identifiers* configuration parameter:

- If the use of delimited identifiers is enabled, no special processing occurs.

- If the use of delimited identifiers is disabled, then any forbidden characters are removed from the username to form a schema name. For example, the username “documentation@intersystems.com” would become the schema name “documentationintersystemscom”.

This does not affect the value returned by the SQL **CURRENT_USER** function. It is always the same as *\$USERNAME*.

23.5.2 Use ObjectScript to List Users

You can use %Library.SQLCatalogPriv class queries to list:

- All users SQLUsers()
- All privileges granted to a specified user SQLUserPrivs(“username”)
- All system privileges granted to a specified user SQLUserSysPrivs(“username”)
- All roles granted to a specified user SQLUserRole(“username”)

The following example lists the privileges granted to the current user:

ObjectScript

```
SET statemt=##class(%SQL.Statement).%New()  
SET cqStatus=statemt.%PrepareClassQuery("%Library.SQLCatalogPriv", "SQLUserPrivs")  
IF cqStatus'=1 {WRITE "%PrepareClassQuery failed:" DO $System.Status.DisplayError(cqStatus) QUIT}  
  
SET rset=statemt.%Execute($USERNAME)  
WRITE "Privileges for ", $USERNAME  
DO rset.%Display()
```

23.6 Roles

SQL privileges are assigned to a user or role. A role enables you to set the same privileges for multiple users. Roles are shared by SQL and system level security: a single role can include both system privileges and SQL privileges.

The Management Portal, **System Administration, Security, Roles** page provides a list of role definitions for an InterSystems IRIS instance. To view or change details on a particular role, select the **Name** link for the role. On the **Edit Role** page that appears, there is information regarding the roles privileges, and which users or roles hold it.

The **General** tab lists a role’s privileges for InterSystems security [resources](#). If a role only holds SQL privileges, the General tab’s Resources table lists the role’s privileges as “None defined.”

The **SQL Privileges** tab lists a role’s privileges for InterSystems SQL resources, where a drop-down list of namespaces allows you to view each namespace’s resources. Because privileges are listed by namespace, the listing for a role holding no privileges in a particular namespace displays “None.”

Note: You should define privileges using roles and associate specific users with these roles. There are two reasons for this:

1. It is much more efficient for the SQL Engine to determine privilege levels by checking a relatively small role database than by checking individual user entries.
2. It is much easier to administer a system using a small set of roles as compared with a system with many individual user settings.

For example, you can define a role called “ACCOUNTING” with certain access privileges. As the Accounting Department grows, you can define new users and associate them with the ACCOUNTING role. If you need to modify the privileges for ACCOUNTING, you can do it once and it will automatically cover all the members of the Accounting Department.

A role can hold other roles. For example, the ACCOUNTING role can hold the BILLINGCLERK role. A user granted the ACCOUNTING role would have the privileges of both the ACCOUNTING role and the BILLINGCLERK role.

You can also define users and roles with the following SQL commands: [CREATE USER](#), [CREATE ROLE](#), [ALTER USER](#), [GRANT](#), [DROP USER](#), and [DROP ROLE](#).

Note: If you manually create a role names SQLAdminRole, there is some special behavior if you would like a user with this role to issue a DROP ROLE command. In particular, you will need to give the %Admin_Secure resource to SQLAdminRole to allow users with this role to perform a DROP ROLE.

23.6.1 Use ObjectScript to List Roles

You can use %Library.SQLCatalogPriv class queries to list:

- All roles SQLRoles()
- All privileges granted to a specified role SQLRolePrivileges(“rolename”)
- All roles or users granted to a specified role SQLRoleUser(“rolename”)
- All roles granted to a specified user SQLUserRole(“username”)

23.7 SQL Privileges

SQL privileges are assigned to a user or role. A role enables you to set the same privileges for multiple users.

InterSystems SQL supports two types of privileges: administrative and object.

- Administrative privileges are namespace-specific.

Administrative privileges cover the creation, altering, and deleting of types of objects, such as the %CREATE_TABLE privilege required to create tables. The %ALTER_TABLE privilege is required not only to alter a table, but to create or drop an index, to create or drop a trigger, and to run **TUNE TABLE**.

[Administrative privileges](#) also include %NOCHECK, %NOINDEX, %NOLOCK, %NOJOURN, and %NOTRIGGER, which determine whether the user can apply the corresponding keyword restrictions when performing an **INSERT**, **UPDATE**, **INSERT OR UPDATE**, or **DELETE**. Assigning the %NOTRIGGER administrative privilege is required for a user to perform a **TRUNCATE TABLE**.

- Object privileges are specific to a table, view, or stored procedure. They specify the type of access to specific named SQL objects (in the SQL sense of the word: a table, a view, a column, or a stored procedure). If the user is the Owner (creator) of the SQL object, the user is automatically granted all privileges for that object.

[Table-level object privileges](#) provide access (%ALTER, DELETE, SELECT, INSERT, UPDATE, EXECUTE, REFERENCES, CANCEL) to the data in all columns of a table or view, both those columns that currently exist and any subsequently added columns.

Column-level object privileges provide access to the data in only the specified columns of a table or view. You do not need to assign column-level privileges for columns with system-defined values, such as RowID and Identity.

Stored procedure object privileges permit the assignment of EXECUTE privilege for the procedure to specified users or roles.

For further details, refer to the [GRANT](#) command.

23.7.1 Granting SQL Privileges

You can grant privileges in the following ways:

- Use the Management Portal. From **System Administration** select **Security**, then select either **Users** or **Roles**. Select the desired user or role, then select the appropriate tab: **SQL Privileges** for administrative privileges, **SQL Tables**, **SQL Views**, or **SQL Procedures** for object privileges.
- From SQL, use the [GRANT](#) command to grant specific administrative privileges or object privileges to a specified user or role (or list of users or roles). You can use the [REVOKE](#) command to remove privileges.
- From ObjectScript, use the `$$SYSTEM.SQL.Security.GrantPrivilege()` method to grant specific object privileges to a specified user (or list of users).

23.7.2 Listing SQL Privileges

- Use the Management Portal. From **System Administration** select **Security**, then select either **Users** or **Roles**. Select the desired user or role, then select the appropriate tab: **SQL Privileges** for administrative privileges, **SQL Tables**, **SQL Views**, or **SQL Procedures** for object privileges.
- From SQL, use the [%CHECKPRIV](#) command to determine if the current user has a specific administrative or object privilege.
- From ObjectScript, use the `$$SYSTEM.SQL.Security.CheckPrivilege()` method to determine if a specified user has a specific object privilege.

23.7.3 Auditing Privilege Errors

When an InterSystems IRIS process invokes an SQL statement for which the user is not privileged, the operation fails and an SQLCODE -99 error is generated. When the Audit Event %System/%SQL/PrivilegeFailure is enabled, a record will be placed in the [Audit database](#) for each SQLCODE -99 error encountered. This Audit database option is disabled by default.

24

Row-Level Security

In addition to its general security, InterSystems IRIS® data platform provides SQL security with a granularity of a single row. This is called *row-level security*. With row-level security, each row holds a list of authorized viewers, which can be either [roles](#) or [users](#).

24.1 Introduction

Typically, SQL security is controlled by granting **SELECT** privilege on a table or on a view; the privilege is granted to a role or user. In most cases, view-level security provides adequate control over which rows each user can select; however, when the number of views required becomes very large, another alternative for fine-grained access control is needed.

For example, a hospital may make patient-specific data available online to each patient, and creating a separate view for each patient is not desirable. In such a scenario, row-level security is the practical way to secure the data.

The following are constraints on the use of row-level security:

- Row-level security is only available for persistent classes.
- Row-level security is only available for tables on the InterSystems IRIS server. It is not available for linked or foreign tables.
- Row-level security is only enforced when accessing rows from SQL. It is not enforced when directly accessing globals or when accessing globals via the object interface.

To implement row-level security for a table, you add a special field that holds a list of usernames or roles such that only those usernames or roles can access a given row. This special field can be the generated [%READERLIST](#) field or it can be a [custom field](#). In either case, the data in this field is for internal use and your code cannot access it.

Important: The **%All** role does not automatically have access to rows in a table that are protected with row-level security. If **%All** is to have access to such a row, the **%All** role must be listed along with other roles in the special field used for row-level security.

24.2 Setting Up with the %READERLIST Field

For a given table, to enable row-level security using the [%READERLIST](#) field, make the following changes to the class from which the table is projected.

1. In the class definition, set the value of the *ROWLEVELSECURITY* parameter:

Class Member

```
Parameter ROWLEVELSECURITY = 1;
```

In this case, once you compile the class, the **%RLI** index is automatically added to the class, although it is not visible in the class definition. This is a collection index on the **%READERLIST** property. The query optimizer uses it to minimize the performance impact when row-level security is enabled.

2. If you want to store the values in another property in the same class, then set the parameter equal to the name of that property.

Class Member

```
Parameter ROWLEVELSECURITY = "rlsprop";
```

Where *rlsprop* is the name of a property in the same class.

3. If you used a custom property name (that is, *ROWLEVELSECURITY* is not 1), add an index named **%RLI** index on the property.

If you expect to never have more than one username or role in this property, then this index can be an ordinary (non-collection) bitmap index. This generally provides optimal performance.

4. In the same class, also define a **%SecurityPolicy()** class method, which specifies the role and usernames that are permitted to select the row, subject to view and table **SELECT** privileges.

This class method must return an empty string, a username, a role, or a comma-separated list of usernames or roles. If the method returns an empty string, there is no specific restriction for this row; the row is visible to all users who hold the **SELECT** privilege on the table.

Important: The **%All** role does not automatically have access to rows in a table that are protected with row-level security. If **%All** is to have access to such a row, the **%SecurityPolicy()** method must explicitly return the **%All** role, along with others as needed.

The structure of the **%SecurityPolicy()** method is:

```
ClassMethod %SecurityPolicy() As %String [ SqlProc ]
{
    RETURN ""
}
```

Its characteristics are:

- It is a class method with the required name **%SecurityPolicy**.
- It returns a string (type **%String**).
- It takes zero or more arguments. If this method takes any arguments, each argument name must match a property name in the class. If any property names do not exactly match their SQL field names, instead use the SQL field names as argument names.
- The **SqlProc** keyword specifies that the method can be invoked as a stored procedure.

5. Compile the class and any dependent classes.

When you compile this class, two changes are made to it automatically (but are not visible in the class definition):

- It has a new property named %READERLIST, which is a calculated field. The value of this field is determined by the %SecurityPolicy() method. Whenever an **INSERT** or **UPDATE** occurs, %SecurityPolicy() is invoked for that row and populates the value of %READERLIST.
- It has an index named %RLI index, which is a collection index on the %READERLIST property.

24.3 Setting Up with a Custom Field

For a given table, to enable row-level security using a custom field, make the following changes to the class from which the table is projected.

1. Identify a property in the same class that contains data that can be used for row-level security. This property can be an empty string or can contain a role, a username, or a comma-separated list of roles or usernames. If the property is an empty string, there is no specific restriction for this row; the row is visible to all users who hold the **SELECT** privilege on the table.
2. Add a suitable index (with any index name) to this class for this property. If you expect to never have more than one username or role in this property, then this can be an ordinary (non-collection) bitmap index. This generally provides optimal performance.
3. In the class definition, set the value of the *ROWLEVELSECURITY* parameter:

Class Member

```
Parameter ROWLEVELSECURITY = "rlsprop";
```

Where *rlsprop* is the name of the property identified in step 1.

4. Compile the class and any dependent classes.

24.4 After Adding Row-Level Security to a Table with Existing Data

If you have added row-level security to a table that already contained data, you need to update the value of the property that lists the users and roles who can view each row.

One way to do this is as follows:

1. From the Management Portal home page, go to the **SQL** page (**System Explorer > SQL**) page.
2. Select the namespace that contains the table.
3. Click **Execute Query**.
4. In the editable area, issue a statement to update the table. It should have the following form:

```
UPDATE MySchema.MyClass SET rlsprop =
    MySchema.SecurityPolicy(MySQLColumnName1, ...)
```

where

- *MySchema* is the schema (package) containing the class.

- *MyClass* is the name of the class.
- *rlsprop* is the field containing the list of users and roles who can read the row. This is %READERLIST by default and the property name specified in the declaration of the *ROWLEVELSECURITY* parameter otherwise.
- *SecurityPolicy* is the SqlName of the **%SecurityPolicy()** method. If SqlName is not specified, the default is *classname_sys_SecurityPolicy*. For example, if the class is *MySchema.MyClass*, then the default SqlName of the **%SecurityPolicy()** method is **myClass_sys_SecurityPolicy** (and the fully qualified name is **MySchema.MyClass_sys_SecurityPolicy**).
- *MySQLColumnName1, ...* is the set of SQL column names corresponding to the arguments, if any, defined in the **%SecurityPolicy()** class method.

5. Click **Execute**.

24.5 Security Tips

Keep in mind the following security factors when using row-level security:

- Row-level security operates in addition to table-level security. To execute a **SELECT**, **INSERT**, **UPDATE**, or **DELETE** statement, a user must have been granted both table-level access and row-level access for the relevant row.
- User privileges are checked dynamically at runtime — when there is an attempt to execute an SQL command.
- If you create an updateable view and specify **WITH CHECK OPTION**, then an **INSERT** operation on that view checks if the row to be inserted passes the **WHERE** clause that is specified in the view. Further, if you are creating the view from a table with row-level security, then the **INSERT** fails if either the **WHERE** clause of the view or the implicit row-level security predicate would cause that row to not be visible if you were to issue a command of **SELECT * FROM** on the view.
- If you have access to a row, it is possible to change the value of the %READERLIST field (or whatever field holds the list of users and roles who can view the row). This means that you can perform an action that, directly or indirectly, removes your access to the row.
- If you attempt to insert a row that would have violated a **UNIQUE** constraint if row-level security had not been defined, then it will still violate the constraint if row-level security is defined, regardless of whether the row causing the constraint failure is visible to the updating transaction.

24.6 See Also

- [Users and Roles](#)
- [Defining Indexes](#)

25

Using Vector Search

Vector search is a foundational concept to systems that use machine learning and artificial intelligence. After using an embedding model to transform unstructured data, such as text or images, into embedding vectors, users can perform operations on these vectors to process input and return the vectors that are the most semantically similar.

InterSystems IRIS SQL supports the storage of vectors in the compact and performant VECTOR and EMBEDDING types, enabling you to efficiently store vectorized data as a part of your traditional relational schemas. By leveraging the EMBEDDING type, InterSystems IRIS SQL converts text into embedding vectors through familiar SQL syntax without interacting with an embedding model directly.

25.1 Vectors and Embeddings

Vectors can be used to represent the semantic meaning in embeddings. These embeddings are determined by an embedding model, which is a machine learning model that maps text, images, or audio to a geometric space with high dimensionality. This page talks specifically using an embedding model with text, but the procedures described here can be used on many different types of data.

Modern embedding vectors typically range between hundreds and thousands of dimensions. Words that share similar semantic meaning occupy nearby positions in that space, while words with disparate semantic meaning occupy positions distant from each other. These spatial positions allow an application to algorithmically determine the similarity between two words or sentences by computing the distance between them on their embedding vectors.

In vector search, a user compares an input vector with the vectors stored in the database using a distance function, such as the dot product. A vector search enables you to algorithmically determine which pieces of text are semantically most similar to an input. As a result, vector search is a great fit for tasks that involve information retrieval.

InterSystems IRIS SQL supports a dedicated [VECTOR type](#) that leverages SIMD CPU instructions to efficiently perform distance computations and an EMBEDDING type that uses all the optimizations of the VECTOR type, but also simplifies the conversion of a string value from a source field in the same table (typically of type VARCHAR) into an embedding vector. There are five numeric vector types: FLOAT (the default), DOUBLE, DECIMAL, and INTEGER.

As VECTOR and EMBEDDING are standard SQL data types, you can store them alongside other data in a relational table, converting a SQL database transparently into a hybrid vector database. To insert VECTOR-typed data with an **INSERT** statement, use the [TO_VECTOR](#) function; note that this requires you to manually create embeddings before you can include them in the **INSERT** statement. To insert EMBEDDING-typed data, define an [embedding configuration](#), then use an **INSERT** statement to insert the string into the EMBEDDING column's source field. InterSystems recommends using the EMBEDDING type for ease of use.

25.2 Inserting VECTOR-typed Data

25.2.1 Translate Text to Embeddings

Before you can store embeddings as vectors in InterSystems IRIS, you first need to create them from a source. In general, you can transform a piece of text into an embedding in three steps.

1. Import a package to use to turn your text into a series of embeddings.
2. Pre-process your text to best fit your chosen embedding model's input specifications.
3. Instantiate the model and convert your text to the embeddings, using your chosen package's workflow.

With [Embedded Python](#), you can execute Python code that converts text to embeddings (using whichever package you like) alongside ObjectScript code that can insert those embeddings directly into your database. For information on importing a Python package into InterSystems IRIS, see [Install and Import Python Packages](#).

25.2.1.1 Example: Create Embeddings with Embedded Python

The following example takes an input list of sentences, passed as a built-in Python list, converts them to embeddings using the `sentence_transformers` package, and returns a list of the embeddings for later insertion into a table. See [Performing an INSERT](#) for more information on inserting the embeddings. This example assumes that the input sentences have already been pre-processed to match the embedding model's specifications.

Class Member

```
ClassMethod GetEmbeddingPy(sentences) [ Language = python ]
{
    import json

    # import the package
    import sentence_transformers

    # perform any preprocessing

    # create the model and form the embeddings
    model = sentence_transformers.SentenceTransformer('all-MiniLM-L6-v2')
    embeddings = model.encode(sentences)

    return embeddings
}
```

25.2.2 Perform an INSERT

Once you have a list of strings that represent embeddings, you can insert them into your table as VECTORS either with an **INSERT** statement or by creating an object and storing the embedding as a property of the object. The following example demonstrates how to insert data with an **INSERT**.

25.2.2.1 Example

For each embedding, execute an **INSERT** statement that adds the embedding to the desired table. Use **TO_VECTOR** to convert the string representation of the embedding to a VECTOR.

In the following command, one embedding is inserted into a table called `Sample.Description` that has two columns: one for the embedding that represents a textual description and the other for a unique identifier that can be used to link the embedding with the text it is derived from (implicitly stored in a separate table with the same unique identifier). (Note that the example uses `?`s as placeholders for the embedding and the unique identifier, as these are typically supplied programmatically as parameters to the statement.)

SQL

```
INSERT INTO Sample.Descriptions (DescriptionEmbedding, UID)
VALUES (TO_VECTOR(?,FLOAT), ?)
```

The following code samples use this query to insert a single embedding into the table.

ObjectScript

```
ClassMethod InsertEmbeddings(embedding As %String, uid As %Integer)
{
    set sc=$$$$OK
    try {
        set myquery = "INSERT INTO Sample.Descriptions (DescriptionEmbedding, UID)"
            _"VALUES (TO_VECTOR(?,FLOAT), ?)"
        set tStatement = ##class(%SQL.Statement).%New()
        $$$ThrowOnError(tStatement.%Prepare(query))

        set rset = tStatement.%Execute(embedding, uid)
        if (rset.%SQLCODE < 0) {
            throw ##class(%Exception.SQL).CreateFromSQLCODE(rset.%SQLCODE,rset.%Message)
        }
    }
    catch e {
        set sc = e.AsStatus()
        return sc
    }
    return 0
}
```

Java/JDBC

```
public void InsertEmbeddings(String embeddings, Integer uid) {
    try {
        // set connection parameters
        IRISDataSource ds = new IRISDataSource();
        ds.setServerName("127.0.0.1");
        ds.setPortNumber(51776);
        ds.setDatabaseName("USER");
        ds.setUser("_SYSTEM");
        ds.setPassword("SYS");
        IRISConnection conn = ds.GetConnection();

        String sql = "INSERT INTO Sample.Embeddings (Embedding, UID) " +
            "VALUES (TO_VECTOR(?,FLOAT), ?)";
        PreparedStatement pstmt = conn.prepareStatement(sql);

        pstmt.setString(embeddings);
        pstmt.setInt(uid);

        pstmt.executeUpdate();

        pstmt.close();
    } catch (Exception ex) {
        System.out.println("caught exception: "
            + ex.getClass().getName() + ": " + ex.getMessage());
    }
}
```

Python/DB-API

```
def insertEmbeddings(embeddings, uid):
    // set the connection parameters
    conn_string = "localhost:1972/USER"
    username = "_system"
    password = "SYS"
    connection = iris.connect(conn_string, username, password)
    cursor = connection.cursor()

    try:
        sql = "INSERT INTO Sample.Embeddings (Embedding, UID) " +
            "VALUES (TO_VECTOR(?,FLOAT), ?)"
        params = [embeddings,uid]
        cursor.execute(sql,params)
        cursor.close()
    except Exception as ex:
        print(ex)
    finally:
        if cursor:
```

```
        cursor.close()
    if connection:
        connection.close()
```

.NET/ODBC

```
static void InsertEmbeddings(string emb, Integer uid)
{
    // set the connection parameters
    string host = "127.0.0.1";
    string port = "51776";
    string username = "_SYSTEM";
    string password = "SYS";
    string namespace = "USER";

    IRISConnection conn = new IRISConnection();
    IRISConnect.ConnectionString = "Server = " + host
        + "; Port = " + port + "; Namespace = " + namespace
        + "; Password = " + password + "; User ID = " + username;

    conn.Open();

    String sql = "INSERT INTO Sample.Embeddings (Embedding, UID) " +
        "VALUES (TO_VECTOR(?,FLOAT), ?)";
    IRISCommand cmd = new IRISCommand(sql, conn);
    cmd.ExecuteNonQuery();

    cmd.Dispose();
    conn.Close();
}
```

25.3 Inserting EMBEDDING-typed Data

25.3.1 Create an Embedding Configuration

Before you can insert embeddings into a table, you must decide on an embedding model you wish to use to convert text into an embedding. Once you have selected a model, you can insert metadata for that model into the %Embedding.Config table to create an embedding configuration. The embedding configuration stores the information needed to make API calls to your chosen embedding model.

The %Embedding.Config table has five columns:

- *Name*: The unique and valid identifier used to refer to the model.
- *Configuration*: A JSON-formatted string that includes particular data for a particular source. See [OpenAI Configuration Settings](#) and [SentenceTransformers Configuration Settings](#) for more information on using this parameter with the default embedding classes.
- *EmbeddingClass*: The name of the ObjectScript class that extends %Embedding.Interface and defines logic for retrieving embeddings from the endpoint. InterSystems IRIS provides two such classes out of the box: %Embedding.OpenAI and %Embedding.SentenceTransformers. To use an embedding model from a different source, you must manually define a new class that extends the %Embedding.Interface class.
- *VectorLength*: The length of the vector (number of dimensions) the embedding model returns.
- *Description*: An optional description of this endpoint.

The following example inserts an embedding configuration for an [OpenAI embedding model](#) into the %Embedding.Config table.

SQL

```
INSERT INTO %Embedding.Config (Name, Configuration, EmbeddingClass, VectorLength, Description)
VALUES ('my-openai-config',
       '{"apiKey": "<api key>",
        "sslConfig": "llm_ssl",
        "modelName": "text-embedding-3-small"}',
       '%Embedding.OpenAI',
       1536,
       'a small embedding model provided by OpenAI')
```

The following example inserts an embedding configuration for a [SentenceTransformers model](#) in the %Embedding.Config table. Note that the VectorLength column is implied and is populated automatically, based on the model

SQL

```
INSERT INTO %Embedding.Config (Name, Configuration, EmbeddingClass, Description)
VALUES ('sentence-transformers-config',
       '{"modelName": "sentence-transformers/all-MiniLM-L6-v2",
        "hfCachePath": "/Users/InterSystems/VEC147/hfCache",
        "maxTokens": 256,
        "checkTokenCount": true}',
       '%Embedding.SentenceTransformers',
       'a small SentenceTransformers embedding model')
```

25.3.1.1 OpenAI Configuration Settings

Table 25–1:

Setting Name	Description	Required?
modelName	The name of the OpenAI model.	Yes
sslConfig	The SSL Configuration used to make the API request to OpenAI.	Yes
apiKey	The OpenAI API key.	Yes
maxTokens	The maximum number of input tokens	No (default value: 8191)
checkTokenCount	A boolean. If true, then inputs are checked against the maximum number of tokens.	No (default value: false)

25.3.1.2 SentenceTransformers Configuration Settings

Table 25–2:

Setting Name	Description	Required?
modelName	The name of the OpenAI model.	Yes
hfCachePath	The path on your machine to the embedding model.	Yes
maxTokens	The maximum number of input tokens.	No (automatically populated by your chosen model)
checkTokenCount	A boolean. If true, then inputs are checked against the maximum number of tokens.	No (default value: false)

25.3.2 Define a Table with EMBEDDING-type Columns

Once you have stored an embedding configuration in the %Embedding.Config table, you can create a table that uses this configuration to populate EMBEDDING-typed columns.

When defining a column with the embedding type, you must specify two arguments for the *model* and *source* parameters. The *model* parameter is the name of the embedding configuration that converts text into an embedding. The *source* parameter is a comma separated list of properties in the class that are used to calculate the embedding.

The following example creates a table with four columns, two of which store strings and two of which store embeddings of those strings. Both of the EMBEDDING-typed fields use an embedding configuration called *my-openai-config*.

SQL

```
CREATE TABLE Embedding.Example (
    Description VARCHAR(200),
    Name VARCHAR(30),
    DescriptionEmbedding EMBEDDING('my-openai-config', 'Description'),
    NameEmbedding EMBEDDING('my-openai-config', 'Name')
)
```

25.4 Define a Vector Index

After storing data in InterSystems IRIS in the VECTOR type, you may define a vector index to improve the efficiency of searches issued against your stored vectors. Vectorized data is most often indexed in an approximate nearest neighbor (ANN) vector index.

In a standard vector search, comparisons against an input vector are made against every individual vector in the database. While this approach guarantees that your searches are completely accurate, it is computationally inefficient. An ANN vector index approximates the exact result by leveraging nearest neighbor algorithms that search a specialized graph data structure to limit the number of comparison operations performed between an input vector and the stored vectors. As a result, when a search is performed, the system does not make comparisons with each stored vector, but instead uses the graph to eliminate vectors that are not close to input vector. This approach dramatically improves the performance of searches on a vector database, particularly when dealing with large amounts of high-dimensional data. This performance benefit outweighs the small loss in precision due to approximation.

As with [standard indexes](#), the query optimizer may decide that the most efficient query plan does not use the vector index you have defined. To see if a query uses the vector index, [examine the query plan](#) with the **EXPLAIN** command.

Note: Unlike other kinds of indexes, an ANN vector index is only used in queries that include a TOP clause, an **ORDER BY ... DESC** clause, and a vector distance function (such as **VECTOR_DOT_PRODUCT**) to find the best matching records. Other index types are routinely used to handle other clauses, including **WHERE**, **GROUP BY**, and **JOIN** conditions. InterSystems recommends [examining such query plans](#) to ensure that the index is being employed as expected.

25.4.1 Hierarchical Navigable Small World Index

InterSystems SQL allows you to define a Hierarchical Navigable Small World (HNSW) index, which creates a vector index for use with search algorithms (such as [ACORN-1](#)) that can greatly improve performance. By default, queries on tables that use an HNSW index use the ACORN-1 algorithm.

Note: InterSystems intends on making improvements to the efficiency of the HNSW index that may impact its execution and structure, potentially requiring you to rebuild the index in a future version.

If, after upgrading to a later version, a query that attempts to use your HNSW index fails with the error message “HNSW index <IndexName> was built using an unsupported HNSW storage version (versions <old version number>); current HNSW code supports storage version <current version number>.”, simply rebuild the index.

You can define an HNSW index using a [CREATE INDEX](#) statement. An HNSW index definition is subject to the following considerations:

- The HNSW index must be defined on a VECTOR- or EMBEDDING-typed field with a fixed length that is of type double or decimal.
- The table the index is defined on must have [IDs that are bitmap supported](#).
- The table the index is defined on must use default storage.
- On a partitioned table, the HNSW index will be a partitioned index (that is, the index is itself partitioned).

When defining an HNSW index, you must specify the distance function used by the index in the `Distance` index parameter. There are two possible values for this parameter: `Cosine` and `DotProduct`. The distance function you use should match the one that you use in queries on this table; for example, if you are using `VECTOR_COSINE` in your queries, you should use `Cosine`. Furthermore, you should only use `DotProduct` if vectors stored in the table are normalized. This parameter is case insensitive and must be surrounded by single quotes.

There are two additional parameters that you can optionally specify when defining an HNSW index:

- `M` (optional): The number of bi-directional links created for every new element during construction. This value should be a positive integer larger than 1; the value will fall between 2–100. Higher `M` values work better on datasets with high dimensionality or recall, while lower `M` values work better with low dimensionality or recall. The default value is 16.
- `efConstruction` (optional): The size of the dynamic list for the nearest neighbors. This value should be a positive integer that is larger than `M`. Larger `efConstruction` values generally lead to better index quality, but longer construction time. There is a maximum value past which `efConstruction` does not improve the quality of the index. The default value is 64.

The following examples define HNSW indexes with various values in the parameters.

SQL

```
CREATE INDEX HNSWIndex ON TABLE Company.People (Biography)
AS HNSW(Distance='Cosine')
```

SQL

```
CREATE INDEX HNSWIndex ON TABLE Company.People (Biography)
AS HNSW(M=24, Distance='DotProduct')
```

SQL

```
CREATE INDEX HNSWIndex ON TABLE Company.People (Biography)
AS HNSW(M=32, efConstruction=100, Distance='Cosine')
```

25.4.1.1 Special Cases

If you build an HNSW index using `Cosine` as the `Distance` parameter, then all-zero vectors stored in a table are silently omitted from the index, so searches that use the HNSW indexes do not include all-zero vectors. Similarly, if you query a

table with an HNSW index that uses `Cosine` as the Distance parameter to find vectors that are similar to an all-zero vector, the result set will contain zero rows.

25.5 Perform Vector Search

Vector search enables you to use one vector to search for other similar vectors stored within the database. In InterSystems SQL, you can perform such a search with a single **SELECT** query.

InterSystems SQL currently supports two functions that determine the similarity between two vectors: **VECTOR_DOT_PRODUCT** and **VECTOR_COSINE**. The larger the value of these functions, the more similar the vectors are. Use an **ORDER BY** clause with the `DESC` option to return a sorted set of vectors, with the most similar vectors returned at the top of the result set.

If you have defined a vector index, be sure to specify that results should be returned in descending order by using the `DESC` option in your **ORDER BY** clause.

25.5.1 Example

The following example demonstrates how to use SQL to issue a query that uses **VECTOR_DOT_PRODUCT** to find the most semantically similar descriptions to an input sentence. Convert an input search term to an embedding with the **EMBEDDING** function and use either **VECTOR_DOT_PRODUCT** or **VECTOR_COSINE** within an **ORDER BY** clause to return the most similar pieces of text. In this example, note that you do not need to specify the name of the embedding model in the **EMBEDDING** function because the function is used on an `EMBEDDING`-typed field in a table and the system automatically uses the embedding model associated with the field. To select only the most similar results, use a **TOP** clause. This example shows a SQL statement that selects the five descriptions that are the most similar to the input user query. (Note that the example uses `?` as a placeholder for the embedding of the search term, as this value is typically provided as a parameter, not as a literal.)

SQL

```
SELECT TOP 5 Description FROM Embedding.Example
ORDER BY VECTOR_DOT_PRODUCT(DescriptionEmbedding,
                           EMBEDDING(?)) DESC
```

The following demonstrates an ObjectScript method that executes this query in Dynamic SQL, iterating through the result set to append the descriptions in a long string:

Class Member

```
ClassMethod GetSimilarDesc(searchTerm As %String)
{
    set sc = $$$OK
    try {
        set query = "SELECT TOP 5 Description FROM Embedding.Example
                    ORDER BY VECTOR_DOT_PRODUCT(DescriptionEmbedding,
                    EMBEDDING(?, 'my-openai-config')) DESC"
        set tStatement = ##class(%SQL.Statement).%New()
        $$$ThrowOnError(tStatement.%Prepare(query))

        set rset = tStatement.%Execute(searchTerm)
        if (rset.%SQLCODE < 0) {
            throw ##class(%Exception.SQL).CreateFromSQLCODE(rset.%SQLCODE, rset.%Message)
        }

        // process retrieved descriptions here and return the result
        set retrievedInfo = ""
        while rset.%Next() {
            set retrievedInfo = retrievedInfo_" Description: "_rset.%Get("Description")
        }
        return retrievedInfo
    }
}
```

```
catch e {  
  set sc = e.AsStatus()  
  return sc  
}
```

25.6 See More

Data Types

- [VECTOR](#)
- [EMBEDDING](#)

Commands

- [VECTOR_COSINE](#)
- [VECTOR_DOT_PRODUCT](#)
- [TO_VECTOR](#)
- [CREATE TABLE](#)
- [INSERT](#)
- [SELECT](#)

26

SQL Settings Reference

InterSystems SQL enables you to control various settings to set up your SQL environment. These settings can be controlled through various interfaces, including the [Configuration Parameter File](#), the Management Portal, the **SET OPTION** command, and methods in the %SYSTEM.SQL.Util class. However, not all options are available through all interfaces and some have different names across the multiple interfaces they might be available through. This page acts as a reference for how you can change each of the settings.

26.1 How to Change Each SQL Setting

Since there are many different SQL settings and multiple mechanisms to change them, the table below provides a summary of how you may change each one.

To find the SQL settings page in the Management Portal, navigate to **System Administration > Configuration > SQL and Object Settings > SQL**.

Table 26–1:

Option Name / Management Portal Name (if applicable)	CPF Name	API	SET OPTION Name
AdaptiveMode / Turn off Adaptive Mode to disable run time plan choice and automatic tuning	AdaptiveMode	SetOption()	n/a
AllowExtrinsicFunction	ExtrinsicFunctions	SetOption()	n/a
AllowRowIDUpdate	AllowRowIDUpdate	n/a	n/a
ANSIPrecedence	ANSIPrecedence	SetOption()	n/a
AutoCommit	n/a	SetOption()	n/a
AutoParallel / Execute queries in a single process	AutoParallel	SetOption()	n/a
AutoParallelThreshold	AutoParallelThreshold	SetOption()	AUTO_PARALLEL_THRESHOLD
BiasQueriesAsOutlier	BiasQueriesAsOutlier	n/a	n/a
BitmapFriendlyCheck	n/a	SetOption()	n/a
CachedQueryLockTimeout	n/a	SetOption()	n/a

Option Name / Management Portal Name (if applicable)	CPF Name	API	SET OPTION Name
CachedQuerySaveSource / Retain cached query source	SaveMAC	SetOption()	n/a
ClientMaxIdleTime / Client maximum idle time (seconds)	ClientMaxIdleTime	SetOption()	n/a
CollectionProjection	n/a	SetOption()	n/a
CompileModeDeferred	n/a	SetOption()	COMPILEMODE
CompileModeImmediate	n/a	SetOption()	COMPILEMODE
CompileModeInstall	n/a	SetOption()	COMPILEMODE
CompileModeNocheck	n/a	SetOption()	COMPILEMODE
DDLDefineBitmapExtent	DDLDefineBitmapExtent	SetOption()	n/a
DDLDropTabDelData	DropDelete	SetOption()	n/a
DDLFinal	DDLFinal	SetOption()	n/a
DDLNo201 / Ignore redundant DDL	DDLNo201	n/a	n/a
DDLNo30 / Ignore redundant DDL	DDLNo30	n/a	n/a
DDLNo307 / Ignore redundant DDL	DDLNo307	n/a	n/a
DDLNo311 / Ignore redundant DDL	DDLNo311	n/a	n/a
DDLNo315 / Ignore redundant DDL	DDLNo315	n/a	n/a
DDLNo324 / Ignore redundant DDL	DDLNo324	n/a	n/a
DDLNo333 / Ignore redundant DDL	DDLNo333	n/a	n/a
DDLPrimaryKeyNotIDKey / Define primary key as ID key for tables created via DDL	IDKey	SetOption()	PKEY_IS_IDKEY
DDLSQLOnlyCompile	DDLSQLOnlyCompile	n/a	n/a
DDLUseExtentSet	DDLUseExtentSet	SetOption()	n/a
DDLUseSequence	DDLUseSequence	SetOption()	n/a
DefaultSchema / Default schema	DefaultSchema	n/a	DEFAULT_SCHEMA
DefaultTimePrecision	TimePrecision	SetOption()	n/a
DelimitedIdentifiers	DelimitedIds	SetOption()	SUPPORT_DELIMITED_IDENTIFIERS
ECPSync	ECPSync	SetOption()	n/a

Option Name / Management Portal Name (if applicable)	CPF Name	API	SET OPTION Name
FastDistinct / GROUP BY and DISTINCT queries must produce original values	FastDistinct	SetOption()	EXACT_DISTINCT
FilterRefIntegrity	ReferentialChecks	SetOption()	n/a
IdentityInsert	n/a	SetOption()	n/a
IdTrxFrom	IdTrxFrom	SetDDLIdentifierTranslations()	n/a
IdTrxTo	IdTrxTo	SetDDLIdentifierTranslations()	n/a
IsolationMode	n/a	SetOption()	n/a
LockThreshold / Lock escalation threshold	LockThreshold	SetOption()	LOCK_ESCALATION_THRESHOLD
LockTimeout / Lock timeout (seconds)	LockTimeout	SetOption()	LOCK_TIMEOUT
ODBCVarcharMaxlen / Default length for VARCHAR	ODBCVarcharMaxlen	n/a	n/a
ProcessLockTimeout	n/a	SetOption()	n/a
ParameterSampling / Turn on parameter sampling to sample the parameter value for query execution	ParameterSampling	SetOption()	n/a
QueryProcedures	QueryProcedures	SetOption()	n/a
RetainSQL	Comment	SetOption()	n/a
RTPC	RTPC	SetOption()	n/a
SQLFunctionArgConversion	n/a	SetOption()	n/a
SQLSecurity	n/a	SetOption()	n/a
SelectMode	n/a	SetOption()	n/a
ServerDisconnectCode	n/a	SetOption()	n/a
ServerInitCode	n/a	SetOption()	n/a
TCPKeepAlive / TCP keepalive for client connections (seconds)	TCPKeepAlive	SetOption()	n/a
ToDateDefaultFormat / TO_DATE default format	TODATEDefaultFormat	SetOption()	n/a

26.2 Settings that Require Permissions

In order to change the following settings, a user must have the %Admin:USE permission:

Table 26–2:

AdaptiveMode	ExtrinsicFunctions	ANSIPrecedence	AutoParallel
SaveMAC	DropDelete	DDLFinal	IDKey
DDLUseExtentSet	DDLUseSequence	TimePrecision	DelimitedIds
ECPSync	FastDistinct	ReferentialChecks	IdTrxFrom
IdTrxTo	LockThreshold	LockTimeout	ParameterSampling
QueryProcedures	Comment	RTPC	DBMSSecurity
TCPKeepAlive	TODATEDefaultFormat		

27

Importing SQL Code

This topic describes how to import SQL code from a text file into InterSystems SQL. When you import SQL code, InterSystems IRIS® data platform prepares and executes each line of SQL. If it encounters a line of code it cannot parse, SQL import skips over that line of code and continues to prepare and execute subsequent lines until it reaches the end of the file. All SQL code import operations import to the current namespace.

SQL Import is primarily used to import Data Definition Language (DDL) commands, such as **CREATE TABLE**, and to populate tables using **INSERT**, **UPDATE**, and **DELETE** commands.

InterSystems recommends importing SQL code from another system or vendor with the [LOAD SQL](#) command. However, you can choose to invoke either the **ImportDDL()** or **Run()** methods from the %SYSTEM.SQL.Schema class as well.

Note: It is worth noting that the %SYSTEM.SQL.Schema class also allows for exporting DDL commands using the **ExportDDL()** method. In brief, this method allows you to export a DDL script file that can later be imported via methods described in this page.

- [Importing SQL with LOAD SQL](#)
- [Importing InterSystems SQL with ObjectScript](#)
- [Importing non-InterSystems SQL with ObjectScript](#): FDBMS, Informix, InterBase, MSSQLServer, MySQL, Oracle, Sybase

27.1 Importing SQL with LOAD SQL

With the [LOAD SQL](#) command, you can easily and efficiently load schemas and table definitions from one database to another, regardless of the system it originates from, as well as run standard DML operations, like **INSERT**. Any errors caused by the loading are written to the %SQL_Diag.Result log. The command can load DDL statements from a single file or from all the files within a single directory (including any subdirectories).

SQL commands should be stored in a standard, non-formatted file (like a text file). Commands may span multiple lines, but are differentiated from each other by a common delimiter that indicates where one command ends and another begins. The default delimiter differs depending on the origin dialect of the commands. See the [LOAD SQL](#) page for more information on delimiters and supported dialects.

27.2 Importing InterSystems SQL with ObjectScript

You can import InterSystems SQL code from a text file using either of the following %SYSTEM.SQL.Schema methods:

- **ImportDDL()** is a general-purpose SQL import method. This method runs as a background (non-interactive) process. To import InterSystems SQL you specify “IRIS” as the third parameter.
- **Run()** is an InterSystems SQL import method. This method runs interactively from the Terminal. It prompts you to specify the location of the import text file, the location to create the Errors.log file and the Unsupported.log file, and other information.

Note: This import and execution of SQL DDL code should not be confused with the **Import Statements Action** from the Management Portal SQL interface. That operation imports [SQL Statements](#) in XML format.

The following example imports the InterSystems IRIS SQL code file pathname mysqlcode.txt, executing the SQL commands listed in that file in the current namespace:

ObjectScript

```
DO $SYSTEM.SQL.Schema.ImportDDL("c:\InterSystems\mysqlcode.txt",,"IRIS")
```

By default, **ImportDDL()** creates an errors log file, as specified in the second parameter. This example, which omits the second parameter, creates by default a file named mysqlcode_Errors.log in the same directory as the SQL code file. This log file is created even when there is nothing written to it.

When executing **ImportDDL()** from the Terminal, it first lists the input file, then the error log file, then lists each SQL command imported, as shown in the following example:

```
Importing SQL Statements from file: c:\InterSystems\mysqlcode.txt
Recording any errors to principal device and log file: c:\InterSystems\mysqlcode_Errors.log

SQL statement to process (number 1):
CREATE TABLE Sample.NewTab (Name VARCHAR(40))
Preparing SQL statement...
Executing SQL statement...
DONE

SQL statement to process (number 2):
CREATE INDEX NameIDX ON Sample.NewTab (Name)
Preparing SQL statement...
Executing SQL statement...
DONE

Elapsed time: 7.342532 seconds
```

If an error occurs in any SQL command, the Terminal display the error, as shown in the following example:

```
SQL statement to process (number 3):
INSERT INTO Sample.MyStudents (StudentName,StudentDOB) SELECT Name,
DOB FROM Sample.Person WHERE Age <= '21'
Preparing SQL statement...
ERROR #5540: SQLCODE: -30 Message: Table 'SAMPLE.PERSON' not found
Pausing 5 seconds - read error message! (Type Q to Quit)
```

If you do not Quit within 5 seconds, **ImportDDL()** proceeds to execute the next SQL command. The error is recorded in the errors log file with a timestamp, the user name, and the namespace name.

27.2.1 Import File Format

An SQL text file must be an unformatted file, such as a .txt file. Each SQL command must begin on its own line. An SQL command may be broken into multiple lines and indentation is permitted. Each SQL command must be followed by a GO statement on its own line.

The following is an example of a valid InterSystems SQL import file text:

```
CREATE TABLE Sample.MyStudents (StudentName VARCHAR(32),StudentDOB DATE)
GO
CREATE INDEX Nameldx ON TABLE Sample.MyStudents (StudentName)
GO
INSERT INTO Sample.MyStudents (StudentName,StudentDOB) SELECT Name,
DOB FROM Sample.Person WHERE Age <= '21'
GO
INSERT INTO Sample.MyStudents (StudentName,StudentDOB)
VALUES ('Jones,Mary',60123)
GO
UPDATE Sample.MyStudents SET StudentName='Smith-Jones,Mary' WHERE StudentName='Jones,Mary'
GO
DELETE FROM Sample.MyStudents WHERE StudentName %STARTSWITH 'A'
GO
```

27.2.2 Supported SQL Commands

Not all valid InterSystems SQL commands can be imported. The following is a list of supported InterSystems SQL commands:

- **CREATE TABLE, ALTER TABLE, DROP TABLE**
- **CREATE VIEW, ALTER VIEW, DROP VIEW**
- **CREATE INDEX** all index types, except bitslice
- **CREATE USER, DROP USER**
- **CREATE ROLE**
- **GRANT, REVOKE**
- **INSERT, UPDATE, INSERT OR UPDATE, DELETE**
- **SET OPTION**
- **SELECT** for optimizer plan mode only

27.3 Importing non-InterSystems SQL with ObjectScript

SQL import can be used to import InterSystems SQL code or to import SQL code from other vendors (FDBMS, Informix, InterBase, MSSQLServer, MySQL, Oracle, Sybase). Code from other vendors is converted to InterSystems SQL code and executed. InterSystems SQL import utilities only import commands and clauses that are compatible with the InterSystems IRIS implementation of the SQL standard; incompatible commands are commonly parsed, but ignored. The following methods are provided:

- **ImportDDL()** is a general-purpose SQL import method. This method runs as a background (non-interactive) process. Refer to [Importing InterSystems SQL](#) for general information on using this method.

To import SQL in a specific format you specify the name of that format as the third parameter: FDBMS, Informix, InterBase, MSSQLServer (or MSSQL), MySQL, Oracle, or Sybase.

The following example imports the MSSQL code file `mssqlcode.txt`, executing the SQL commands listed in that file in the current namespace:

ObjectScript

```
DO $SYSTEM.SQL.Schema.ImportDDL($lb("C:\temp\somsql.sql", "UTF8"),, "MSSQL")
```

Note that if the third parameter is MSSQL, Sybase, Informix, or MySQL, the first parameter can be either an SQL code file pathname or a two-element %List with the first element the SQL code file pathname and the second element is the [I/O translation table](#) to use.

- Individual interactive methods are provided in %SYSTEM.SQL.Schema to import the following types of SQL: **LoadFDBMS()**, **LoadInformix()**, **LoadInterBase()**, **LoadMSSQLServer()**, **LoadOracle()**, and **LoadSybase()**. These methods runs interactively from the Terminal. It prompts you to specify the location of the import text file, the location to create the Errors.log file and the Unsupported.log file, and other information.
- **ImportDDLDir()** allow you to import SQL code from multiple files in a directory. This method runs as a background (non-interactive) process. It supports Informix, MSSQLServer, and Sybase. All files to be imported must have a .sql extension suffix.
- **ImportDir()** allow you to import SQL code from multiple files in a directory. Provides more options than **ImportDDLDir()**. This method runs as a background (non-interactive) process. It supports MSSQLServer, and Sybase. You can specify a list of allowed file extension suffixes.

A

Importing and Exporting SQL Data

In the InterSystems IRIS® data platform Management Portal, there are tools for importing and exporting data. Internally these tools use [Dynamic SQL](#), which means that queries are prepared and executed at runtime. InterSystems IRIS cannot import a row that exceeds the [string length limit](#), and within InterSystems IRIS, no row can exceed that limit.

You can also import data using the %SQL.Import.Mgr class, and export data using the %SQL.Export.Mgr class.

A.1 Importing Data with LOAD DATA

You can import data into InterSystems IRIS using the [LOAD DATA](#) SQL command. This utility can be used to import data either from a file or from a table accessed with JDBC. Before calling **LOAD DATA**, you must first use [CREATE TABLE](#) to define the table and its columns.

If the table is empty when loading data, **LOAD DATA** populates the table with rows from the data source. If the table already contains data, **LOAD DATA** inserts rows into the table without overwriting any data in the table.

The following example creates a table and loads data into it using a file stored on a local system, named people.csv:

```
>>> CREATE TABLE Sample.Person (  
      Name VARCHAR(25),  
      Age INT,  
      DOB DATE)  
  
>>> LOAD DATA FROM FILE 'C://sampledata/people.csv' INTO Sample.Person
```

LOAD DATA also provides options for speeding up the loading operation through use of the **BULK** keyword. Refer to the [LOAD DATA](#) reference for a full description of this behavior.

A.2 Exporting Data to a Text File

You can export data for a given class to a text file. To do so:

1. From the Management Portal, select **System Explorer**, then **SQL**. Select a namespace by clicking the name of the current namespace displayed at the top of the page; this displays the list of available namespaces.
2. At the top of the page, click the **Wizards** drop-down list, and select **Data Export**.
3. On the first page of the wizard:
 - Enter the complete path and filename of the file that you are going to create to hold the exported data.

- From the drop-down lists, select a Namespace, Schema Name, and Table Name from which you want to export the data.
- Optionally select a character set from the **Charset** drop-down list; the default is Device Default.

Then click **Next**.

4. On the second page of the wizard, select which columns to export. Then click **Next**.
5. On the third page of the wizard, describe the format of the external file.
 - For **What delimiter separates your columns?**, click the option corresponding to the delimiter in this file.
 - Click the **Export column headers?** check box if you want to export column headers as the first line of the file.
 - For **String quote**, click an option to indicate how to start and end string data in this file.
 - For **Date format**, click an option to indicate the date format to use in this file.
 - For **Time format**, click an option to indicate the time format to use in this file.
 - Optionally click **Preview Data** to see what the results will look like.

Then click **Next**.

6. Review your entries and click **Finish**. The wizard displays the **Data Export Result** dialog box.
7. Click **Close**. Or click the given link to view the background tasks page.

In either case, the wizard starts a background task to do the work.